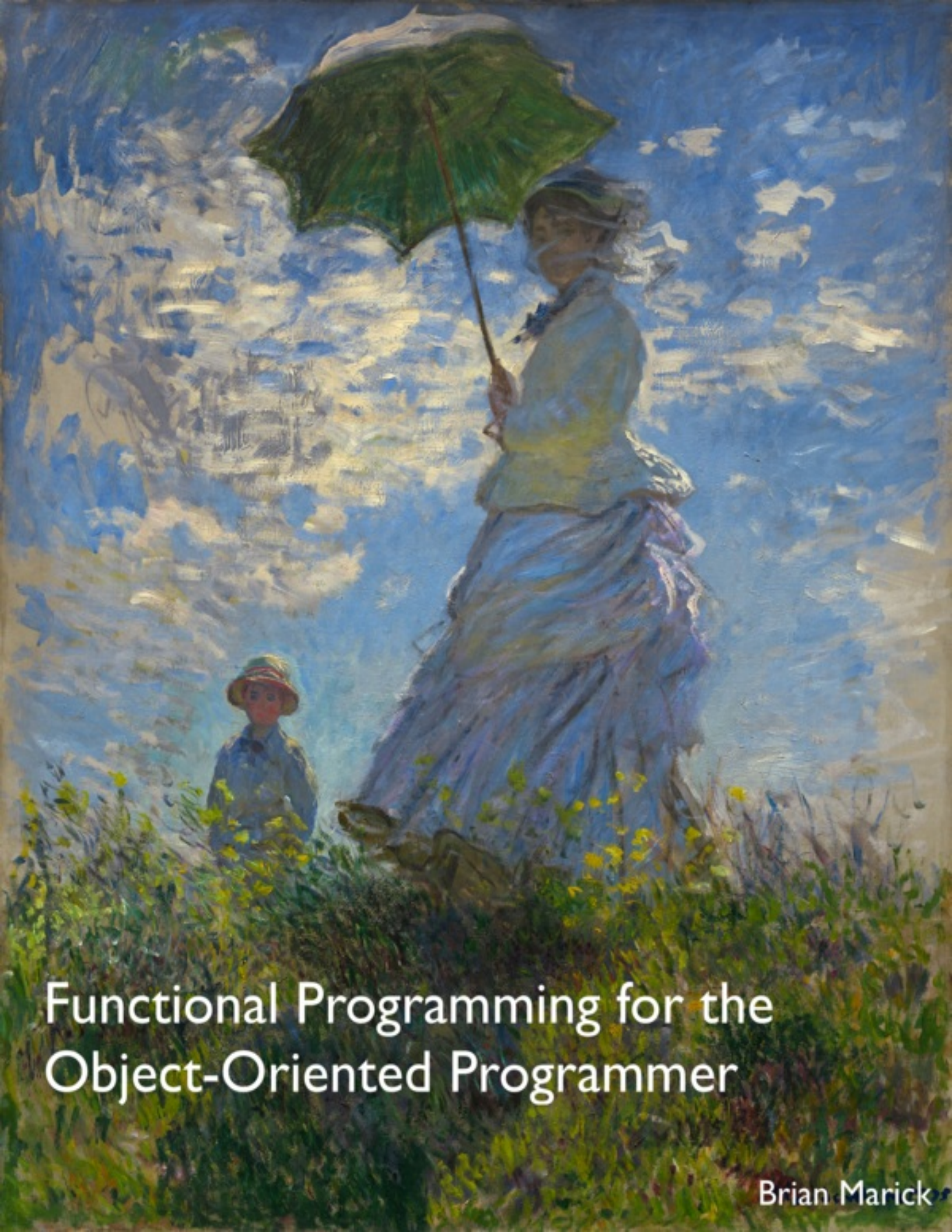




Functional Programming for the Object-Oriented Programmer

Brian Marick

Functional Programming for the Object-Oriented Programmer

Brian Marick

This book is for sale at <http://leanpub.com/fp-oo>

This version was published on 2015-04-30



* * * * *

This is a [Leanpub](#) book. Leanpub empowers authors and publishers with the Lean Publishing process. [Lean Publishing](#) is the act of publishing an in-progress ebook using lightweight tools and many iterations to get reader feedback, pivot until you have the right book and build traction once you do.

* * * * *

© 2012 - 2015 Brian Marick

To my father, who taught me to care about the work.

Table of Contents

[Introduction](#)

[Prerequisites](#)
[The flow of the book](#)
[About the exercises](#)
[About the cover](#)
[About links](#)
[There is a glossary](#)
[Getting help](#)
[Notes to reviewers](#)
[Changes to earlier versions](#)
[Acknowledgments](#)
[Advertisement](#)

[1. Just Enough Clojure](#)

[1.1 Installing Clojure](#)
[1.2 Working with Clojure](#)
[1.3 The Read-Eval-Print Loop](#)
[1.4 A note on terminology: “value”](#)
[1.5 Functions are values](#)
[1.6 Evaluation is substitution](#)
[1.7 Making functions](#)
[1.8 More substitution as evaluation](#)
[1.9 Naming things](#)
[1.10 Lists](#)
[1.11 Vectors](#)
[1.12 Vector? List? Who cares?](#)
[1.13 More on evaluation and quoting](#)
[1.14 Conditionals](#)
[1.15 Rest arguments](#)
[1.16 Explicitly applying functions](#)
[1.17 Loops](#)
[1.18 More exercises](#)

[I Embedding an Object-Oriented Language](#)

[2. Objects as a Software-Mediated Consensual Hallucination](#)

[3. A Barely Believable Object](#)

[3.1 Maps](#)
[3.2 I present to you an object](#)
[3.3 The class begins as documentation](#)
[3.4 Exercises](#)

[4. All the Class in a Constructor](#)

[4.1 One exercise](#)

5. Moving the Class Out of the Constructor

5.1 Let

5.2 Implementing instance creation

5.3 Message dispatch

5.4 Exercises

6. Inheritance (and Recursion)

6.1 Assertions

6.2 Method dispatch

6.3 Recursion

6.4 Exercises

6.5 Finishing up

6.6 Choose your own adventure

II The Elements of Functional Style

7. Basic Datatypes that Flow through Functions

7.1 The problem

7.2 The general strategy

7.3 Sets

7.4 Annotating maps

7.5 The arrow operator, ->

7.6 Processing sequences of maps

7.7 Destructuring arguments

7.8 Finishing up

7.9 Exercises

7.10 Avoiding argument passing

7.11 Information hiding

8. Embedding Functional Code in an OO Codebase

8.1 Tasty functional nuggets in a tub of OO ice-cream

8.2 Object-relational mapping: threat or menace?

8.3 The big picture

9. Functions That Make Functions

9.1 Closing over values

9.2 Lifting functions

9.3 Point-free definitions

9.4 Exercises

9.5 Higher-order functions from the object-oriented perspective

10. Branching and Looping in Dataflow Style

10.1 That pesky nil again

10.2 Continuation-passing style

10.3 Exercises

10.4 Expansions in evaluation

10.5 Extending continuation-passing style

10.6 Monads as data-driven extended continuation-passing style

10.7 A peek at metadata

10.8 Cond

10.9 Exercises

10.10 Lifting functions with monads

[10.11 Turning loops into straight-line flows](#)

[10.12 Exercises](#)

[10.13 Choose your own adventure](#)

[11. Pretending State is Mutable](#)

[11.1 Trees](#)

[11.2 Zippers](#)

[11.3 The do special form](#)

[11.4 A little problem of self-reference](#)

[11.5 Exercises](#)

[11.6 Editing trees](#)

[11.7 Zipper functions](#)

[11.8 Exercises](#)

[11.9 Thinking about zippers](#)

[11.10 Implementing zippers](#)

[11.11 The Worm Ouroborous](#)

[12. Pushing Bookkeeping into the Runtime: Laziness and Immutability](#)

[12.1 Laziness](#)

[12.2 Infinite data](#)

[12.3 Implementing your own lazy sequences](#)

[12.4 Exercises](#)

[12.5 Turning the external world into a lazy sequence](#)

[12.6 Exercises](#)

[12.7 Immutability through structure sharing](#)

[12.8 Implications for the object-oriented programmer](#)

[13. Pattern Matching](#)

[13.1 Moving conditionals into arguments](#)

[13.2 Sequence structure](#)

[13.3 Arbitrary arguments](#)

[13.4 Summary](#)

[13.5 Exercises](#)

[14. Generic Functions](#)

[14.1 Generic functions for object-oriented programming](#)

[14.2 A digression on verbs](#)

[14.3 Polymorphism without a privileged argument](#)

[14.4 On your own](#)

[III Coda](#)

[IV A Mite More on Monads \(Optional\)](#)

[15. Implementing the Sequence Monad](#)

[15.1 Monadic values and binding values](#)

[15.2 Defining the monad](#)

[15.3 The monad bestiary](#)

[15.4 Exercise](#)

- [15.5 Refactoring to a monad transformer](#)
- [16. Functions as Monadic Values](#)
 - [16.1 A function wrapper](#)
 - [16.2 A counting monad](#)
 - [16.3 Exercises](#)
 - [16.4 The State monad](#)
 - [16.5 Exercises](#)

[V To Ruby... And Beyond! \(Optional\)](#)

- [17. The Class as an Object](#)
 - [17.1 The wrong implementation](#)
 - [17.2 A better implementation](#)
 - [17.3 The story so far](#)
 - [17.4 Exercises](#)
- [18. The Class That Makes Classes](#)
 - [18.1 klass in pictures \(version 1\)](#)
 - [18.2 The first klass implementation](#)
 - [18.3 Class-defining functions](#)
 - [18.4 klass in pictures \(version 2\)](#)
 - [18.5 The revised klass implementation](#)
 - [18.6 Exercises](#)
- [19. Multiple Inheritance, Ruby-style](#)
 - [19.1 New terminology](#)
 - [19.2 How modules work](#)
 - [19.3 How dispatch works with modules](#)
 - [19.4 Adding module to the class structure](#)
 - [19.5 Exercises](#)
- [20. Dynamic Scoping and send-super](#)
 - [20.1 Dynamic scope](#)
 - [20.2 Implicit this](#)
 - [20.3 Ruby's super](#)
 - [20.4 A design](#)
 - [20.5 Exercises](#)
- [21. Making an Object out of a Message in Transit](#)
 - [21.1 The map](#)
 - [21.2 The programmer interface](#)
 - [21.3 Implementation](#)
 - [21.4 Exercises](#)

[VI Glossary](#)

Introduction

Many, many of the legendary programmers know many programming languages. What they know from one language helps them write better code in another one. But it's not really the language that matters: adding knowledge of C# to your knowledge of Java doesn't make you much better. Those languages are too similar: they encourage you to look at problems in pretty much the same way. You need to know languages that conceptualize both problems and solutions in substantially different ways.

Once upon a time, object-oriented programming was a radical departure from what most programmers knew. So learning it was both hard and mind-expanding. Nowadays, the OO style (or some approximation to it) is the dominant one, so ambitious people need to seek out different styles.

The functional programming style is nicely different from the OO style, but there are many interesting points of comparison between them. This book aims to teach you key elements of the functional style, helping you take them back to your OO programming.

There's a bit more, though: although the functional style has been around for many years, it's recently become trendy, partly because language implementations keep improving, and partly because functional languages are better suited for the problem of running one program on multiple cores. Some trends with a lot of excitement behind them wither, but others (like object-oriented programming) succeed immensely. If the functional style becomes commonplace, this book will position you to be around the leading edge of that wave.

There are many functional languages. There are arguments for learning the purest of them (Haskell, probably). But it's also worthwhile to learn a slightly-less-pure language if there are more jobs available for it or more opportunities to fold it into your existing projects. According that standard,

Clojure and Scala—both of which piggyback on the Java runtime—stand out. This book will use Clojure.

Prerequisites

You need to know at least one object-oriented programming language. Anything will do: Objective-C, C++, C#, Java, Ruby, Python: you name it.

You need to be able to start a program from the command line.

The flow of the book

I'll start by teaching you [just enough Clojure](#) to be dangerous. Then, in Part 1 of the book, we'll use that knowledge to [embed an object-oriented language](#) within Clojure in a more-or-less conventional way. That'll give you an understanding of how objects typically work “under the hood.” It will also introduce you to more Clojure features and give you practice with some fundamentals of functional style (functions as data, [recursion](#), the use of general-purpose data types).

After about 50 pages of object system implementation, we'll have implemented an object system something like Java's, and we'll also have exhausted the example's usefulness for teaching functional style. Those interested in object models as a topic in themselves can temporarily branch off to the optional [Part V](#), which fleshes out the Java-like object model into one inspired by Ruby's.

Part 2, [Elements of Functional Style](#), is where I fulfill the promise to show you how functional languages help you “conceptualize both problems and solutions in substantially different ways.”

- The first two chapters are about functional programmers' habit of using [basic data types that “flow”](#) through a series of functions. You'll have seen that in Part 1, but these chapters focus on it explicitly and also cover how this style can be hidden inside an object-oriented program.
- The next chapter, [Functions That Make Functions](#), shows one of the main tools of abstraction for functional programs: functions that create

other functions in a parameterized way.

- When data is flowing through functions, `if` statements and loops complicate the code. The [next chapter](#) is about how to eliminate them from the visible parts of your programs. Most of the emphasis will be an introduction to *monads*. Much as classes abstract away details like the concrete representation of data, monads abstract away details like how the results of a series of computations should be combined. This chapter introduces only some simple monads, but an [optional part of the book](#) describes more interesting ones and will give you a deeper understanding of how they work.
- The languages you're used to don't allow you to change the value of 1 to be, say, 2. Clojure, along with many other functional languages, extends the same *immutability* to all data. Once a list is created, you can't add to it or change it. In Part 1, you'll have seen that's not as crazy as it seems—at least for relatively flat data structures. However, things get more difficult when working with deeper structures. New approaches are needed for tasks like “change that 4 way down in that tree to a 5.” In the [next chapter](#), I'll show how the “zipper” datatype and its associated functions can be used for just such a task. I'll also work through the implementation of zippers, for two reasons:

First, zippers illustrate how functional programmers often solve problems by writing code that builds a data structure that represents a computation, then pass that structure around to be used only when (and if) it's needed.

Second, it provides a more complex example of using basic data types than does the first chapter. We'll see how it's useful to think about the *shape* of the data rather than its type or class.

- Throughout the book, I will have teased you with what seems to be deliberately and ridiculously inefficient code. [In the next chapter](#), I'll show how that apparent inefficiency is an illusion. In reality, we're relying on the runtime to make two kinds of optimizations for us:

Lazy evaluation: In functional languages, it's common for some or all values to be *lazy*, meaning that they (and their component parts) are only calculated when demanded. I'll show how this collapses what

appear to be many loops into just one. More importantly, I'll show how it allows you to let free of the idea that you must be able to calculate in advance *how much* data you'll need. Instead, you can just ask for *all of it* and let the runtime avoid generating too much.

Sharing structure behind the scenes: While you can't mutate functional data structures, the language runtime can. I'll show an example of how the runtime can optimize away the wasteful copying your program appears to be doing. I'll also discuss the implications of adopting immutability in object-oriented programs.

- When you start looking at data in terms of its shape, it begins to seem reasonable to have functions decide what code to run based not on explicit `if` tests but rather on [matching patterns against shapes](#).
- [Generic functions](#) support a verb-centered way of thinking about the world: there are actions that can apply very broadly. The specifics of an action depend on some properties (determined at runtime) of the values it's applied to. Generic functions are the flip side of the noun-centered approach taken by object-oriented languages.

That finishes the book, except for the optional parts on object models and monads.

About the exercises

I've taught some of the material in this book in a two-day tutorial. Most of the classroom time is spent doing exercises. They add a lot of value; you should do them. Failing that, you should at least read them and perhaps look at my solutions, as I'm not shy about having later sections build on functions defined by the exercises.

You can find exercise solutions in this book's [Github repository](#). If you use the Git version management tool, you can fetch the code like this:

```
1 git clone git://github.com/marick/fp-oo.git
```

You can also use your browser to download [Zip or Tar files](#).

At last resort, you can browse the repository through your browser. The exercise descriptions include a clickable link to the solutions.

The repository also includes some code that you'll load before starting some of the exercises. The instructions for loading are given [later](#).

Testing

I'm a big fan of test-driven design and testing in general, so I've written tests for the presupplied code and my exercise solutions. If you're interested in how to test Clojure code, you can find those tests [also on Github](#).

These tests use my own testing library, [Midje](#). With it, you can run my tests like this:

```
1 709 $ lein midje
2 ## Many lines of output, normally something I hate
3 ## to see from tests. They appear in this case because
4 ## my solution files print the results of examples when
5 ## they're loaded.
6 All claimed facts (1117) have been confirmed.
7 710 $
```

To see how to install Midje, see its [quickstart](#).

About the cover

The painting is “Woman with a Parasol” (1875) by Claude Monet, a French Impressionist painter. Impressionist paintings are characterized by a fascination with color and shadow, which you can certainly see here.

I chose this painting because I'm struck by the way the woman (Monet's wife) is looking down on us with something of a haughty expression. We've surprised her—interrupted her—and we need to give an account of ourselves.

This “prove you're worthy of my attention” attitude has been, I'm sorry to say, all too often characteristic of functional programmers' interactions with the rest of the programming world. Although that's changing, it contributes to functional programming's reputation as forbiddingly rigid, mathematical, and unforgiving of human limitations.

I want this book to be the opposite: informal yet serious, viewing programs as code to be grown rather than mathematical objects to contemplate. That's the reason I went looking for an Impressionist painting for the cover. The loose brush strokes, the non-rigid edges, the fascination with variety over formal choices—even the embrace of awkwardness (I really don't like the darker blue squiggles on the left side of the cloud): all these things fit my style of explanation.

About links

In the PDF version, links within a book appear as colored text, and links to external sites appear in footnotes. Both are clickable.

In the Kindle version, all links appear inline. That makes tasks like installing Clojure or doing exercises awkward: you can't easily read the URLs on the Kindle and type them on your computer. (You have to actually follow the links on the Kindle to see the URLs.) I recommend getting both the Kindle and PDF versions. Use the latter when you need to work with links. (That should only be in exercises, which you can't do on the Kindle anyway.)

There is a glossary

I never notice a book has a glossary until I turn the last page of the last chapter and discover it. If you're like me, you'll be glad to read now that there's a [glossary](#). (However, if you're like me, you also skim introductions and so will miss this paragraph.)

Getting help

There's a [mailing list](#).

Notes to reviewers

You can put your comments on the mailing list or by [filing issues](#) on Github. It doesn't matter to me. Comments that would benefit from group discussion are better sent to the mailing list.

Please tag your comments with the version of the book to which they apply. The version you're reading now is **garrulous gastropod**.

Changes to earlier versions

garrulous gastropod

- [Chapters 19](#) (Ruby-style multiple inheritance), [20](#) (dynamic binding and send-super), and [21](#) (objectifying messages in transit) are new.
- The Class as an Object chapter (formerly chapter 7) has been moved into the optional Part V.
- Deleted the last set of exercises in “Inheritance” (chapter 6), moving them into Part V.
- Shouldn't have used “seq” as shorthand for “lazy sequence” since non-lazy lists are also technically seqs. Replaced with “lazyseq”. This only matters in [chapter 13](#) (and even then the ideas are unaffected).
- Changed the name of the method that makes new objects from a to make.

fastidious flounder

- A new chapter (15) on [generic functions](#).
- Bug fixes.
- A [coda](#) containing one of my favorite quotes (after chapter 15).

ecstatic earthworm

- Three new chapters (12, 13, and 14): The zipper data structure as an example of [working around the constraint of immutability](#), the implications of [lazy evaluation](#), and [pattern matching](#).
- Added material about using Light Table as an alternative to Leiningen.
- Miscellaneous fixes to the text.

discursive diplodocus

- Another reorganization of the unwritten chapters. Part 2 of the book now focuses much more explicitly on characterizing “the elements of functional style”. That more clearly fulfills the promise the title of the

book makes. See the description of the [flow of the book](#) (above) to keep yourself oriented.

- Two new chapters begin [Part 2](#). There's then an unfinished chapter. None of the later chapters will depend on it.
- Five new exercises in [“Functions That Make Functions”](#). That chapter has been somewhat rewritten. Those who've read it before need only read the new sections on [lifting functions](#) and [higher-order functions from the object-oriented perspective](#).
- That chapter is followed by [a chapter](#) on avoiding `if` expressions. It also introduces monads.
- Two optional [chapters on monads](#).

crafty chameleon

- The flow of the book has changed, as described [earlier](#).
- [“Inheritance \(and Recursion\)”](#) and [“The Class as an Object”](#) complete Part 1. [“Functions That Make Functions”](#) begins Part 2. [“The Class That Makes Classes”](#) begins Part 4.
- I've added tests for [exercise sources and solutions](#).

bashful barracuda

- Use `(load-file "foo.clj")` instead of `(load "foo")`
- Added the glossary.
- Added the first four chapters of Part 1.
- Added an eighth exercise to the first chapter.

Acknowledgments

I thank these people for commenting on the mailing list and filing bug reports:

Adrian Mowat,

Aidy Lewis,

Ben Moss,

Chris Pearson,

Greg Spurrier,

Jim Cooper,

Juan Manuel Gimeno Illa,

Julian Gamble,
Matt Mower,
Meza,
Mike Suarez,
Oliver Friedrich,
Ondrej Beluský,
Robert D. Pitts,
Robert “Uncle Bob” Martin,
Roberto Mannai,
Stephen Kitt,
Suvash Thapaliya,
Ulrik Sandberg,
and
Wouter Hibma.

And for other help, thanks to Dawn Marick, John MacIntyre, Paul Marick,
and Sophie Marick.

Advertisement

I would be happy to do Ruby or Clojure contract programming or
consulting for you. I’m also competent to coach teams on working in the
Agile style.

My contact address is marick@exampler.com.

1. Just Enough Clojure

Clojure is a variant of Lisp, one of the oldest computer languages. You can view it as a small core surrounded by a lot of library functions. In this chapter, you'll learn most of the core and functions you'll need in the rest of the book.

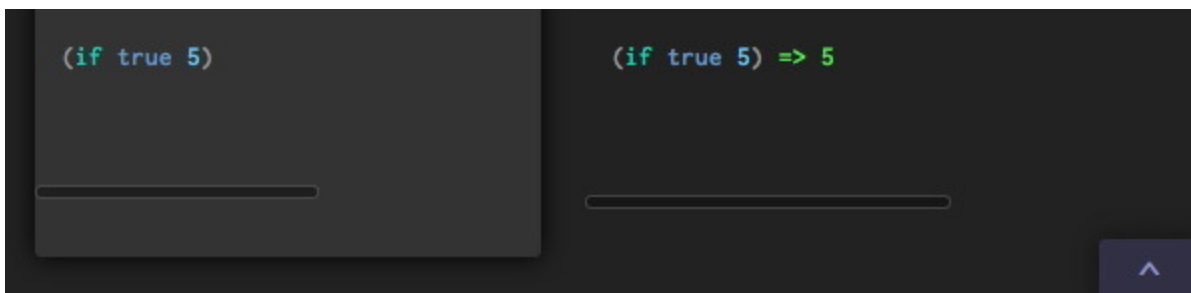
1.1 Installing Clojure

At this moment, there seem to be two attractive choices for running Clojure. In this section, I'll describe how to get started with either. If you have trouble, visit the [installation troubleshooting page](#) or ask for help on the [mailing list](#).

Light Table

One simple way to install Clojure is to install the [Light Table playground](#) instead. It is something like an IDE for Clojure, with the interesting property that it evaluates expressions as you type them. You can find out more at [Chris Granger's site](#).

You'll work in something Light Table calls the “Instarepl”. That's its version of the Clojure *read-eval-print loop* (typically called “the repl”). You type text in the left half of the window, and the result appears in the right. It looks like this:



Light Table

The book doesn't show input and output in the same split screen style. Instead, it shows input preceded by a prompt, and output starting on the next line:

```
1 user=> (if true 5)
2 5
```

Important: as of this writing, Light Table does not automatically include all the normal repl functions. You have to manually include them with this magic incantation:

```
1 (use 'clojure.repl)
```

(Note the single-quote mark: it's required.)

Leiningen

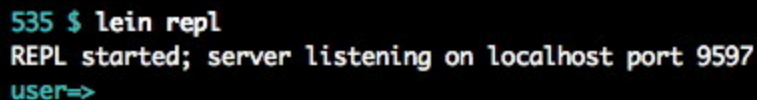
If you want to run Clojure from the command line, first install [Leiningen](#). Go to its page and follow the installation instructions.

When you're finished, you can type the following to your command line:

```
1 lein repl
```

That asks Leiningen to start the *read-eval-print loop* (typically called “the repl”). Don't be alarmed if nothing happens for a few seconds: because the Java Virtual Machine is slow to start, Clojure is too.

All is well when you see something like this:

A terminal window with a black background. The prompt is '535 \$'. The command 'lein repl' has been entered. The output is 'REPL started; server listening on localhost port 9597'. Below the output, the prompt 'user=>' is visible.

```
535 $ lein repl
REPL started; server listening on localhost port 9597
user=>
```

A command line repl, with a fashionable black background

From now on, I won't use screen shots to show repl input and output. Instead, I'll show it like this:

```
1 user=> (if true 5)
2 5
```

The most important thing you need to know now is how to get *out* of the repl. That's done like this:

```
1 user=> (exit)
```

On Unix machines, you can also use Control-D to exit.

1.2 Working with Clojure

All the exercises in this book can be done directly in the repl. However, many editors have a “clojure mode” that knows about indentation conventions, helps you avoid misparenthesizations, and helpfully colorizes Clojure code.

- Emacs: [clojure-mode](#)
- Vim: one is [VimClojure](#)

You can copy and paste Clojure text into the repl. It handles multiple lines just fine.

Many people like to follow along with the book by copying lines from the PDF version and pasting them into the repl. That's usually safe. However, my experience is that such text is sometimes (but not always!) missing some newlines. That can cause perplexing errors when it results in source code ending up on the same line as a to-the-end-of-the-line comment. In later chapters (where there are more lines with comments), I'll provide text files you can paste from.

If you want to use a Clojure command to load a file like (for example) `solutions/add-and-make.clj`, use this:

```
1 user> (load-file "solutions/add-and-make.clj")
```

Warning: I'm used to using `load` in other languages, so I often reflexively use it instead of `load-file`. That leads to this puzzling message:

```
1 user=> (load "sources/without-class-class.clj")
2 FileNotFoundException Could not locate sources/without-class-class.
3 clj__init.class or sources/without-class-class.clj.clj on classpath:
4 clojure.lang.RT.load (RT.java:432)
```

The clue to my mistake is the “.clj.clj” on the next-to-last line.

1.3 The Read-Eval-Print Loop

Here’s a use of the repl:

```
1 user=> 1
2 1
```

More is happening than just echoing the input to output, though. This profound calculation requires three steps.

First, the *reader* does the usual parser thing: it separates the input into discrete tokens. In this case, there’s one token: the string "1". The reader knows what numbers look like, so it produces the number 1.

The reader passes its result to the *evaluator*. The evaluator knows that numbers evaluate to themselves, so it does nothing.

The evaluator passes its result to the *printer*. The printer knows how to print numbers, so it does.

Strings, truth values, and a number of other types play the same self-evaluation game:

```
1 user=> "hi mom!"
2 "hi mom!"
3
4 user=> true
5 true
```

Let’s try something more exciting:

```
1 user=> *file*
2 "NO_SOURCE_PATH"
```

`*file*` is a *symbol*, which plays roughly the same role in Clojure that identifiers do in other languages. The asterisks have no special significance: Clojure allows a wider variety of names than most languages. Most importantly, Clojure allows dashes in symbols, so Clojure programmers

prefer them to underscores. StudlyCaps or interCaps style is uncommon in Clojure.

Let's step through the read-eval-print loop for this example. The reader constructs the symbol from its input characters. It gives that symbol to the evaluator. The evaluator knows that symbols do *not* evaluate to themselves. Instead, they are associated with (or *bound to*) a value. `*file*` is bound to the name of the file being processed or to "NO_SOURCE_PATH" when we're working at the repl.

Here's a slightly more interesting case:

```
1 user=> +  
2 #<core$PLUS_clojure.core$PLUS_@38a92aaa>
```

The value of the symbol `+` is a *function*. (Unlike many languages, arithmetic operators are no different than any other function.) Since functions are executable code, there's not really a good representation for them. So, as do other languages, Clojure prints a mangled representation that hints at the name.

Now let's walk through what happens when you ask the repl to add one and two to get three:

```
1 user> (+ 1 2)  
2 3
```

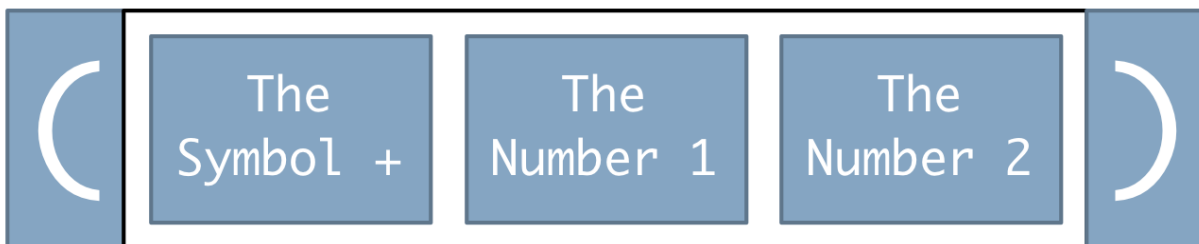
In this case, the first token is a parenthesis, which tells the reader to start a list, which I'll represent like this:



The second token represents the symbol `+`. It's put in the list:



The next two tokens represent numbers, and they are added to the list. The closing parenthesis signals that the list is complete:



The reader's job is now done, and it gives the list to the evaluator. Lists are a case we haven't described before. The evaluator handles them in two steps:

1. It recursively evaluates each of the list elements.
 - The symbol `+` evaluates to the function that adds.
 - As before, numbers evaluate to themselves.
2. The first value from a list *must* be a function (or something that behaves like a function). The evaluator *applies* the function to the remaining values (its arguments). The result is the number 3.

The printer handles 3 the same way it handled 1.

To emphasize how seriously the evaluator expects the first element of the list to be a function, here's what happens if it's not:

```
1 user=> (1 2 3)
2 java.lang.ClassCastException: java.lang.Integer cannot be cast to
3 clojure.langIFn (NO_SOURCE_FILE:0)
```

(I'm afraid that Clojure error messages are sometimes not as clear as they might be.)

1.4 A note on terminology: “value”

Since Clojure is implemented on top of the Java runtime, things like functions and numbers are Java objects. I’m going to call them *values*, though. In a book making distinctions between object-oriented and functional programming, using the word “object” in both contexts would be confusing. Moreover, Clojure values are (usually) used very differently than Java objects.

1.5 Functions are values

Clojure can do more than add. It can also ask questions about values. For example, here’s how you ask if a number is odd:

```
1 user> (odd? 2)
2 false
```

There are other predicates that let you ask questions about what kind of value a value is:

```
1 user> (number? 1)
2 true
```

You can ask the number? question of a function:

```
1 user> (number? +)
2 false
```

If you want to know if a value is a function, you use fn?:

```
1 user> (fn? +)
2 true
3 user> (fn? 1)
4 false
```

It’s important to understand that functions aren’t special. Consider these two expressions:

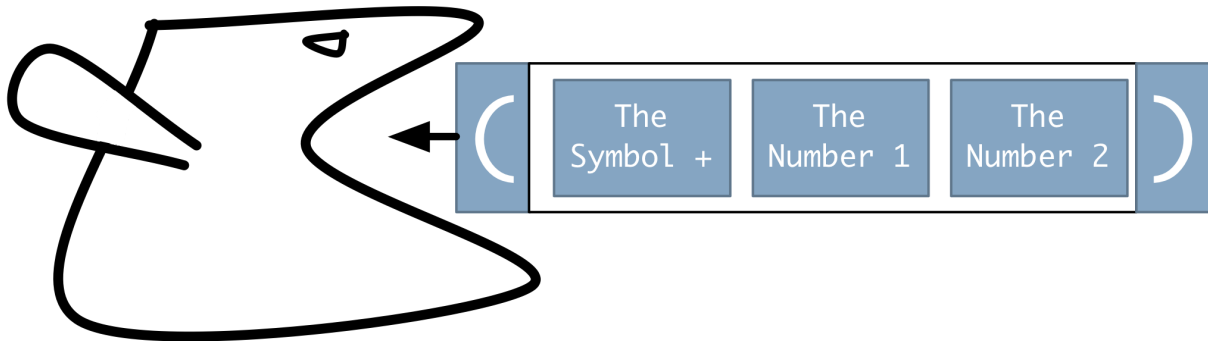
```
1 (+ 1 2)
2 (fn? +)
```

In one case, the `+` function gets called; in the other, it's examined. But the difference between the two cases is solely due to the *position* of the symbol `+`. In the first case, its position tells the evaluator that its function is to be executed; in the second, that the function is to be given as an argument to `fn?`.

1.6 Evaluation is substitution

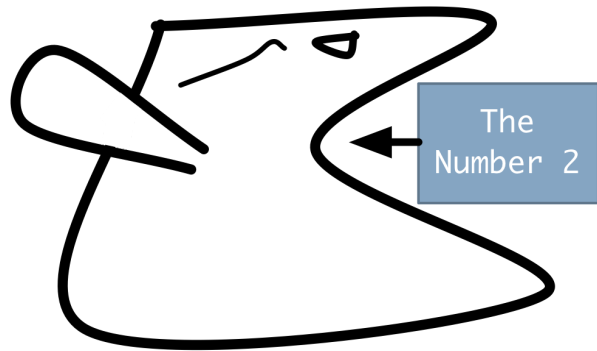
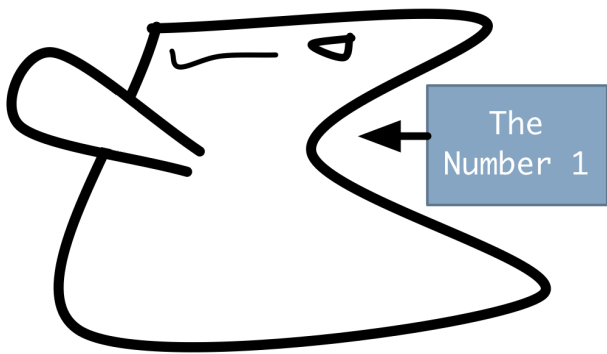
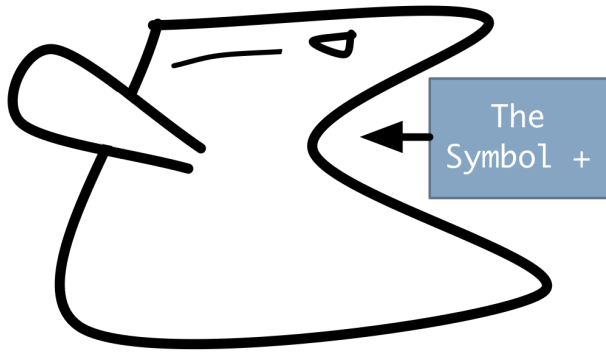
Let's look in more detail at the evaluator. This may seem like overkill, but one of the core strengths of functional languages is the underlying simplicity of their evaluation strategies.

To add a little visual interest, let's personify the evaluator as a bird. That's appropriate because parent birds take in food, process it a little, then feed the result to their babies. The evaluator takes in data structures from the reader, processes them, and feeds the result to the printer. Here's a picture of the evaluator at taking in a list:



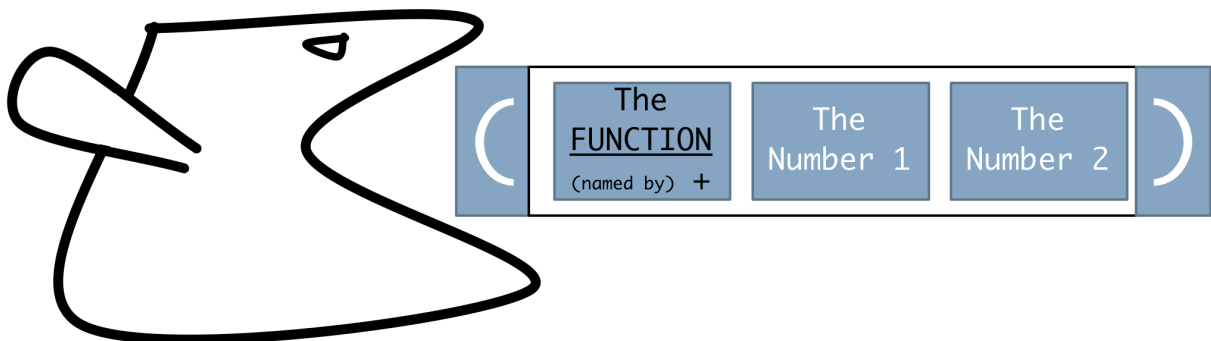
An evaluator is a lazy bird, though. Whenever it sees a compound data structure, it summons other evaluators to do part of the work.

A list is a compound data structure, so (in this case) three evaluators are set to work:



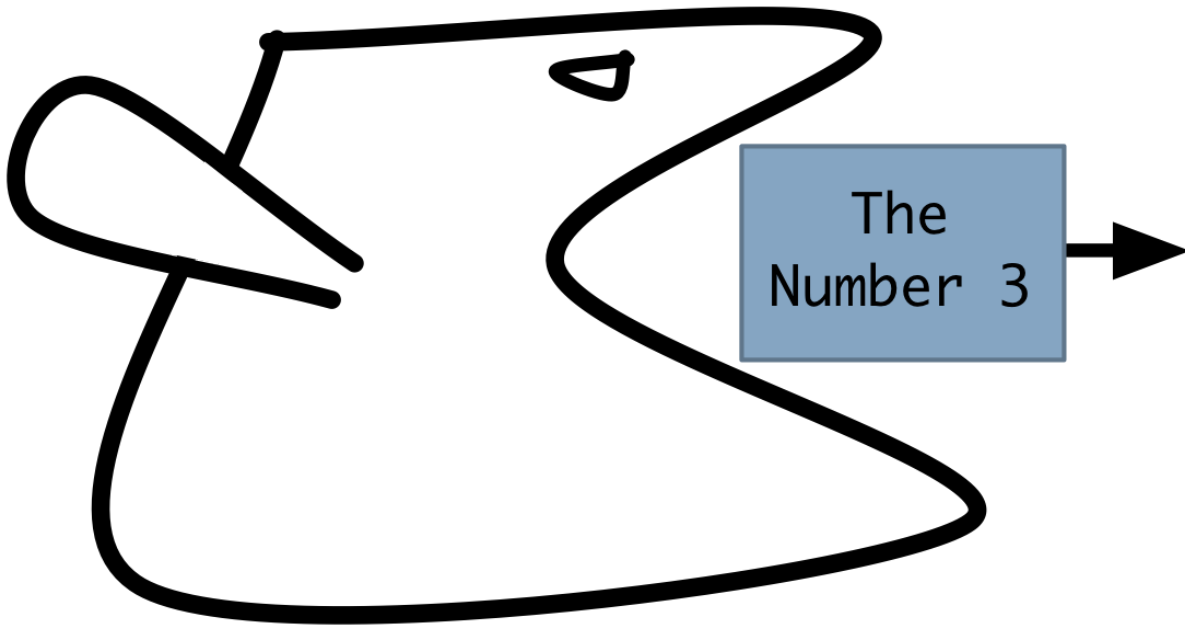
The bottom two have an easy job: numbers are already digested (evaluate to themselves), so nothing need be done. The top bird must convert the symbol into the function it names.

Each of these sub-evaluators feeds its result to the original evaluator, which substitutes those values for the originals, making a list that is almost—but not quite—the same:



(The symbol has been substituted with a function.)

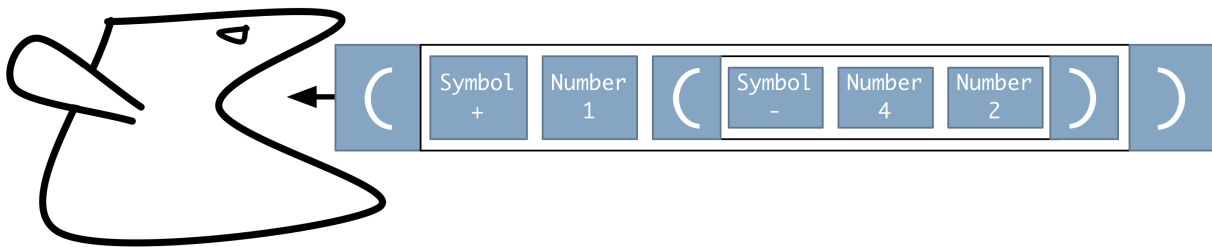
The original evaluator must process this list according to the rules for that data structure. That means calling the function in the first position and giving it the rest of the list as arguments. Since this is the top-level evaluator, that result is provided as nourishment to the printer:



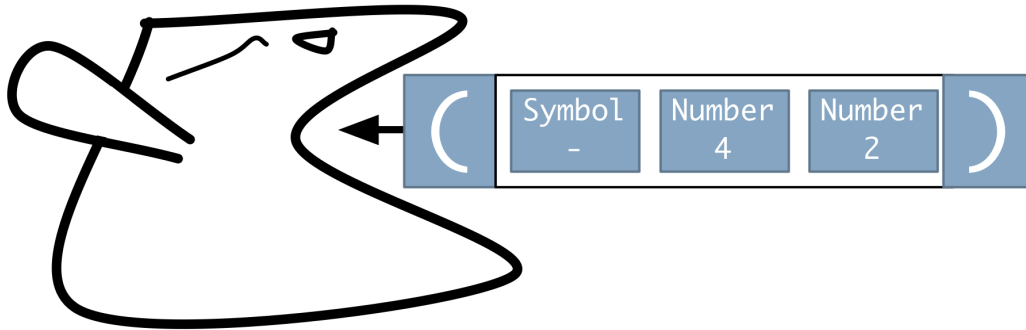
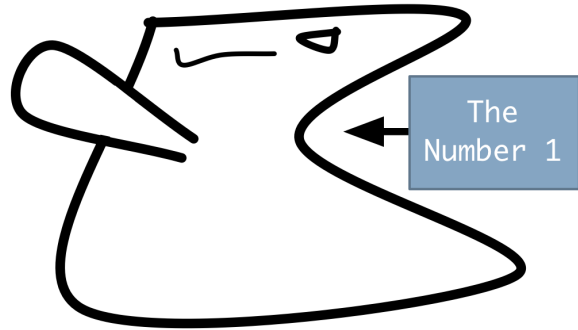
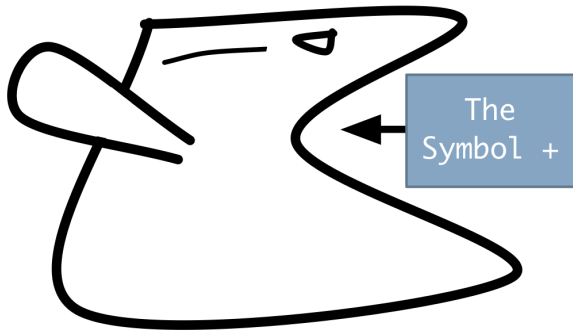
Now consider a longer code snippet, like this:

```
1 user=> (+ 1 (- 4 2))
2 3
```

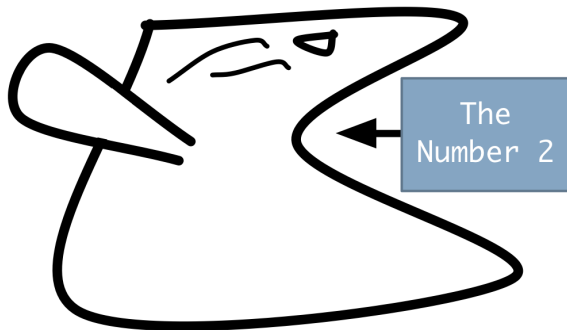
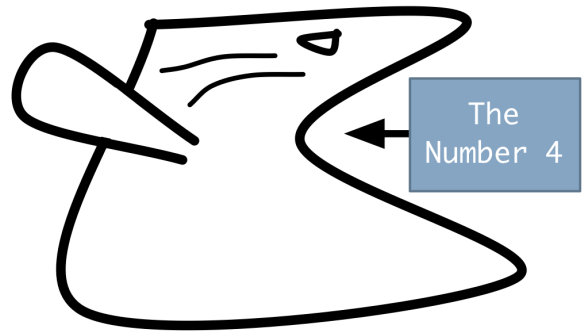
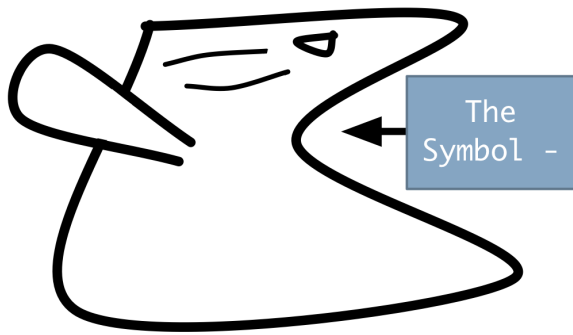
The first (top-level) evaluator is to consume this:



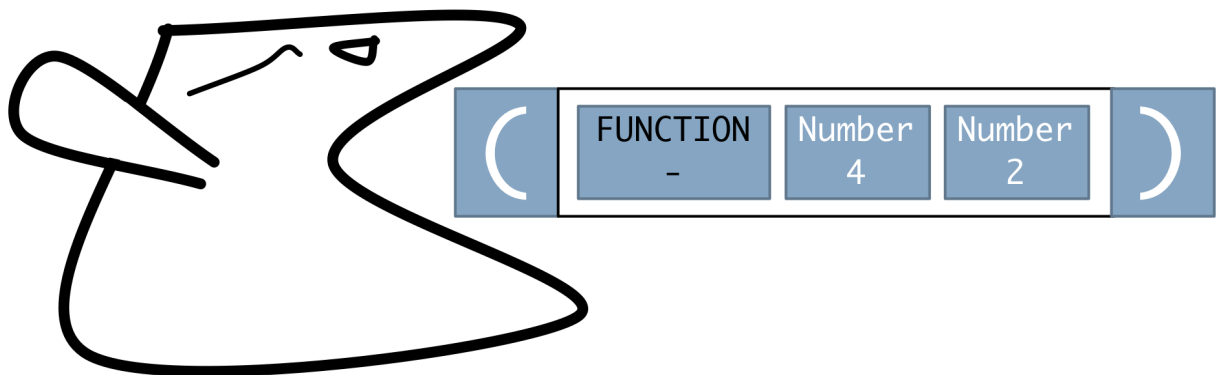
“Too complicated!” it thinks, and recruits three fellows for the three list elements:



The first two are happy with what they've been fed, but the last one has gotten another list, so it recruits three *more* fellows:

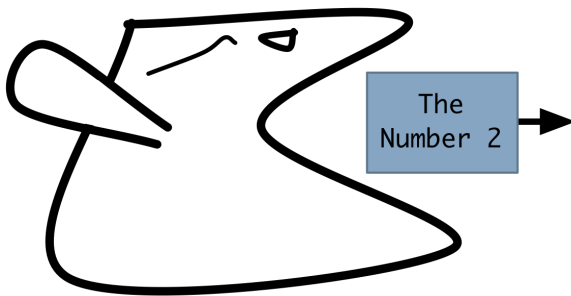
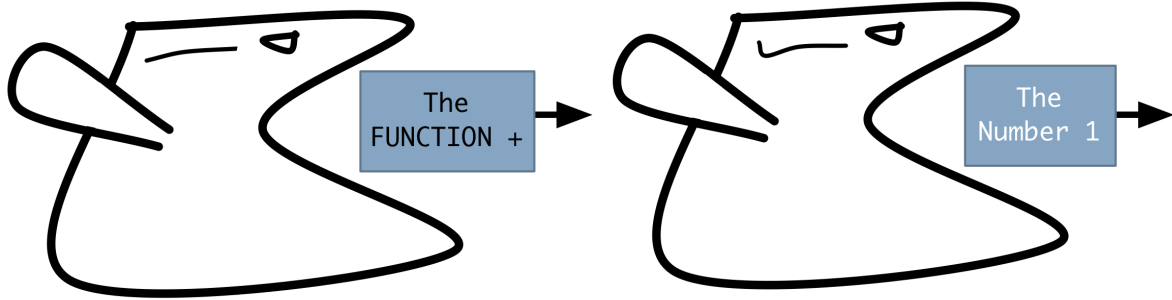


When the third-level birds finish, the lazy second-level bird substitutes the values they provide, so it now has this list:

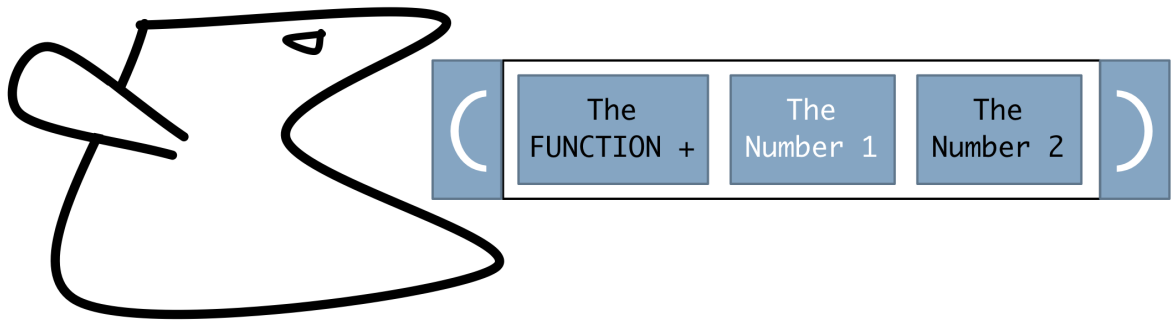


It applies the - function to 4 and 2, producing the new value 2.

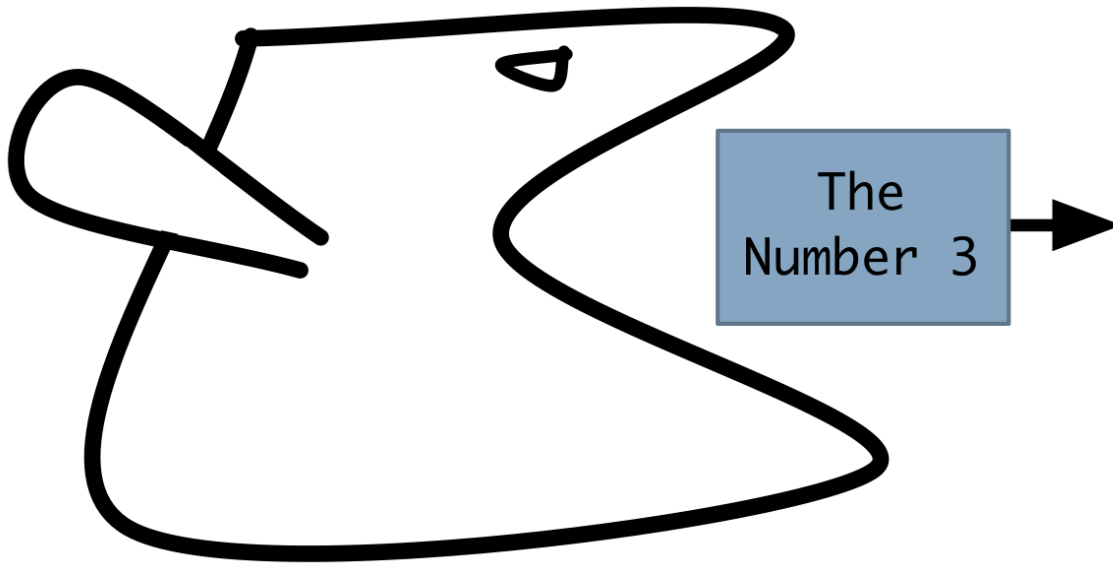
When all the second-level birds are done, they feed their values to the top-level evaluator:



The top-level evaluator substitutes them in:



It applies the function to the two arguments, yielding 3, which it feeds to the printer:

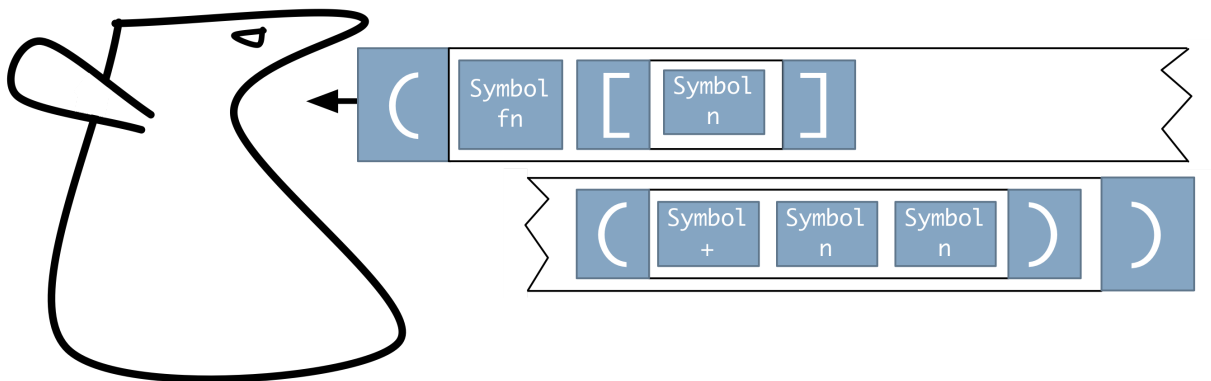


1.7 Making functions

Suppose you type this:

```
1 user> (fn [n] (+ n n))
```

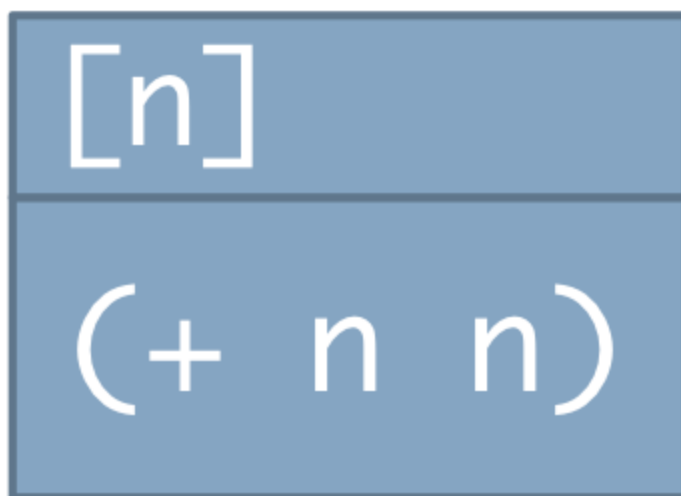
That's another list handed to the evaluator, like this:



As another list headed by a symbol, this looks something like the function applications we've seen before. However, `fn` is a *special* symbol, handled specially by any evaluator. There are a smallish number of special symbols in Clojure. The expressions headed by a special symbol are called *special forms*.

In the case of this special form, the evaluator doesn't recruit a flock to handle the individual elements. Instead, it conjures up a new function. In this case, that function takes a single parameter¹, *n*, and has as its *body* the list `(+ n n)`. Note that the parameter list is surrounded by square brackets, not parentheses. (That makes it a bit easier to see the structure of a big block of code.)

I'll draw function values like this:



The functions you create are just as “real” as the functions that come pre-supplied with Clojure. For example, they print just as helpfully:

```
1 user=> (fn [n] (+ n n))
2 #<user$eval66$fn__67 user$eval66$fn__67@5ad75c47>
```

Once a function is made, it can be used. How? By putting it in the first position of a list:

```
1 user> ( (fn [n] (+ n n)) 4)
2      _____
3      8
```

(I've used the underlining to highlight the first position in the list.)

Although more cumbersome, the form above is conceptually no different than this:

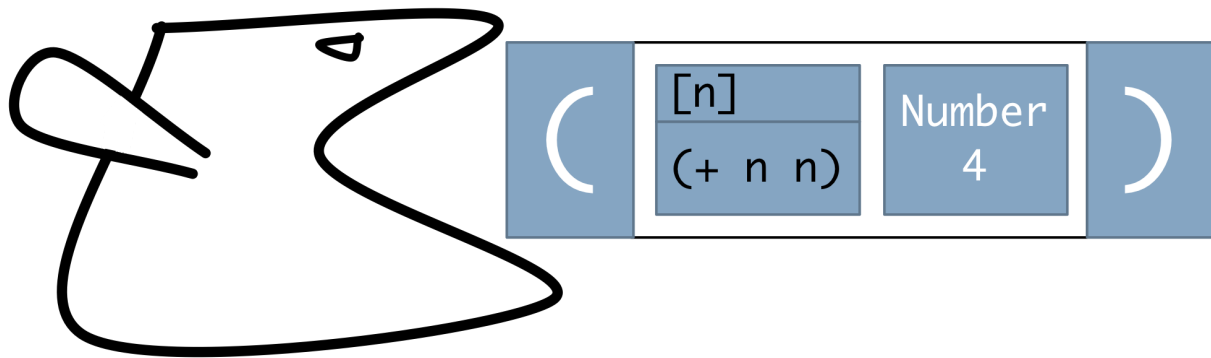
```
1 user> (+ 1 2)
2 3
```

In both cases, a function value will be applied to some arguments.

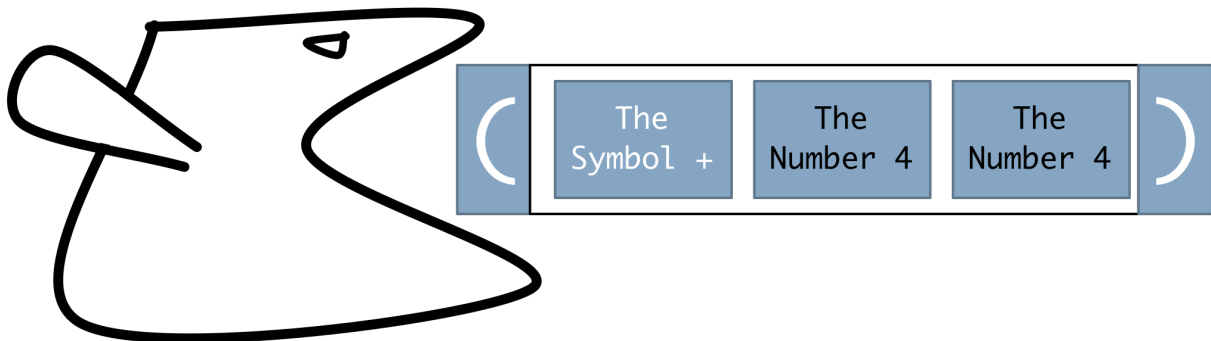
1.8 More substitution as evaluation

In `( (fn [n] (+ n n)) 4)`, our doubling function is applied to 4. How, precisely, does that work?

The whole thing is a list, so the top-level evaluator recruits two lower-level evaluators. One evaluates a list headed by the symbol `fn`; the other evaluates a number. When they hand their values to the top-level evaluator, it substitutes, so it now has this:



It processes this function by *substituting* the actual argument, 4, for its matching parameter, *n*, *anywhere in the body of the function*:



Hey! Look! A list! We know how to handle a *list*. The list elements are evaluated by sub-evaluators, and the resulting function (the `+` function value) is applied to the resulting arguments. Were the `+` function value a

user-written function, it would also be evaluated by substitution. So would be most of the Clojure library functions. There are some *primitive* functions, though, that are evaluated by Java code. (It can't be turtles *all* the way down.)

Despite being tedious, this evaluation procedure has the virtue of being simple. (Of course, the real Clojure compiler does all sorts of optimizations.) But you may be thinking that it has the disadvantage that it **can't possibly work**. What if the code contained an assignment statement, something like the following?

```
1 (fn [n]
2   (assign n (+ 1 n))
3   (+ n n))
```

It doesn't make sense to substitute a number into the left-hand side of an assignment statement. (What are you going to do, assign 4 the new value 5?) And even if you *did* change *n*'s value on the first line of the body, the two instances of *n* on the second line have already been substituted away, something like this:

```
1 (assign n (+ 1 4))
2 (+ 4 4))
```

Therefore, the assignment can't have an effect on any following code.

Clojure avoids this problem by not having an assignment statement. With one exception that you'll see shortly, there is no way to change the binding between a symbol and a value.

"Still," you might object, "what if the argument is some data structure that the code modifies? If you pre-substitute the whole data structure wherever the parameter appears, changes to the data structure in one part of the function won't be seen in other parts of the function!" Code suffering from this problem would look something like this:

```
1 (fn [tree]
2   (replace-left-branch tree 5)
3   (duplicate-left-branch tree))
```

Because of substitution, the tree on the last line that's having its left branch duplicated is not the tree that had its left branch replaced on the previous line.

Clojure avoids this problem by not allowing you to change trees, sets, vectors, lists, hashmaps, strings, or anything at all except a few special datatypes. In Clojure, you don't modify a tree, you create an entirely new tree containing the modifications. So the function above would look like this:

```
1 (fn [tree]
2   (tree-with-duplicated-left-branch
3     (tree-with-replaced-left-branch tree 5)))
```

We'll be discussing the details of all this in the chapter on [immutability](#). For now, ignore your efficiency concerns—"Duplicating a million-element vector to change *one* element?!"—and delegate them to the language implementor. Also hold off on thinking that programming without an assignment statement has to be crazy hard—it's part of this book's job to show you it's not.

1.9 Naming things

You surely don't want to create the doubling function every time you use it. Instead, you want to create it once, then bind it to some sort of global symbol.

```
1 user> (def twice (fn [n] (+ n n)))
2 #'user/twice
```

Once named, a user-defined function can be used as conveniently as a built-in function:

```
1 user=> (twice 10)
2 20
```

Since functions are values not essentially different than other values, you might expect that you can give names to strings, numbers, and whatnot. Indeed you can:

```
1 user> (def two 2)
2 #'user/two
3 user> (twice two)
4 4
```

You can use `def` with a particular symbol more than once. That's the only exception to Clojure's “no changing a binding” rule². It's useful for correcting mistakes:

```
1 user=> (def twice (fn [n] (- n n)))
2 user=> (twice 10)
3 0
4 user=> ;; Darn! (This, by the way, is a Clojure comment.)
5 user=> (def twice (fn [n] (+ n n)))
6 user=> (twice 10)
7 20
```

1.10 Lists

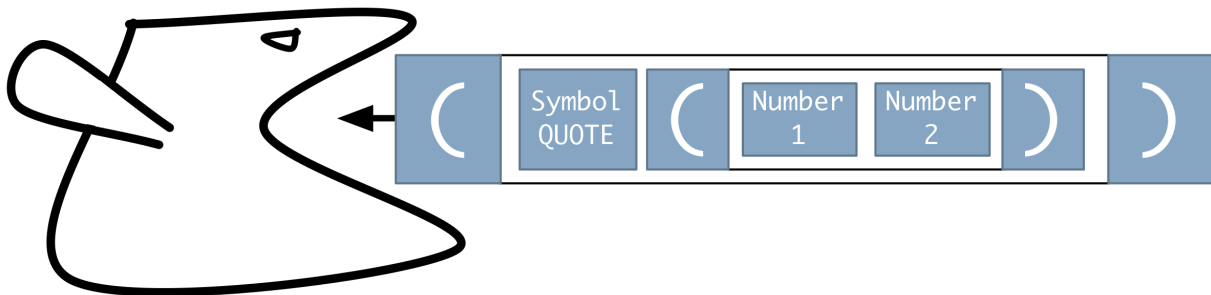
We've seen that lists are surrounded by parentheses. The evaluator function interprets a list as an excuse to apply a function to arguments. But lists are also a useful data structure. How do you say that you want a list to be treated as data, not as code? Like this:

```
1 user> '(1 2)
2 (1 2)
```

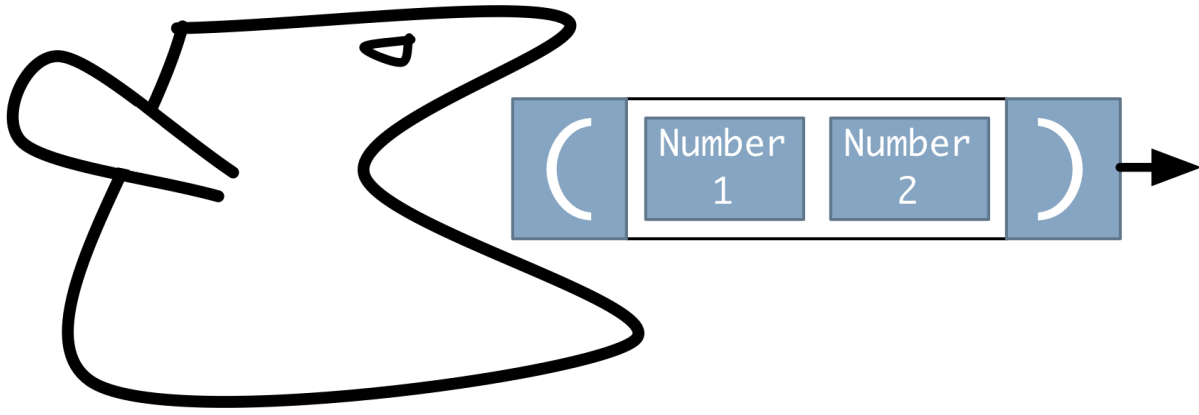
The quote tells the evaluator not to interpret the list as a function call. That character is actually syntactic sugar for a more verbose notation:

```
1 user> (quote (1 2))
2 (1 2)
```

The reader gives the evaluator this list:



The evaluator notices that the first element is the special symbol quote. Instead of unleashing sub-evaluators, it digests the form into its single argument, which is what it feeds to the printer:



You can also create a list with a function:

```
1 user> (list 1 2 3 4)
2 (1 2 3 4)
```

You can take apart lists. Here's a way to get the first element:

```
1 user> (first '(1 2 3 4))
2 1
```

Here's a way to get everything *after* the first element:

```
1 user> (rest '(1 2 3 4))
2 (2 3 4)
```

You can pick out an element at a particular (zero-based) position in a list:

```
1 user> (nth '(1 2 3 4) 2)
2 3
```

Exercise 1: Given what you know now, can you define a function `second` that returns the second element of a list? That is, fill in the blank in this:

```
1 user> (def second (fn [list] ____))
```

Be sure to try your solution at the repl. (When you do, you'll notice that you've just overridden Clojure's built-in second function. Don't worry about that.)

You can find solutions to this chapter's exercises in [solutions/just-enough-clojure.clj](#).

Exercise 2: Give two implementations of `third`, which returns the third element of a list.

1.11 Vectors

Lists are (roughly) the classic linked list that many of us encountered when we first learned programming. That means code has to traverse the whole list to get to the last element. Clojure's creator cares about efficiency, so Clojure also makes it easy to use vectors, where it takes no more time to access the last element than the first.

Vectors have a literal notation, in which the elements are surrounded by square brackets:

```
1 user> [1 2 3 4]
2 [1 2 3 4]
```

Note that I didn't have to quote the vector to prevent the evaluator from trying to use the value of 1 as a function. That only happens with lists.

There's also a function-call notation for creating vectors:

```
1 user> (vector 1 2 3 4)
2 [1 2 3 4]
```

The `first`, `rest`, and `nth` functions also work with vectors. Indeed, most functions that apply to lists also apply to vectors.

1.12 Vector? List? Who cares?

Both vectors and lists are *sequential* datatypes.


```
1 user=> (sequential? [1 2 3])
2 true
3 user=> (sequential? '(1 2 3))
4 true
```

There's a third datatype called the *lazyseq* (for “lazy sequence”) that's also sequential. That datatype won't be relevant until we discuss [laziness](#). I mention it because some functions that you might think produce vectors actually produce lazyseqs. For example, consider this:

```
1 user=> (rest [1 2 3])
2 (2 3)
```

The first time I typed something like that, I expected the result to be the vector `[2 3]`, and the parentheses confused me. The result of `rest` is a lazyseq, which prints the same way as a list. Here's how you can tell the difference:

```
1 user=> (list? (rest [1 2 3]))
2 false
3 user=> (seq? (rest [1 2 3]))
4 true
5 user=> (vector? (rest [1 2 3]))
6 false
```

Such changes of type seem like they'd lead to bugs. In fact, the differences almost never matter. For example, equality doesn't depend on the type of a sequential data structure, only on the contents. Therefore:

```
1 user=> (= [2 3] '(2 3))
2 true
3 user=> (= [2 3] (rest [1 2 3]))
4 true
```

The single most obvious difference between a list and vector is that you have to quote lists.

It will never matter in this book whether you create a list or vector, so suit your fancy. I will often use “sequence” from now on when the difference is irrelevant.

seqs

The predicate `seq?` doesn't actually check specifically for a `lazyseq`. It responds `true` for both lists and `lazyseqs`, and the word `seq` is used as an umbrella term for both types. If you really need to know the complete set of sequential types and the names that refer to them, see the table below. However, the definition of a `seq` will never matter for this book.

	Lists	Vectors	Lazyseqs
<code>sequential?</code>	YES	YES	YES
<code>seq?</code>	YES	no	YES
<code>list?</code>	YES	no	no
<code>vector?</code>	no	YES	no
<code>coll?</code> (for “collection”)	YES	YES	YES

1.13 More on evaluation and quoting

When the evaluator is given a list from the reader, it first evaluates each element of the list, then calls the first element as a function. When it's given a vector, it first evaluates each element and then packages the resulting values as a (different) vector.

That means that literal vectors can (and often do) contain code snippets:

```
1 user=> [ (+ 1 1) (- 1 1) ]
2 [2 0]
```

It also means quoting is sometimes required for vectors as well as lists. Can you guess the results of these two code snippets?

- `[inc dec]`
- `'[inc dec]`

The first is a vector of two *functions*:

```
1 user=> [inc dec]
2 [#<core$inc clojure.core$inc@13ab6c1c>
3  #<core$dec clojure.core$dec@7cdd7786>]
```

The second is a vector of two *symbols* (which happen to name functions):

```
1 user=> '[inc dec]
2 [inc dec]
```

At first, you're likely to be confused about when you need to quote. Basically, if you see an error like this:

```
1 java.lang.Exception: Unable to resolve symbol: foo in this context
2 (NO_SOURCE_FILE:67)
```

... it probably means you forgot a quote.

Quoting doesn't only apply to entire lists and vectors. You can quote symbols:

```
1 user=> 'a
2 a
```

You can also quote individual elements of vectors:

```
1 user=> [ 'my 'number (+ 1 3) ]
2 [my number 4]
```

1.14 Conditionals

Despite the [anti-if campaign](#), the conditional statement is one of primordial operations of the Turing Machine (that is, computer). Conditionals in Clojure look like this:

```
1 user=> (if (odd? 3)
2         (prn "Odd!")
3         (prn "Even!"))
4 "Odd!"
5 nil
```

The `prn` function prints to the output. Unlike in some languages, `ifs` are expressions that produce values and can be embedded within other expressions. The value of an `if` expression is the value of the “then” or “else” case (whichever is chosen). Since `prn` always returns the value `nil`, that's what the repl printed in the example above. (`nil` is called `null` in some other languages—it's the object that is no object, or the pointer that points to nothing, or what Tony Hoare called his [billion-dollar mistake](#).)

1.15 Rest arguments

Clojure functions can take a variable number of arguments:

```
1 user> (add-squares 1 2)
2 5
3 user> (add-squares 1 2 3)
4 14
```

As with other languages, there's a special token in a function's parameter list to say "Take any arguments after this point and wrap them up in a sequential collection (list, vector, whatever)". Clojure's looks like this:

```
1 user> ( (fn [& args] args) 1 2 3 4)
2
3 (1 2 3 4)
```

That function gathers all the arguments into the list `args`, which it then returns.

Note the space after the `&`. It's required.

Now that we know how to define *rest arguments*, here's what `add-squares`' definition would look like:

```
1 (def add-squares
2   (fn [& numbers]
3     (...something... numbers)))
```

What could the "something" be? The next section gives us a clue.

1.16 Explicitly applying functions

Suppose we have a vector of numbers we want to add up:

```
1 [1 4 9 16]
```

That is, we want to somehow turn that vector into the same value as this `+` expression:

```
1 user=> (+ 1 4 9 16)
2 30
```

(Notice that + can take any number of arguments.)

The following is *almost* what we want, but not quite:

```
1 user=> (+ [1 4 9 16])
2 java.lang.ClassCastException (NO_SOURCE_FILE:0)
```

What we need is some function that hands all the elements of the vector to + as if they were arguments directly following it in a list. Here's that function:

```
1 user=> (apply + [1 4 9 16])
2 30
```

apply isn't magic; we can define it ourselves. I think of it as turning the second argument into a list, sticking the first argument at the front, and then evaluating the result in the normal way a list is evaluated. Or:

```
1 (def my-apply
2   (fn [function sequence]
3     (eval (cons function sequence))))
```

Let's look at this in steps.

1. cons (the name chosen [more than 50 years ago](#) as an abbreviation for the verb “construct”) produces a list³ whose first element is cons's first argument and whose rest is cons's second argument:

```
1 user=> (cons "the first element" [1 2 3])
2 ("the first element" 1 2 3)
```

(Notice that cons, like rest earlier, takes a vector in but doesn't produce one.)

2. eval is our old friend the bird-like evaluator. In my-apply, it's been given a list headed by a function, so it knows to apply the function to the arguments.

According to the substitution rule, (my-apply + [1 2 3]) is first converted to this:

```
1      (eval
2      (cons + [1 2 3]))
```

After that, it is evaluated “from the inside out”, each result being substituted into an enclosing expression, finally yielding 6.⁴

1.17 Loops

How do you write loops in Clojure?

You don’t (mostly).

Instead, like Ruby and other languages, Clojure encourages the use of functions that are applied to all elements of a sequence. For example, if you want to find all the odd numbers in a sequence, you’d write something like this:

```
1 user> (filter odd? [1 2 3 4])
2 (1 3)
```

The `filter` function applies its first argument, which should be a function, to each element of its second argument. Only those that “pass” are included in the output.

Question: How would you find the first odd element of a list?

Answer:

```
1 user> (first (filter odd? [1 2 3 4]))
2 1
```

Question: Isn’t that grossly inefficient? After all, `filter` produces a whole list of odd numbers, but you only want the first one. Isn’t the work of producing the rest a big fat waste of time?

Answer: No. But you’ll have to read the later discussion of [laziness](#) to find out why.

The `map` function is perhaps the most common loop-like function. (If you know Ruby, it’s the same as `collect`.) It applies its first argument (a

function) to each element of a sequence and produces a sequence of the results. For example, Clojure has an `inc` function that returns one plus its argument. So if you want to increment a whole sequence of numbers, you'd do this:

```
1 user> (map inc [0 1 2 3])
2 (1 2 3 4)
```

The `map` function can take more than one sequence argument. Consider this:

```
1 user> (map * [0 1 2 3]
2             [100 200 300 400])
3 (0 200 600 1200)
```

That is equivalent to this:

```
1 user=> (list (apply * [0 100])
2             (apply * [1 200])
3             (apply * [2 300])
4             (apply * [3 400]))
```

1.18 More exercises

Exercise 3: Implement `add-squares`.

```
1 user=> (add-squares 1 2 5)
2 30
```

Exercise 4: The `range` function produces a sequence of numbers:

```
1 user=> (range 1 5)
2 (1 2 3 4)
```

Using it and `apply`, implement a bizarre version of factorial that uses neither iteration nor recursion.

Hint: The factorial of 5 is $1*2*3*4*5$.

Exercise 5: Below, I give a list of functions that work on lists or vectors. For each one, think of a problem it could solve, and solve it. For example, we've already solved two problems:

```

1 user> ;; Return the odd elements of a list of numbers.
2 user> (filter odd? [1 2 3 4])
3 (1 3)
4 user> ;; (One or more semicolons starts a comment.
5 user>
6 user> ;; Increment each element of a list of numbers,
7 user> ;; producing a new list.
8 user=> (map inc [1 2 3 4])
9 (2 3 4 5)

```

You'll probably need other Clojure functions to solve the problems you put to yourself. Therefore, I also describe some of them below.

Clojure has a built-in documentation tool. If you want documentation on `filter`, for example, type this at the repl:

```

1 user=> (doc filter)
2 -----
3 clojure.core/filter
4 ([pred coll])
5   Returns a lazy sequence of the items in coll for which
6   (pred item) returns true. pred must be free of side-effects.

```

Many of the function descriptions will refer to “sequences”, “seqs”, “lazy seqs”, “colls”, or “collections”. Don't worry about those distinctions. For now, consider all those synonyms for “either a vector or a list”.

In addition to the built-in doc, clojuredocs.org has examples of many Clojure functions.

Functions to try

- `take`
- `distinct`
- `concat`
- `repeat`
- `interleave`
- `drop` and `drop-last`
- `flatten`
- `partition` only the `[n coll]` case, like: `(partition 2 [1 2 3 4])`
- `every?`
- `remove` and create the function argument with `fn`

Other functions

- `(= a b)` – Equality
- `(count sequence)` – Length
- `and`, `or`, `not` – Boolean functions
- `(cons elt sequence)` – Make a new sequence with the `elt` on the front
- `inc`, `dec` – Add and subtract one
- `(empty? sequence)` – Is the sequence empty?
- `(nth sequence index)` – Uses zero-based indexing
- `(last sequence)` – The last element
- `(reverse sequence)` – Reverses a sequence
- `print`, `println`, `prn` – Print things. `print` and `println` print in a human-friendly format. For example, strings are printed without quotes. `prn` prints values in the literal format you’d type to create them. For example, strings are printed with double quotes. `println` and `prn` add a trailing newline; `print` does not. All of these can take more than one argument.
- `pprint` – Short for “pretty print”, it prints a nicely formatted representation of its single argument.

Note: if the problems you think of are anything like the ones I think of, you’ll want to use the same sequence in more than one place, which would lead to annoying duplication. Since you don’t know how to create local variables yet, the easiest way to avoid duplication is to create and call a function:

```
1 (def solver
2   (fn [x]
3     (... x ... .. x ... ..)))
4
5 (solver [1 2 3 4 5 6 7])
```

You can find my problems and solutions in [solutions/just-enough-clojure.clj](https://github.com/just-enough-clojure).

Exercise 6: Implement this function:

(prefix-of? candidate sequence): Both arguments are sequences. Returns true if the elements in the candidate are the first elements in the sequence:

```
1 user> (prefix-of? [1 2] [1 2 3 4])
2 true
3 user> (prefix-of? '(2 3) [1 2 3 4])
4 false
5 user> (prefix-of? '(1 2) [1 2 3 4])
6 true
```

Exercise 7: Implement this function:

(tails sequence): Returns a sequence of successively smaller subsequences of the argument.

```
1 user> (tails '(1 2 3 4))
2 ((1 2 3 4) (2 3 4) (3 4) (4) ())
```

To implement tails, use range, which produces a sequence of integers. For example, (range 4) is (0 1 2 3).

This one is tricky. My solution is very much in the functional style, in that it depends on sequences being easy to create and work with. So I'll provide some hints. Here and hereafter, I encourage you to try to finish without using the hints, but not to the point where you get frustrated. Programming is supposed to be fun.

Hint: What is the result of evaluating this?

```
1 [(drop 0 [1 2 3])
2  (drop 1 [1 2 3])
3  (drop 2 [1 2 3])
4  (drop 3 [1 2 3])]
```

Hint: map can take more than one sequence. If you give it two sequences, it passes the first of each to its function, then the second of each, and so on.

Exercise 8: In the first exercise in the chapter, I asked you to complete this function:

```
1 (def second (fn [list] ____))
```

Notice that `list` is a parameter to the function. We also know that `list` is (globally), a function in its own right. That raises an interesting question. What is the result of using the following function?

```
1 user=> (def puzzle (fn [list] (list list)))
2 user=> (puzzle '(1 2 3))
3 ????
```

Why does that happen?

Hint: Use the [substitution rule for functions](#).

1. I will consistently use “argument” for real values given to functions and “parameter” for symbols used to name arguments in function definitions. So `n` is a parameter, while a real number given to our doubling function would be an argument. ↩
2. `def` isn’t actually an exception. It looks like just another way of associating a symbol with a value, but it’s actually doing something different. The difference, though, is irrelevant to this book, and would just complicate your understanding to no good end, so I’m going to ignore it. If you’re curious, see the description of [var](#) in the Clojure documentation or any other book on Clojure. ↩
3. Strictly, `cons` produces a lazyseq, not a list, but the evaluator treats them the same. ↩
4. The substitution as printed isn’t quite true. After it receives `(my-apply + ...)` from the reader, the evaluator processes the symbols `function` and `+` to find function values. Therefore, in the expansion of `my-apply`, the `function` parameter is substituted with an argument that’s a function value. And so the list given to `eval` starts with a function value, not a symbol. That’s different than what we’ve seen before. But it still works fine, because a function value self-evaluates the way a number does. I opted for the easier-to-read expansion. ↩

I EMBEDDING AN OBJECT-ORIENTED LANGUAGE

Many object-oriented languages were first implemented by embedding them into procedural languages. Objective-C was originally a C preprocessor hack. The C++ compiler originally emitted C code for the C compiler to compile. And during the gold rush phase of objects, many downright loopy object systems were written in Lisp. These embedded languages were cheap and useful ways to learn what object orientation was all about.

In this part of the book, we'll use Clojure to implement an object system patterned after Java's. (In the optional [Part V](#), we'll extend it to be more like Ruby's.)

Before we start, though, it's important for us to agree on what words mean. An *instance* is synonymous with “object”, but it's often used to emphasize that the object has been instantiated from a *class*, whose role is to describe (in some sense) a lot of similar objects.

Instances contain *instance variables*, which are named bits of data. These variables may or may not be defined by the class. (In Java, they are. In Ruby, they are not.)

In Clojure, computation is done by applying functions to arguments. For objects, we'll use a different metaphor: *messages* are *sent* to *receivers*. As a result, a *method* (which is, roughly, a function) is applied to the receiver and any arguments sent along with the message. Methods are defined by classes.

Messages are just names. The same name may apply to many different methods (each in a different class). A *dispatch function* selects one of those methods by examining the class of the receiver.

Instances are created with *constructors*. In some object systems, constructors are ordinary methods; in others (like Java's), they are not.

After a short [introduction](#) describing the relationship between functions and methods, we'll proceed in these steps:

1. Creating a [barely believable object](#) that's nothing but instance variables, a rudimentary constructor, and a vestigial class.
2. [Embedding methods](#) within the constructor, making it look a bit like a class definition.
3. Define methods [in the class](#) rather than in the constructor.
4. Adding superclasses and [inheritance](#).

While this section appears to be about object-oriented programming, not functional programming, it gives you the opportunity to practice programming in a functional language. That experience will make the topics in the next part of the book easier to grasp.

2. Objects as a Software-Mediated Consensual Hallucination

Cyberspace. A consensual hallucination experienced daily by billions of legitimate operators, in every nation... A graphic representation of data abstracted from the banks of every computer in the human system. Unthinkable complexity. Lines of light ranged in the nonspace of the mind, clusters and constellations of data. Like city lights, receding.

— William Gibson, *Neuromancer*, 1984.

Sometime in the early 1980's, I introduced Ralph Johnson to the head of the Research Division of a computer manufacturer that doesn't exist any more. This was in the days when Smalltalk was *the* object-oriented language, and Ralph was an expert in Smalltalk. At one point during the discussion, the division head said, "We were doing object-oriented programming 20 years ago! In assembly language!" That comment was both:

- true, and
- irrelevant.

It was true because you certainly can do object-oriented programming in assembler. In some sense, you have to, because computer hardware doesn't *have* objects. It has numbered locations that store words of data. It doesn't have methods, either. It just has instructions that provide a limited kind of function. Object-orientation has to be built up out of that, these days usually with more than one layer of intermediate language.

It was irrelevant because doing OO in assembler is *so hard* that the kind of designs that flow naturally out of a good language just don't happen.

However, it's an important point that object-orientation is really nothing more than some conventions for using data, conventions that are both

supported and hidden by the language. Consider this Java method that shifts a point by some increments:

```
1 public class Point {
2     private int x;
3     private int y;
4
5     void shift(int xinc, int yinc) {
6         x += xinc;
7         y += yinc;
8     }
9 }
```

In Python, the equivalent method would be written like this:

```
1 class Point:
2     def shift(self, xinc, yinc)
3         self.x += xinc
4         self.y += yinc
```

The Python method has a `self` parameter, whereas the Java one appears not to. Python is being more honest (though less convenient) than Java. Every Java method really *does* have a `self` parameter (though Java calls it `this`). But since that parameter must always be there, the language designers decided to make it implicit.

The Java designers also decided to change the syntax of functions to single out the first argument. In the old-fashioned math you learned at school, functions take their arguments at the right:

```
1 shift(point, xinc, yinc)
2 reflect-across-origin(point)
3 ...
```

In object-oriented programming, we know we'd have a pile of functions that would always take the same kind of first argument, so OO languages use a human-friendly out-of-order syntax:

```
1 point.shift(xinc, yinc)
2 point.reflect-across-origin
```

Even Python uses this syntax. But, somewhere behind the scenes, that format is converted into the old-fashioned format (so that the `self`

argument is in the right place).

Another way in which Java hides conventions is that it pretends that instance variables are more distinct than they are. Consider these two variants of the same method:

```
1 public class Point {
2     private int x;
3     private int y;
4
5     void shift1(int xinc, int yinc) {
6         x += xinc;
7         y += yinc;
8     }
9
10    void shift2(int xinc, int yinc) {
11        this.x += xinc;
12        this.y += yinc;
13    }
```

The first definition makes the instance variables seem like very distinct things within the broad scope of the class. The second hints at what they really are: a *single* data structure that contains a mapping (or dictionary) from names to values.

So: what's an object? It's a clump of name->value mappings, some functions that take such clumps as their first arguments, and a dispatch function that decides which function the programmer meant to call. This reality is usefully obscured by the language so that programmers can, without thinking, do wonderful things, blissfully pretending that the pictures in their head are what the computer is really doing.

3. A Barely Believable Object

Let's start implementing our consensual hallucination of objects.

3.1 Maps

In Clojure, the *map* is the data structure of choice for connecting names to values. It's like Java's HashMaps, Ruby's Hashes, or Python and Smalltalk's Dictionaries. Maps have this literal notation:

```
1 user> {:a 1, "b" 2}
2 {:a 1, "b" 2}
```

- The first, third, and following odd-numbered elements are the *keys*. They can be any type of object, but it's very common to use colon-prefixed *keywords* because, unlike symbols, they're self-evaluating (don't need to be quoted).
- The second, fourth, and following even-numbered elements are the *values*. They can also be of any type (including nested maps).
- It's common to use commas to separate key-value pairs. In Clojure, commas are *white space*: the reader ignores them.

There's also a function that creates maps:

```
1 user> (hash-map :a 1 :b 2)
2 {:a 1, :b 2}
```

It's not unusual to apply hash-map to a sequence, like this:

```
1 user=> (apply hash-map [:a 1 :b 2])
2 {:a 1, :b 2}
```

That allows something like keyword arguments:

```
1 user=> (do-something-with-a-colored-point :x 1 :y 2 :color "red")
2
3 user=> ;; Arguments can be in any order
```

```

4 user=> (do-something-with-a-colored-point :color "red", :x 1, :y 2)
5
6 user=> (def do-something-with-a-colored-point
7         (fn [& args]
8           ... (apply hash-map args) ...))

```

To retrieve a value from a map, you can use `get`:

```

1 user> (get {:a 1, :b 2} :a)
2 1

```

If you try to fetch the value of a key that’s not in the map, you get `nil`:

```

1 user=> (get {} :x)
2 nil

```

`get` is actually somewhat rare in Clojure code, though. Keywords have the nice property that they act like functions when placed as the first element of a list:

```

1 user> (:a {:a 1, :b 2})
2 1

```

Consider `:a` in the above to mean “get the value using me as the key”. In this book, we’ll be using this style a lot, and we’ll often write functions like this one:

```

1 (def do-something-to-map
2   (fn [function]
3     (function {:a "a value", :b "b value"})))

```

`do-something-to-map` can legitimately be given either a true function or a keyword:

```

1 user=> (do-something-to-map :a)
2 "a value"
3 user=> (do-something-to-map count)
4 2

```

To keep from having to say “either a function or a keyword” in the rest of the book, I’ll use *callable* as the umbrella term for anything that behaves like a function when it’s in the first position of a evaluated list^{[1](#)}.

Clojure gives you no way to modify existing maps, so they're "changed" by building new maps from old ones. To add a single element, you use `assoc`:

```
1 user=> (assoc {:a 1, :b 2} :c 3)
2 {:c 3, :a 1, :b 2}
```

You can use `assoc` to add multiple key-value pairs:

```
1 user=> (assoc {:a 1, :b 2} :c 3 :d 4 :e 5)
2 {:e 5, :d 4, :c 3, :a 1, :b 2}
```

You can also merge two or more maps:

```
1 user=> (merge {:a 1, :b 2} {:c 3, :d 4} {:e 5})
2 {:e 5, :d 4, :c 3, :a 1, :b 2}
```

You can exclude key-value pairs from a map with `dissoc`:

```
1 user=> (dissoc {:a 1, :b 2, :c 3} :b :c)
2 {:a 1}
```

Question: How do `assoc` and `merge` handle duplicate keys? Try it and see!

```
1 user=> (assoc {:a 1} :a 2222)
2 ???
3 user=> (merge {:a 1, :b 2, :c 3} {:a 111, :b 222, :d 4} {:b "two"})
4 ???
```

3.2 I present to you an object

Let's make a constructor for a `Point` class. Here's a minimal version:

```
1 (def Point
2   (fn [x y]
3     {:x x
4      :y y}))
```

What do we see here? We have two instance variables `x` and `y`. Instance variables aren't encapsulated, so we can access them like this:

```
1 user=> (:x (Point 1 2))
2 1
```

That would be the equivalent of `object.field` in Java.

In this book, we won't implement the encapsulation rules of Smalltalk or Ruby (neither of which allow direct access to instance variables). We will often implement and use accessor (getter) methods, though. Here's one:

```
1 (def x
2     (fn [this] (get this :x)))
3
4 user=> (x (Point 1 2))
5 1
```

However, since keywords are callables, we could define `x` more simply:

```
1 (def x
2     (fn [this] (:x this)))
```

or even just this:

```
1 (def x :x)
```

3.3 The class begins as documentation

The result of `(Point 1 2)` prints like this:

```
1 {:x 1, :y 2}
```

If that was the first line of code you saw in this book, you could probably guess it represented a mathematical point, but that's only because you've already seen a million examples of them. You'd have much more trouble with other kinds of objects.

So let's add the class. For the moment, it'll just be used as documentation. Because it's a kind of metadata (data about data), I'll give it a funny name:

```
1 (def Point
2     (fn [x y]
3         {:x x
4          :y y
5          :__class_symbol__ 'Point}))    ;; <==
```

From Clojure's point of view, a keyword like `:__class_symbol__` is no different than any other. To us, though, this double-underscore convention will signal that a keyword is part of the workings of the object system, not something that ordinary client code would ever use. (In more polished object systems, this kind of metadata is typically inaccessible to client code.)

Now we can get an object's class easily:

```
1 (def class-of :__class_symbol__)
2
3 user=> (class-of (Point 1 2))
4 Point
```

(I'm using `class-of` instead of `class` because Clojure already has a `class` function.)

Given a large enough dose of magic mushrooms, we can hallucinate that the `class-of`, `x`, and `y` callables are instance methods of `Point`. (It requires mushrooms because we're used to thinking of methods as being somehow attached to objects, and these are free-floating. We'll fix that in the next chapter.)

Here's a less boring method: `shift`. It takes a particular point and shifts it according to an `x`-increment and a `y`-increment. Since Clojure doesn't let us modify maps, `shift` has to create a new `Point`.

```
1 (def shift
2   (fn [this xinc yinc]
3     (Point (+ (x this) xinc)
4            (+ (y this) yinc))))
5
6 user=> (shift (Point 1 200) -1 -200)
7 {:x 0, :y 0, :__class_symbol__ Point}
```

3.4 Exercises

You can find starting Clojure code in [sources/add-and-make.clj](#). You can either copy-and-paste the code into the repl, or you can type the following to a repl started in the root directory of a cloned repository:

```
1 user=> (load-file "sources/add-and-make.clj")
```

Note that Clojure requires that you def any name (like a function name) before you can use it. If you paste the definition of `shift` (which uses `Point`) before the definition of `Point`, you'll get this error:

```
1 java.lang.Exception: Unable to resolve symbol: Point in this context
2 (NO_SOURCE_FILE:3)
```

You can find my solutions for these exercises in [solutions/add-and-make.clj](#).

Exercise 1: Implement add.

Implement an `add` function that adds two points, producing a third. First implement it without using `shift`. Then implement it using `shift`. (If you think of `add` as an instance method, calling `shift` is like using another instance method in the same class.)

Exercise 2: A new operator

Our `Point` function matches the way Python constructors are called. Java would use `new Point` and Ruby would use `Point.new`. That suggests an alternative syntax:

```
1 (new Point 3 5)
```

However, Clojure reserves `new` for the creation of Java objects, so we'll use `make` instead:

```
1 (make Point 1 2)
```

Write the function `make`, assuming that the function `Point` already exists.

The exercise source contains a `Triangle` constructor. Does your `make` work with it? For example:

```
1 (make Triangle (make Point 1 2)
2               (make Point 1 3)
3               (make Point 3 1))
```

If not, make it work.

Exercise 3: `sources/add-and-make.clj` also defines three triangles: `right-triangle`, `equal-right-triangle`, and `different-triangle`. Write a function `equal-triangles?` that produces these results:

```
1 user=> ;; Identical objects
2 user=> (equal-triangles? right-triangle right-triangle)
3 true
4 user=> ;; Not identical, but contents are equal
5 user=> (equal-triangles? right-triangle equal-right-triangle)
6 true
7 user=> ;; different
8 user=> (equal-triangles? right-triangle different-triangle)
9 false
```

It's easier than you probably think!

Exercise 4: Change `equal-triangles` so that it can compare more than two triangles:

```
1 user=> (equal-triangles? right-triangle
2                               equal-right-triangle
3                               different-triangle)
4 false
```

It's way easier than you might think!

Exercise 5: Start to write a function `valid-triangle?` that takes three Points and returns either `true` or `false`. In this exercise, only check to see whether any of the points are duplicates. Your first thought might be to do this:

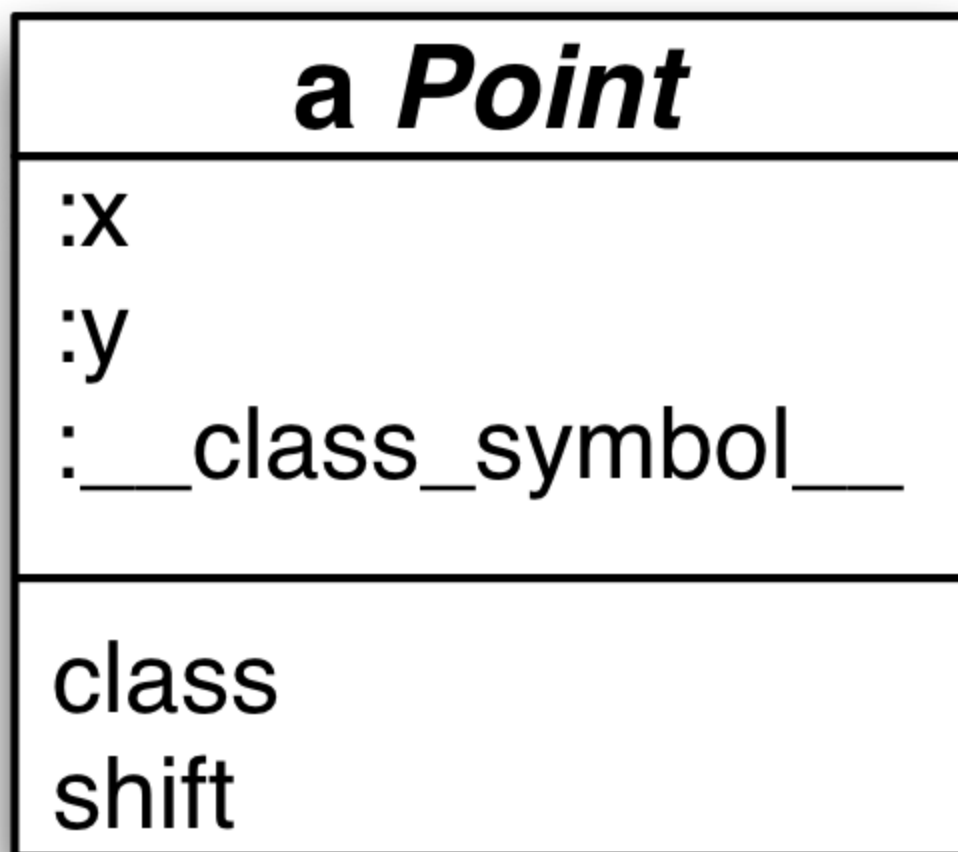
```
1 (def valid-triangle?
2   (fn [point1 point2 point3]
3     (and (not= point1 point2)
4           (not= point1 point3)
5           (not= point2 point3))))
```

However, that approach is prone to typos². You can use one of the sequence functions you explored in the previous chapter to do a more concise job.

1. Vectors and maps are also callables, though we won't use that fact in this book. If you're curious, try `({:a 1, :b 2} :a)` and `(["zero" "one"] 1)`.[↩](#)
2. I once found a bug in a fairly complicated Unix kernel function. It had three “inode” parameters: `i`, `ii`, and `iii`. Guess what the bug was?[↩](#)

4. All the Class in a Constructor

We don't usually think of objects having their methods scattered all over the global namespace the way `shift` and `add` are. We usually think of methods as somehow *belonging to* objects:



Let's have our constructor embed functions in the map it returns:

```
1 (def Point
2   (fn [x y]
3     ;; initializing instance variables
4     :x x
```

```

5         :y y
6
7         ;; Metadata
8         :__class_symbol__ 'Point
9         :__methods__ {
10             :class :__class_symbol__
11
12             ;; Not implementing getters for `x` and `y` yet.
13
14             :shift
15             (fn [this xinc yinc]
16                 (make Point (+ (:x this) xinc)
17                             (+ (:y this) yinc))))))

```

- That’s starting to look a little like a class definition: it names instance variables and defines methods.
- Notice that I’m using the “make Point” style object construction that you implemented in the previous chapter’s exercises.
- For tidiness’ sake, I’m not defining the methods directly alongside the instance variables. Instead, I’m isolating them inside an embedded map.

Because of the embedded map, retrieving a method is a bit complicated:

```

1 user=> (:shift (:__methods__ (make Point 1 2)))
2 #<user$Point$fn__588 user$Point$fn__588@3764f8d4>

```

And applying one is not exactly graceful:

```

1 user=> (def point (make Point 1 2))
2 user=> ( (:shift (:__methods__ point)) point -2 -3)
3
4 { :x -1, :y -1, :__class_symbol__ Point, :__methods__ {...}}

```

That clearly won’t do. Let’s add a send-to function that looks more like a message send in a regular object-oriented language. Like this:

```

1 user=> (send-to point :shift -2 -3)
2 { :x -1, :y -1, :__class_symbol__ Point, :__methods__ {...}}

```

send-to is easiest to implement using a feature of apply I didn’t show you earlier. apply can take more than a function and a sequence as arguments. It

can also have one or more additional arguments between the two; those are prepended to the front of the sequence. Like this:

```
1 user=> (apply + 1 [2 3])
2 6
3 user=> (apply + 1 2 [3])
4 6
5 user=> (apply + 1 2 3 [])
6 6
```

That given, the definition of send-to is this:

```
1 (def send-to
2   (fn [object message & args]
3     (apply (message (:__methods__ object)) object args)))
```

Make sure you understand send-to, since we'll be changing it for the rest of this part of the book.

4.1 One exercise

Change the Point constructor to add x and y accessors (getters). Use them in shift. Implement add, and have it use shift. You can find the source for the code so far in [sources/embedded-functions.clj](#). My solution is in [solutions/embedded-functions.clj](#).

5. Moving the Class Out of the Constructor

So far, our implementation isn't typical of object systems. It's unusual for every single instance of a class to contain references to all of its methods, using the class itself as merely an identifying name. Instead, in most object systems the instance has a single reference to its class, and it finds its methods via that link. The methods are better thought of as belonging to the class than to the instance. In this chapter, we'll shift over to that implementation.

First, we'll pull out the methods from the instance into a new `Point` object:

```
1 (def Point
2   ;;      (fn [x y]
3   ;;      {;; initializing instance variables
4   ;;      :x x
5   ;;      :y y
6   ;;
7   ;;      ;; Metadata
8   ;;      :__class_symbol__ 'Point
9   {
10    :__instance_methods__
11    {
12     :class :__class_symbol__
13     :shift
14     (fn [this xinc yinc]
15       (make Point (+ (:x this) xinc)
16                   (+ (:y this) yinc)))
17    }
18  })
```

I've changed `__methods__` to `__instance_methods__` to emphasize that the methods are to be used with an instance, not the class. (It is the instance that is passed to them as their `this` argument.)

We have to find a replacement for what the commented-out code used to do. We no longer have a `Point` constructor, but you've [already implemented](#) an alternative notation for us to use:

```
1 user=> (make Point 1 2)
```

Your old implementation will have to be updated, though. Let's pattern the revised make after the three steps new performs in many object-oriented languages:

1. Objects usually use a contiguous block of memory to store instance variables. So that memory is first allocated.

In our case, we'll continue to use maps, so our "allocation" will look like this: {}

2. The empty object is seeded with metadata needed by the language runtime (such as information about what its class is).

In our case, that's easy:

```
1 (assoc allocated :__class_symbol__ 'Point)
```

3. Finally, (what amounts to) an instance method is called to fill in the starting values of instance variables. (That's the constructor in Java, or initialize in Ruby.)

Since Clojure doesn't allow modification of existing maps, the called function must create a new map with alloc or merge. To (slightly) emphasize that, I won't call the method initialize. Instead, the user-supplied function will be called add-instance-values.

Before implementing make, let's add add-instance-values to the class:

```
1 (def Point
2 {
3   :__instance_methods__
4   {
5     :add-instance-values      ;; <=> new
6     (fn [this x y]
7       (assoc this :x x :y y))
8
9     :shift
10    (fn [this xinc yinc]
11      (make Point (+ (:x this) xinc)
12                  (+ (:y this) yinc)))
13   }
14 })
```

We can now implement make. Its code will be clearer if we use a new Clojure special form.

5.1 Let

One way to introduce something like a local variable is to use nested functions. Suppose we wanted to implement the following without the duplication:

```
1 user=> (* (+ 1 2)
2          (+ 1 2)
3          5)
4 45
```

We could do this:

```
1 user=> ( (fn [name] (* name name 5)) (+ 1 2))
2
3 45
```

But that's not exactly graceful or readable. So Clojure provides an alternate way to name sub-computations. It's called `let`:

```
1 user=> (let [name (+ 1 2)]
2          (* name name 5))
3 45
```

Notice that `let` expressions return a value. So you can nest them inside other expressions:

```
1 user=> (+ 1 (let [one 1] (* one one)) 3)
2
3 5
```

You can create more than one name with a `let`:

```
1 user=> (let [one 1
2             two 2]
3          (+ one two))
4 3
```

Notice that all the name-value pairs are surrounded by a single set of square brackets. That's like the way that brackets are used to hold function

parameters.

Names take effect immediately, so a later computation in a `let` parameter list can use an earlier one:

```
1 user=> (let [little-map {:a 1}
2             bigger-map (assoc little-map :b 2)]
3         bigger-map)
4 {:b 2, :a 1}
```

A hint

`let` expressions define a computation as a series of steps. If you find multi-step `let` expressions confusing, execute each step at the repl. To show what I mean by that, here's a silly function that operates on a map like `{:a 1}`:

```
1 (fn [starting-map]
2   (let [bigger-map (assoc starting-map :b 2)
3         overridden-a (assoc bigger-map :a 33)]
4     overridden-a))
```

You can puzzle out the steps like this:

```
1 user=> (def starting-map {:a 1})
2 user=> (def bigger-map (assoc starting-map :b 2))
3 user=> bigger-map
4 {:b 2, :a 1}
5 user=> (assoc bigger-map :a 33)
6 {:b 2, :a 33}
```

5.2 Implementing instance creation

Recall that the instance creation function has three steps: allocation, seeding with metadata, then calling the user-provided constructor-like method. So the structure for our new `make` function can use a `let` to give a name to each step's result:

```
1 (def make
2   (fn [class & args]
3     (let [allocated {}
4           seeded ...
5           finished-instance ...]
6       finished-instance)))
```

The allocated step was easy. Earlier, I said we could use code like this for seeded:

```
1 (assoc allocated :__class_symbol__ 'Point)
```

But we have a problem. Where do we get 'Point from? In the evaluation of the expression (make Point 1 2), the value bound to the symbol Point will be substituted for make's first parameter (class). But that value is a map, and the symbol Point appears nowhere in it. We'll have to add it as a new bit of metadata:

```
1 (def Point
2 {
3   :__own_symbol__ 'Point      ;; <== new
4   :__instance_methods__
5   {...}
6 })
```

It's annoying that Point is bound to a map that refers straight back to Point, but we're stuck with that. Languages with syntactic sugar for class definitions hide such circularity from the programmer¹.

So the “seeded” instance will be created with this:

```
1      (let [allocated {}]
2          seeded (assoc allocated
3                      :__class_symbol__ (:__own_symbol__ class)))
```

In order to create the finished instance, we have to find and apply a particular method, namely :add-instance-values. Finding the method looks like this:

```
1 (:add-instance-values (:__instance_methods__ class))
```

Let's use let to name that constructor:

```
1      (let [allocated {}]
2          seeded (assoc allocated
3                      :__class_symbol__ (:__own_symbol__ class))
4          constructor (:add-instance-values
5                      (:__instance_methods__ class)) ; <==== new
```


Once we have the constructor, we need only apply it to the half-finished (“seeded”) constructor and whatever arguments the user gave:

```
1      (let [allocated {}
2            seeded (assoc allocated
3                          :__class_symbol__ (:__own_symbol__ class))
4            constructor (:add-instance-values
5                        (:__instance_methods__ class))]
6      (apply constructor seeded args))) ; <<<<=
new
```

Since the constructor returns the new map that is to be (hallucinated to be) the object, that’s the return value of make.

All that given, the final implementation for make looks like this:

```
1 (def make
2   (fn [class & args]
3     (let [allocated {}
4           seeded (assoc allocated
5                         :__class_symbol__ (:__own_symbol__ class))
6           constructor (:add-instance-values
7                       (:__instance_methods__ class))]
8       (apply constructor seeded args))))
```

5.3 Message dispatch

We now need to make calls like this work:

```
1 user=> (send-to (make Point 1 2) :shift -2 -3)
```

Message dispatch will be similar to the last part of make, but not identical:

- :add-instance-values isn’t the only message we can send.
- Whereas make is given a class to work with, send-to is given an instance and has to look up the class.

That suggests a definition something like this:

```
1 (def send-to
2   (fn [object message & args]
3     (let [class (??? object ???)
4           method (message (:__instance_methods__ class))]
5       (apply method object args))))
```

The only new bit is how you go from an instance to its class. This is complicated because the instance's `__class__` is the *symbol* `Point`, not the value that `symbol` is bound to. To make the distinction clear(er), let's put both a symbol and its value into a map:

```
1 user=> (def SomeClass "...pretend there's a class definition here...")
2 user=> (def some-instance {__class__ 'SomeClass
3                               :__class__ SomeClass})
4 user=> some-instance
5 {__class__ SomeClass,
6  :__class__ "...pretend there's a class definition here..."}
```

We got the symbol `SomeClass` into the map by protecting it from evaluation with `quote`. Since `eval` removes quotes, it follows that we should get the class the symbol names with `eval`. (We saw a similar use of `eval` when we used it to [define apply](#).) Like this:

```
1 user=> (eval (__class__ some-instance))
2 "...pretend there's a class definition here..."
```

So that finishes `send-to`:

```
1 (def send-to
2   (fn [instance message & args]
3     (let [class (eval (__class__ instance))
4           method (message (__instance_methods__ class))]
5       (apply method instance args))))
```

One cosmetic change

In this chapter, I wanted to explain the three-step object-creation-and-initialization process common in OO languages, but separating two of those steps is rather silly in Clojure:

```
1 (let [allocated {}
2       seeded (assoc allocated
3                       :__class__ (__own_symbol__ class))
```

It would be better to write this:

```
1 (let [seeded {__class__ (__own_symbol__ class)}]
```

This works because, as with lists and vectors, `eval` recursively evaluates all the entries between literal curly-brace tokens before forming the map. So we can put arbitrary expressions inside curly braces. Here's another example:

```
1 user=> {:two (+ 1 1)}
2 {:two 2}
```

5.4 Exercises

You can find the source for `make` and `send-to`, as well as a definition of the `Point` class, in [sources/class.clj](#). My solutions are in [solutions/class.clj](#).

Exercise 1: The last two steps of `make` and `send-to` are very similar. Both look up an instance method in a class, then apply that method to an object and arguments. Extract a common function `apply-message-to` that takes a class, an instance, a message, and a sequence of arguments. Like this:

```
1 (def apply-message-to
2   (fn [class instance message args]
3     ...))
4
5 (apply-message-to Point a-point :shift [1 3])
```

It should use the `class` (a map, not a symbol) and the message (a keyword) to find a method/function. It should apply that method to the instance and the args. (You may have noticed that you can get the `class` from the instance, so those two separate parameters are not strictly needed. I thought it worthwhile to give each of the key values names in the parameter list.)

Feel free to define helper functions.

Next, use `apply-message-to` within both `make` and `send-to`.

Exercise 2: Up until now, the `:class` message has returned the symbol naming the class. That was OK while an object's class was nothing but a symbol. But now it'd be more appropriate to have `class-name` return the


```
9  })
10
11 user> (send-to (make Holder "stuff") :held)
12 "stuff"
```

Hint: What is the value of `(:not-there {:a 1, :b 2})`?

Hint: What are the values of the following expressions?

```
1 user=> (and true nil)
2 user=> (and true false)
3 user=> (or nil 3)
```

Exercise 5: Having implemented the previous exercise, what do you predict is the result of the following?

```
1 user=> (send-to (make Point 1 2) :some-unknown-message)
```

I think you'll agree that's not the best possible result. However, we're not going to keep the previous exercise's automatic accessors going forward, so it's not worth fixing.

1. Clojure and other Lisps let you define your own special forms with *macros*. It's one of their greatest strengths. The Lisp message is: "Don't like the syntax? Make up your own! (As long as it has lots of parentheses.)" If I were really embedding this OO language in Clojure, I'd use macros to let you write something closer to the class definitions other languages provide. But attractive class definitions is not the point of this book. [↩](#)

6. Inheritance (and Recursion)

Are you a person who likes to follow along in the repl while reading the book? If so, start by either using your solution from the last chapter or using mine:

```
1 user=> (load-file "sources/class.clj")
2 user=> (load-file "solutions/class.clj")
```

When I show longer redefinitions that have only a few lines of changes, I'll leave parts out. You can find the full redefinitions in [sources/java.clj](#). That's the implementation we're building in this chapter.

The source is also useful when copying-and-pasting from the book's PDF isn't working because of missing newlines.

Our `Point` class has its own `class` and `class-name` methods. But those methods should work with any object. So now is a good time to implement inheritance.

But first...

6.1 Assertions

As we've been working along, we've been using maps as classes and symbols to refer to classes. As our class system got more complicated during the writing of this chapter and the optional ones in Part V, I found myself sometimes passing symbols to functions that expected maps and vice-versa.

That leads to two annoying results:

1. Keywords as callables are excessively tolerant. If you pass a symbol to a function that expects a map, you're likely to execute code like this:

```
1 user=> (:__instance_methods__ 'Point)
2 nil
```

I would prefer getting an exception over getting a `nil`. As it is, the `nil` result tends to propagate through the code for a while before something blows up. Coupled with Clojure’s less-than-optimal stack traces, that makes debugging the problem unnecessarily hard.

2. Maps are self-evaluating. Therefore, these two expressions produce the same class map:

```
1 user=> (eval 'Point)
2 user=> (eval Point)
```

In some sense, that’s nice: you can write the wrong code—passing a map to a function that expects a symbol—and have it still work. What I’ve found, though, is that hampers later change when a function that has been working for two chapters suddenly fails in a mysterious way.

Therefore, starting in this chapter, I’ll be adding Clojure *assertions* to selected helper functions. That looks like this:

```
1 (def class-from-instance
2   (fn [instance]
3     (assert (map? instance))           ;; <== New
4     (eval (:__class_symbol__ instance))))
```

I recommend you do the same.

I’ve been involved in the endless debate between statically type-checked languages (like Java or Haskell) and dynamically-typed languages (like Clojure or Ruby) for almost 30 years. I started out on the static side but have found myself on the dynamic side, although in a live-and-let-live, whatever-floats-your-boat kind of way. The unpleasantness in this chapter hasn’t changed my mind, since it’s literally the first time I’ve felt the need to add `assert`s to my Clojure code.

6.2 Method dispatch

We’ll start our implementation by adding a symbol to the `Point` class that names its superclass:

```
1 (def Point
2   {
```

```

3   :__own_symbol__ 'Point
4   :__superclass_symbol__ 'Anything           ;; <=< New
5   :__instance_methods__ {...}})

```

I'm calling the superclass Anything because Clojure predefines Object to refer to the Java class of the same name.

```

1 user=> Object
2 java.lang.Object

```

Here is Anything:

```

1 (def Anything
2 {
3   :__own_symbol__ 'Anything
4   :__instance_methods__
5   {
6     ;; default constructor
7     :add-instance-values identity
8
9     ;; these two methods have been pulled up from Point.
10    :class-name :__class_symbol__
11    :class (fn [this] (class-from-instance this))
12  }
13 })

```

Notice that I gave Anything a default constructor. That way, classes that don't need any initialization don't have to define their own `:add-instance-values`. Since the default constructor should do nothing by default—merely return the same map it was given—I defined it with Clojure's `identity` function, which does just that (for any kind of argument).

Point's `:__superclass_symbol__` is the key that message dispatch will work with. The way I usually think of message dispatch under inheritance is as walking up the inheritance hierarchy. That is, this message send:

```

1 user> (send-to (make Point 1 2) :class-name)

```

... should first look in Point's `:__instance_methods__` map. When it fails to find `:class-name` there, it should move up to Anything and look there, where it will succeed.

Since that's so familiar, let's implement inheritance differently. Suppose we had a function, `lineage`, that produces a list of class symbols, starting at `Anything` and working down:

```
1 user=> (lineage 'Point)
2 (Anything Point)
```

If we also had a function, `class-instance-methods`, say, that could go from a class symbol to the `:__instance_methods__` map, we could get a sequence of maps. In the following example, I'll use `pprint` (short for “pretty print”) to print the maps more readably:

```
1 user=> (def maps (map class-instance-methods (lineage 'Point)))
2 user=> (pprint maps)
3 ({:add-instance-values #<user$fn...>,
4   :class-name :__class_symbol__,
5   :class #<user$fn...>}
6  {:add-instance-values #<user$fn...>,
7   :shift #<user$fn...>,
8   :add #<user$fn...>})
9 nil
```

Now let's merge those maps together:

```
1 user=> (def merged (apply merge maps))
2 user=> (pprint merged)
3 {:add #<user$fn>,
4  :shift #<user$fn>,
5  :add-instance-values #<user$fn>,
6  :class-name :__class_symbol__,
7  :class #<user$fn>}
```

We can apply methods from the merged map, which contains elements from both classes:

```
1 user=> ( (:class-name merged) (make Point 1 2))
2 Point
3 user=> ( (:shift merged) (make Point 1 2) 100 200)
4 {:y 202, :x 101, :__class_symbol__ Point}
```

What if the subclass overrides a method in the superclass? Do we have the right version in our merged map? Yes, because `merge` resolves key clashes by picking the value from the rightmost map. We can test that with `:add-`

instance-values. If Point's version is the one in the merged map, we should see an application of that function add :x and :y instance values to any map it receives as its this argument. Do we?

```
1 user=> ( (:add-instance-values merged) {} 1 2)
2 {:y 2, :x 1}
```

We do.

What follows is the process we just worked through manually put into a named function. Although we've been working with class names (symbols) so far, I'm going to have the function take a class as its argument. That will save some typing later.

```
1 (def method-cache
2   (fn [class]
3     (let [class-symbol (:__own_symbol__ class)
4           method-maps (map class-instance-methods
5                             (lineage class-symbol))]]
6       (apply merge method-maps))))
```

I picked the rather odd name method-cache to give me an excuse to point out that this implementation isn't *completely* silly. Suppose a Java class doesn't define toString(). The first time anyone sends the toString message to any object of that class, the runtime has to walk up the inheritance hierarchy. But the next time the runtime needs to find toString for that same class, it seems wasteful to walk the hierarchy again. Instead, it might cache the results in the class (or even in the object) to make method lookup faster. As long as caches get invalidated when superclasses change, the programmer could never notice the difference.

Such a cache would be a mapping from message names to functions, and it might look very much like the results of method-cache.

Real OO systems do a lot of caching of this sort, though I'm sure Java's is more sophisticated than what I've described.

In our case, we won't really cache the results of the merge, because showing you how you'd do that in Clojure doesn't fit the theme of this book.

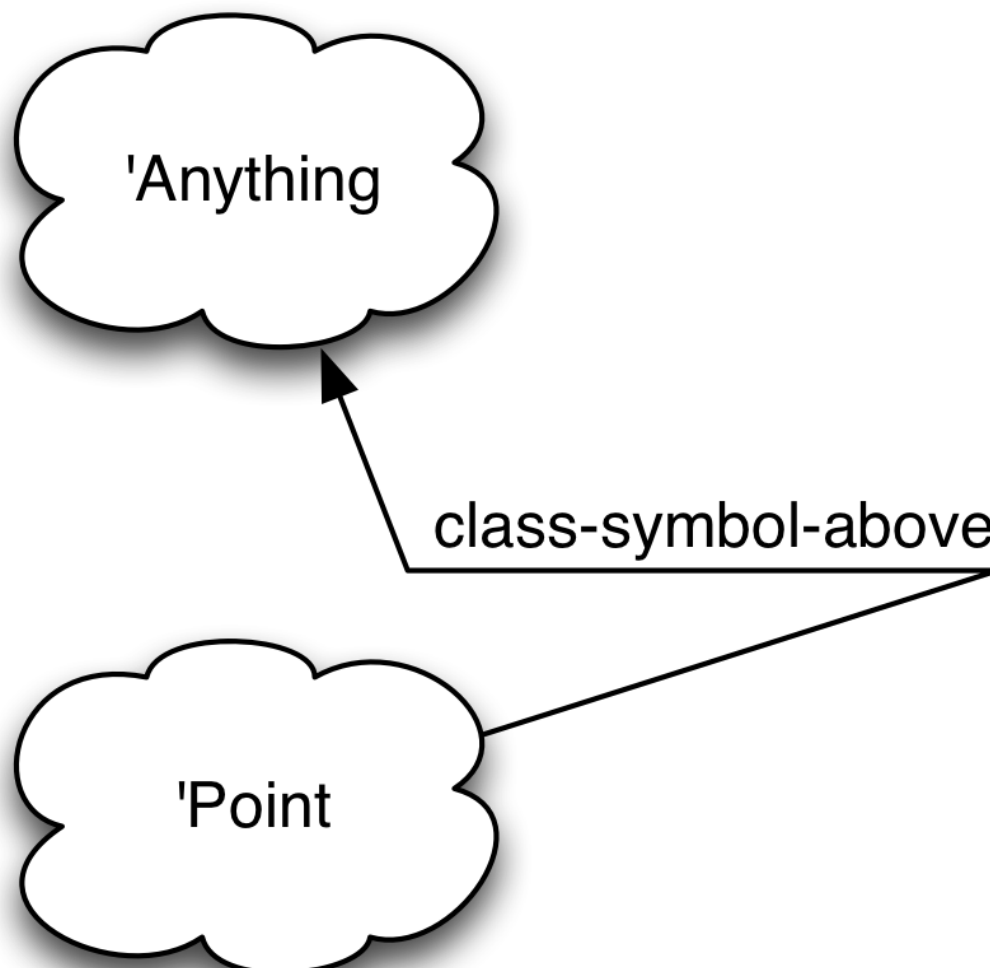
To implement method-cache, we need to implement two functions, `class-instance-methods` and `lineage`. Of the two, `class-instance-methods` is the easiest, because it's a use of `eval` like we saw in the previous chapter:

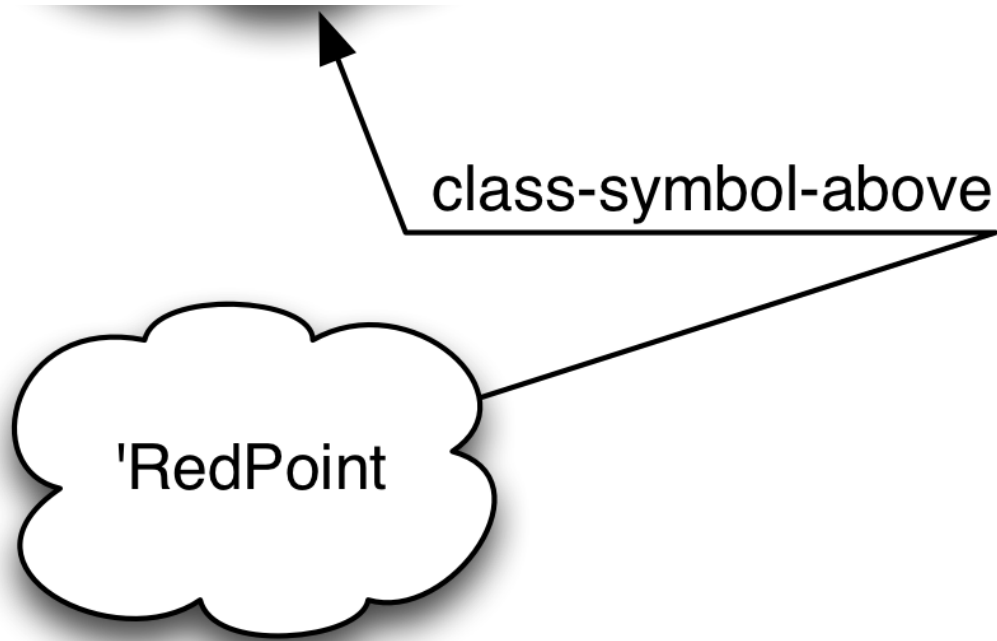
```
1 (def class-instance-methods
2   (fn [class-symbol]
3     (:__instance_methods__ (eval class-symbol))))
```

`lineage` is trickier.

6.3 Recursion

To make the explanation of `lineage` clearer, I'm going to pretend that we have a subclass of `Point` called `RedPoint`. Here's a picture of our class hierarchy:





I'm drawing the classes as clouds because there's a lot of stuff in them that's irrelevant to calculating `RedPoint`'s lineage. I'm drawing crooked lines from subclass to superclass because we don't have a direct pointer or link; instead we have to do the now-familiar `eval` trickery to move from one to the other. However, that can all be handled within a function that we can treat like a direct link:

```
1 (def class-symbol-above
2   (fn [class-symbol]
3     (:__superclass_symbol__ (eval class-symbol))))
```

We'd be happy if we could use sequence functions like `map` to traverse those links, but we can't: these classes aren't contained within sequences. That means we fall back to the functional programming equivalent of loops: *recursion*. A recursive function is one that calls itself. A typical recursive function does five things:

1. Breaks a big problem into an easy problem and a slightly-less-big problem.
2. Solves the easy problem.
3. Solves the slightly-less-big problem by applying itself (the recursive function) to it (the slightly-less-big problem).

4. Notices when the big problem has become so tiny it's now an easy problem.
5. Combines the solutions to all the easy problems.

But there's a fair amount of artfulness in how you apply those steps to any given problem.

It's perfectly possible to use recursion in object-oriented languages, but lots of OO programmers don't have much experience with it, so they view it as something strange and hard. If that describes you, I can't entirely erase that impression—that takes more practice¹ than this book can give you—but I'll try to help you make some significant steps toward seeing recursion as a comfortable tool.

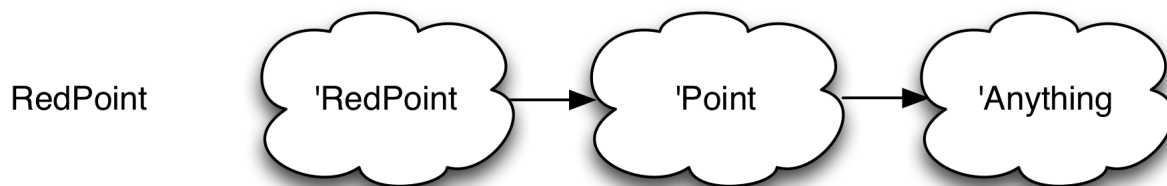
A first example of recursion

Since most explanations of recursion are mathematical or textual, I'm going to explain it visually.

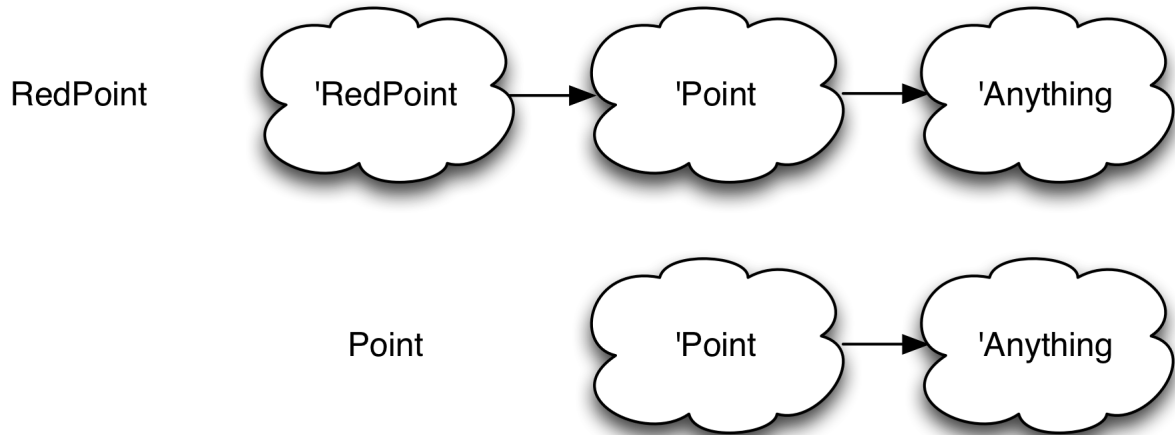
lineage begins with a symbol that refers to a class:

```
1 (lineage 'RedPoint)
```

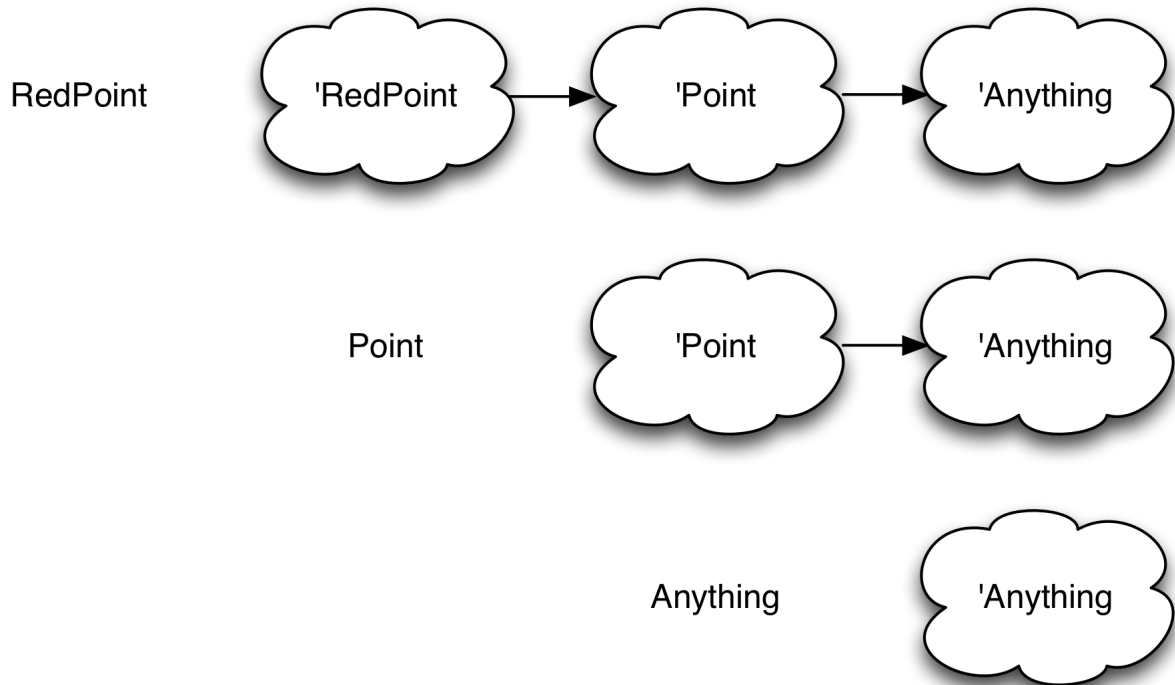
That symbol identifies a point in a class hierarchy. Let's juxtapose the symbol and the classes above it (including its own class):



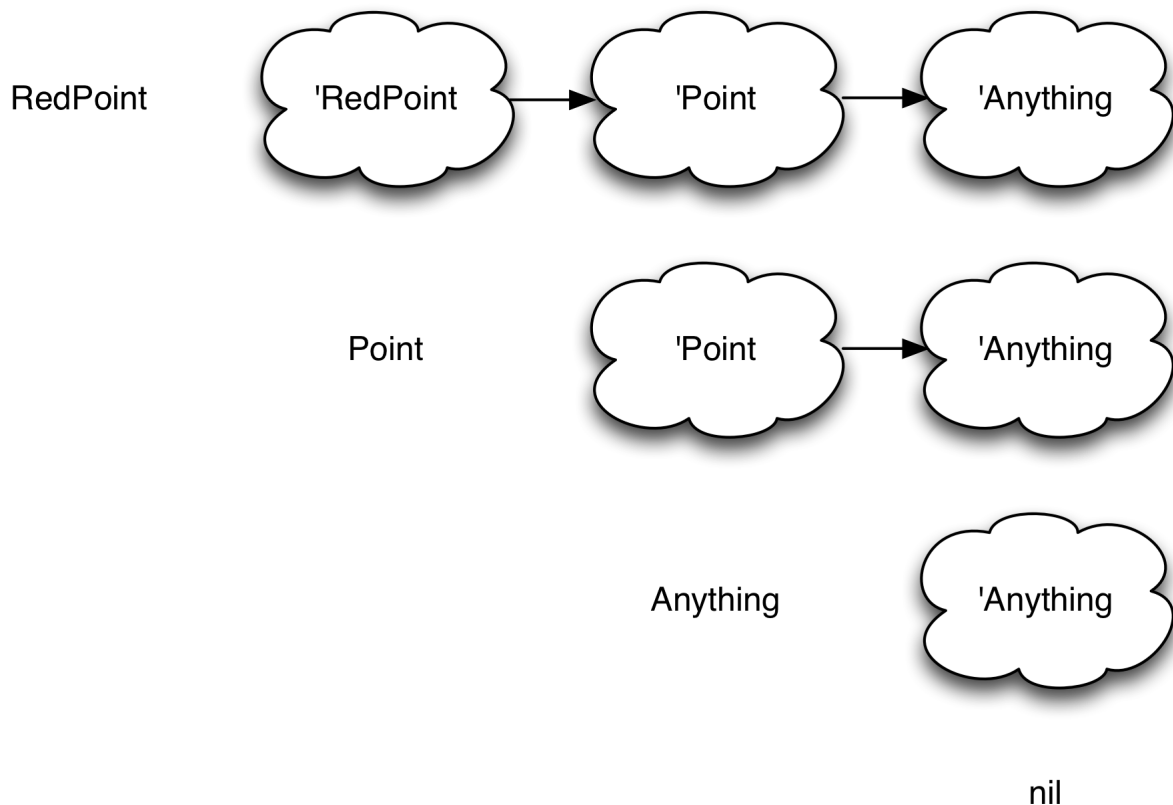
If we apply `class-symbol-above` to `RedPoint`, we'll get `Point`, which corresponds to a shorter class-cloud list:



If we apply `class-symbol-above` to `Point`, we get a single-element list:



The `Anything` class has no superclass. If you evaluate `class-symbol-above` for a class with no superclass, you'll get the result `nil`. We can add that to our growing diagram:



One way to think of what we’ve done is that we’ve followed the `class-symbol`-above links until we couldn’t any more. Another way is that we have a compound data structure (the list of clouds) that we’re making progressively smaller until we couldn’t any more—but at each step in the process, the smaller list retains the same “shape”, so that we know we can continue processing it in the same way. This “smaller but same shape” criterion is what recursion is about.

As we’ve drawn different levels of the diagram, the names along the left formed this sequence:

```
1 RedPoint      Point      Anything      nil
```

How can we turn that into the return value we desire? Well, let’s consider the last two elements. If you `cons` something onto `nil`, you get a one-element list:

```
1 user=> (cons 'Anything nil)
2 (Anything)
```

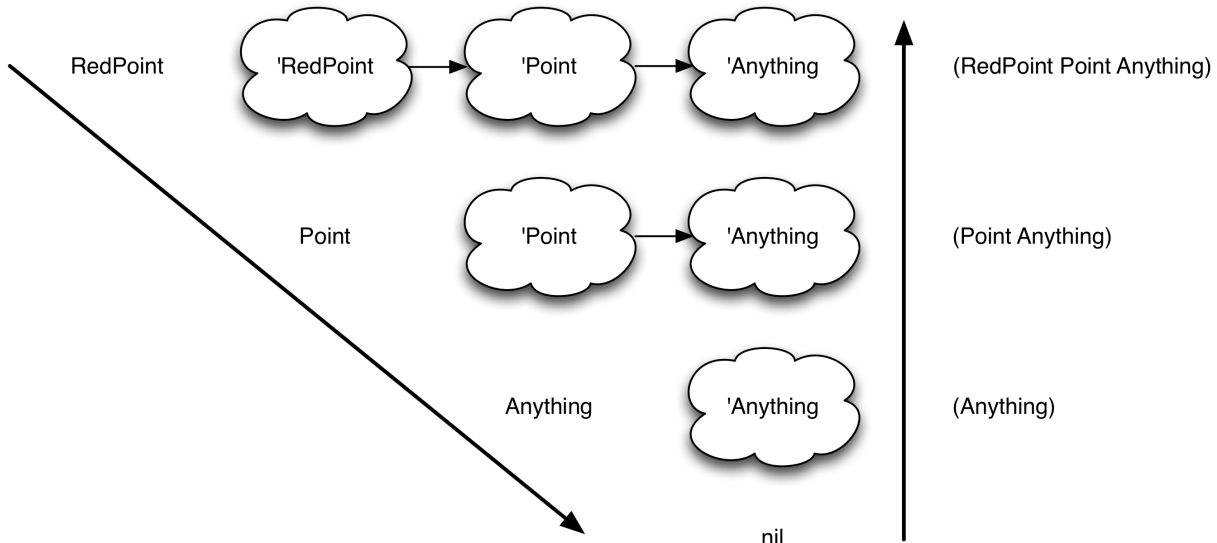
So if we keep “consing”, we’d get this:

```
1 user=> (cons 'RedPoint (cons 'Point (cons 'Anything nil)))
2 (RedPoint Point Anything)
```

That’s the reverse of what we’re looking for. (We want the most specific class on the right.) The fix is easy:

```
1 user=> (reverse (cons 'RedPoint (cons 'Point (cons 'Anything nil))))
2 (Anything Point RedPoint)
```

What we’ve done here is to *descend* (in the direction of smaller lists), finding names as we go, and then to *ascend* back up the lists, collecting what we’ve found. That’s a typical pattern in recursion:



That idiomatic pattern of computation has an idiomatic shape of function that produces it. I’ll show it now and explain it in detail in the next section:

```
1 (def recursive-function
2   (fn [something]
3     (if (**ending-case?** something)
4         something
5         (cons something
6               (recursive-function (**smaller-structure-from** something))))
7   )))
```

All we have to do is find the right functions to substitute for *ending-case?* and *smaller-structure-from*. For our problem, we end

when we find a `nil`, which the `nil?` predicate can tell us. We get the smaller structure from `class-symbol-above`.

```
1 (def recursive-function
2   (fn [class-symbol]
3     (if (nil? class-symbol)
4         nil
5         (cons class-symbol
6               (recursive-function (class-symbol-above class-symbol))))))
```

Checking the function with the substitution rule

We can hand-execute this function by applying the [substitution rule](#). Consider `(recursive-function 'RedPoint)`. When the symbol argument is substituted into the body of the function, we get a result that I'll represent like this:

```
1 (if (nil? 'RedPoint)
2     nil
3     (cons 'RedPoint
4           (recursive-function (class-symbol-above 'RedPoint))))
```

`if` heads a special form. It evaluates its first term. Then, depending on the result, it replaces itself with either the second or third terms. In this case, `RedPoint` is a symbol, which is not `nil?`. So the expansion of the `if` is the expansion of the `cons` list:

```
1 (cons 'RedPoint
2       (recursive-function (class-symbol-above 'RedPoint)))
```

Applying `class-symbol-above`, we get:

```
1 (cons 'RedPoint
2       (recursive-function 'Point))
```

The same process happens with `Point` as with `RedPoint`, yielding this:

```
1 (cons 'RedPoint
2       (cons 'Point
3             (recursive-function (class-symbol-above 'Point))))
```

... and then, from the call to `class-symbol-above`:

```
1 (cons 'RedPoint
2       (cons 'Point
3             (recursive-function 'Anything))
```

... which expands to:

```
1 (cons 'RedPoint
2       (cons 'Point
3             (cons 'Anything
4                   (recursive-function (class-symbol-above 'Anything)))))
```

...and then:

```
1 (cons 'RedPoint
2       (cons 'Point
3             (cons 'Anything
4                   (recursive-function nil))))
```

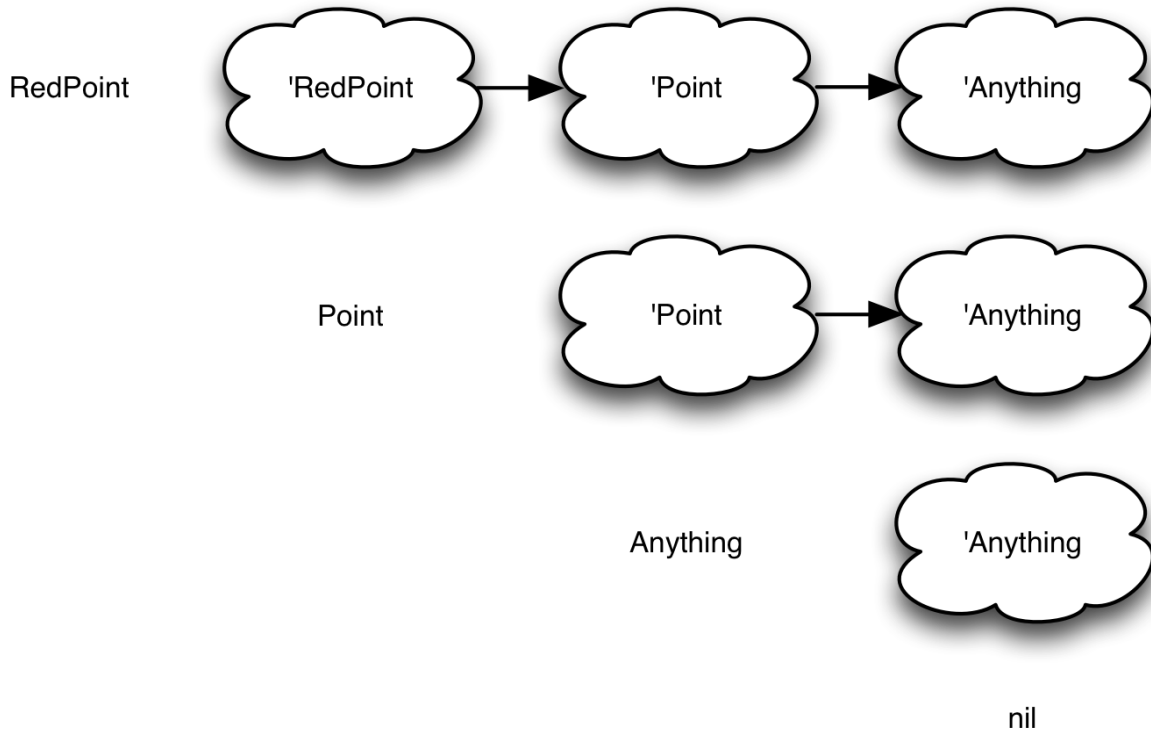
...and then, expanding recursive-function one last time:

```
1 (cons 'RedPoint
2       (cons 'Point
3             (cons 'Anything
4                   (if (nil? nil)
5                       nil
6                       (recursive-function (class-symbol-above nil))))))
```

In this case, nil really is nil?, so the expansion becomes:

```
1 (cons 'RedPoint
2       (cons 'Point
3             (cons 'Anything
4                   nil)))
```

... which builds the structure we want. Whew! Notice the shape of the expanded code echoes the shape of our levels of clouds:



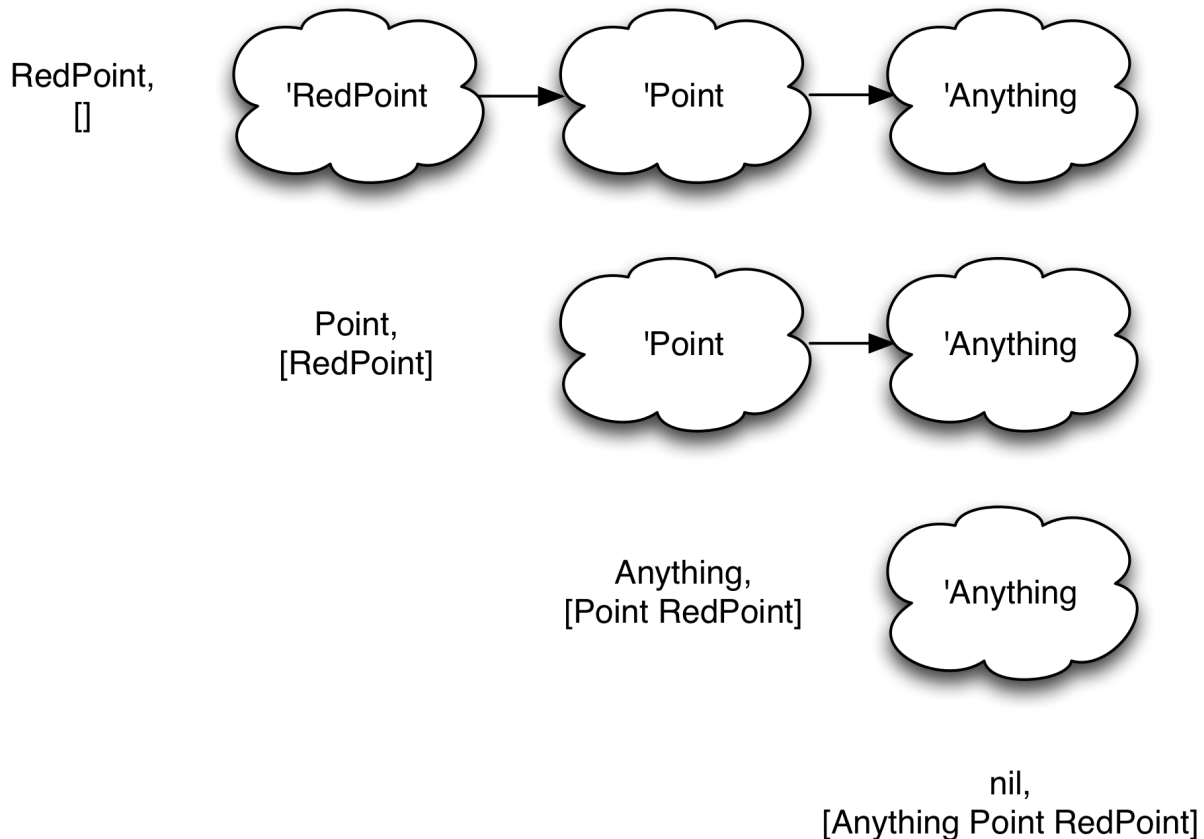
That's not a coincidence.

A second recursion pattern

The previous pattern builds up a pattern of computation that lies in wait until the bottommost function returns:

- “When the one below me returns, I’m going to cons on a RedPoint.”
 - “When the one below me returns, I’m going to cons on a Point.”
 - “When the one below me returns, I’m going to cons on an Anything.”
 - “I’m going to return nil. Watch what happens!”

Another pattern of recursion is to use a *collecting parameter* in addition to the structure you’re working on. Instead of putting the final answer together as it ascends the levels, it passes a partial solution down the levels and has each level improve on it:



Nothing happens as you ascend the levels: the finished solution is just passed along.

Notice that (in this case) the solution is in the right order: we don't have to reverse it.

Here's the pattern for recursion of this type:

```
1 (def recursive-function
2   (fn [something so-far]
3     (if (**ending-case?** something)
4         so-far
5         (recursive-function (**smaller-structure-from** something)
6                             (cons something so-far)))))
```

For our particular use of this pattern, the **ending-case?** can again be `nil?`, and we again make the smaller structure by using `class-symbol-` above. So we can calculate the lineage like this:

```

1 (def lineage-1
2   (fn [class-symbol so-far]
3     (if (nil? class-symbol)
4         so-far
5         (lineage-1 (class-symbol-above class-symbol)
6                     (cons class-symbol so-far)))))

```

Notice that I named the function `lineage-1`. That's because our original `lineage` only took one argument, and this pattern uses two. I just have the real `lineage` call `lineage-1` with an empty sequence:

```

1 (def lineage
2   (fn [class-symbol]
3     (lineage-1 class-symbol [])))

```

(As do many languages, Clojure has a way of providing default values for omitted arguments. It's a bit awkward for the simple cases, so I'm not using it in this book.)

Tail recursion

Let's look at `lineage-1` again:

```

1 (def lineage-1
2   (fn [class-symbol so-far]
3     (if (nil? class-symbol)
4         so-far
5         (lineage-1 (class-symbol-above class-symbol)
6                     (cons class-symbol so-far)))))

```

Suppose we had a class hierarchy with a million classes. That will mean a million and one calls to `lineage-1`. At the moment of that last call, the first call will be waiting on the second, the second will be waiting on the third, the third... ..and the millionth call will be waiting on the the million-and-first. Every one of those calls will be consuming memory while waiting.

And what will the millionth call do with the result of the million-and-first? Nothing. The recursive call is the last thing it does, so it just passes the result back to its caller, which just passes it back to *its* caller, and so on, all the way back to the original caller.

This pattern—where the last thing a recursive function does is call itself—is called *tail recursion*. Tail recursion is significant because a smart compiler can abandon the substitution rule and rewrite the function into a loop. Instead of making a new call to `lineage-1`, it would just `cons` the current `class-symbol` onto the value of `so-far`, change the value of `class-symbol` to the result of `class-symbol-above`, jump back to the beginning of the function, and redo it with the new values. *You* can't change symbol-to-value bindings, but the compiler can.

Not only do loops not risk running out of memory, they run faster.

Many functional languages detect tail recursion and turn it into loops. Because of limitations of the Java Virtual Machine, Clojure can't do that automatically. You have to tell it when you have tail recursion. That's done like this:

```
1 (def lineage-1
2   (fn [class-symbol so-far]
3     (if (nil? class-symbol)
4       so-far
5       ;VVVVV                                     next line
6       (recur (class-symbol-above class-symbol)
7              (cons class-symbol so-far)))))
```

The change is only one word: not a huge price to pay.

More general recursion patterns

Above, I showed you patterns with only two choices: the *ending-case?* function and the *smaller-structure-from* function. You'll need more general patterns for the exercises, ones that offer two more choices. The first applies to both patterns, and lets you choose a *combiner* function other than `cons`.

The second could apply to both patterns, but I found it added complexity in the exercises for no gain, so we'll only use it for the first pattern. It lets you pick an *ending-value* other than `nil`.

So here are our two patterns:

```

1 (def recursive-function
2   (fn [something]
3     (if (**ending-case?** something)
4         (**ending-value** something)
5         (**combiner** something
6           (recursive-function
7             (**smaller-structure-from** something))))))

1 (def recursive-function
2   (fn [something so-far]
3     (if (**ending-case?** something)
4         so-far
5         (recursive-function (**smaller-structure-from** something)
6                             (**combiner** something so-far))))

```

Notice that I made the `**ending-value**` in the first pattern a function applied to the ending “something”. That gives more flexibility than a constant. In some cases, you’ll use a constant instead of a function.

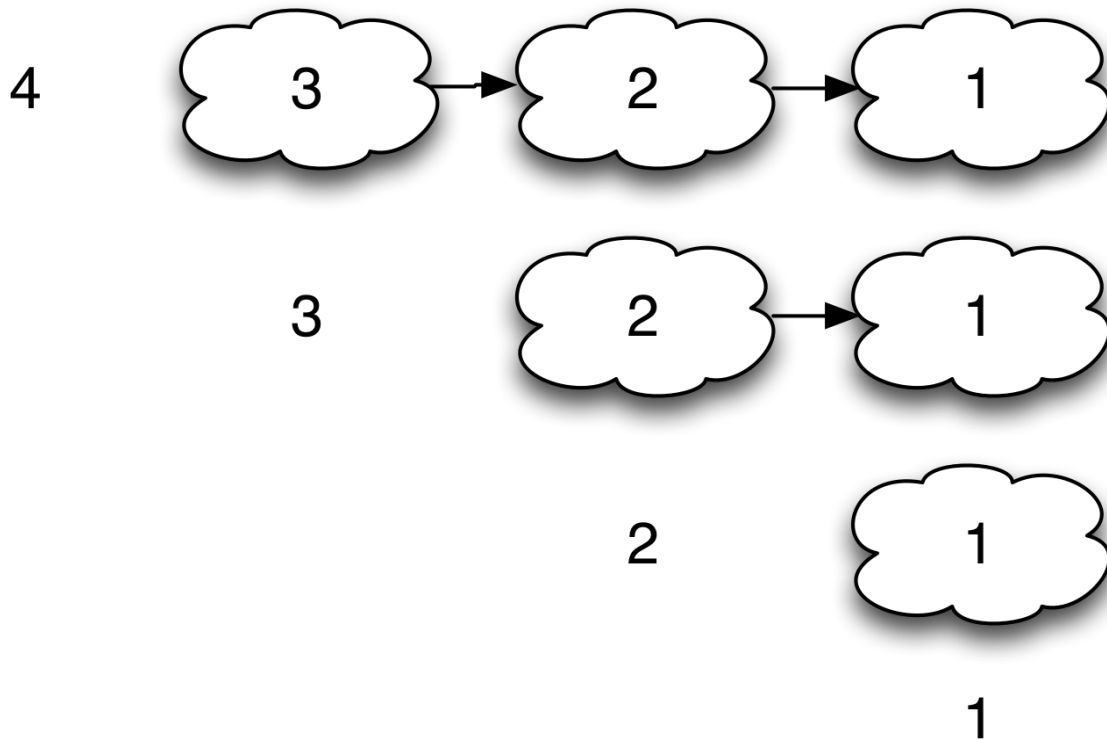
6.4 Exercises

My solutions are in [solutions/recursion.clj](https://github.com/ericniebler/solutions/blob/master/recursion.clj).

Exercise 1:

The `factorial` function is a classic example of recursion. To recap: factorial of 0 is 1, factorial of 1 is 1, factorial 2 is $2*1$, factorial of 3 is $3*2*1$, factorial of 4 is $4*3*2*1$ ².

Factorial can fit our first recursive pattern, where the sequence of descending numbers is the structure to make smaller.



Here's that pattern. Write a factorial that follows it:

```

1 (def factorial
2   (fn [n]
3     (if (**ending-case? n)
4         (**ending-value n)
5         (**combiner n
6           (recursive-function
7             (**smaller-structure-from n))))))

```

Hint: The zero case is an annoying detail. Don't worry about it at first.

Hint: The combining function is multiplication.

Hint: You make the “structure” smaller by decrementing n .

Hint: The ****ending-case?**b> is when n is either 0 or 1.**

Hint: The **ending-value** doesn't have to be a function. It is the constant value 1.

Exercise 2:

Here is the second pattern:

```
1 (def recursive-function
2   (fn [something so-far]
3     (if (**ending-case?** something)
4         so-far
5         (recursive-function (**smaller-structure-from** something)
6                               (**combiner** something so-far)))))
```

Implement factorial using it.

Exercise 3:

Use the second pattern to make a recursive-function that can add a sequence of numbers. Like this:

```
1 user=> (recursive-function [1 2 3 4] 0)
2 10
```

Hint: Do you remember the empty? function from the first chapter?

Hint: What function takes a sequence and produces the same sequence except without the first element?

Exercise 4:

Now change the previous exercise's function so that it can multiply a list of numbers. Like this:

```
1 user=> (recursive-function [1 2 3 4] 1)
2 24
```

(Note that the second argument changed to a 1.)

What is the difference between the two functions? Extract that difference, and make it the *first* argument to recursive-function.

Exercise 5:

Without changing recursive-function, choose starting values for the two wildcard parameters below that will cause it to convert a sequence of

keywords into this rather silly map:

```
1 user> (recursive-function **combiner**
2           [:a :b :c]
3           **starting-so-far**)
4 {:a 0, :b 0, :c 0}
```

Hint: I don't think there's any built-in Clojure function that you can pass in. You'll have to write your own with `fn`.

A bit trickier is producing a map that associates each keyword with its position in the list:

```
1 user> (recursive-function **combiner**
2           [:a :b :c]
3           **starting-so-far**)
4 {:a 0, :b 1, :c 2}
```

Hint: Try applying `count` to a map.

Exercise 6:

Pat yourself on the back! You have both used *and implemented* the built-in function `reduce`, perhaps the most dreaded of all sequence functions.

`reduce` has a different order of arguments, but it does the same thing:

```
1 user=> (reduce + 0 [1 2 3 4])
2 10
3 user=> (reduce * 1 [1 2 3 4])
4 24
5 user=> (reduce (fn [so-far val] (assoc so-far val 0))
6           {}
7           [:a :b :c])
8 {:c 0, :b 0, :a 0}
9 user=> (reduce (fn [so-far val] (assoc so-far val (count so-far)))
10          {}
11          [:a :b :c])
12 {:c 2, :b 1, :a 0}
```

Note: when you're down at the pub and you hear someone mention `fold`, know that they're talking about the same function as `reduce`, but they're

either not a Clojure programmer or they are but want to show off to a Haskell programmer of the appropriate sex.

6.5 Finishing up

We've written a method, `method-cache`, that provides a message-to-method map for all the messages that an instance can accept, taking inheritance into account. That's not yet been hooked into either `send-to` or `make`, though. Let's do that.

In the exercises for the last chapter, you created a method, `apply-message-to`, that applies the method named by a message to an instance and an argument list. My solution looked like this:

```
1 (def apply-message-to
2   (fn [class instance message args]
3     (apply (method-from-message message class)
4            instance args)))
```

We can replace the use of `method-from-message` by looking up the message in the `method-cache`.

```
1 (def apply-message-to
2   (fn [class instance message args]
3     (apply (message (method-cache class)) ;;<== changed
4            instance args)))
```

Note: `message` will have a keyword substituted into it, so it is used to look up a method by key in the `method-cache` map.

After that, `send-to` and `make` will work. Their code doesn't have to be changed from the previous chapters.

```
1 user=> (send-to (make Point 1 2) :class-name)
2 Point
3 user=> (send-to (make Point 1 2) :shift 3 4)
4 {:y 6, :x 4, :__class_symbol__ Point}
```

6.6 Choose your own adventure

The object system we have now is small, certainly inefficient, but also reasonably close to Java's (although it doesn't have class methods or

anything like the `super` keyword to call shadowed methods). By working through the exercises, you've learned more Clojure and also programmed in a functional style: recursion, basing code on generic, widely-useful datatypes³ (maps) rather than on custom classes, and a resolute treatment of functions as values that you can (for example) stick deep into a map to pull out later.

That prepares you for the [next part of the book](#), which abandons objects to concentrate on the different idioms and habits of functional programming. If, however, you'd like to learn more about object systems, I suggest first taking a side trip to [Part V](#), where you'll extend this Java-like model to look like Ruby's.

1. The book I usually recommend for learning recursion is *The Little Schemer* by Daniel P. Friedman, Matthias Felleisen, Duane Bibby, and Gerald J. Sussman (though I have not read the later editions).[↵](#)
2. I'm defining the 0 and 1 cases of factorial independently. You could define the 1 case in terms of the 0 case, but keeping them separate works better for my purposes in a chapter 14 example.[↵](#)
3. "It is better to have 100 functions operate on one data structure than 10 functions on 10 data structures."—Alan J. Perlis[↵](#)

II THE ELEMENTS OF FUNCTIONAL STYLE

Only a fool tries to define something as nebulous as a style of programming. So here goes! The functional style has three characteristics:

- Functions that create other functions. More narrowly, functions that take arguments that they use to parameterize the functions they create.
- The use of basic datatypes, notably maps, rather than the creation of classes. In the object-oriented style, you have many types (classes). Limiting the number of methods in a class is seen as a good thing. In the functional style, you have few types and a larger number and variety of general-purpose functions that act on them.
- Trying to solve problems by setting up data that can be seen as flowing through functions. This involves abstracting away control flow, and also carefully isolating code that might need to change state.

Pedantically, none of those characteristics can uniquely identify a functional style. For example, there are many object-oriented languages in which you can write function-generating functions. And in languages in which you can't (like Java), you can accomplish the equivalent with classes that generate parameterized objects.

And most any language has basic datatypes. When I begin to talk about Clojure programs that use maps, you could easily point to Ruby's Hash datatype or Java's HashMap: both classes that correspond nicely to Clojure's maps. So how can the use of something every language has really characterize "functional programming"?

Exactly in this way: people *can* do these things in Ruby or Java, and they often *should* do them, but they *don't*. As a result, there's a tradition—a lore

—of functional programming, as it is today, that is barely known to object-oriented programmers. This part of the book aims explain that lore to you so that, when you’re facing a design problem, you can think “Oh, I could solve this in a functional style, and that would be *sweet*.” My recent contracting work has been in Ruby and Rails, and I find myself doing that a pleasing amount of the time.

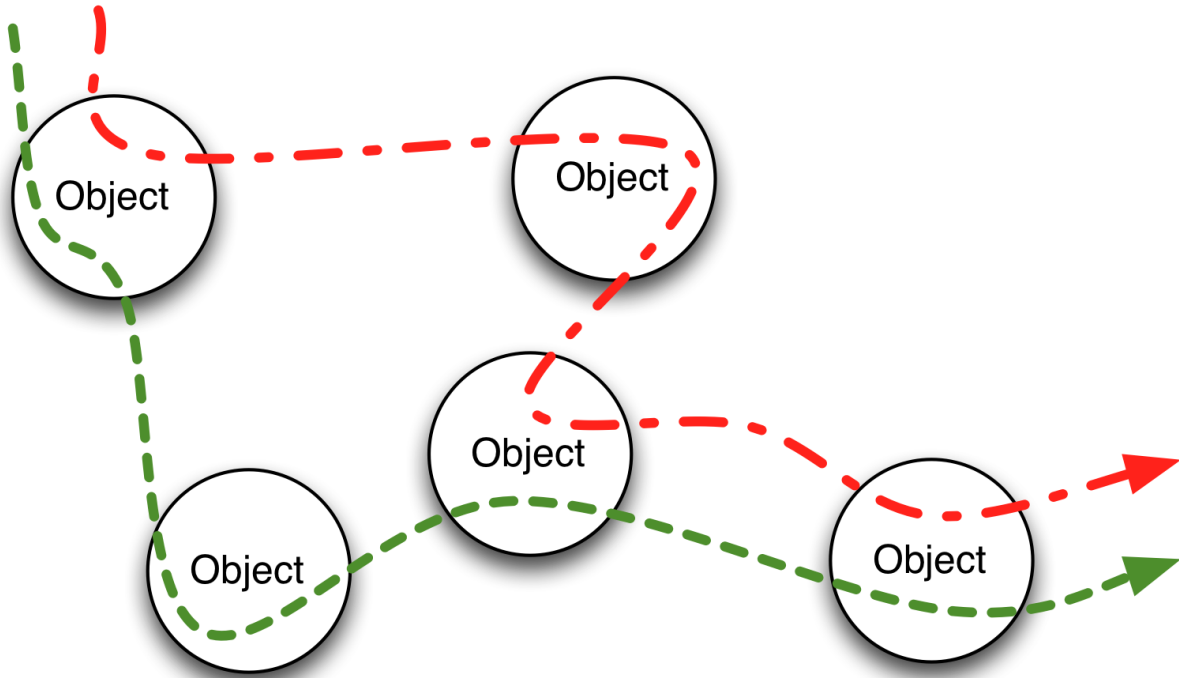
7. Basic Datatypes that Flow through Functions

In the previous part of the book, we built an object-oriented sub-language. As is typical with object-oriented languages, ours used a metaphor of methods (functions) being somehow attached to clumps of data (maps) and only accessible via that data.

What do we programmers gain from this way of structuring programs?

- It fits into a popular way of modeling the world, which is as taxonomies of categories and subcategories. It turns out that's not the way the world works, especially not the world inside people's heads^{[1](#)}, but as George Box famously said, "All models are wrong, but some are useful."
- It gives us a way to organize what might be many, many functions. If we have a map that's tagged as a `Point`, we know how to follow references to find *all* of the functions that we can apply to it. We also get a disciplined way of adding new functions to such clumps (via subclassing). In a world overflowing with choices, reducing the number of things we have to think about is valuable.

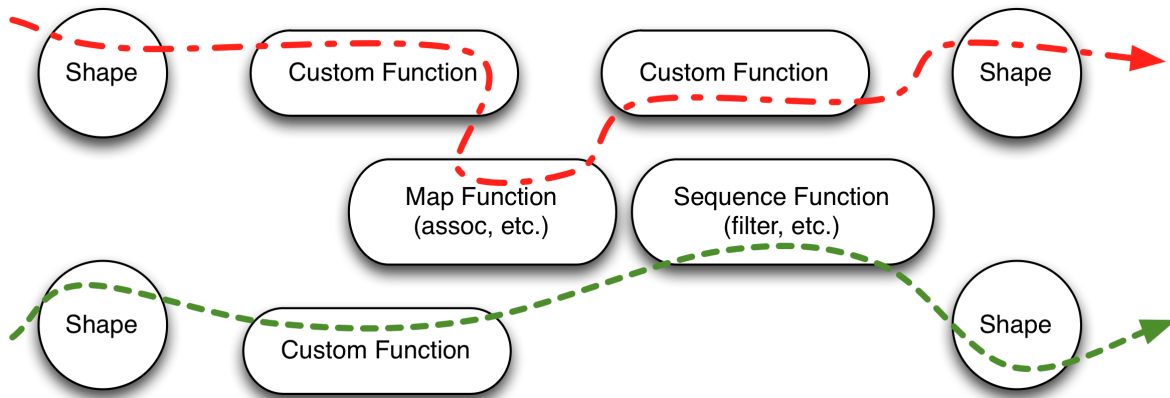
When I think of an object-oriented program in execution, I think of this:



Stable relationships and varying paths

Each object has a relatively small, relatively fixed set of neighbors. Each execution of the program may produce a somewhat different object graph, but there'll be a strong family resemblance between all of them. What's different about two executions is mainly about how the objects create and use their neighbors in response to variations in the input.

Instead of favoring a variety of data-dependent flows through an object graph, functional style favors a single flow through a linear pipeline of functions. Each path might have its own specialized functions, but each will also make heavy use of generic map and sequence functions. The picture looks like this:



Many specialized flows and shapes

I'll call this *dataflow style*. Note that, just as the different flows may have different helper functions, each flow will operate on its own custom “shape” of data, tailored to its particular purpose.

One way to think about this is that a class will have many client classes, so its emphasis is on behaviors that will be helpful to any of them. A flow does not try to be so accommodating. It's solving a narrow problem, using functions and data tailored to that problem.

Both styles—object-oriented and dataflow—have to deal with the temptation to duplicate code. In object-oriented programming, duplication leads to the creation of common superclasses and the splitting of one class into more than one. In dataflow programming, the emphasis is on extracting general-purpose functions and rearrangement of the contents (rather than type or behavior) of a shape. But before we can talk about that, we need an example of the style.

7.1 The problem

Your company has decided to have a big day of training. They've secured three instructors, any of whom can teach any of seven courses. Courses last half a day, and any given course can be repeated in the afternoon.

Courses are assigned first-come, first-served. If the morning versions of courses 1, 2, and 7 are signed up for first, that uses up the instructors, and the other courses are not available that morning (though they may be in the afternoon).

To complicate things, different courses can hold different numbers of “registrants”. Once a course is full, new registrants have to pick a different one. If there is no other course with room, they’re out of luck.

For this chapter, we’re going to deal with the flow provoked when a registrant asks to see which courses are available to her. The result should be two sequences, one for the morning courses and one for the afternoon. (The morning one should come first.)

The sequences should include all the courses our registrant is already signed up for. They should not include full classes (unless she’s registered for them.) If all the instructors are allocated, the sequences should not include courses no one has signed up for.

For each course, the results should contain the course name, the number of people registered, the number of spaces still available, and whether the registrant is registered. Specifically, the sequences should contain maps that look like this:

```
1 {:course-name "Zigging",  
2  :morning? true,  
3  :registered 5,  
4  :spaces-left 2,  
5  :already-in? true}
```

Both the morning and afternoon sequences should be alphabetized by course name.

A warning

My solution will, with abandon, use `map`, `filter`, and other sequence functions. If you think of each of those functions as looping over a sequence, constructing a new sequence, and passing it on to the next `map` or `filter`, my solution will seem so inefficient as to border on professional malpractice. Hold off on that judgment until you’ve read the chapter on [pushing bookkeeping into the runtime](#).

7.2 The general strategy

The interface to the dataflow will be a Clojure function rejoicing in the name `solution`. It will take three arguments.

One is a sequence of course maps, called `courses`:

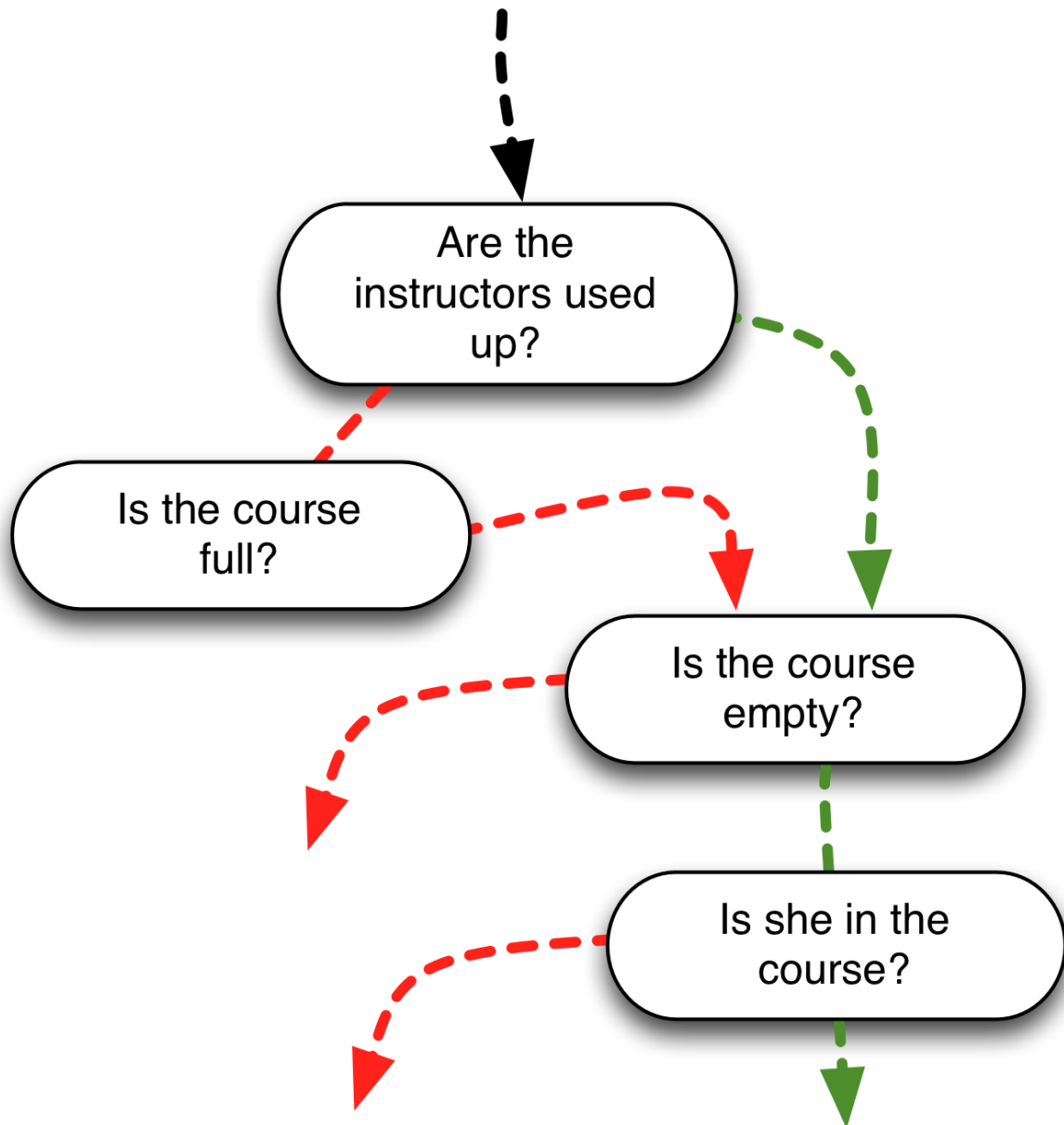
```
1 {:course-name "Zigging",  
2  :morning? true,  
3  :limit 5  
4  :registered 3}
```

Another is a sequence of strings. Each one is the name of a course our registrant has signed up for. It will look like this:

```
1 ["Zigging" "Zagging" "Thinking"]
```

The final argument is the number of instructors available.

The description of the problem implies a number of yes or no questions: Is the course full? Are the instructors all used up? Is this registrant already in the course? But earlier, I said that this chapter was about “a single flow through a linear pipeline of functions”. How do we work with binary answers without branching code (without `if` expressions)? We want to avoid code like the following?



There are two (sequential) parts to the answer. First, we annotate the maps given to `solution` with useful information. Second, we use sequence functions like `filter` to hide the `ifs` from our sight (in much the way that inheritance hides branching on object class from the object-oriented programmer's sight).

So, for example, to work with all the full courses, we wouldn't write code like this:

```
1 (if (= (:limit course) (:registered course)) ...
```

Instead, we'd first write code like this:

```
1 (assoc course
2   :full? (= (:limit course) (:registered course)))
```

... and then later use code like this:

```
1 (filter :full? courses)
```

In the annotation phase of `solution`, we'll add (via `assoc`) five fields to the course map:

- **spaces-left**: The solution map needs this.
- **already-in?**: Is the registrant already in the course? The solution map needs this.
- **empty?**: Does the course have zero registrants?
- **full?**: Does the course have no more room?
- **unavailable?**: Are no registrants allowed to sign up for the course (either because it's full or there's no instructor for it)?

Those given, a straightforward—and straight!—pipeline of sequence operations can produce the answer.

Or, rather, it gives us much more than the answer. We'll produce maps with a lot of information that the output isn't supposed to contain. But there's a map function (`select-keys`) that makes it easy to get rid of the unwanted information.

In later parts of the chapter, I'll detail the solution. For one small part of the solution, it'll use a new datatype. Since that datatype is another generally useful one, it's worth explaining in a bit of detail.

7.3 Sets

The `set` function takes any sequence as an argument and converts it to a set, removing all duplicates in the process.

```
1 user=> (set [1 2 1 3 1])
2 #{1 2 3}
```

The printed value shows the literal notation for sets.

You can use the `contains?` function to find if an element is in the set:

```
1 user=> (contains? #{1 2 3} 2)
2 true
```

A set is also a callable. It's given a single value as an argument. If the value is in the set, it's returned. If it's not, `nil` is returned:

```
1 user=> (#{1 2 3} 1)
2 1
3 user=> (#{1 2 3} 4)
4 nil
```

Such calls are often used instead of `contains?` (so long as you don't care that return values are merely "truthy" and "falsey", not specifically `true` and `false`).

Many of the usual set operations aren't pre-loaded in Clojure. You retrieve them like this:

```
1 user=> (use 'clojure.set)
```

Then you have functions like these:

```
1 user=> (union #{1 2} #{2 3})
2 #{1 2 3}
3 user=> (intersection #{1 2} #{2 3})
4 #{2}
5 user=> (difference #{1 2} #{2 3})
6 #{1}
```

You can use `filter` and other sequence operations on sets, but the result is a sequence, not a set:

```
1 user=> (filter odd? #{1 2 3})
2 (1 3)
3 user=> (set? (filter odd? #{1 2 3}))
4 false
```

For that reason, there's a `select` function that acts like `filter` but returns a set:

```
1 user=> (select odd? #{1 2 3})
2 #{1 3}
```

7.4 Annotating maps

Note: all the code shown in the rest of this chapter can be found in [sources/scheduling.clj](https://github.com/tonyoo/scheduling.clj).

I'll separate map annotation into three functions. The first uses the sequence of course maps and the other sequence of course names to add two fields that the answer requires:

```
1 (def answer-annotations
2   (fn [courses registrants-courses]
3     (let [checking-set (set registrants-courses)]
4       (map (fn [course]
5              (assoc course
6                    :spaces-left (- (:limit course)
7                                   (:registered course))
6              :already-in? (contains? checking-set
6                                (:course-name
6                                course))))
10      courses))))
```

Notice the use of `contains?` for an easy check if the registrant is in a course.

Here's an example of `answer-annotations` in action:

```
1 user=> (answer-annotations [{:course-name "zigging" :limit 4,
2                             :registered 3}
3                             {:course-name "zagging" :limit 1,
4                             :registered 1}]
5   ["zagging"])
6
7 ({:already-in? false, :spaces-left 1,
8   :registered 3, :limit 4, :course-name "zigging"}
9  {:already-in? true, :spaces-left 0,
10   :registered 1, :limit 1, :course-name "zagging"})
```

That was simple enough, but it illustrates an important benefit of using maps instead of classes. Because I was working with maps, my test data could include only the three keys that matter. That's in sharp contrast to many conceptually simple tests in object-oriented systems. When you construct an object, the coding of the constructor assumes that you're constructing it for *any legitimate use*, not for one particular use. As such, the constructor is obliged to fill in all fields, even those not used in the test. Since, often, those fields are other objects, a test may require the construction of a large object graph. That leads to test fragility and systems that are hard to change.

There are techniques that avoid fragility, ably described in Freeman and Pryce's fine [*Growing Object-Oriented Software, Guided by Tests*](#), but they require skill and discipline. This functional style “flattens out” object relationships, which I find is easier on my brain and saves me time. (How we get the flattened graphs is the topic of the next chapter.)

The next function adds two useful keys. I'm doing that mainly to avoid ugly `ifs` in later code, but—as is so often the case—making code nice is the same thing as modeling the problem. “Empty” and “full” are adjectives from the problem domain, and it's appropriate that we put them in the solution domain.

```
1 (def domain-annotations
2   (fn [courses]
3     (map (fn [course]
4           (assoc course
5                 :empty? (zero? (:registered course))
6                 :full? (zero? (:spaces-left course))))
7       courses)))
```

Here's a test:

```
1 user=> (domain-annotations [{:registered 1, :spaces-left 1},
2                             {:registered 0, :spaces-left 1},
3                             {:registered 1, :spaces-left 0}])
4 ({:registered 1, :spaces-left 1, :full? false, :empty? false,}
5  {:registered 0, :spaces-left 1, :full? false, :empty? true, }
6  {:registered 1, :spaces-left 0 :full? true, :empty? false, })
```


Once again, the test can focus on exactly the test data that matters, which is a considerable win for test clarity.

Another idea from the problem domain is the idea of an unavailable course, which I will represent with an `:unavailable?` key. I didn't include it in `domain-annotations` for two reasons:

- It's the only key that depends upon the instructor count, and the separate function makes that clear.
- Its value depends on whether the course map is full, empty, or neither. Setting it in `domain-annotations` would mean that `domain-annotations` and `answer-annotations` would look pretty different. Putting it in its own function means all three functions look roughly the same.

```
1 (def note-unavailability
2   (fn [courses instructor-count]
3     (let [out-of-instructors?
4           (= instructor-count
5             (count (filter (fn [course] (not (:empty? course)))
6                           courses)))]
7       (map (fn [course]
8             (assoc course
9                   :unavailable? (or (:full? course)
10                                   (and out-of-instructors?
11                                       (:empty? course))))))
12         courses))))
```

To put the three functions together, we just have to chain them so that the sequence returned by one is used as an argument to the next. But here we run into a problem. Here's a solution that nests the three functions:

```
1 (def annotate
2   (fn [courses registrants-courses instructor-count]
3     (note-unavailability (domain-annotations
4                           (answer-annotations courses
5                                                 registrants-courses))
6                           instructor-count)))
```

That's ugly, and it obscures the step-by-step flow. Those of us whose native languages are written left to right typically think of time as flowing in that direction and then from top to bottom. In `annotate`, the flow is from the middle and up and to the left.

We can get a more culturally-appropriate flow like this:

```
1 (def annotate
2   (fn [courses registrants-courses instructor-count]
3     (let [answers (answer-annotations courses
4                                     registrants-courses)
5           domain (domain-annotations answers)
6           complete (note-unavailability domain
7                                     instructor-count)]
8       complete)))
```

I'm not wild about that either, for two reasons:

- The symbol names (`answers`, `domain`, and `complete`) don't add anything to the names of the functions. If anything, they subtract clarity.
- You have to make sure that the name on step N gets used on the right-hand side of step $N+1$. I screw that up a really annoying amount of the time.

We need a new notation.

7.5 The arrow operator, `->`

The arrow operator passes values through a pipeline of functions. Here's how it'd be used in `annotate`:

```
1 (def annotate
2   (fn [courses registrants-courses instructor-count]
3     (-> courses
4         (answer-annotations registrants-courses)
5         domain-annotations
6         (note-unavailability instructor-count))))
```

`->` translates the code after it into a nested series of function calls, following these rules:

1. The first element is inserted (as defined by the next two steps) into the second, making a new element, which is inserted into the third, and so on.
2. When the next element is a list, the previous element is inserted as its second element. Thus,
(-> 1 (- 2)) is the same as (- 1 2).
3. When the next element is not a list, it's converted to a single-element list, whereupon the previous rule applies. Thus, (-> 1 inc) becomes (inc 1).

Being able to read this style is important for the rest of this chapter and book, so take a few moments to practice.

Exercises

My solutions are in [solutions/arrows.clj](#).

Exercise 1:

Use -> to process [1] by removing the number from the vector, incrementing it, and wrapping it in a list. The sequence of values would be this:

```
1 [1]
2 1
3 2
4 (2)
```

Exercise 2:

Add a step to the previous example. After incrementing the value, multiply it by 3, for this sequence of values:

```
1 [1]
2 1
3 2
4 6
5 (6)
```

Hint: The multiplication will be done by a form similar to the first and third steps in annotate.

Exercise 3:

Here's a function that doubles a number:

```
1 (fn [n] (* 2 n))
```

Use that function instead of `(* 2)` in this chain:

```
1 (-> 3 (* 2) inc)
```

Hint: Use the second `->` rule in a painfully literal-minded way.

Exercise 4:

Convert `(+ (* (+ 1 2) 3) 4)` into a three-stage computation using `->`:

1. The first step calculates `(+ 1 2)`
2. The result is passed to a step that multiplies the result by 3.
3. And that result is passed to a step that adds 4.

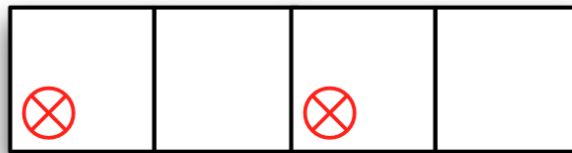
7.6 Processing sequences of maps

Here's the basic shape of what we have to do once we've annotated the maps. We begin with a sequence, in no particular order, like this:



The checkmarks are courses our recipient is signed up for (`:already-in? true`). The crossouts are courses our recipient may not sign up for (`:unavailable? true`).

Since `:already-in?` courses should always be visible, and the remainder sometimes should not be, we can begin by separating them:



Inexplicably, Clojure doesn't have a separate-by-predicate function, but it's easy enough to write one:

```
1 (def separate
2   (fn [predicate sequence]
3     [(filter predicate sequence) (remove predicate sequence)]))
```

It would be used like this:

```
1 (separate :already-in? courses)]
```

Once the two kinds of courses are separated, the one on the right should have `:unavailable?` courses removed. That looks like this:



That can easily be done with `(remove :unavailable? ...)`.

And, finally, the two halves can be reunited with `concat`:



All that's simple enough. But to make the code look properly pleasing, we need another Clojure feature.

7.7 Destructuring arguments

The separate function produces two sequences:

```
1 user=> (separate odd? [1 2 3 4 5])
2 [(1 3 5) (2 4)]
```

You *could* capture the two halves for later work like this:

```
1 (let [both (separate odd? [1 2 3 4 5])
2       odds (first both)
3       evens (second both)]
4   ...)
```

But surely we don't live in a world so depressing and hostile that it requires three lines for such a simple operation? Indeed we do not:

```
1 (let [ [odds evens] (separate odd? [1 2 3 4 5])]
2   ...)
```

This is called *destructuring binding*. Instead of putting a symbol on the left-hand side of a `let` step, we put a vector of two symbols. That tells Clojure that the right-hand side should deliver up a two-element sequence. The first element is bound to `odds` and the second to `evens`.

Destructuring bindings can also be used when declaring function parameters. You can also destructure maps and capture sequences in different ways. I'll leave those explanations to the Clojure documentation or Clojure books².

Given destructuring binding, the sequence of pictures in the previous section can be implemented like this:

```
1 (def visible-courses
2   (fn [courses]
3     (let [[guaranteed possibles] (separate :already-in? courses)]
4       (concat guaranteed (remove :unavailable? possibles)))))
```

Destructuring binding is a special case of pattern matching parameters, which will be discussed in its [own chapter](#).

7.8 Finishing up

There's little of interesting remaining. Given that we've annotated the course maps with much information that's only of temporary interest, we need a function that strips that out:

```
1 (def final-shape
2   (fn [courses]
3     (let [desired-keys [:course-name :morning? :registered :spaces-left
4                        :already-in?]]
5       (map (fn [course]
6              (select-keys course desired-keys))
7            courses))))
```

The two halves of the day should be independent. The availability of the morning version of a course has nothing to do with its availability in the afternoon. So it makes sense to process the two halves of the day separately. That'll be done in `half-day-solution`. It performs several steps:

1. It annotates the input courses.
2. It uses `visible` to find the courses to show.
3. It sorts them by the course name.
4. It puts them into their `final-shape`.

That flow looks like this:

```
1 (def half-day-solution
2   (fn [courses registrants-courses instructor-count]
3     (-> courses
4         (annotate registrants-courses instructor-count)
5         visible-courses
6         ((fn [courses] (sort-by :course-name courses))))
7         final-shape)))
```

Note that we have to rely on `fn` when sorting because `sort-by` doesn't take a collection as the first argument.

With all that done, our solution function need only separate its input into morning and afternoon courses, process each, and return both results:

```
1 (def solution
2   (fn [courses registrants-courses instructor-count]
3     (map (fn [courses]
4           (half-day-solution courses registrants-courses
5                                instructor-count))
6         (separate :morning? courses))))
```

There you have it: a pretty functional solution to our problem. But that solution is of no use by itself. It probably lives within a non-functional world. The next chapter is about how that might be arranged.

7.9 Exercises

You can start from this source: [sources/scheduling.clj](#). My solutions are in [solutions/scheduling.clj](#).

Exercise 1

Managers are not allowed to take afternoon courses because that would make them put in too many hours in a day. Implement that restriction.

(Note: if your solution is like mine, you'll be annoyed by how a simple change ripples through the code. That wouldn't happen with instance variables! I'll address that in the final section of this chapter.)

Exercise 2

Any course can have one or more prerequisite courses. Make it so that a registrant can't see a course if she doesn't have the prerequisites.

7.10 Avoiding argument passing

In my solution to the first exercise, I changed registrants-courses to registrants, and that name change had to ripple through three functions:

```
1 (def answer-annotations
2   (fn [courses **REGISTRANTS-COURSES**]
3     (let [checking-set (set **REGISTRANTS-COURSES**)]
```



```

5             (assoc course
6                 :unavailable? (or (:full? course)
7                                 (and out-of-instructors? ;;
<<==
8                                     (:empty? course))))))
9         courses))))

```

In this example, the body of the function referred to an external binding created with `let`. The same thing works with function parameters. We'll talk about the implications of that in the chapter on [functions that make functions](#). In this chapter, let's just note that that means we can nest all the helper functions inside `solution` and remove the constants from the parameter lists:

```

1 (def solution
2   (fn [courses registrants-courses instructor-count]
3     (let [answer-annotations
4           (fn [courses]
5             (let [checking-set (set registrants-courses)]
6               (map ... courses)))
7
8             domain-annotations (fn [courses] ...)
9             note-unavailability (fn [courses] ...)
10            annotate (fn [courses] ...)
11            visible-courses (fn [courses] ...)
12            final-shape (fn [courses] ...)]
13
14       (map (fn [courses]
15              (half-day-solution courses))
16            (separate :morning? courses))))

```

If you'd like to examine this solution in more detail, see [sources/wrapped-scheduling.clj](#).

It's useful to nest helper functions within other functions, but it also has its downsides. For one, you can't test the nested functions independently. And it's harder to grab one and use it in a different flow. (You have to add back in the parameters, but then you have to decide: should I use the new function in place of the original version, even though it's got parameters unnecessary in that context, or do I want to live with two versions of the same function?) I think nested functions are a bit more difficult to read, both because of the nesting and because the origin of their values isn't all in one place (their own parameter list).

As a result, I will often err on the side of having more top-level functions. To me, deciding how to partition work into top-level functions feels similar to deciding which classes to have: how do I divide the solution so that its structure is clear and meaningful?

Because of that, I decided to change the previous solution so that the three core parts of the flow (annotating, solving, and removing excess keys) were highlighted by being their own top-level functions:

```
1 (def solution
2   (fn [courses registrants-courses instructor-count]
3     (let [core-flow
4           (fn [courses]
5             (-> courses
6               (annotate (set registrants-courses) instructor-count)
7               half-day-solution
8               final-shape)))]
9       (map core-flow
10          (separate :morning? courses))))))
11
12 (def annotate
13   (fn [courses registrants-courses instructor-count] ...))
14
15 (def half-day-solution
16   (fn [courses] ...))
17
18 (def final-shape
19   (fn [courses] ...))
```

The remaining helper functions were divided between `annotate` and `half-day-solution`.

If you want to see the rest of the solution, look in [sources/scheduling-variant.clj](https://github.com/rpbj/scheduling-variant.clj).

When reading it, keep in mind some advice I first heard from Richard P. Gabriel: don't read Lisp functions from top to bottom. Instead, look for the most important code (typically closer to the bottom of the function, often the most visually dense) and read it first. Refer upwards to definitions when you need to understand them. In this code, what I consider the core code just happens to be written with `->` expressions.

You'll see that I went to some effort to make the `->` expressions consistently represent each function's core flow—the code you should look at first when trying to understand the function.

I also redid the previous exercises, using the code I just talked about as the base. You can see these new exercise solutions in [solutions/scheduling-variant.clj](#).

7.11 Information hiding

People with an object-oriented background who come to functional programming often find the plethora of functions a bit scary. If your system is one giant pile of functions, how do you avoid name clashes when writing new ones? If there are no classes to hide private methods and data, how do you avoid building fragile dependencies into your code?

We've seen one solution in this chapter: private functions can be nested inside public functions. They are completely inaccessible from the outside, so it's impossible to build fragile dependencies on them.

In addition, functional languages typically have at least the file-based modularity of C. At the least, you can have a unit ("module") of encapsulation (which happens to be a file in the file system), and functions within that module can be marked as either private or public. Clients of the module cannot refer to the private functions, and they need only worry about name clashes with public functions in the modules they explicitly use.

That may seem crude, but let's not forget that systems written with millions of lines of C run most of the Internet (in the form of the Linux kernel) and are, as I write, trundling about on the surface of Mars (the Curiosity rover). File-based modularity plus conventions plus care can get you a long way.

Since there's nothing like a typical functional way of handling modules, I won't say more about them in this book.

1. See Lakoff's *Women, Fire, and Dangerous Things: What Categories Reveal about the Mind*. I wrote a brief summary of his argument at <http://exampler.com/writing/pnsgc-2005-marick.pdf>.↵

2. Here are some Clojure books: *The Joy of Clojure* by Michael Fogus and Chris Houser (my favorite), *Clojure in Action* by Amit Rathore, and *Programming Clojure* by Stuart Halloway. [↩](#)

8. Embedding Functional Code in an OO Codebase

I recently solved two problems using a functional approach not too different from that of the last chapter (except that I wrote in Ruby instead of Clojure). But that didn't mean I was working in apps written in an entirely functional style: they were written using the Ruby on Rails framework, which is object-oriented. I embedded my functional code in object-oriented apps, using two approaches this chapter describes.

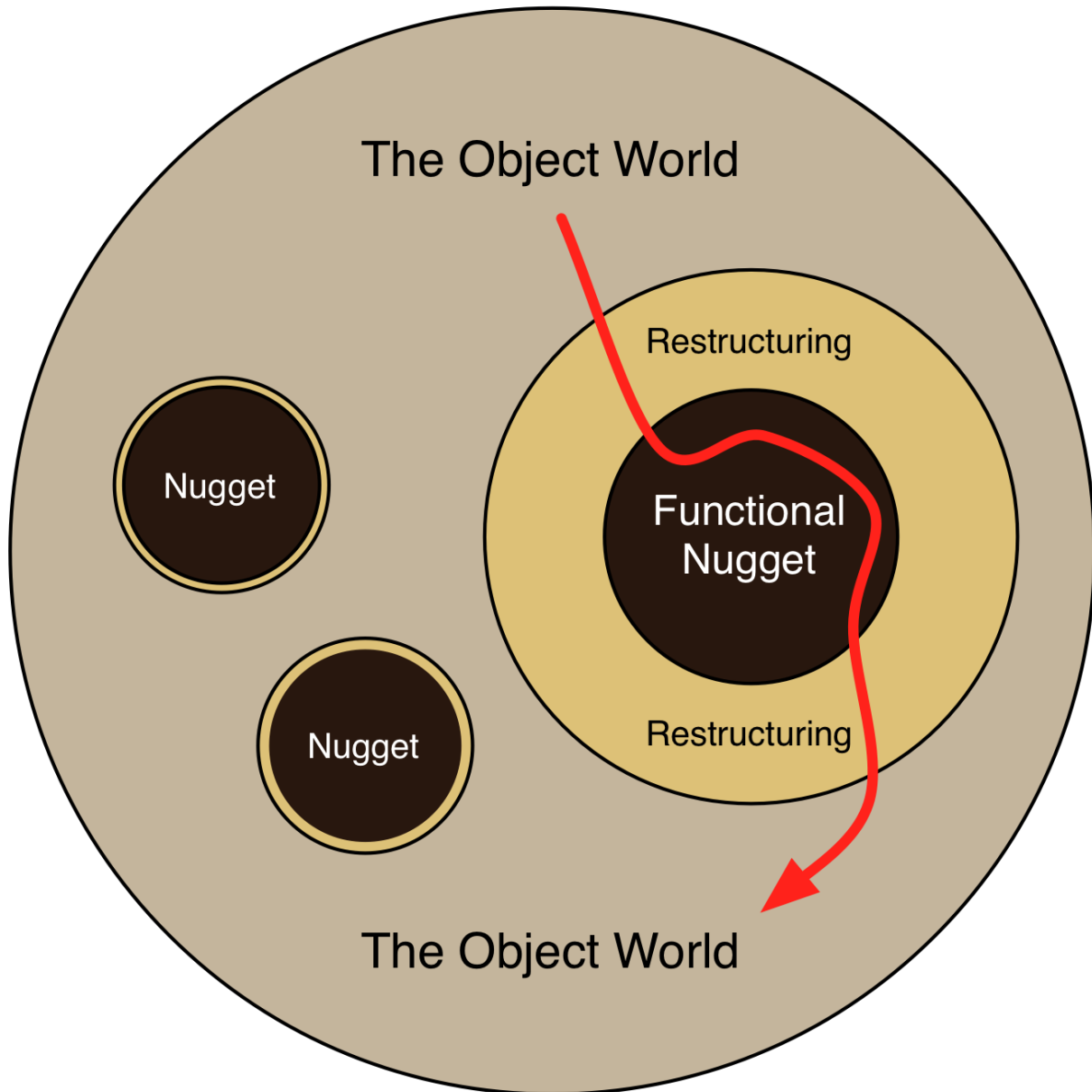
8.1 Tasty functional nuggets in a tub of OO ice-cream

In an [interview](#), the neuroscientist author of *The Compass of Pleasure* explained that our brains are wired to like food with contrasts of both flavor and texture. That accounts for the popularity of ice cream flavors like [ginger peach ice cream with chocolate fudge nuggets](#)¹ over plain flavors like chocolate.



Yum

When I think of this chapter's first approach, I think of it as embedding "nuggets" of functional code inside an OO app:



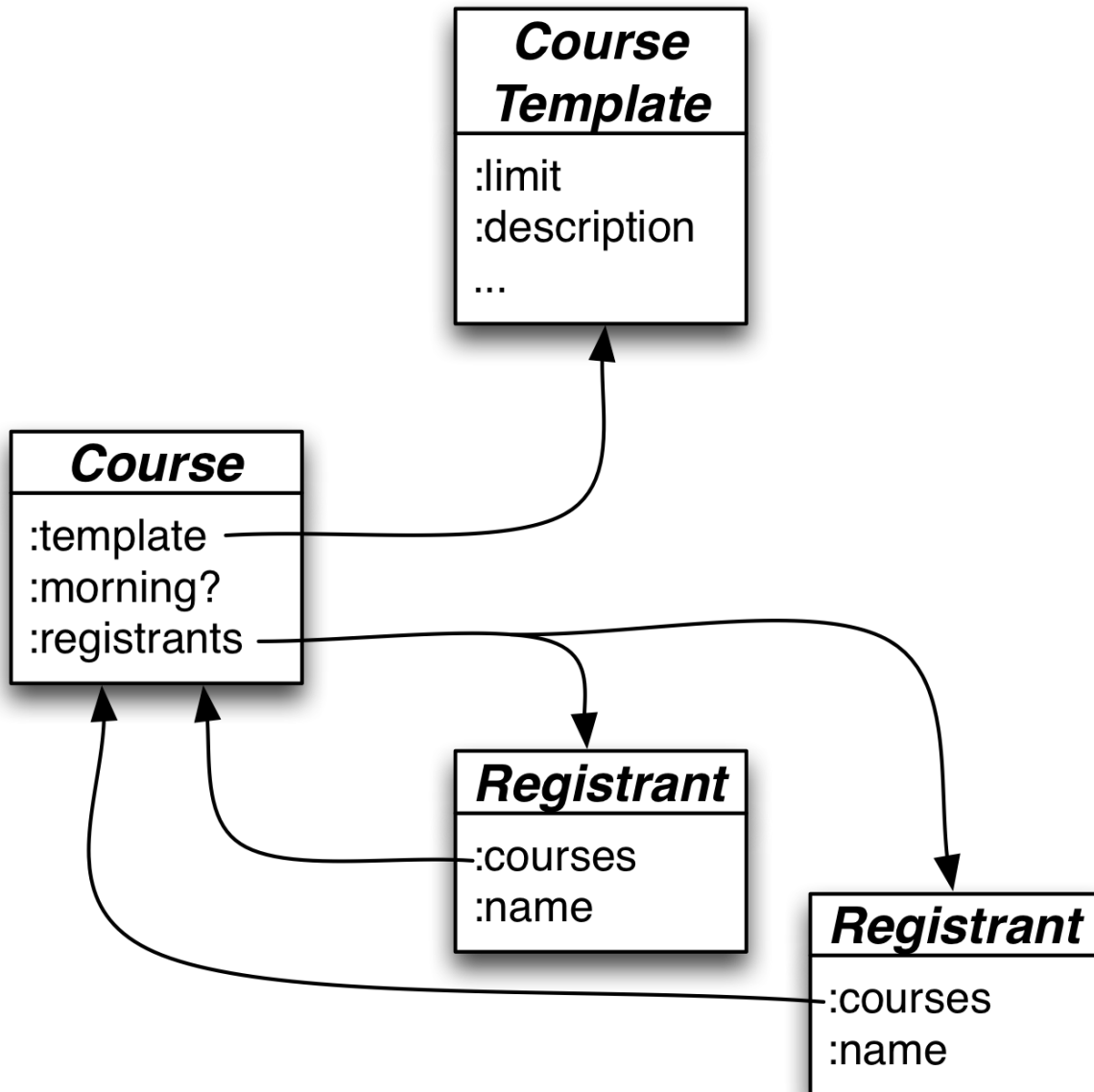
There are clearly separated functional and object parts of the program, with the object part in control. The object part occasionally flows data through a functional nugget to solve some problem. To do so, it converts, or *restructures*, object graphs into whatever shapes the dataflow accepts, then applies an interface function (like last chapter's solution) to the result. The

interface function returns shapes (maps and sequences, most likely) that the restructuring layer turns into objects.

My experience has been monolingual: it's all Ruby, just written in two different styles. To make things more interesting, I'll give a bilingual solution, in which Java code calls Clojure code.

The object model

I claim the following is a not-ridiculous object-oriented model of the last chapter's problem:



- We probably need to distinguish between a `CourseTemplate` class and a `Course` class. The first holds information like how many people a course can accommodate, the course description, prerequisites, and so on. The latter holds information like whether it's the morning or afternoon version and who is signed up for the course.
- Since all the instructors are the same, we don't seem to need a class for them (yet).
- We need a `Registrant` class. We can ask a `Registrant` instance which Courses that person is signed up for. We can also ask it for data like the name and email address.

- All this gets stored in a database, but we have an object-relational mapping layer (ORM) that hides that from us. For example, we don't care that a Registrant's courses field is built from a join table.

The Java code to convert such objects into Clojure values is a bit tedious, but straightforward. The Clojure code needs maps. Clojure provides several variants, all of which implement the `clojure.lang.IPersistentMap` interface. We'll use the concrete class `clojure.lang.PersistentHashMap`. Similarly, the Clojure sequence interface is defined by the `Sequential` interface, and `PersistentVector` is an implementation. That given, populating a Clojure map is a small matter of iteration:

```

1 // Create what the `solution` function will call `courses`:
2 // a sequence of maps.
3 public Sequential restructureCourses(List<Course> courses) {
4     ArrayList<Map> result = new ArrayList<Map>();
5     for (Course course : courses) {
6         HashMap<Keyword, Object> map = new HashMap<Keyword, Object>();
7         map.put(Keyword.intern("course-name"),
8                 course.getTemplate().getName());
9         map.put(Keyword.intern("course-limit"),
10                course.getTemplate().getLimit());
11         map.put(Keyword.intern("morning?"),
12                 course.isMorning());
13         map.put(Keyword.intern("registered"),
14                 course.getRegistrants().size());
15         result.add(map);
16     }
17     return PersistentVector.create(result);
18 }

```

The code also uses `clojure.lang.Keyword` to create the Clojure keywords for the maps. It does not have to convert Strings and Booleans because they are the same object in both Java and Clojure.

Converting a registrant's courses into a vector of strings is even easier:

```

1 // Create what the `solution` function will call `registrants-courses`:
2 // a sequence of strings.
3 public Sequential restructureSignups(Registrant registrant) {
4     ArrayList<String> result = new ArrayList<String>();
5     for (Course course: registrant.getCourses()) {
6         result.add(course.getName());
7     }

```

```
8     return PersistentVector.create(result);
9 }
```

Note: *I haven't actually compiled this, but it's roughly correct.*

Calling a Clojure function from Java looks roughly like this:

```
1 IFn f = (IFn) RT.var("solution");
2 Object result = f.invoke(restructureCoursesList(courses),
3                          restructureSignups(registrant));
```

Restructuring Clojure values into objects is also straightforward. Both sequences and maps have an `iterator` method. For a sequence, it returns the entries in order. For a map, the iterator returns `MapEntry` objects, which have `getKey` and `getValue` methods.

8.2 Object-relational mapping: threat or menace?

Our imaginary application uses a relational database. Database queries will return “result sets”, which are essentially sequences of key/value pairs. Here's the result of a database query:

Output pane					
Data Output		Explain	Messages	History	
	course-name text	morning? boolean	limit integer	registered bigint	
1	Zigging	t	4	0	
2	Zigging	f	4	2	
3	Zagging	t	2	1	
4	Zagging	f	2	1	

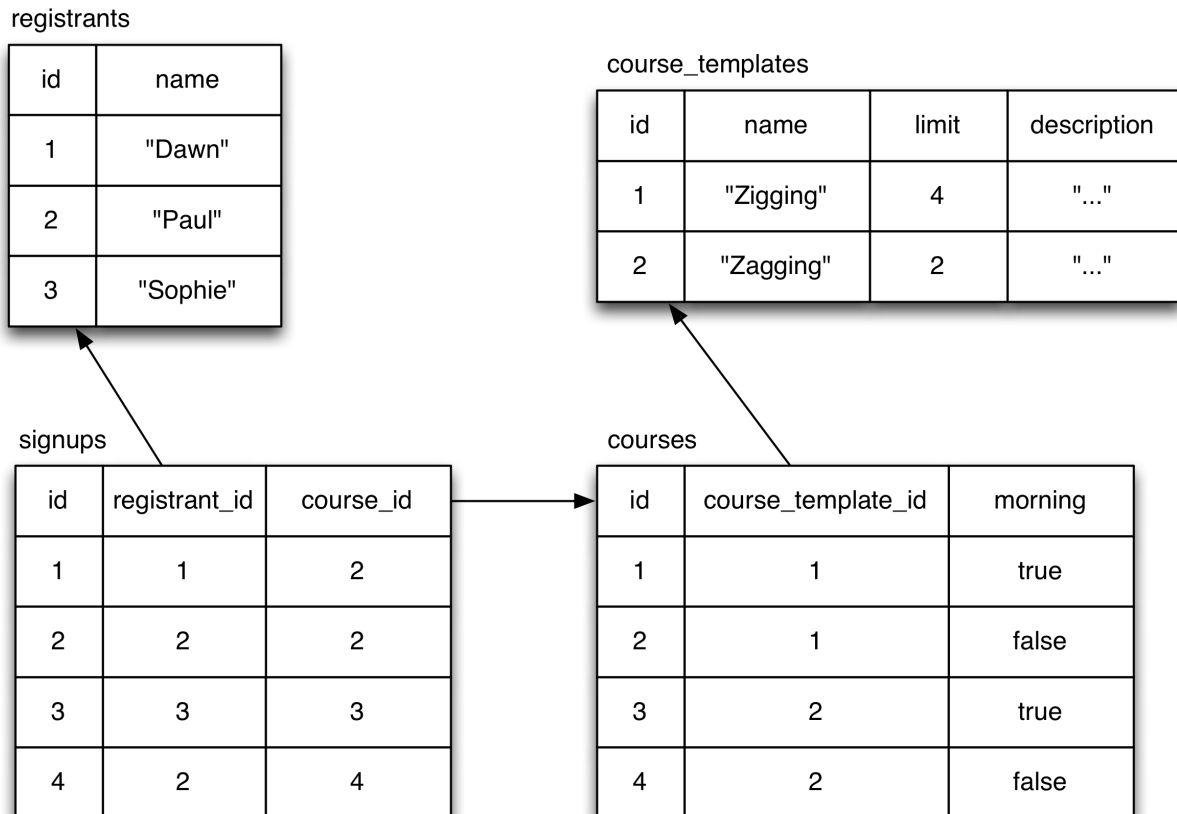
Here's a sequence of maps:

```

1 ({:course-name "Zigging", :morning? true, :limit 4, :registered 0},
2  {:course-name "Zigging", :morning? false, :limit 4, :registered 2},
3  {:course-name "Zagging", :morning? true, :limit 2, :registered 1},
4  {:course-name "Zagging", :morning? false, :limit 2, :registered 1})

```

They're structurally the same, just rotated 90 degrees. That means that the approach of the previous section is wasteful. We start with a database like this:



Our ORM will want to instantiate nine objects of four classes, slurping up a lot more information than is needed, in more queries than are required.

Most likely, most of the time, the waste is totally insignificant. It's not the performance bottleneck in the system, you've already got the object-to-relational mappings defined (or will need them for other reasons), and your team knows your mapping framework a lot better than they do SQL.

But if performance is an issue, or if you're having to write obscure or contorted code to get the ORM to behave as you wish, you might consider

giving in and using SQL.

Here's a single query for our problem:

```
1 SELECT
2   course_templates.name AS "course-name",
3   courses.morning AS "morning?",
4   course_templates."limit",
5   registration_counts.count AS registered
6 FROM courses
7 INNER JOIN
8   (SELECT
9     courses.id AS course_id,
10    count(signups.id)
11   FROM courses
12   LEFT JOIN signups ON (signups.course_id = courses.id)
13   GROUP BY courses.id
14  ) AS registration_counts ON (registration_counts.course_id = courses.id)
15 INNER JOIN
16   course_templates ON (course_templates.id = courses.course_template_id)
```

Is it pretty? No. Was it as easy for me to write as the previous section's restructuring code? No way. But it wasn't all *that* hard.

Depending on what SQL-generating libraries your language offers, it can perhaps be made more pretty. Here's an example using Ruby's [sequel](#) library, which provides a more functional style for composing queries:

```
1 registration_counts = DB[:courses].
2   left_join(:signups, :signups__course_id => :courses__id).
3   group(:courses__id).
4   select(:courses__id.as(:course_id)).
5   select_more{count(:signups__id).as(:registered)}
6
7 courses = DB[:courses].
8   join(registration_counts,
9     :course_id => :courses__id).
10  join(:course_templates,
11    :course_templates__id => :courses__course_template_id).
12  select(:course_templates__name.as("course-name"),
13    :courses__morning.as("morning?"),
14    :limit,
15    :registered)
16
17 pp courses.all
```

That's perhaps only slightly prettier than the raw SQL, but it produces a courses vector that would be trivial to convert to Clojure:

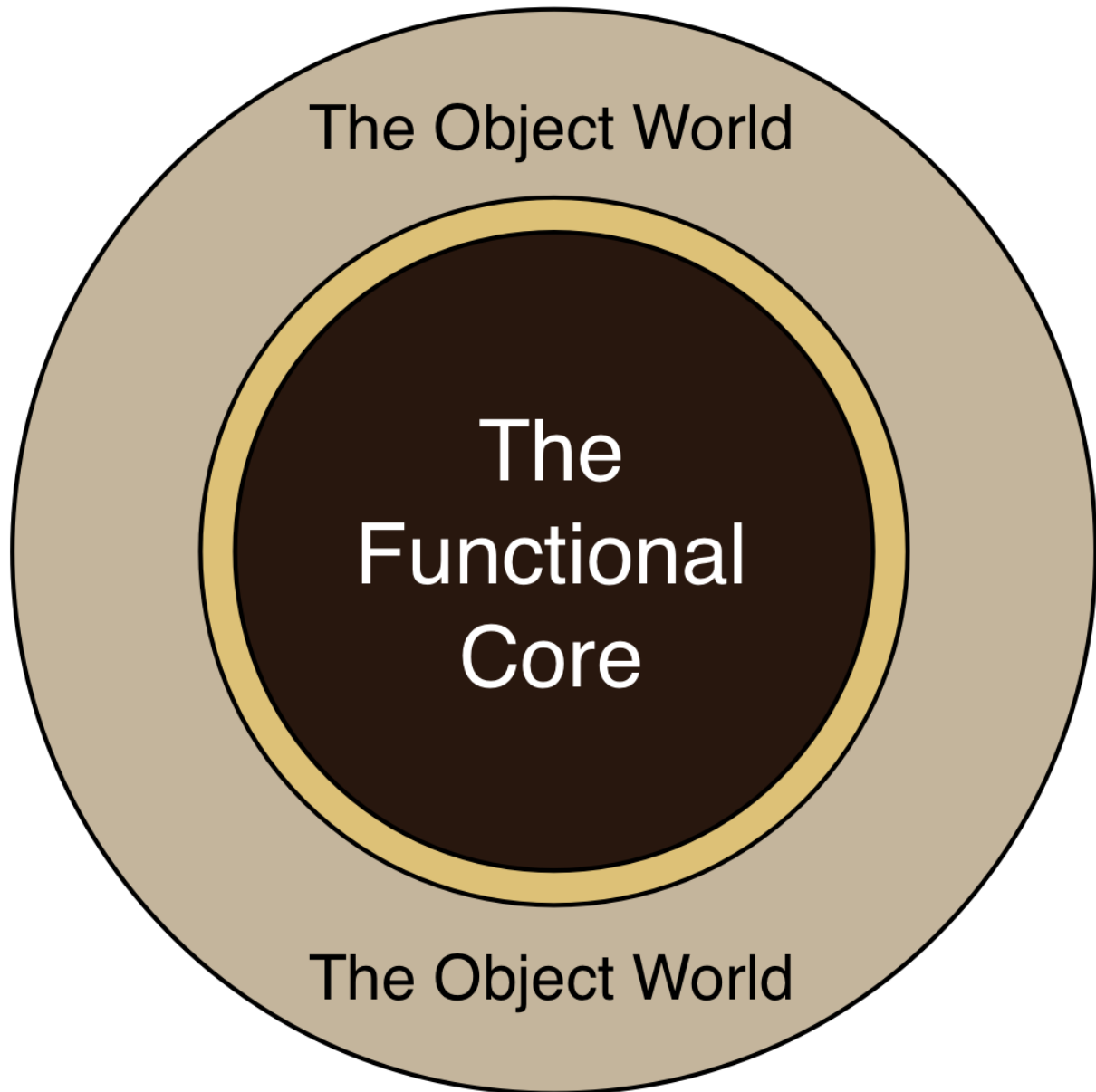
```
1 [{:registered=>0, :limit=>4, :course-name=>"Zigging", :morning?=>true},
2  {:registered=>2, :limit=>4, :course-name=>"Zigging", :morning?=>false},
3  {:registered=>1, :limit=>2, :course-name=>"Zagging", :morning?=>true},
4  {:registered=>1, :limit=>2, :course-name=>"Zagging", :morning?=>false}]
```

8.3 The big picture

In this chapter, I've started talking about larger-scale architectural style. That's even *more* foolish than talking about functional style in general. As functional programming encounters new groups of programmers and problems, it's being redefined². That's even more true for architecture-scale functional style.

That's not to deny that large systems have been written entirely in functional languages. But large scale systems were written in object-oriented languages, too, and it wasn't until the publication of books like Martin Fowler's *Patterns of Enterprise Application Architecture* and Eric Evans's *Domain-Driven Design* that I think you could say "There is a thing that we're justified in saying is an object-oriented architectural style." We're not at that point for the functional style yet, so far as I can tell.

Nevertheless, I think the approach of embedding functional nuggets within an object-oriented app is reasonable. A related strategy is to wrap object-orientation around a cohesive functional core:



To generalize that picture a bit, think of the outer core as being in charge of mutable state (whether it be handled with objects or not), and the inner core as being functional and immutable.

Since I've mentioned Fowler's *Patterns of Enterprise Application Architecture*, it's worth noting that what I'm calling "dataflow style" is akin to what he calls "Transaction Scripts". In his book, Fowler points out that the transaction script approach often doesn't scale (in terms of program size) as well as an approach that emphasizes objects. It remains to be seen how much of that problem is inherent and how much was a consequence of

weak languages and everyone's lack of experience (at that time) with the ideas and techniques of refactoring coherent designs out of working code. If not much of the problem is inherent, I look forward to a *Functional Patterns for Enterprise Application Architecture*.

1. Copyright Amie Sue, Nouveauraw.com. Used with permission.↵
2. As a parallel example, consider Smalltalk. After it moved from the research labs into the enterprise, a stylistic emphasis on deep class hierarchies seemed to have shifted to preferring shallow ones.↵

9. Functions That Make Functions

One way to eliminate duplication in functional programs is to not write functions yourself, but rather write functions that create parameterized functions. This chapter is about that.

9.1 Closing over values

You know `inc` as a function that adds 1 to its argument. But what if you needed to increment by different values: sometimes by 5, sometimes by 3, sometimes by 8? That'd be easy enough to code up:

```
1 user=> (def inc5 (fn [x] (+ 5 x)))
2 #'user/inc5
3 user=> (def inc3 (fn [x] (+ 3 x)))
4 #'user/inc3
5 user=> (def inc8 (fn [x] (+ 8 x)))
6 #'user/inc8
7 user=> (inc3 3)
8 6
```

But that seems awfully repetitive: wasteful typing, prone to error. It would be better if there were a function that could make incrementer functions. Like this:

```
1 user=> (def inc5 (make-incrementer 5))
2 user=> (def inc3 (make-incrementer 3))
3 user=> (def inc8 (make-incrementer 8))
4 user=> (inc8 0)
5 8
```

That turns out to be remarkably easy to do. Here's the definition of `make-incrementer`:

```
1 (def make-incrementer
2   (fn [increment]
3     (fn [x] (+ increment x))))
```

Consider a call like `(make-incrementer 3)`. By the substitution rule, that call expands by substituting the actual argument 3 for the parameter `increment`:

```
1 (fn [x] (+ 3 x))
```

... and that form, once evaluated, is a function that adds 3 to its argument.

This effect—making values hang around after the exit of the function to which they’re passed—is called *closing over a value*, or, in languages that allow variables to change value, “closing over variables”. (No doubt the term “closing” seemed perfectly apt to someone somewhere.) A function that can close over external values is called a *closure*. I mention that because now you know why Clojure is called “Clojure”. It’s a portmanteau word combining “closure” and “Java”.

`make-incrementer` follows a common pattern. The innermost calculation is a function, `+`, that’s given two arguments. By nesting functions, we’ve “constantized” the first argument as the constant 3.

That behavior—converting a function that takes n arguments to one that takes $n-m$ arguments, where the first m arguments become fixed as constants, is called *partial application*. You’ll also hear it called “currying”, though that is technically incorrect.

Partial application is a common enough operation that it has shorthand:

```
1 user=> (def add3 (partial + 3))
2 #'user/add3
3 user=> (add3 8)
4 11
```

Here’s a slightly more complicated example, in which we produce a function that increments each element of a sequence, returning the incremented sequence:

```
1 user=> (def increment-all (partial map inc))
2 #'user/increment-all
3 user=> (increment-all [1 2 3])
4 (2 3 4)
```

`partial` can take more than two arguments. What follows is the definition of a function that takes a sequence and adds 100 to the first element, 200 to the second element, and 300 to the last. It takes advantage of the fact that `map` can take more than one sequence argument:

```
1 user=> (def incish (partial map + [100 200 300]))
2 user=> (incish [1 2 3])
3 (101 202 303)
```

9.2 Lifting functions

Consider the humble function `not`:

```
1 user=> (even? 2)
2 true
3 user=> (not (even? 2))
4 false
```

Suppose that Rich Hickey (the author of Clojure) gave us only an `even?` predicate. We could write an `odd?` easily enough:

```
1 user=> (def odd?
2         (fn [x]
3           (not (even? x))))
4 user=> (odd? 2)
5 false
```

But there are an unending number of boolean-valued functions we might want to invert, and it would be a shame to have to write all that text each time. It would be better to have a function that takes another function and gives us the inverted version of that original:

```
1 user=> (def odd? (complement even?))
2 user=> (odd? 2)
3 false
```

`complement` is, indeed, predefined in Clojure.

It's interesting to think about the connection between `not` and `complement`. `not` operates on individual boolean values; `complement` works on whole functions that produce individual values. The latter seems more exalted: instead of dealing with merely a huge number of values, it's dealing with a

huge number of functions, *each of which* deals with a huge number of values.

So it seems justified to say that functions like `complement` are of a “higher order” than functions like `not`. Functional programming jargon doesn’t quite match this intuition. It defines *higher-order functions* as any functions that either produce functions as results *or* consume functions as arguments. Because I try to think of functions as mostly being ordinary values, there seems nothing “higher” about using them as arguments. But the jargon is what it is¹.

`complement` would be easy enough for us to define:

```
1 user=> (def my-complement
2         (fn [function]
3           (fn [x] (not (function x)))))
4 user=> ( (my-complement even?) 2)
5 false
```

But that raises a question: what’s special about `my-complement`? It’s applies one particular modifier function (`not`) to other functions. There are probably a zillion functions that need modification, and a zillion modifiers to modify them with. For example, what if you had a consistent need to negate the results of arithmetic computations? You could write this `complement`-like function:

```
1 user=> (def negate
2         (fn [function]
3           (fn [& args] (- (apply function args)))))
4 user=> ( (negate +) 1 2 3)
5 -6
```

And if you often needed to add a modest 5% to the results of financial functions, like this one:

```
1 user=> (honest-return {:account 34343})
2 3.0
```

... you could write another `complement`-like function to use in such situations:

```

1 user=> (def madoffize
2         (fn [function]
3           (fn [& args] (* 1.05 (apply function args)))))
4 user=> ( (madoffize honest-return) {:account 34343})
5 3.1500000000000004

```

Or would you rather write the following?

```

1 user=> (def negate (lift -))
2 user=> (def madoffize (lift (partial * 1.05)))

```

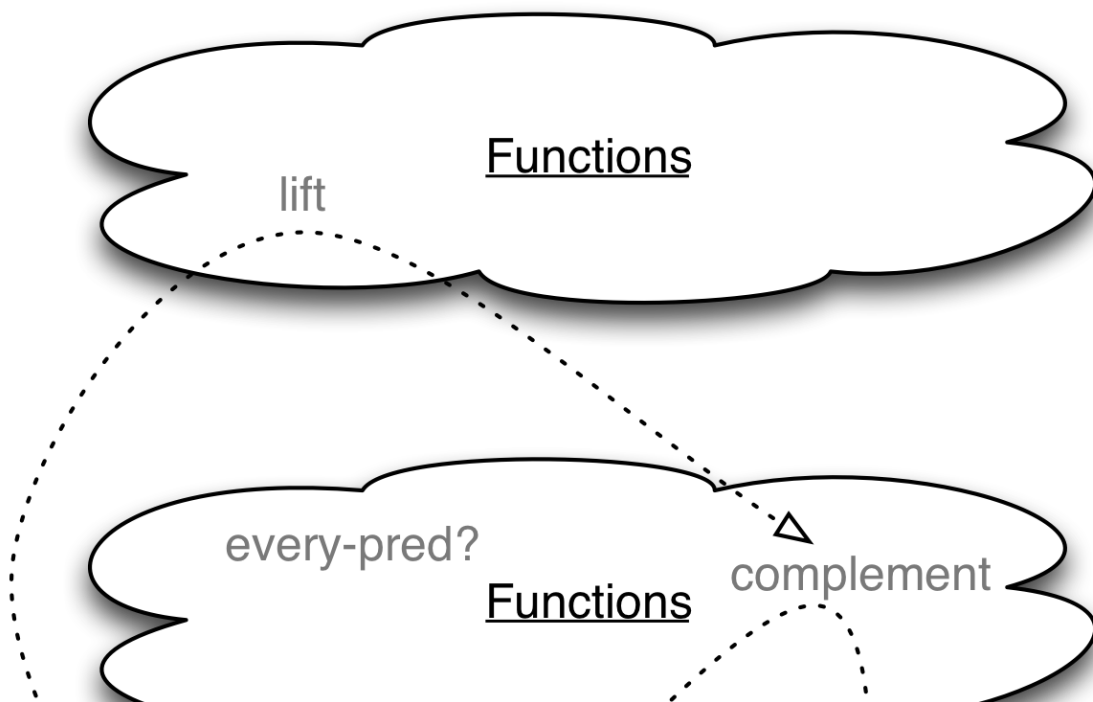
The definition of `lift` can be easily derived from `my-complement`: add another level of function definition that allows you to replace `not` with an arbitrary modifier:

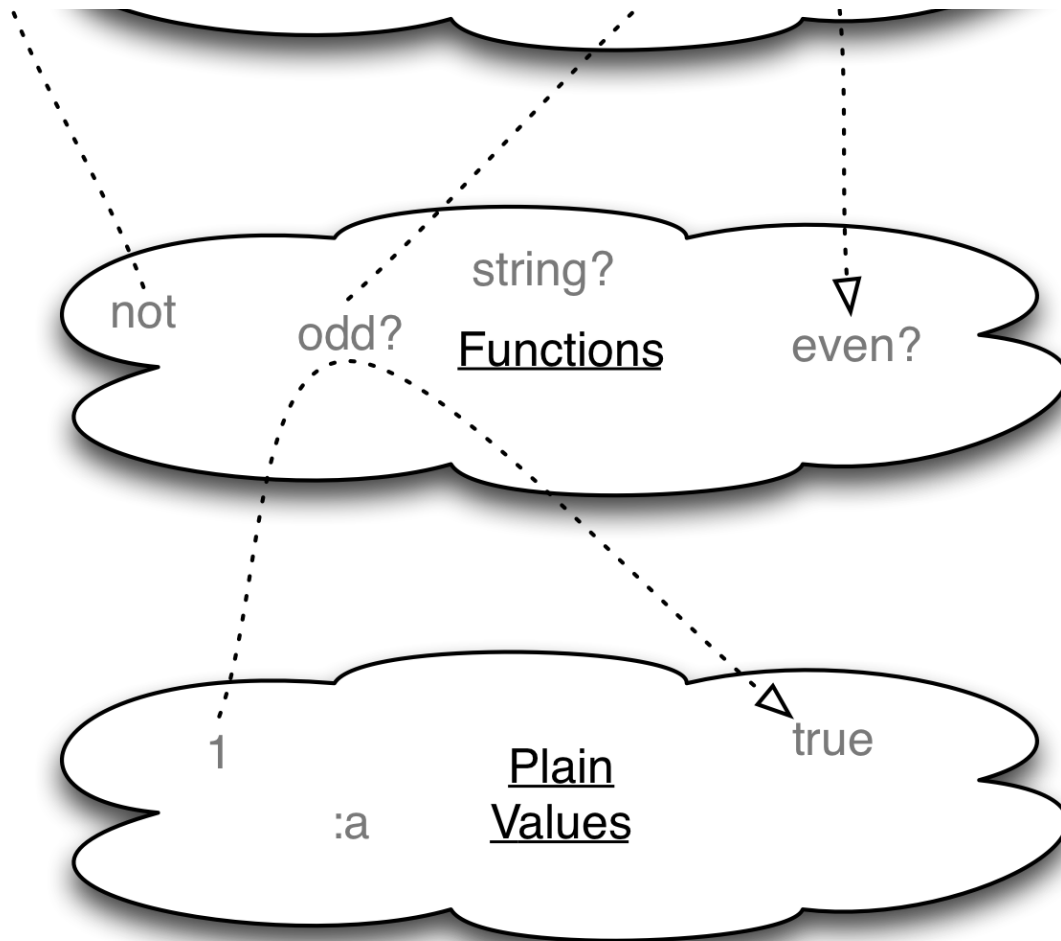
```

1 user=> (def lift
2         (fn [modifier]
3           (fn [base-function]
4             (fn [& args] (modifier (apply base-function args))))))

```

We can think of characterizing the values in a program into distinct strata: plain values, functions that work on plain values, higher-order functions that work on functions, higher-higher-order functions that work on higher-order functions, and so on:





That's a false picture in that there are no such clouds at runtime. There's no program you can write to separate out the higher-higher-order functions from all the rest of the functions. But it's a useful way to think of problem solving: the higher you go in the hierarchy, the more broadly applicable your tools. If your goal is to write functions that operate on plain values, you may end up with a flood of functions whose details override their commonality. If you abstract the commonality out into higher-order functions, you are more likely to see new areas where that commonality can be applied. In a sense, each step up the hierarchy squeezes out a different kind of inessential detail.

9.3 Point-free definitions

At this point in the book, you've seen two different ways to define functions. The first, `fn`, names its parameters:

```

1 (def increment-by-5
2   (fn [n] (+ 5 n)))
3
4 (def string-addition
5   (fn [& numbers]
6     (str (apply + numbers))))

```

The second doesn't mention parameters:

```

1 (def increment-by-5 (partial + 5))

```

The latter style could easily be called “parameter-free”, and so of course it's called something else: *point-free*. (No doubt the analogy of formal parameters to geometrical points seemed perfectly apt to someone somewhere.)

In addition to `partial`, the `comp` function is often used for point-free function definitions. `comp` composes several functions. The functions are applied in right to left order. The rightmost function takes any number of arguments and returns a value. The next one over is applied to that value, and so on. Here's an example I've lifted from clojuredocs.org:

```

1 user=> ( (comp str +) 8 8 8)
2 "24"

```

A mild rant on style

It's common in functional programming for you to “cons up” (jargon for “create”) temporary functions to use within the body of a larger function. You'll often have four stylistic choices.

1. **Naming neither the function nor the parameters.** For example, in some of my code I have a function that is given a sequence that might look like this:

```

1 [[:times 5] [:line-number 3] [other stuff]]

```

It needs to process all pairs until it finds one that doesn't begin with a keyword. It selects the pairs to process like this:

```

1 (take-while (comp keyword? first) pairs)

```

I don't think that the generated function needs documentation in either the form of a name saying what it's for, or a parameter saying what it works with.

2. **Naming only the function.** I have a different use of `comp` that looks like this:

```
1 (comp not not predicate)
```

I use that because I need to make sure the predicate really returns either `true` or `false` (and not, say, `nil` as a “falsey” value). Because I thought that purpose might not be clear, I gave the function a name:

```
1 (let [strict-predicate (comp not not predicate)] ...)
```

3. **Naming only the parameters.** If I look at a point-free definition and have to puzzle out what kind of argument (or, worse, arguments) it would have to take to make sense, I replace it with an explicit `fn`.

4. **Naming both.**

I mention these choices because many programming subcultures are afflicted by a cult of cleverness wherein cult members show off by writing the tersest possible code—and much of functional programming is one of those subcultures. I encourage you to resist cult members' siren songs of reputational glory and reproductive success, because I may someday encounter your code. I want to read it, not decipher it.

9.4 Exercises

My solutions are in [solutions/higher-order-functions.clj](https://github.com/haskell/haskell-exercises/blob/master/solutions/higher-order-functions.clj).

Exercise 1

In a free country, you could add 2 to each element of a sequence like this:

```
1 (map (fn [n] (+ 2 n)) [1 2 3])
```

However, ever since the Point-Free Programmer's Brigade seized control of the government, `fn` has been banned. How else could you accomplish your

goal? Can you think of more than one way?

Exercise 2

juxt turns n functions into a single function that returns a vector whose first element comes from the first function, the second from the second function, and so on.

```
1 user=> ( (juxt empty? reverse count) [:a :b :c])
2 [false (:c :b :a) 3]
```

In an earlier chapter, we defined `separate` like this:

```
1 (def separate
2   (fn [pred sequence]
3     [(filter pred sequence) (remove pred sequence)]))
```

Define it using `juxt`.

Exercise 3

Consider this code

```
1 (def myfun
2   (let [x 3]
3     (fn [] x)))
```

Predict the result of typing these two lines:

```
1 user> x
2 user> (myfun)
```

Can you explain, using the substitution rule, why you get those results?

Exercise 4

If `let` didn't exist, could you use functions to achieve the same effect as in the previous exercise? That is, what code could you wrap around `(fn [] x)` to produce an `x` that was bound to no value outside the function and bound to 3 inside it?

Hint: `let` is a form that binds names to values and then executes a body. A function is a form that binds parameter names to values and then executes a body.

Exercise 5

Clojure has datatypes that support mutability in the presence of concurrency. For example, an *atom* is an object protected from overlapping updates by multiple threads. You update an atom by applying a function to its current value:

```
1 user=> (def my-atom (atom 0))
2 user=> (swap! my-atom inc)
3 user=> (deref my-atom)
4 1
```

Suppose you wanted to set an atom's value to 33, regardless of its current value. How would you do that? Keep in mind that `swap!` demands a function, not (say) an integer.

Exercise 6

In the previous exercise, you probably hand-crafted (with `fn`) a function that returned a constant value. That meant you wrote a function with a parameter that was ignored in favor of the constant. If anything calls for point-free style, this is it.

Write a function `always` that takes a value and produces a function that returns that value no matter what its arguments are. That is:

```
1 user=> ( (always 8) 1 'a :foo)
2 8
```

Compare your solution to Clojure's `constantly`.

Exercise 7 [2](#)

To practice for this book, I wrote three earlier ones. Here are their ISBNs: 0131774115, 0977716614, and 1934356190—except that one of them contains a typo. In the next two exercises, you're to find which one.

First, use map to write a function check-sum that performs the following calculation on the sequence [4 8 9 3 2]:

```
1 (+ (* 1 4)
2    (* 2 8)
3    (* 3 9)
4    (* 4 3)
5    (* 5 2))
```

check-sum should take any number of digits.

Exercise 8

A valid ISBN has a check sum that can be divided by 11 to leave a remainder of 0. Write a function isbn? that checks if a string is a valid ISBN. You can use (rem N 11) to find the remainder. In the source, [sources/higher-order-functions.clj](#), you'll find the three strings, as well as a function, reversed-digits, that converts them into sequences appropriate for check-sum.

(Note: if your isbn? claims two of the numbers aren't ISBNs, you probably have an off-by-one error.)

Exercise 9

Universal Product Codes (UPCs) need a slightly more complicated check-sum. Here's an example of the calculation for [4 8 9 3 2]:

```
1 (+ (* 1 4)
2    (* 3 8)
3    (* 1 9)
4    (* 3 4)
5    (* 1 2))
```

If the position is odd, the number is multiplied by 1, otherwise 3. The divisor is 10, not 11.

Implement check-sum and upc? and check it against these numbers (also in the sources):

Exercise 10

The same general “shape” of function will work for checking credit cards, money orders, and so on. Extract the commonality of isbn? and upc? (and their respective check-sums) into a function number-checker that can be used to create either of them. Like this:

```
1 (def isbn? (number-checker ...))
2 (def upc? (number-checker ...))
```

9.5 Higher-order functions from the object-oriented perspective

Higher-order functions can be looked at as on-the-fly classes with a single public method. Consider this function:

```
1 self=> (def make-incrementer
2           (fn [increment]
3             (fn [value] (+ increment value))))
4 self=> (def fiver (make-incrementer 5))
5 self=> (fiver 3)
6 8
```

I claim that is no different than this Ruby code:

```
1 irb(main):001:0> class Incrementer
2 irb(main):002:1>   def initialize(increment)
3 irb(main):003:2>     @increment = increment
4 irb(main):004:2>   end
5 irb(main):005:1>
6 irb(main):006:1*   def increment(value)
7 irb(main):007:2>     @increment + value
8 irb(main):008:2>   end
9 irb(main):009:1> end
10 irb(main):010:0> fiver = Incrementer.new(5)
11 irb(main):011:0> fiver.increment(3)
12 => 8
```

In object-oriented programs, It’s not horribly unusual to see classes with a single public method. In such cases, consider replacing them with plain functions. In the Ruby example, that might look like this:

```
1 1.9.3p194 :002?> module FunctionMakers
2 1.9.3p194 :002?>   def make_incrementer(increment)
3 1.9.3p194 :003?>     ->(value) { increment + value }
4 1.9.3p194 :004?>   end
```

```

5 1.9.3p194 :005?> end
6 1.9.3p194 :006 >
7 1.9.3p194 :007 > # ...
8 1.9.3p194 :008 >
9 1.9.3p194 :009 > include FunctionMakers
10 1.9.3p194 :010 >
11 1.9.3p194 :011 > fiver = make_incrementer(5)
12 1.9.3p194 :012 > fiver.(3)
13 => 8

```

What's the advantage of that, besides a little bit less typing? Well, I agree with the authors of [Growing Object-Oriented Software](#) that the point of object oriented design is getting a good mental picture of the relationships between objects. Having lots of little utility classes obscures what's important behind what's merely necessary.

I might even go a bit further. Sometimes an object is passed to another object, and the receiver only uses one of the first object's methods. How about hiding the object in a function?

Here's an example. Suppose I have a Booking class and a Reservation class. A Booking object can ask a Reservation to add people. The Reservation might accept none, some, or all of them. The original Booking code might look like this:

```

1 class Booking
2   def try_booking(people, reservation)
3     ...
4     booked, rejected = reservation.accept(people)
5     booked.each do ... end
6     rejected.each do ... end
7     ...
8   end
9   ...

```

Instead, let's have try_booking not take a Reservation. Instead, it will take a function that we'll call accept_some_people. Instead of asking the Reservation to accept the people, try_booking will apply the accept_some_people function:

```

1 class Booking
2   def try_booking(people, accept_some_people)
3     ...

```

```

4      booked, rejected = accept_some_people.(people)  # <== New
5      booked.each do ... end
6      rejected.each do ... end
7      ...
8  end
9  ...

```

One of Booking's clients would invoke `try_booking` like this:

```

1  ...
2  result = try_booking(people, ->(people) { reservation.accept(people) })
3  ...

```

That's is somewhat ugly, but the Booking class is now less coupled to the Reservation class. It can work with any function that accepts people. Even in loosey-goosey Ruby, the original Booking class could only work with classes that have a specific method (`accept`). In statically-typed languages like Java, you'd be further restricted to collaborators that match a certain interface or class.

As a practical consequence of reduced coupling, testing becomes easier. If you want to test the behavior of `try_booking` when all the people are rejected, your test looks like this:

```

1  ...
2  actual = booking.try_booking(people,
3                                ->(people) { [ [], people] })
4  ...

```

There's no need to create a real Reservation object and arrange for it to reject everyone or, alternately, to program a mock object to reject everything.

If you're a Rails programmer interested in this approach, buy Avdi Grimm's [Objects on Rails](#).

With loosely-coupled objects that communicate only by passing behaviors (functions) around (rather than entire bundles of state and behavior in the form of objects), the early part of design may become easier. That's the part where you're not sure what the right classes are, or which behaviors belong to which classes. The less coupling you have at first, the less quickly your

object design will ossify, and the less discipline you'll need to keep it flexible.

I emphasize early design because at some point, you'll likely say, "The heck with it: Booking just *does* depend on Reservation". Then you'll add a Reservation parameter to the constructor and let Booking become coupled to a class rather than to a set of behaviors.

1. Note: if you read other sources on functional programming, you'll sometimes also see higher-order functions called "functionals" or "functors". "Functor", especially, sounds much cooler. ↩
2. The next four exercises were inspired by *Concrete Abstractions*, by Max Hailperin, Barbara Kaiser, and Karl Knight. ↩

10. Branching and Looping in Dataflow Style

We have two notations for dataflow style: the arrow and the `let`. An arrow flow looks like this:

```
1 (-> (+ 1 2) (* 3) (+ 4))
```

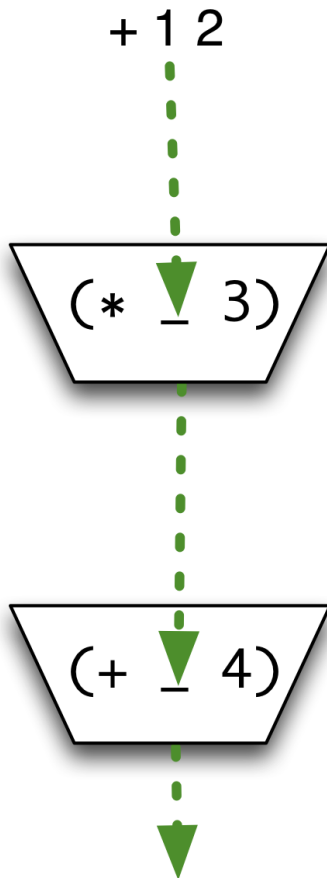
Each value *must* be used by the next step, and no step can use the value of any step other than its immediate predecessor.

`let` relaxes those constraints. Since the value of a step is bound to a symbol, it can be accessed (or not) by any later step:

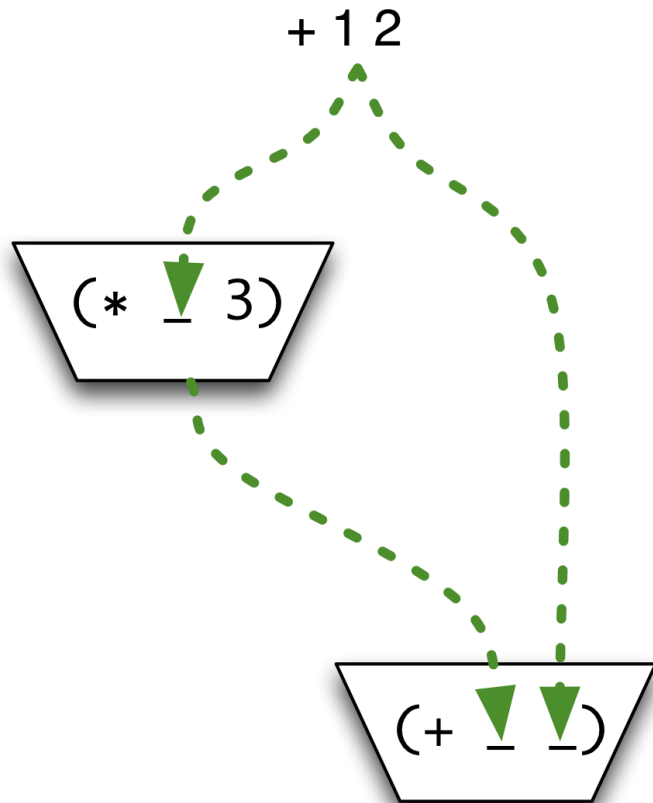
```
1 user=> (let [step1-value (+ 1 2)
2           step2-value (* step1-value 3)
3           step3-value (+ step2-value 4)]
4       step3-value)
5 13
```

I visualize the difference between the arrow and `let` like this:

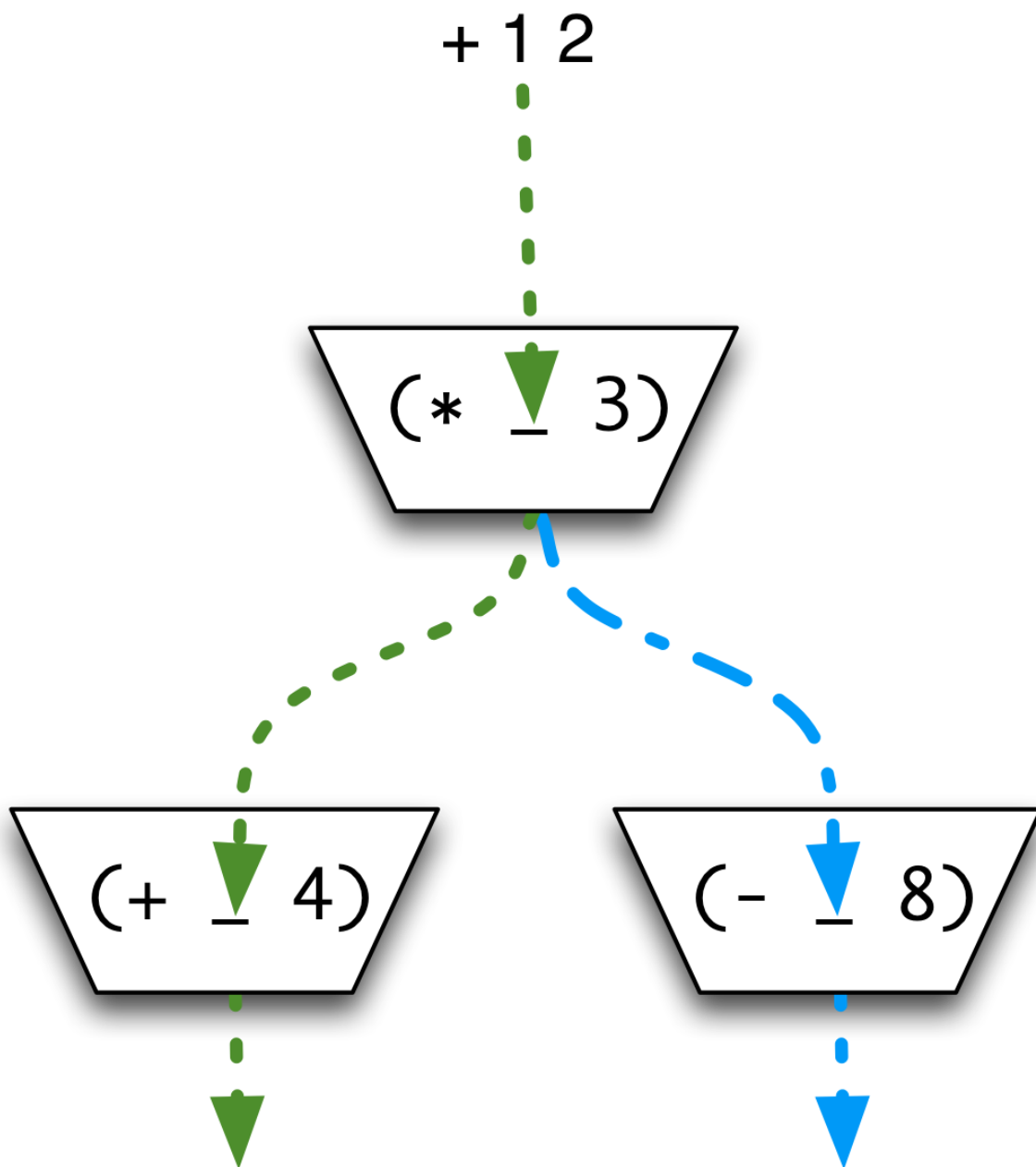
->



LET



In both cases, though, there's no decision points in the flow. Arrows go where they go, and that's all there is to it. There's no way to change the direction of the flow depending on the value, as symbolized by this:



That limitation can be overcome by imagination and the clever use of functions, though, as the rest of this chapter will show.

I should note that the desire to rid ourselves of explicit branching is not unique to functional programming. In object-oriented programming, polymorphic dispatch selects one of N methods according to both the

message name and receiver type. That lets the programmer convert the *idea* behind code like this:

```
1 if (some_property_of(x) {
2   perform_some_action();
3 } else if (some_other_property_of(x)) {
4   perform_some_other_action();
5 } else if (yet_a_third_property_of(x)) {
6   perform_yet_another_action();
7 }
```

... into a more rigid (and, so, very often more comprehensible) inheritance structure, where the multi-way `if` is hidden inside the normally-invisible dispatch function.

10.1 That pesky `nil` again

Like many languages, Clojure has functions that can return either a useful value or `nil` (meaning that no useful value can be calculated). Putting one of those functions in a flow introduces the chance that a `nil` might enter a function that doesn't expect it. Like this:

```
1 user=> (-> 1 function-that-might-produce-nil inc (* 3) dec)
2 java.lang.NullPointerException (NO_SOURCE_FILE:0)
```

In many cases, a reasonable response to a `nil` is to use some default value instead. (This is similar to what, in object-oriented programming is called the [null object pattern](#).) So let's "lift" `inc` to a version that treats nulls as 0. (There are probably vanishingly few cases where that's a good idea, but this is just an example.) Here's the code:

```
1 (def tolerant-inc
2   (fn [n]
3     (if (nil? n)
4       (inc 0)
5       (inc n))))
```

... and here's a version without the duplication of `inc`:

```
1 (def tolerant-inc
2   (fn [n]
3     (inc (if (nil? n) 0 n)))))
```

That's an annoyingly limited solution, though. We're smearing together two ideas: the idea of incrementing a number and the idea of substituting a value for `nil`. Those ideas should be separated into two separate functions.

We already have a function that increments: `inc`. So let's write a function, `nil-patch`, that takes a function to produce a function just like it, except for its handling of `nil`. Then we can write our original flow like this:

```
1 user=> (-> 1
2         function-that-might-produce-nil
3         ((nil-patch inc 0))           ; <== new
4         (* 3)
5         dec)
6 2
```

(Note the double parentheses around the use of `nil-patch` to force `->` to put the value in the right place, as the single argument to a generated function.)

`nil-patch` can be a higher-order function modeled after `tolerant-inc`:

```
1 (def nil-patch
2   (fn [function replacement]
3     (fn [original]
4       (function (if (nil? original) replacement original)))))
```

Note: `nil-patch` is already built into Clojure under the name `fnil`. `fnil` can substitute values for functions that take more than one argument.

`fnil` is a cute solution, but `nil-patching` probably isn't usually what you want to do with stray `nils`. More often, you'll want to simply skip the entire rest of the computation and return `nil`. You can do that with an optional Clojure library called `clojure.algo.monads`. It lets you write this `let`-like expression:

```
1 (with-monad maybe-m
2   (domonad [step1-value (function-that-might-produce-nil 1)
3                   step2-value (* (inc step1-value) 3)]
4     (dec step2-value)))
```

The `domonad` follows the same rules as `let`: the steps are evaluated in sequence, ending in the evaluation of the body (the `dec` expression). However, the `(with-monad maybe-m...)` wrapped around the `domonad`

effectively inserts `nil`-checking code after each step. If any step produces `nil`, the whole `domonad` immediately returns `nil`.

This is useful in its own right, but is more interesting because of the possibilities it opens up: if you can insert `nil`-checking code after steps, what *else* could you usefully insert? Quite a lot, it turns out.

A *monad* is a collections of functions that can be used to modify stepwise calculations. Above, I showed you a use of the *Maybe monad* (named, in Clojure, `maybe-m`). A related monad, the *Error monad*, also stops the flow if something bad happens, but in a more useful way. In this chapter, you'll learn enough to be able to write the Error monad for yourself. I'll also show you how to use (but not implement) the *Sequence monad*, which lets you implement multiply-nested loops with a single series of steps.

While monads are a good example of the power of functions, they need to be explained carefully if you're to understand them. We'll start by looking at yet another variant of flow style.

10.2 Continuation-passing style

Let's start with a simple expression:

```
1 (+ (* (+ 1 2) 3) 4)
```

Here's a way to describe what's happening: "We have a multi-step computation. The first step is to calculate `(+ 1 2)`. The result of that calculation is passed to the rest of a computation." By doing that, we've made explicit an idea that was implicit before: when we're in the middle of a computation, there's this thing we can call "the rest of the computation". The usual name for that is the *continuation*.

In object-oriented programming, you make ideas real by turning them into objects. In functional programming, you make ideas real by turning them into functions. So here's the continuation of `(+ 1 2)`:

```
1 (fn [step1-value]
2   (+ (* step1-value 3) 4))
```

And the entire calculation can be written in *continuation-passing style* by flowing the result of the first step into the continuation:

```
1 (-> (+ 1 2)
2     ((fn [step1-value]
3         (+ (* step1-value 3) 4))))
```

Now, the calculation inside the continuation is itself a multi-step one (first multiplication, then addition), so we can also rewrite it in continuation passing style:

```
1 (-> (+ 1 2)
2     ((fn [step1-value]
3         (-> (* step1-value 3)           ;; expanded to
4             ((fn [step2-value]         ;; continuation-passing
5                 (+ step2-value 4)))))) ;; style
```

Notice that the nesting of the functions lets any of them refer back to the results of earlier steps, since those results are bound to symbols in the nested functions' parameter lists. So, for example, the body of the bottom function could use both `step1-value` and `step2-value`:

```
1 (-> (+ 1 2)
2     ((fn [step1-value]
3         (-> (* step1-value 3)
4             ((fn [step2-value]
5                 (+ step2-value           ;; This function's parameter
6                     step1-value)))))) ;; An enclosing function's parameter
```

10.3 Exercises

My solutions are in [solutions/continuation-passing.clj](#).

Exercise 1

Convert this into continuation-passing style:

```
1 (let [a (concat '(a b c) '(d e f))
2       b (count a)]
3     (odd? b))
```

Hint: Don't forget the need for double parentheses when `fn` expressions are used within arrow expressions.

Exercise 2

This code computes the same value as the previous code. Rewrite it into continuation-passing style.

```
1 (odd? (count (concat '(a b c) '(d e f))))
```

Is the resulting code the same as your answer to the first exercise (except for trivialities like choice of parameter names)?

If so, is it *necessarily* the same, or could you make an argument that you could (or should) choose different continuations?

Hint: How might you write `(concat '(a b c) '(d e f))` in continuation-passing style?

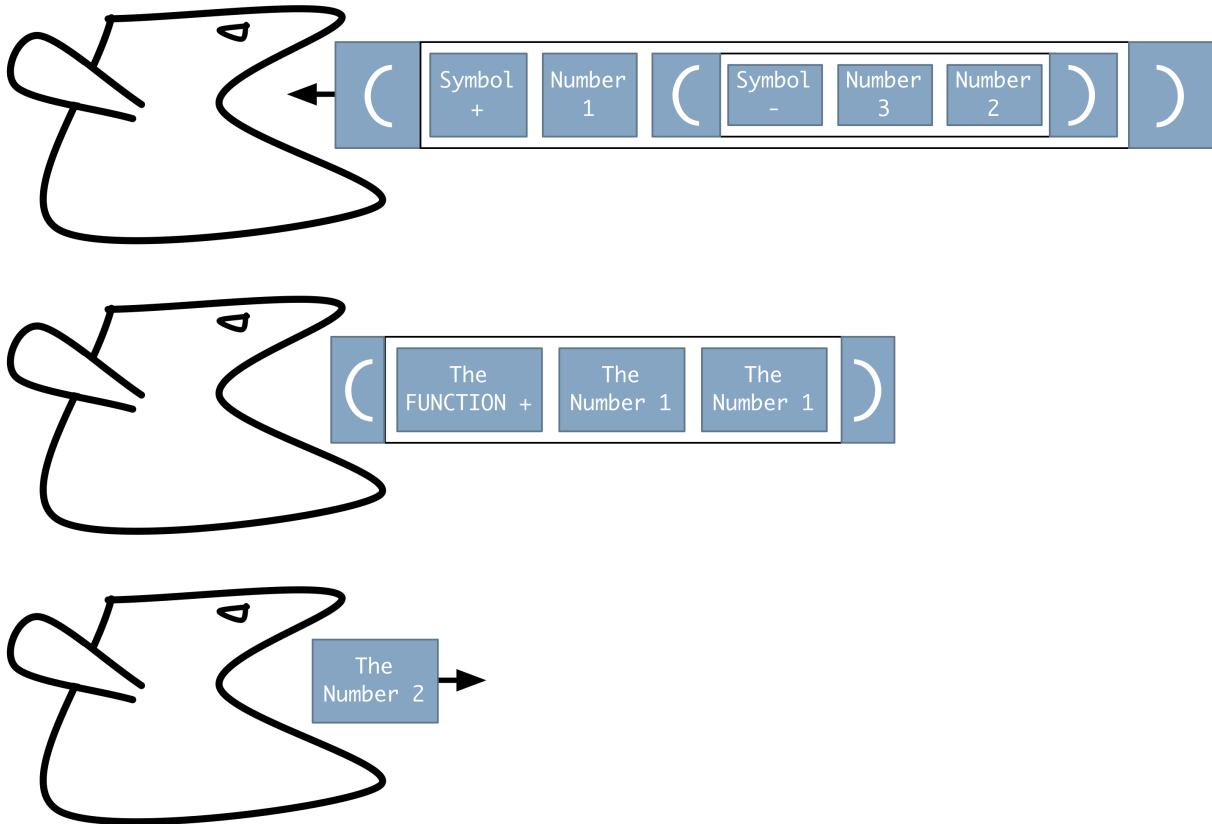
Exercise 3

Convert this into continuation-passing style:

```
1 (-> 3
2    (+ 2)
3    inc)
```

10.4 Expansions in evaluation

You should be used to the idea of evaluation as a process of producing progressively smaller lists until you obtain a single value:



It's also possible for the evaluation process to temporarily *expand* a form into a larger but equivalent form. It would be valid for the evaluator to transform this:

```
1 (* (+ 1 2) 3)
```

... into continuation-passing form:

```
1 (-> (+ 1 2)
2      ((fn [temp__1]
3         (* temp__1 3))))
```

... and then reduce it down to a single value in the normal way. ([Some compilers](#) make such expansions in order to simplify their job, but those details normally don't concern us.)

It would also be reasonable for the evaluator to transform this `let`:

```
1 (let [step1-value (function-that-would-never-produce-nil 1)
2       step2-value (* (inc step1-value) 3)]
3   (dec step2-value))
```


... into this continuation passing form:

```
1 (-> (function-that-would-never-produce-nil 1)
2      ((fn [step1-value]
3          (-> (* (inc step1-value) 3)
4              ((fn [step2-value]
5                  (dec step2-value)))))))
```

... and then reduce it to a single value in the normal way. Similarly, a domonad form like this:

```
1 (domonad [step1-value (function-that-might-produce-nil 1)
2             step2-value (* (inc step1-value) 3)]
3      (dec step2-value))
```

... is transformed into a slightly extended continuation-passing style. To peek ahead, it looks something like this:

```
1 (-> (function-that-might-produce-nil 1)
2      (mystery-function                               ; <== extension
3      (fn [step1-value]
4          (-> (* (inc step1-value) 3)
5              (mystery-function                       ; <== extension
6                  (fn [step2-value]
7                      (dec step2-value)))))))
```

(The mystery-function is explained in the next section.)

Understanding that domonad is an expansion into continuation-passing style will help you understand how monads work.

In Clojure and other Lisps, expansions can be defined by programmers. domonad, with-monad, and the monad form you'll see shortly were all defined by a programmer named Konrad Hinsen, the author of the `clojure.algo.monads` library.

In other languages, all expansions have to be predefined by the language implementers. A user's monad implementation can't hide the underlying continuations nearly as well.

Because user-defined expansions, what the Lisps call *macros*, aren't a common feature in functional languages, I won't explain them in this book. They're one of the most powerful features of Lisps, though, and make Lisp the best language for domain-specific languages (as long as you don't mind parentheses).

10.5 Extending continuation-passing style

Let's look at how we can make continuation-passing style more flexible. Consider this:

```
1 (-> (+ 1 2)
2     ((fn [step1-value]
3         (-> (* step1-value 3)
4             ((fn [step2-value]
5                 (+ step2-value 4)))))))
```

Excitingly nested though it is, it's dull in one important way: it always does the same thing with the value of a step. It passes it to the continuation. Let's introduce a function that is given both the value and the continuation and decides what to do with them. We'll call it the decider.

Let's start by making the decision what it already was: to pass the value to the continuation. Here's such a definition:

```
1 (def decider
2   (fn [step-value continuation]
3     (continuation step-value)))
```

Inserting decider into the previous expression looks like this:

```
1 (-> (+ 1 2)
2     (decider (fn [step1-value]
3                 (-> (* step1-value 3)
4                     (decider (fn [step2-value]
5                                 (+ step2-value step1-value)))))))
```

In the above, the arrow flows the result of (+ 1 2) into the first argument of decider, which has a second argument that's the continuation. Without the arrow, that would look like this:

```
1 (decider (+ 1 2)
2         (fn [step1-value]
3           ...))
```

Now we'll change decider to refuse to apply the continuation if the step value is nil:

```

1 (def decider
2   (fn [step-value continuation]
3     (if (nil? step-value)
4         nil
5         (continuation step-value))))

```

So at any step in the computation, decider can “short-circuit” it and just return `nil`. Let’s see an example.

Here’s a computation

```

1 (+ (function-that-returns-nil) 3)

```

An expansion into plain continuation-passing style would look like this:

```

1 (-> (function-that-returns-nil)
2     ((fn [step1-value]
3        (+ step1-value 3))))

```

Adding the decider looks like this:

```

1 (-> (function-that-returns-nil)
2     (decider (fn [step1-value]
3               (+ step1-value 3))))

```

If `nil` flows into decider as its first argument, the continuation is never called and `nil` is returned.

10.6 Monads as data-driven extended continuation-passing style

Now we have the background to dissect a use of a monad. Warning: I’m going to lie to you at first, then present the slightly more complicated reality.

Recall that the Maybe monad stops the computation if any step produces `nil`. That is, it’s a way of producing an expansion that looks like that which ended the last section. Here’s a sample use:

```

1 (with-monad maybe-m
2   (domonad [a (some-function)]

```

```

3           b 2]
4   (+ a b)))

```

It has three working parts: `with-monad`, `maybe-m`, and `domonad`. Why?

Because that makes behavior *easily substitutable*. The code above checks for `nil`. You'll soon write an Error monad that stops the computation if any step produces a particular datum that means "error". You could swap that Error monad in for the Maybe monad by changing a single word:

```

1 (with-monad error-monad           ; <== one word
2   (domonad [a (some-function)
3             b 2]
4   (+ a b)))

```

And you could swap out that monad for one, called the *Identity monad*, that does exactly what a `let` does:

```

1 (with-monad identity-m           ; <== one word
2   (domonad [a (some-function)
3             b 2]
4   (+ a b)))

```

That code would execute all the steps in order, no matter what.

Now that I've justified the three pieces, how do they work together?

The lie

The monad (`maybe-m`, `identity-m`) is just a name for the decider function. Like this:

```

1 (def maybe-m
2   (fn [step-value continuation]
3     (if (nil? step-value)
4         nil
5         (continuation step-value))))
6
7 (def identity-m
8   (fn [step-value continuation]
9     (continuation step-value)))

```

with-monad just turns into a let that binds its first argument to the name decider. Given this:

```
1 (with-monad maybe-m
2     ...)
```

... we get this expansion:

```
1 (let [decider (fn [step-value continuation]
2                 (if (nil? step-value)
3                     nil
4                     (continuation step-value)))]
5     ...)
```

domonad produces the previous section's extended continuation-passing style. Given this:

```
1 (domonad [a (some-function)
2           b 2]
3     (+ a b))
```

... we get this:

```
1 (-> (some-function)
2     (decider (fn [a]
3                 (-> 2
4                     (decider (fn [b]
5                                 (+ a b))))))))
```

Putting the three pieces together, given this:

```
1 (def maybe-m
2   (fn [step-value continuation]
3     (if (nil? step-value)
4         nil
5         (continuation step-value))))
6
7 (with-monad maybe-m
8   (domonad [a (some-function)
9             b 2]
10      (+ a b)))
```

... we get this:

```

1 (let [decider (fn [step-value continuation]
2               (if (nil? step-value)
3                   nil
4                   (continuation step-value)))]
5   (-> (some-function)
6       (decider (fn [a]
7                 (-> 2
8                     (decider (fn [b]
9                               (+ a b))))))))))

```

Make sure you understand why this whole expression returns `nil` if `some-function` returns `nil`. And make sure you understand why it returns `3` if `some-function` returns `1`.

Less of a lie

As it happens, some monads need more than just a decider. In the [optional chapter on the Sequence monad](#), I'll call that other function the “monadifier”.

To be more exact: all monads require both a monadifier and decider, but some can just use identity for their monadifier. (`identity` just returns its argument, unchanged).

Because monads have two functions, we need to store them in a map:

```

1 (def maybe-m
2   {:decider (fn [step-value continuation]
3               (if (nil? step-value)
4                   nil
5                   (continuation step-value)))
6   :monadifier identity})

```

`with-monad` must use `let` to bind both of them:

```

1 (let [decider (fn [step-value continuation]
2               (if (nil? step-value)
3                   nil
4                   (continuation step-value)))
5      monadifier identity]
6   ...)

```

And `domonad` must insert the monadifier where required:

```

1 (-> (some-function)
2     (decider (fn [a]
3               (-> 2
4                 (decider (fn [b]
5                           (monadifier ; <== here
6                               (+ a b)))))))

```

The truth

I may have mentioned that I sometimes think functional programming terminology is perhaps slightly less than optimal for novice understanding. That is the case with monad terminology.

What I’ve been calling “the decider” is conventionally called “bind”. You can sort of see where that name comes from: the decider calls a function (the continuation) with a named parameter, thus binding the value of a step to a symbol.

But that kind of binding is just the way all continuation-passing expansion works. What’s special about monads is what *else* the decider does. When I was learning monads, I kept getting confused until I hit upon the idea of thinking of the “bind” function as a decider.

What I call “the monadifier” is conventionally called “return”, “result”, or “unit”^{[1](#)}.

`clojure.algo.monads` follows the conventional terminology, so what `with-monad` produces is actually this:

```

1 (let [m-bind ...
2       m-result ...]
3     ...)

```

... and `domonad` uses those names in its expansion.

Monads can also contain two optional functions, used for things we won’t cover in this book. Those (or a flag that says they aren’t provided) have to be included in the map that defines a monad. Because of that, monads are usually defined with a helper function, `monad`:

```

1 (def maybe-m
2   (monad [m-bind (fn [step-value continuation]
3                     (if (nil? step-value)
4                         nil
5                         (continuation step-value)))
6     m-result identity]))

```

That produces a map like this:

```

1 {:m-plus :clojure.algo.monads/undefined,
2  :m-zero :clojure.algo.monads/undefined,
3  :m-bind #<user$fn__515$m_bind__516 user$fn__515$m_bind__516@66f4652>,
4  :m-result #<core$identity clojure.core$identity@17eda64e>}

```

Although the full truth isn't relevant to this chapter, and hinders, rather than helps, your understanding of monads (at least if you're anything like me), you need to know it. That's because you'll shortly define your own monad. First, though, a couple of Clojure facts will make that exercise more pleasant.

10.7 A peek at metadata

Remember how, in Part 1, we put metadata in object and class maps? Metadata was stored in the same map as instance values: the only way you knew it was metadata was by typographical convention (`#__own_symbol__`).

It turns out that Clojure has a real (not just conventional) notion of metadata, which is stored in a map attached to a value. That map can be retrieved with the `meta` function. Here's the metadata for a function:

```

1 user=> (pprint (meta +))
2 {:ns #<Namespace clojure.core>,
3  :name +,
4  :file "clojure/core.clj",
5  :line 809,
6  :arglists ([[] [x] [x y] [x y & more]]),
7  :added "1.0",
8  :inline-arities #{2},
9  :inline #<core$_PLUS__inliner clojure.core$_PLUS__inliner@ba336d5>,
10 :doc "Returns the sum of nums. (+) returns 0."}

```

As with most things in Clojure, metadata can't be changed. You can, however, make a new copy of an object with different metadata, using the

with-meta function. Like this:

```
1 user=> (def a {:map "me"}) ; no metadata
2 user=> (meta a)
3 nil
4 user=> (def b (with-meta a {:type :error}))
5 user=> (meta b)
6 {:type :error}
```

At this point, a and b are equal values with different metadata. That is, metadata is ignored in equality checks. Indeed, aside from a few functions like meta and with-meta, everything ignores metadata.

Not all values can have metadata. For example, a Clojure string is a Java `java.lang.String`, so there's no place to put metadata. Numbers also don't have metadata.

However, even values that can't have metadata can be asked for it:

```
1 user=> (meta "hi")
2 nil
```

That's handy because it means you don't have to check what kind of value you have before it's safe to ask for its metadata. Since keywords applied to `nil` return `nil`, it's also safe to ask for a boolean value in metadata:

```
1 user=> (:open? (meta a))
2 nil
3 user=> (if (:open? (meta a))
4         (do-something-explosive))
5 nil
```

One last bit of information: I gave a map the metadata `{:type :error}` for a reason. Clojure's type function uses the metadata. If a value has a `:type` key in its metadata, its value is type's result:

```
1 user=> (type (with-meta {} {:type :error}))
2 :error
```

In a [later chapter](#), we'll see how metadata and type can be used to build something that looks like an inheritance hierarchy but is more flexible.

10.8 Cond

`cond` is a multi-way `if`. Here's a function that uses a `cond` expression:

```
1 user=> (def classify
2         (fn [number]
3           (cond (zero? number)
4                 "zero"
5                 (even? number)
6                 "even"
7                 :else
8                 "unclassified"))))
11
12 )
13 user=> (classify 5)
14 "unclassified"
```

`cond` takes any number of pairs of expressions. They are evaluated in order. Given a pair, `cond` evaluates its element. If the result counts as true (that is, is anything but `false` and `nil`), the second element is evaluated and its result is the result of the entire `cond`.

Notice that the last pair begins with `:else`, which counts as true (as do all keywords). That is, it's not a special token, just a conventional way to mark the “otherwise” case.

If none of the first expressions counts as true, the value of the `cond` is `nil`. Here's the simplest possible example:

```
1 user=> (cond)
2 nil
```

10.9 Exercises

Often, receiving a `nil` from the Maybe monad is not so useful. Something bad happened somewhere, but you don't know where or what. Furthermore, not every error is associated with a `nil`. In this exercise, you'll implement a monad that has the short-circuiting behavior of the Maybe monad, but works with error values instead of `nil`s.

For example, suppose we have this function:

```

1 (def factorial
2   (fn [n]
3     (cond (< n 0)
4           (oops! "Factorial can never be less than zero." :number n)
5
6           (< n 2)
7           1
8
9           :else
10          (* n (factorial (dec n))))))

```

factorial is now a sort of dual-typed function: it can return either a number or an error. The Error monad you're about to write will allow you to write code like this:

```

1 user=> (def result
2         (with-monad error-monad
3           (domonad [big-number (factorial -1)
4                       even-bigger (* 2 big-number)]
5                     (repeat :a even-bigger))))
6 user=> (oopsie? result)
7 true
8 user=> (:reason result)
9 "Factorial can never be less than zero."
10 user=> (:number result)
11 -1

```

Because I know you're too dignified to write such functions, I'll predefine oops! and oopsie? for you.

oops! generates a special type of map:

```

1 (def oops!
2   (fn [reason & args]
3     (with-meta (merge {:reason reason}
4                       (apply hash-map args))
5               {:type :error})))

```

oopsie? checks whether a value represents an error by looking at its type:

```

1 (def oopsie?
2   (fn [value]
3     (= (type value) :error)))

```

Your job in this exercise is to write a monad that halts computation when an `oopsie?` is produced by a step.

You can start from [sources/maybe.clj](#), which contains `oops!` and `oopsie?` as well as this definition of the Maybe monad (which I'm calling `maybe-monad` so as not to conflict with the predefined `maybe-m`):

```
1 (use 'clojure.algo.monads)
2
3 (def decider
4   (fn [step-value continuation]
5     (if (nil? step-value)
6         nil
7         (continuation step-value))))
8
9 (def maybe-monad
10  (monad [m-result identity
11         m-bind decider]))
```

You'll have to be sure to use that first line (the `use` statement) in your repl.

My solution is in [solutions/error.clj](#).

10.10 Lifting functions with monads

Consider the humble addition function, `+`. Suppose you knew it always took only two arguments. And then suppose you wanted to write a new function `+`? that is resistant to `nil` in the following way:

```
1 user=> (+? 1 2)
2 3
3 user=> (+? nil 2)
4 nil
5 user=> (+? 1 nil)
6 nil
```

Here's one way to do it:

```
1 (def +?
2   (fn [arg1 arg2]
3     (with-monad maybe-m
4       (domonad [a1 arg1
5                  a2 arg2]
6                 (+ a1 a2)))))
```

But writing such code is very rote, so `clojure.algo.monads` provides `m-lift` to do it for us:

```
1 user=> (def +?
2           (with-monad maybe-m
3             (m-lift 2 +)))
```

The only annoying thing about this is that you have to specify the number of arguments in advance. (That's because Clojure gives you no way to ask a function how many arguments it has.)

`m-lift` is useful because, although nested function calls can be transformed into flow style, sometimes the result is less readable or annoyingly verbose. So if you *both* want to write code like this:

```
1 (defenestrations-of (counselors)
2                     (-> (building 8) (story 3)))
```

... *and* you want errors to short-circuit the calculations, you can do this:

```
1 (with-monad error-monad
2   (let [defenestrations-of (m-lift 2 defenestrations-of)
3         building (m-lift 1 building)
4         story (m-lift 2 story)]
5     (defenestrations-of (counselors)
6                         (-> (building 8) (story 3)))))
```

That hardly seems worth it for one calculation, but if you often want lifted behavior from functions, it's nice that you can add it as easily as this:

```
1 (with-monad error-monad
2   (def story
3     (m-lift 2
4             (fn [building n] ...)))
5   (def defenestrations-of (m-lift 2 ...))
6   ...)
```

10.11 Turning loops into straight-line flows

It is the fate of popular monads to turn into language constructs. In this section, we'll talk about the Sequence monad, which Clojure provides in a simpler form via the `for` construct. I'll show how to build the real Sequence monad [in an optional chapter](#).

Here, in no particular programming language, is how you might compute all the products of three sets of numbers:

```
1 result = []
2 for a in [1, 2]
3     for b in [10, 100]
4         for c in [-1, 1]
5             result.append(a*b*c)
6 (-10, 10, -100, 100, -20, 20, -200, 200)
```

This is not straight-line flow. In Clojure, you could do a similar thing via nested map functions, but I claim there's no solution that isn't really ugly. (Try it! Prove me wrong!) For that reason, Clojure provides a special form that looks like the straight-line flow of `let`:

```
1 user=> (for [a [1, 2]
2             b [10, 100]
3             c [-1, 1]]
4         (* a b c))
5 (-10 10 -100 100 -20 20 -200 200)
```

The idea of looping has been abstracted away. (Indeed, I find that I'm so stuck in thinking in terms of loops that it hampers my understanding of `for` expressions in real code. I read from the top down, trying to understand loop values, instead of concentrating on the body of the `for`, which is where the action is.)

It's important to realize that a property `for` shares with `let` is that later steps can refer to the results of earlier steps. To see that, here's a simple example:

```
1 user=> (for [a [1 2 3]
2             b (repeat a "hi!")]
3         [a b])
4 ([1 "hi!"]
5  [2 "hi!"] [2 "hi!"]
6  [3 "hi!"] [3 "hi!"] [3 "hi!"])
```

`for` is a fine and convenient special form (with some features I haven't mentioned), but its status *as* a special form limits it. Let's compare it to the actual Sequence monad, which is used like this:

```

1 user=> (with-monad sequence-m
2         (domonad [a [1 2 3]
3                  b (repeat a "hi!")]
4                  [a b]))
5 ([1 "hi!"]
6  [2 "hi!"] [2 "hi!"]
7  [3 "hi!"] [3 "hi!"] [3 "hi!"])

```

Suppose I want to define a pairwise-plus that works like this:

```

1 user=> (pairwise-plus [1 2 3] [4 5 6])
2 (5 6 7 6 7 8 7 8 9)

```

I can do that using for, like this:

```

1 user=> (def pairwise-plus
2         (fn [seq1 seq2]
3           (for [a seq1
4                 b seq2]
5                (+ a b))))

```

But I can do it much more easily with the Sequence monad:

```

1 user=> (def pairwise-plus
2         (with-monad sequence-m (m-lift 2 +)))
3 user=> (pairwise-plus [1 2 3] [4 5 6])
4 (5 6 7 6 7 8 7 8 9)

```

And suppose that I want to combine the behavior of the Maybe monad and for... well, I can't. I can with the Sequence monad, though, because all well-behaved monads can be combined:

```

1 user=> (def combined-monad (maybe-t sequence-m))
2 user=> (with-monad combined-monad
3         (domonad [a [1 nil 3]
4                   b [-1 -2]]
5                  (* a b)))
6 (-1 -2 nil -3 -6)

```

10.12 Exercises

My solution is in [solutions/primes.clj](#).

Exercise 1

2 is a prime number. All multiples of 2 are not primes (except for 1×2). Here's how you can use `range` to find all the non-prime multiples of 2 between 4 (the first one) and 100 (inclusive).

```
1 user=> (range (* 2 2) 101 2)
2 (4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48 50 52 54
3 56 58 60 62 64 66 68 70 72 74 76 78 80 82 84 86 88 90 92 94 96 98 100)
```

Here's how you do the same thing with 3:

```
1 user=> (range (* 3 2) 101 3)
2 (6 9 12 15 18 21 24 27 30 33 36 39 42 45 48 51 54 57 60 63 66 69 72 75 78 81
3 84 87 90 93 96 99)
```

We can do the same for 4. That's wasted work, since any multiple of 4 is already a multiple of 2, but what the heck: your CPU spends almost all of its time waiting for you to give it something to do.

```
1 user=> (range (* 4 2) 101 4)
2 (8 12 16 20 24 28 32 36 40 44 48 52 56 60 64 68 72 76 80 84 88 92 96 100)
```

Write a function `multiples` that takes a number and returns a sequence of all its non-prime multiples less than 100.

Exercise 2 [2](#)

Use the Sequence monad or `for` (your choice!) to find all non-primes less than 100. Duplicates are OK.

Hint: You'll need two steps and a body that just returns a value.

Exercise 3

Use [sets](#) to calculate all the primes less than 100.

Hint: Here's how you can tell if 6 is a non-prime less than 10:

```
1 user=> ({4 6 8 9} 6)
2 6
```

Hint: You'll likely want to use `remove`.

10.13 Choose your own adventure

What monads illustrate, I believe, is that the old programming commandment “remove duplication” can be obeyed far more ruthlessly and thoroughly via functional abstractions than in object-oriented languages. You can climb levels of abstraction to dizzying heights.

“Dizzying”, though, is an operative word. How much can you use monads before you’ve made a path that the programmers who come after you can’t follow? Remember: in most cases, you’re not creating abstractions to give *yourself* more power; you’re creating them so the *people extending your code* have power they can readily exploit.

At the current state of the practice, I believe using the Maybe, Error, and Sequence monads is perfectly reasonable. Lifting functions with those monads seems reasonable. Defining new monads, or transforming existing monads to make new ones, will likely leave those who follow after you gasping for air.

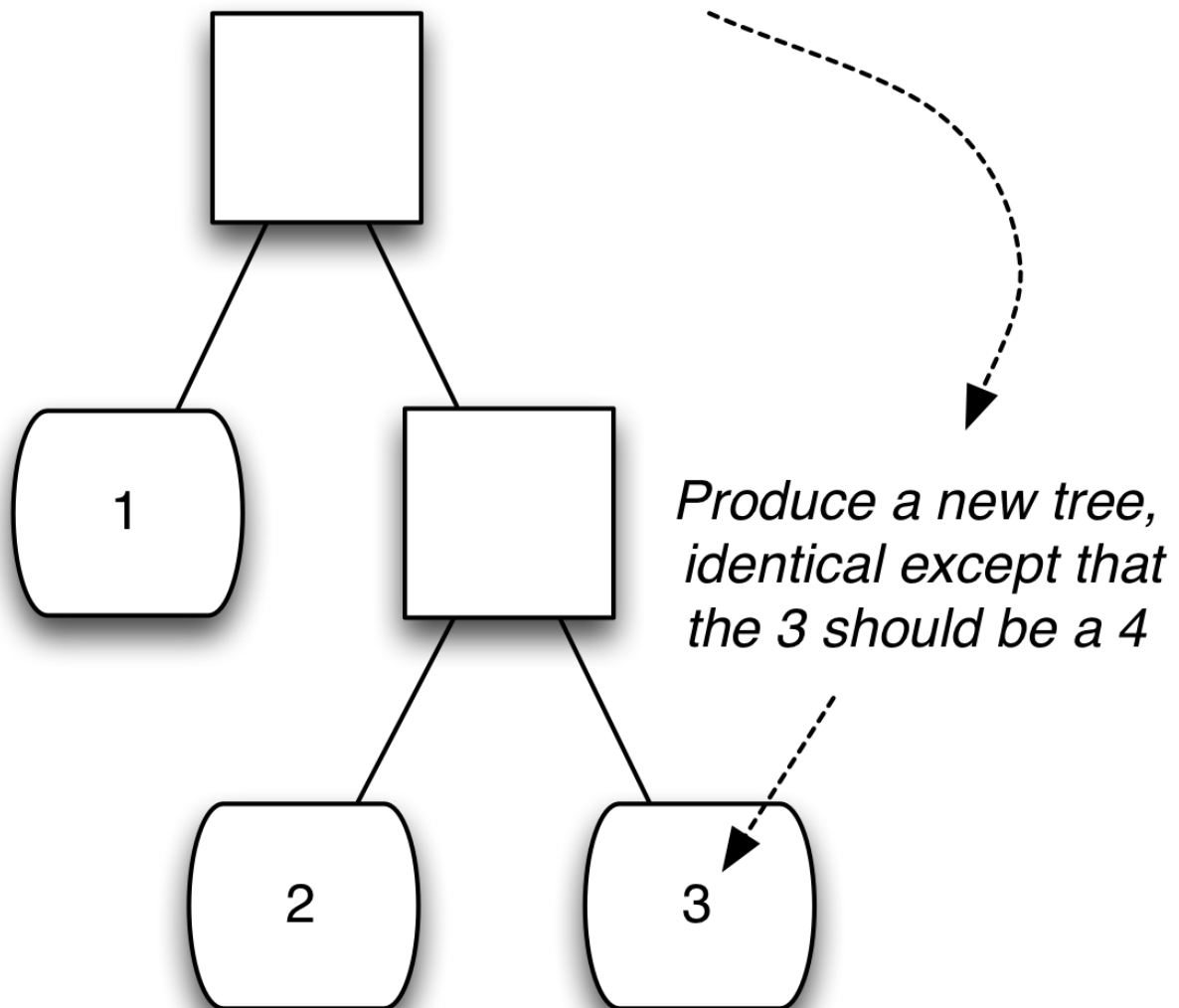
However, let’s keep in mind that today’s commonplaces of object-oriented programming were mysterious and brain-bending twenty or thirty years ago. (Really! I was there! It astounds me what seemed perplexing back then.) Monads and other higher-higher-higher-order abstractions may follow the same trajectory. I hope they do.

So we have another branch point in the book. If you find the promise of monads fascinating and want to understand them better, go to [Part III](#). If you’re content knowing what you know now and want to move on, go to the next chapter.

1. You’ll have to trust me that a name like `monadifier` is better. [↩](#)
2. I borrowed this example from some [python documentation](#), author unknown to me. [↩](#)

11. Pretending State is Mutable

This chapter is about easily solving problems like this:

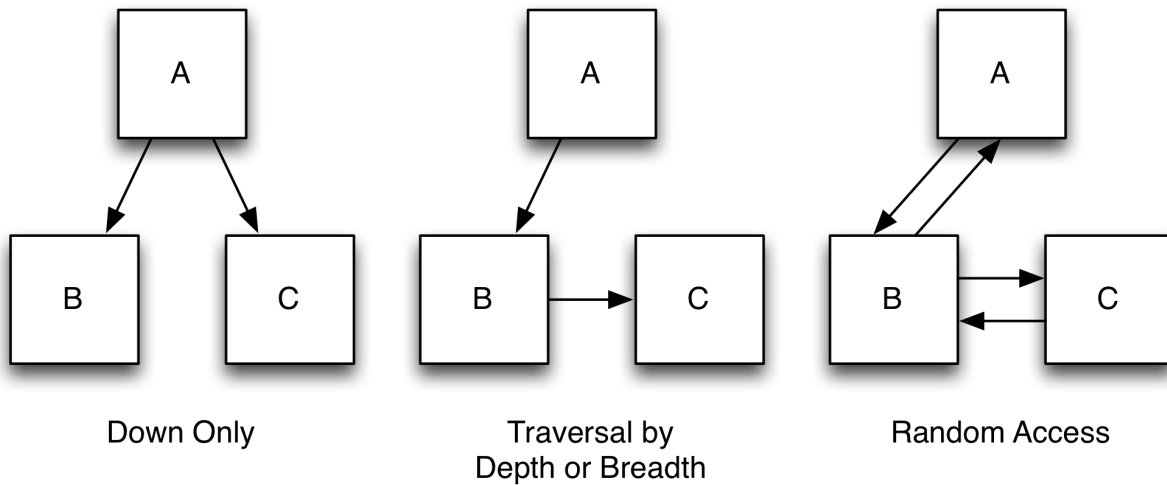


Before we can tackle that problem, I need to lay some groundwork.

11.1 Trees

In object-oriented languages, it's not uncommon to have specialized classes for trees, with nodes having direct pointers to other nodes. The class is

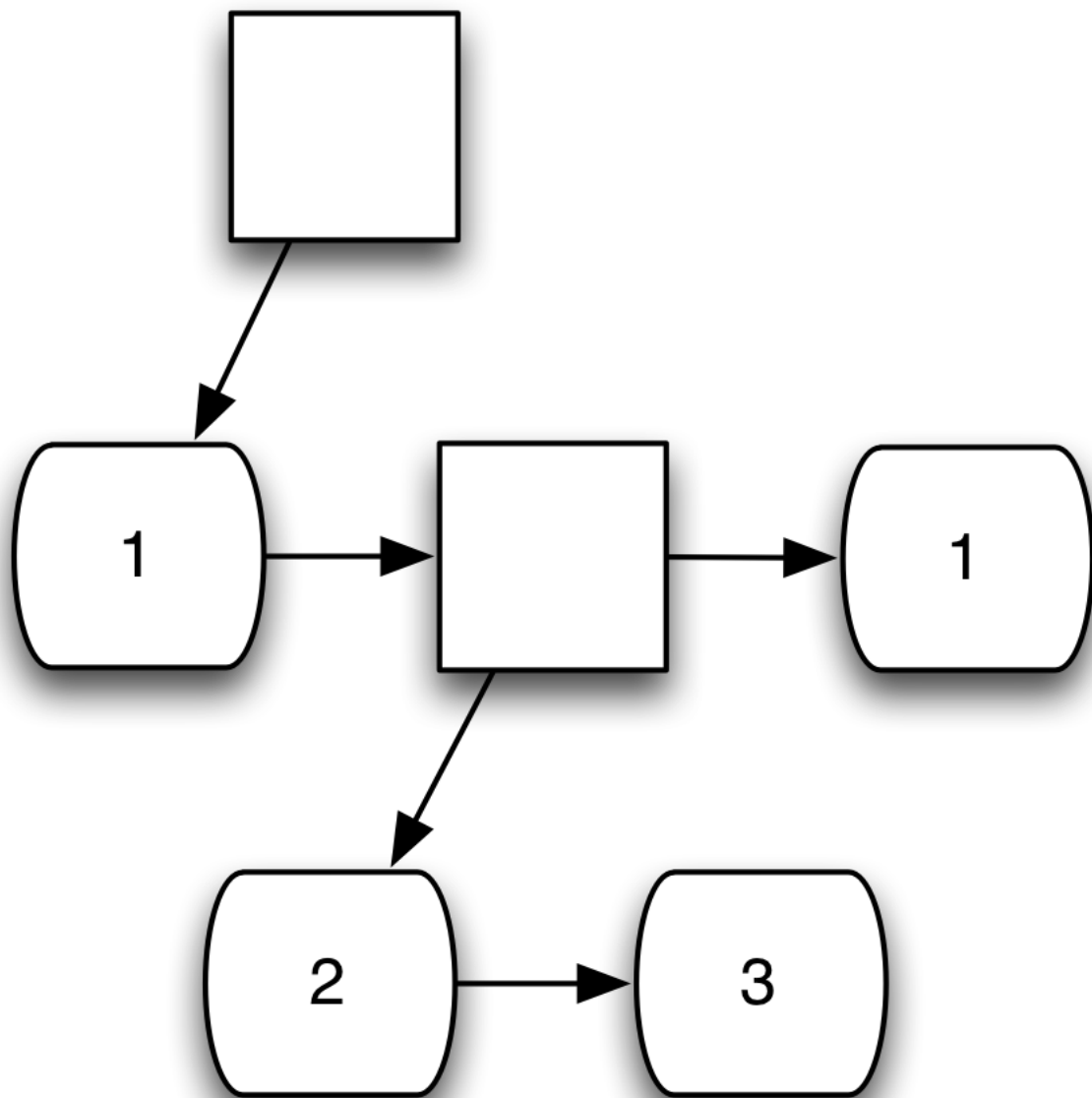
designed according to the expected pattern of traversal:



In functional languages, it's often more convenient to represent trees as nested sequences. Here's an example where each "node" is composed of a value and a sequence of the node's children.

```
1 (A
2   (B ())
3   (C ()))
```

Alternately, suppose that there's a distinction between interior nodes and leaves, and only leaves have values. An object graph might look like this:

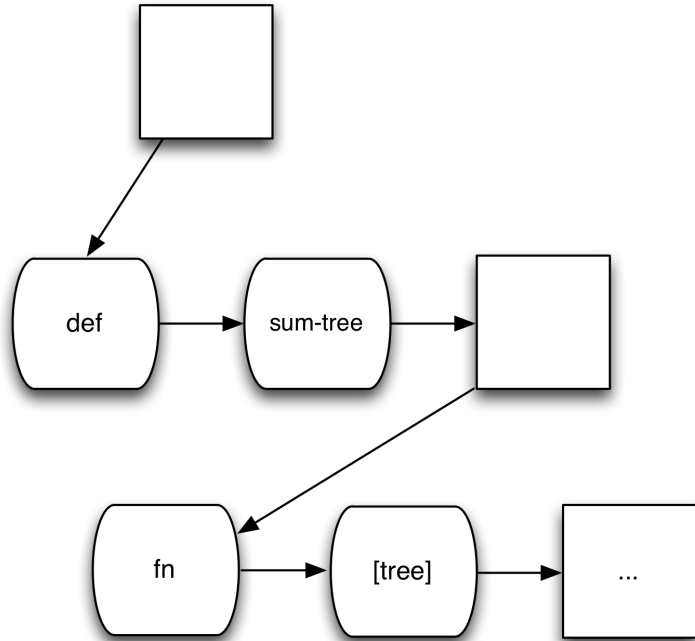


And the nested sequence equivalent would look like this:

```
1 (1 (2 3) 1)
```

The *zipper* data structure, explained next, works with this kind of tree. To make things interesting, we'll work with trees whose contents are code:

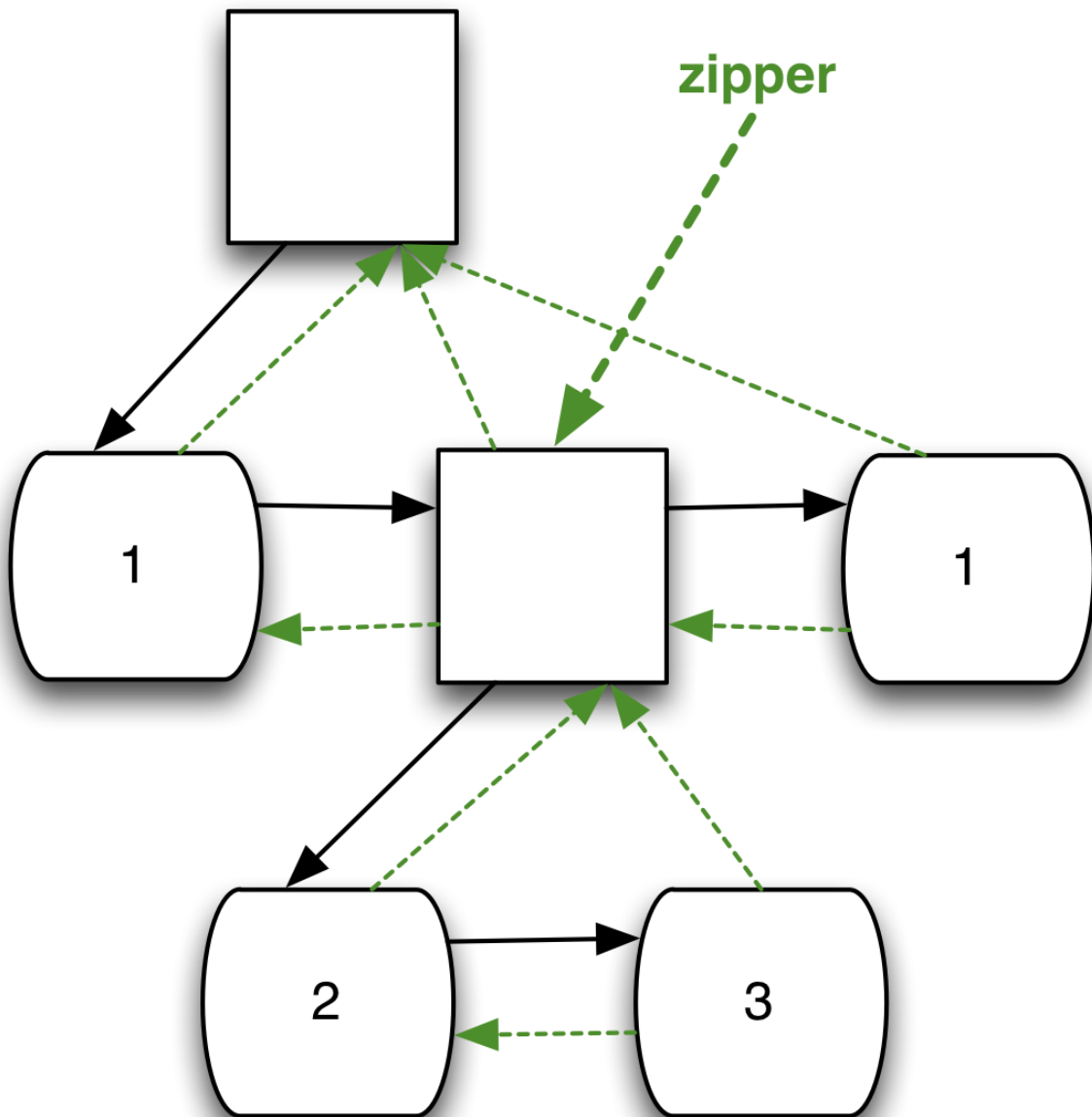
```
(def sum-tree
  (fn [tree]
    (if (sequential? tree)
        (apply + (map sum-tree tree))
        tree)))
```



That code is simultaneously two things: a recursive function that sums up all the numbers in a tree, *and* a tree itself. It is a three-element sequence. The first two are symbols (`def` and `sum-tree`). The third element is a nested subsequence (the `fn` form). This property—that the primary way to represent a program is as a basic datatype—rejoices in the name *homoiconicity*.

11.2 Zippers

For the moment, let's think of zippers as a data structure that overlays pointers on the basic nested sequence shape:



At any given moment, the zipper points to one of the nodes in the tree. From that node, it can move left, right, up, or down (where down always moves to the leftmost node of a sequence).

Here's how you create a zipper:

```
1 user=> (require '[clojure.zip :as zip])
2 user=> (def zipper (zip/seq-zip '(1 (2 3) 1)))
```

The `require` expression makes zipper functions available to us, but only with a `zip/` prefix. (Otherwise some of them would clash with core Clojure functions.)

At the beginning, the zipper's location is at the root of the tree. At any moment, we can print the subtree at the current location:

```
1 user=> (zip/node zipper)
2 (1 (2 3) 1)
```

That's not so exciting, so let's descend the tree and see what's there:

```
1 user=> (-> zipper zip/down zip/node)
2 1
```

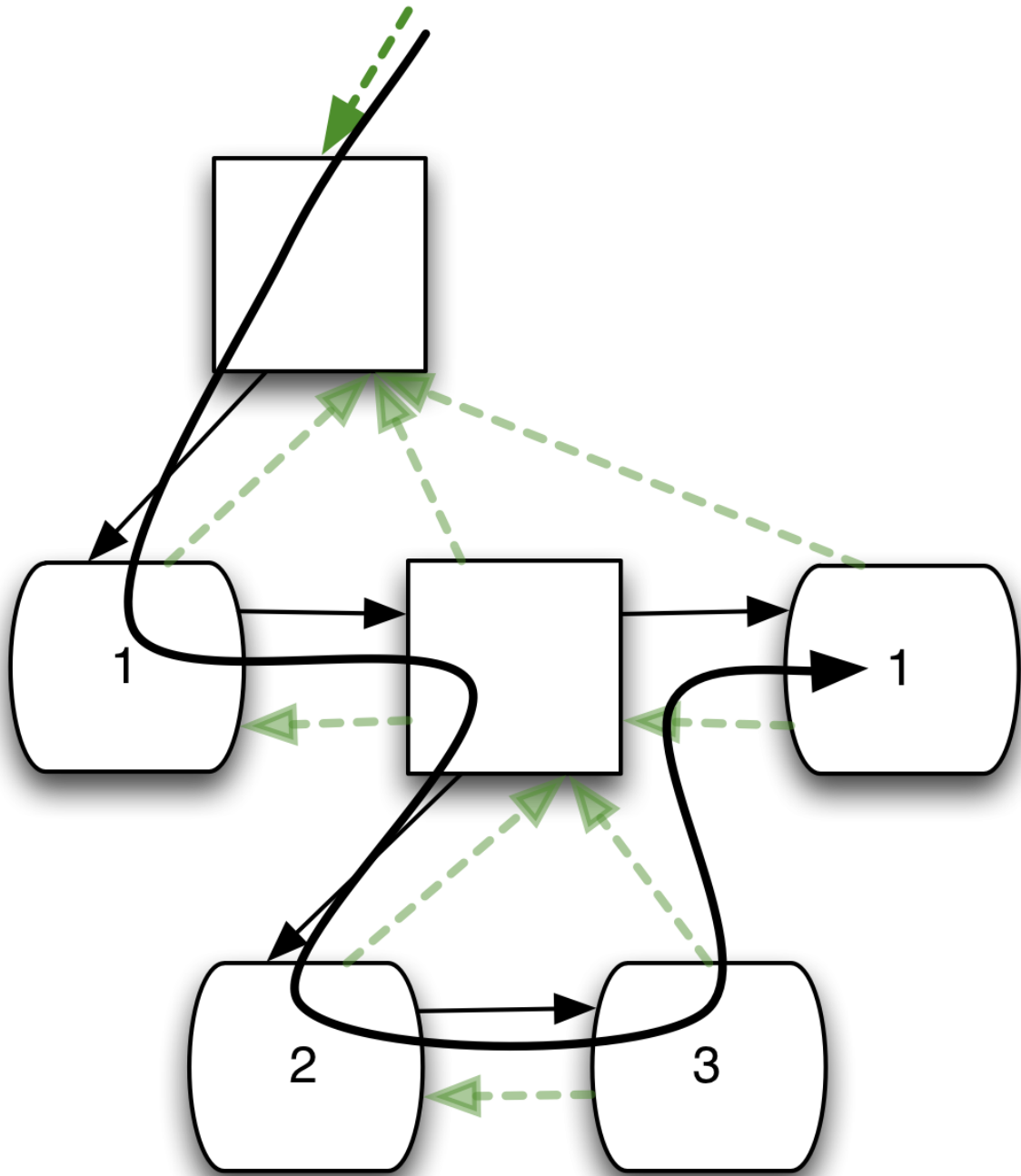
It's important remember that a zipper is an immutable data structure just like any other. That is, `zip/down` doesn't alter the zipper's location, it creates a *new* zipper with a different location. So the symbol `zipper` is still bound to the original zipper, which is still pointing at the original location:

```
1 user=> (zip/node zipper)
2 (1 (2 3) 1)
```

If we want to hold onto the new location, we have to bind it to a symbol:

```
1 user=> (def zipper (-> zipper zip/down zip/right))
2 user=> (zip/node zipper)
3 (2 3)
```

Zipper's have a `zip/next` function that moves them through the tree left-to-right and top-to-bottom in a *depth-first* way.:



During the traversal, the zipper can know if it's at an interior or leaf node with `zip/branch?`. When the zipper produced by `zip/next` has “fallen off the end of the tree”, `zip/end?` is true.

So let's implement a recursive function that collects all the nodes in a tree, effectively flattening it. Here's an example:


```

1 user=> (flattenize '(fn [a] (* 2 (inc a))))
2 (fn [a] * 2 inc a)

```

As a start toward that form, let's think about a function that takes a zipper and produces a flattened list of nodes:

```

1 user=> (def flatten-zipper
2         (fn [so-far zipper]
3           (cond (zip/end? zipper)
4                 so-far
5
6                 (zip/branch? zipper)
7                 (flatten-zipper so-far (zip/next zipper))
8
9                 :else
10                (flatten-zipper (cons (zip/node zipper) so-far)
11                                  (zip/next zipper))))

```

There are two things to note about this code:

1. It prints its output backwards because we cons new entries onto the front of so-far. That's easy to fix, especially since we won't be calling flatten-zipper directly:

```

1 user=> (def flattenize
2         (fn [tree]
3           (reverse (flatten-zipper '() (zip/seq-zip tree)))))

```

2. This zipper doesn't descend into literal vectors or maps; it treats them as leaves, not branches. That's the most useful behavior when you're working with code. (There are other zippers that behave differently.)

That's enough information for a first set of exercises with zippers. First, let me introduce two more Clojure features that might come in handy.

11.3 The do special form

I'm going to make you write recursive functions similar to flatten-zipper. If you're anything like me, you'll make mistakes and wish that you could see what's happening during execution.

It's easy to put debugging print statements inside a function body:

```

1 (def f
2   (fn [a b]
3     (println a "&" b)
4     (println "Is a less than b?" (< a b))
5     (+ a b)))

```

But the “then” and “else” parts of an `if` and the clauses of a `cond` accept only a single form. That means you can’t insert a debugging `println` in this:

```

1 (if (safe-to-launch? a 17)
2   (ignite a))

```

... to produce this:

```

1 (if (safe-to-launch? a 17)
2   (println "Success case. Will execute `ignite`.")
3   (ignite a))

```

... because the `ignite` form has now become the “else” case, which is probably bad.

Fortunately, the `do` special form wraps several forms into one:

```

1 (if (safe-to-launch? a 17)
2   (do
3     (println "Success case. Will execute `ignite`.")
4     (ignite a)))

```

The value returned by a `do` is the value of the last expression.

11.4 A little problem of self-reference

Given that we’re now happy nesting functions within other functions, it seems bizarre to independently define `flattenize` and its helper function `flatten-zipper`. We should write this:

```

1 user=> (def flattenize
2         (fn [tree]
3           (let [flatten-zipper
4                 (fn [so-far zipper]
5                   (cond (zip/end? zipper)
6                         so-far
7

```

```

8             (zip/branch? zipper)
9             (flatten-zipper so-far (zip/next zipper))
10
11         :else
12         (flatten-zipper (cons (zip/node zipper) so-
far)
                        (zip/next zipper))))]
13
14         (reverse (flatten-zipper '() (zip/seq-zip tree))))))
15 java.lang.Exception: Unable to resolve symbol: flatten-zipper

```

Oops. What's happening can be more easily seen by comparison to a `let` form that defines something simpler than a function:

```

1 user=> ;; `something` is undefined
2 user=> (let [something (inc something)]
3         something)
4 java.lang.Exception: Unable to resolve symbol: something

```

Any symbols on the right-hand side of a `let` step refer to previous bindings, not the binding that's about to be made. Remember that `let` is equivalent to the Identity monad, and therefore to this expansion:

```

1 (-> (inc something)
2      (fn [something] something))

```

The binding of `something` inside the continuation has nothing to do with the binding of `something` outside of it. There must be a previous binding of `something` for that expansion to make sense.

In the case of `flatten-zipper`, there *is* no previous binding—the symbol `flatten-zipper` doesn't even exist yet—so we get the error shown above. Because of that, you can't use `let` to define recursive functions.

Still, recursive local functions are awfully useful, and it would be a shame to insist that they can only be used when made globally available with `def`. Therefore, there's a form specifically for defining potentially recursive local functions: `letfn`. Here it is:

```

1 user=> (letfn [(factorial [n]
2                 (if (or (= n 0) (= n 1))
3                     1
4                     (* n (factorial (dec n)))))]

```

```
5      (factorial 5))
6 120
```

The syntax is new. The functions are defined within a vector, as with `let`. Each definition has this form:

```
1 (function-name [args] body...)
```

That looks like a pointless variation from our usual `fn` style, but I can actually now reveal that what we've been using is the *real* oddball. Most Clojure programs use the following form to define top-level functions:

```
1 (defn factorial [n]
2   (if (or (= n 0) (= n 1))
3       1
4       (* n (factorial (dec n)))))
```

That's shorthand for the `(def factorial (fn ...))` form you've seen throughout the book. I've used the latter form to emphasize that functions are values like any others. I'll continue to use it, just for kicks.

Here, then, is a definition of `flattenize` that works:

```
1 (def flattenize
2   (fn [tree]
3     (letfn [(flatten-zipper [so-far zipper]
4               (cond (zip/end? zipper)
5                     so-far
6
7                     (zip/branch? zipper)
8                     (flatten-zipper so-far (zip/next zipper))
9
10                    :else
11                    (flatten-zipper (cons (zip/node zipper) so-far)
12                                       (zip/next zipper)))]
13       (reverse (flatten-zipper '() (zip/seq-zip tree)))))
```

If you want to use recursive helper functions in the exercises, don't forget `letfn`.

11.5 Exercises

You can find the source for `flattenize` in [sources/zipper.clj](#). My exercise solutions are in [solutions/zipper.clj](#).

Exercise 1

Write a function, similar to `flattenize`, that collects all the vectors in a tree:

```
1 user=> (all-vectors '(fn [a b] (concat [a] [b])))
2 ([a b] [a] [b])
```

You can tell if a node is a vector with `vector?`.

Exercise 2

Write a function that returns only the first vector in a tree. It should return `nil` if there is no vector.

The easy way to do that would be this:

```
1 (first (all-vectors ...))
```

Don't do it that way.

11.6 Editing trees

The previous examples and exercises could be done with plain recursion over a tree, with no need for a fancy zipper structure (try it if you like). It's only when you get to “modifying” trees that zippers prove their worth.

I think it's fair to describe zippers as a concession to our experience in the physical world. In that physical world, we are used to taking objects with state and manipulating them to change that state. By cooking steaks on the grill, we don't create *new* cooked steaks, we change the state of the existing steaks. Because of such, it's natural to think of programming in terms of changing state.

In this section, I'll show you several modification functions. More importantly, I'll demonstrate the kind of thinking that goes into using

zippers.

Replacing

So consider the use of zippers to replace all + symbols in a form with PLUS. We know how to identify when a node is a +:

```
1 (= (zip/node zipper) '+)
```

The zip/replace function replaces the current node (which may be a leaf or a branch) with a new one. Note that replace, like everything else, is an immutable operation: the original zipper is left unchanged, and a new one is created. Here's an example of its use:

```
1 user=> (def z1 (zip/seq-zip '(+ 1 2)))
2 user=> (def z2 (-> z1 zip/down (zip/replace 'PLUS)))
```

The new zipper has moved down to the + node and replaced it. zip/replace leaves the zipper pointing at the same node, so if we view it, we can see the change:

```
1 user=> (zip/node z2)
2 PLUS
```

There are two ways we can see the change to the whole tree. First, we could move up and use zip/node:

```
1 user=> (-> z2 zip/up zip/node)
2 (PLUS 1 2)
```

More common, though, would be to use a function that converts a zipper into a regular tree of sequences:

```
1 user=> (zip/root z2)
2 (PLUS 1 2)
```

It's fairly straightforward to write a recursive function that walks an entire tree, replacing as it goes:

```
1 user=> (def tumult
2         (fn [form]
3           (letfn [(helper [zipper]
```

```

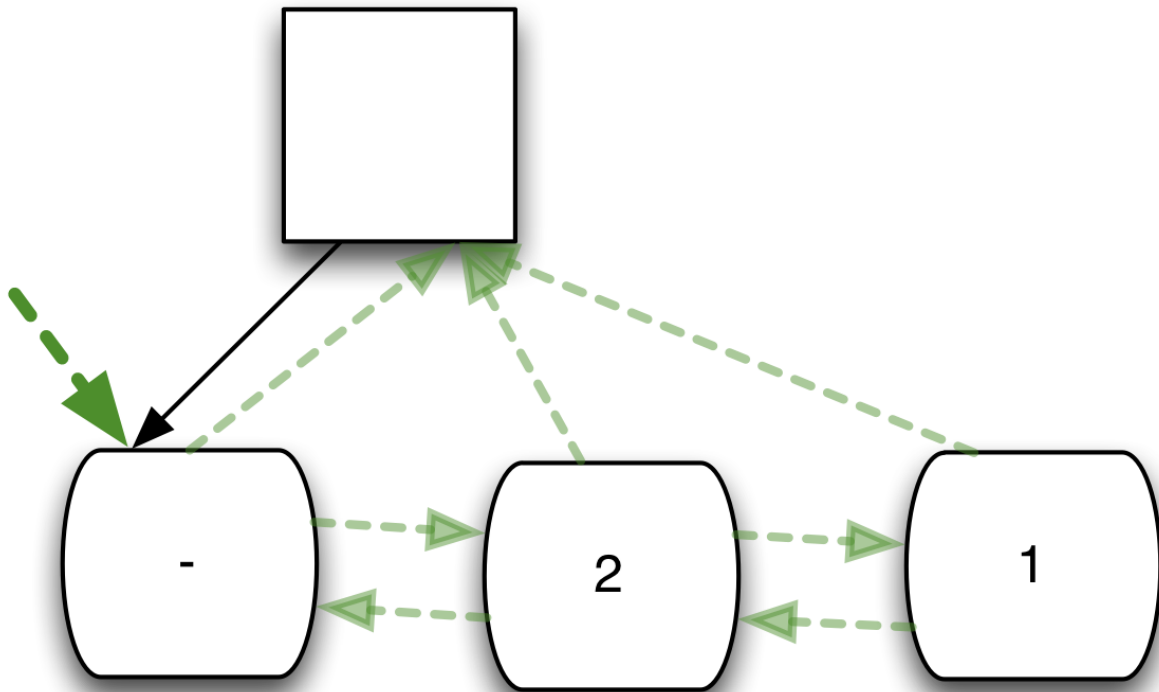
4                               (cond (zip/end? zipper)
5                                   zipper
6
7                                   (= (zip/node zipper) '+)
8                                   (-> zipper
9                                       (zip/replace 'PLUS)
10                                      zip/next
11                                      helper)
12
13                                   :else
14                                   (-> zipper zip/next helper))))]
15                               (-> form zip/seq-zip helper zip/root))))
16 user=> (tumult '(- 3 (+ 6 (+ 3 4) (* 2 1) (/ 8 3))))
17 (- 3 (PLUS 6 (PLUS 3 4) (* 2 1) (/ 8 3)))

```

Appending (and the consequences of immutability)

Editing doesn't have to be restricted to replacing a node. I'll add another clause to the `cond` that appends a 55555 to the end of any list that begins with a `-`. That can be done with `append-child`. When pointing at a branch, `append-child` adds its argument as the rightmost child of that branch. It returns a new zipper that hasn't changed what it points to.

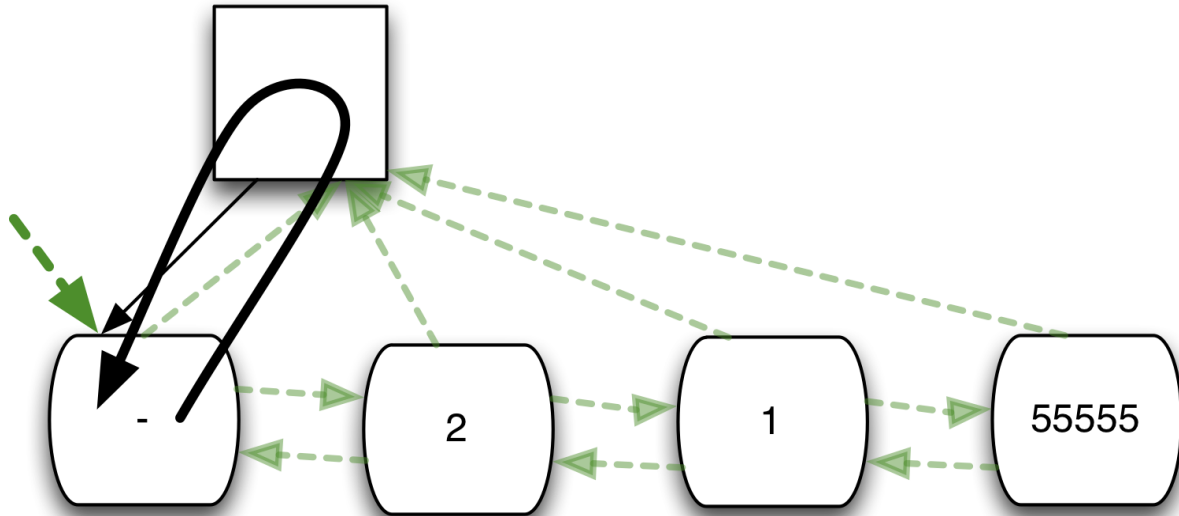
I find it somewhat easy to get confused in cases like this. That's because zippers make it *look* like you're editing a single tree, but you actually have to remember that every zipper operation gives you a new zipper. Consider this case:



In that situation, I find it easy to think “I should go up, then use `zip/append-child`. With that done, I’ll make recursive call to helper on the `zip/next` node.” Which leads to this clause in the `cond`:

```
1 (cond ...
2   (= (zip/node zipper) '-')
3   (-> zipper
4     zip/up
5     (zip/append-child 55555)
6     zip/next
7     helper)
```

Do you see the problem? It’s that the `zip/next` is applied to the results of `zip/append-child`, which is the branch node. So `zip/next` produces a zipper pointing at the `-` node:



That will not end well.

It's better to do the append-child when the recursion first arrives at the branch with a - child:

```
1 (cond ...
2   (and (zip/branch? zipper)
3         (= (-> zipper zip/down zip/node) '-))
4   (-> zipper
5       (zip/append-child 55555)
6       zip/next
7       helper)
```

The tricksiness of removing

Next, I'll demonstrate how to pluck values out of a zipper and insert them elsewhere by writing code to flip the two arguments of '/'. I'll use the `zip/remove` function, which removes the current node. The zipper returned points to the previous node. Let's see what "previous" means by example.

To make sure it's clear what's going on, let's work with a simple list:

```
1 user=> (def zipper (zip/seq-zip '(1 2 3)))
```

Let's position our zipper on the 2.

```
1 user=> (def zipper (-> zipper zip/down zip/right))
2 user=> (zip/node zipper)
```

3 2

Let's remove it:

```
1 user=> (def zipper (zip/remove zipper))
```

Where are we now? On the 1:

```
1 user=> (zip/node zipper)
2 1
```

And if we convert the zipper to a tree, we see the 2 really is gone:

```
1 user=> (zip/root zipper)
2 (1 3)
```

Where do we go if we remove the 1? – up to the root of the tree:

```
1 user=> (def zipper (zip/remove zipper))
2 user=> (zip/node zipper)
3 (3)
```

So the meaning of “previous” is based on `zip/next` (unsurprisingly). At any point in a zipper, if you continue removing nodes, you'll retrace the path `zip/next` took from the root, just in reverse.

It's important to realize the consequences of that. The previous node is not (*not! not!*) necessarily the node to the left. (The emphasis is because I make this mistake all the time.)

Consider this:

```
1 user=> (def zipper (zip/seq-zip '((+ 1 HERE) (+ 2 3))))
```

Let's position the zipper at the second sublist:

```
1 user=> (def zipper (-> zipper zip/down zip/right))
2 user=> (zip/node zipper)
3 (+ 2 3)
```

Let's delete that:

```
1 user=> (def zipper (zip/remove zipper))
```

Are we at the (+ 1 HERE) list? No:

```
1 user=> (zip/node zipper)
2 HERE
```

Remember that zip/next moves depth-first, not breadth-first. The next node after the (+ 1 HERE) branch is +, not (2 3). Instead, it's HERE whose next node is (2 3). Therefore, the previous node to (2 3) is HERE, and that's where zip/remove puts us when we remove (2 3).

So, given a zipper pointing at a / node, here is how *not* to delete both arguments:

```
1 user=> (def start (-> (zip/seq-zip '(/ (+ 1 HERE) 2)) zip/down))
2 user=> (def end (-> start zip/right zip/right zip/remove zip/remove))
3 user=> (zip/node end)
4 1
5 user=> (zip/root end)
6 (/ (+ 1))
```

It has first removed the 2, then the HERE. That's not the two-argument deletion we wanted.

So instead of moving wildly right, I have to be careful to always delete the tree just after the / (because / must always be that tree's previous node).

```
1 user=> (def start (-> (zip/seq-zip '(/ (+ 1 HERE) 2)) zip/down))
2 user=> (def end (-> start zip/right zip/remove zip/right zip/remove))
3 user=> (zip/node end)
4 /
5 user=> (zip/root end)
6 (/)
```

Inserting, and looking back in time

Two new functions, zip/insert-left and zip/insert-right, let you add elements next to the current node and leave the new zipper pointing at the same element. So here is how to turn (/) into (/ 2 1) when the zipper is positioned on the /:

```

1 user=> (-> end (zip/insert-right 1) (zip/insert-right 2) (zip/root))
2 (/ 2 1)

```

Putting together what we know about removing and inserting, we have the following:

```

1 (cond ...
2   (= (zip/node zipper) '/')
; (1)
3   (-> zipper
4     zip/right
5     zip/remove
; (2)
6     zip/right
7     zip/remove
8     (zip/insert-right (-> zipper zip/right zip/node)) ; (3)
9     (zip/insert-right (-> zipper zip/right zip/right zip/node)) ; (4)
10    zip/next
; (5)
11    helper)

```

1. In the previous example, I used `append-child`. It was most conveniently used from the branch above the `-`. Here I'm using `insert-right` while positioned at the `/`. So I check for the `/` when the recursion takes me to it, not when it takes me to the branch above it.
2. Instead of removing and inserting, I could have used `'zip/replace'`, but I needed an excuse to explain the new functions.
3. I'm being cute here. I'm deep in a flow at this point, one with a zipper where the 8 and 3 have been removed. However, zipper still points to the unmodified original zipper, so I can "reach back in time" to grab its values to insert.

Note the consequence: if I used `let` to bind each step in the flow to a symbol, I'd have a complete record of what had happened. I could use `zip/root` to display each version of the tree on the way to the final solution.

4. If I wanted to be a bit more terse, I could use a new function, `zip/rightmost`, instead of two `zip/rights`. That would force me to require that a `/` only take two arguments, though.
5. The whole flow leaves the final zipper pointing at the `/`, so we need to advance before the next recursion.

Subtrees are added to the recursion

In a final example, I'll replace every `*` expression with a complicated `/` expression. That's a relatively straightforward use of `zip/replace`:

```
1 (cond ...
2   (and (zip/branch? zipper)
3         (= (-> zipper zip/down zip/node) '*))
4   (-> zipper
5       (zip/replace '(/ 1 (+ 3 (- 0 9999)))))
6       zip/next
7       helper)
```

Remember that `zip/replace` produces a zipper pointing at what replaced the original node. Is that a structure that the recursion descends? Why, yes. Yes it is:

```
1 user=> (tumult '(- 3 (+ 6 (+ 3 4) (* 2 1) (/ 8 3))))
2 (- 3 (PLUS 6 (PLUS 3 4) (/ (PLUS 3 (- 0 9999 55555)) 1) (/ 3 8)) 55555)
3                                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

Here are the steps in the processing. When the zipper points to `(* 2 1)`, `tumult` sees that the first child is `*`. Therefore, it replaces the whole subtree with this:

```
1 (/ 1 (+ 3 (- 0 9999))) ;; instead of (* 2 1)
```

The zipper is positioned at that subtree. It moves to the `zip/next` node, which is a `/`. `tumult` is coded to flip the two nodes to the right, which produces this:

```
1 (/ (+ 3 (- 0 9999)) 1)
```

The zipper moves on. When it encounters `+`, it changes it to `PLUS`:

```
1 (/ (PLUS 3 (- 0 9999)) 1)
```

It moves on. When it encounters the `(- 0 9999)` subtree, it adds 5555 as the last child:

```
1 (/ (PLUS 3 (- 0 9999 5555)) 1)
```

And that's how a complicated substitution is further processed.

Edit-and-replace

There's one other function worth explaining here: `edit`. In its simple form, it passes the subtree at the current position to a given function, then `zip/replaces` the old value with the new:

```
1 user=> (def tree '(+ 1 2))
2 user=> (def zipper (-> (zip/seq-zip tree) zip/next zip/next))
3 user=> (-> zipper (zip/edit inc) zip/root)
4 (+ 2 2)
```

`edit` can also take arguments that are passed to the function (after the value from the tree):

```
1 user=> (-> zipper (zip/edit - 33) zip/root)
2 (+ -32 2)
```

11.7 Zipper functions

Here are some functions you might use in the exercises. You've seen most of them, but a few are new. There are scare quotes around “moving” and “editing” to remind you that these actually return a new zipper.

“Moving”

- **down**: The leftmost child, or `nil` if there are no children.
- **up**: The parent, or `nil` if at the top.
- **right, left**: `nil` if there's nothing there.
- **rightmost, leftmost**: No change if already there.
- **next, prev**: Traverses hierarchy depth-first.

“Editing”

- **insert-left, insert-right**: No change in position.
- **replace**: Takes an arbitrary subtree. No change in position.
- **edit**: Applies a function to the tree at the node and replaces old value with new.
- **insert-child, append-child**: Adds a new child for this branch, at beginning or end of the children. No movement.

- **remove**: Remove current node, backing up (as with `prev`).

Viewing

- **node**: The subtree at the current location.
- **root**: The whole tree, including all changes.
- **children**: A sequence of the children of a branch.
- **lefts, rights**: Siblings of the current node.

Predicates

- **branch?**: At a branch?
- **end?**: True when the recursion has moved off the end of the tree. (*Not* true of the last element.)

11.8 Exercises

These exercises are a continuation of the last set, so the sources and solutions are in the same files: [sources/zipper.clj](#) and [solutions/zipper.clj](#).

Exercise 3

If you're like me, you understand code better after editing it. There's a lot of duplication in `tumult`. Factor it out into helper functions.

I'm rather pleased by my solution, so I suggest you take a look.

Exercise 4

Change `tumult` so that it replaces forms beginning with `*` with forms beginning with `-`. For example, suppose `(* 1 2)` was converted into `(- 1 2)`. Given the predefined transformations for `-`, what would you expect the behavior of the following to be?

```
1 (tumult '(* 1 2))
```

What is it actually? Can you change it to be as you expect?

Exercise 5

My Clojure unit testing tool, [Midje](#), uses the metaphor that the programmer first states facts about the program, checks that the supposed facts are actually lies, then writes the code to make them true. Here are two examples of facts that are already true:

```
1 (facts
2   (+ 1 2) => 3
3   3 => odd?)
```

Notice that the syntax is somewhat unisplike. Another test tool, [Expectations](#), has a more conventional notation:

```
1 (expect 3 (+ 1 2))
2 (expect odd? 3)
```

Internally, Midje works by expanding the arrow forms into something a bit more like Expectations' format. The expanded form of the previous example would look like this:

```
1 (do
2   (expect (+ 1 2) => 3)
3   (expect 3 => odd?))
```

(The arrows are not syntactic sugar; there are different variants for different purposes.)

In this exercise, write a function transform that converts facts into do forms that wrap expect calls.

Note that the arrow forms can appear deeply nested in a fact. For example, they're often inside let expressions. Here's an example from Midje's own test suite:

```
1 (fact "Metaconstants print as their name"
2   (let [mc (Metaconstant. '...name... {})]
3     (str mc) => "...name..."
4     (pr-str mc) => "...name..."))
```

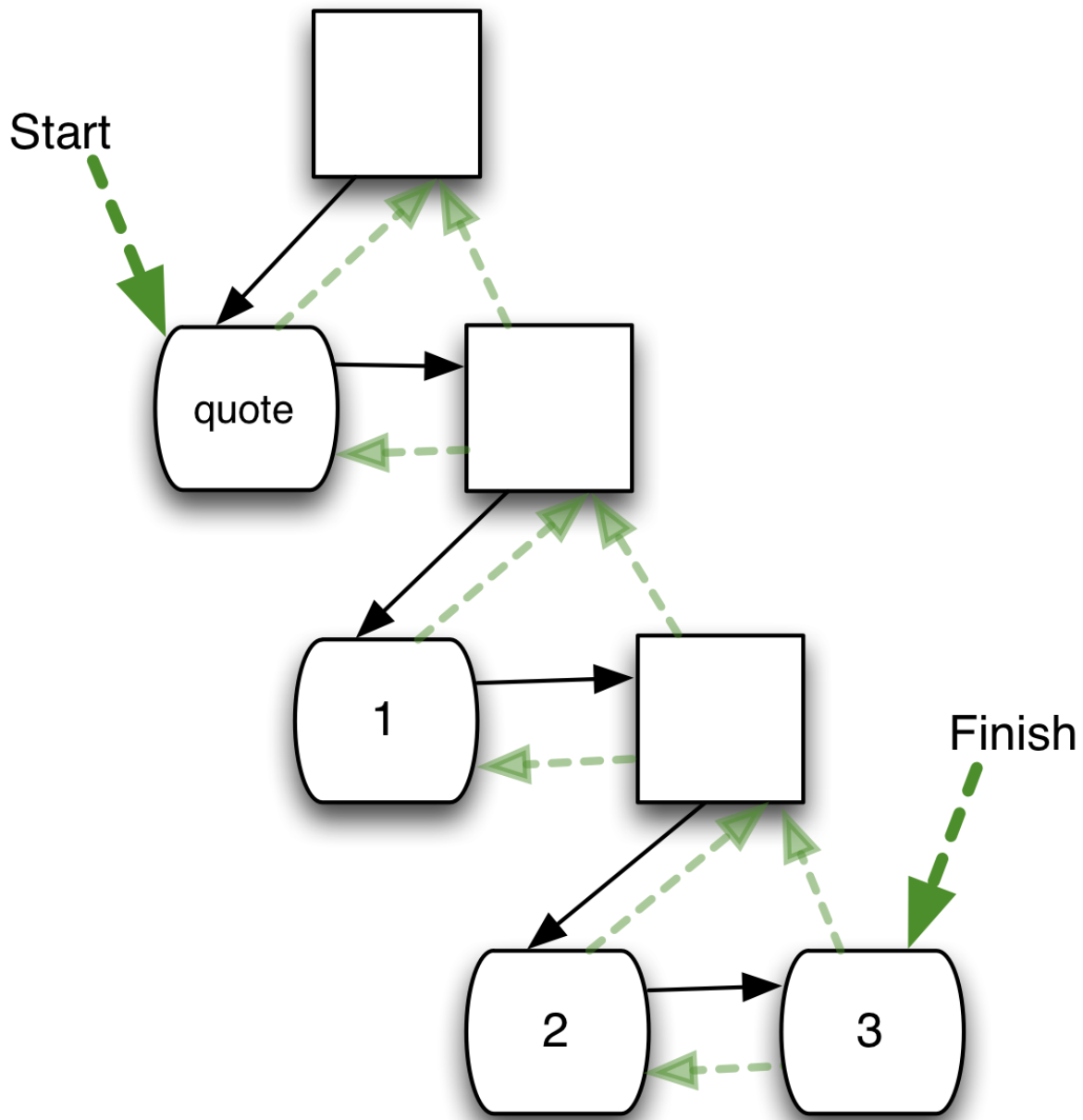

The only exception is that arrows may not be nested inside of either left-hand or right-hand side of an arrow. That is, the following is illegal:

```
1 (fact
2   (3 => odd?) => "what could possibly made sense here?")
```

The implication of that is that once you've transformed an arrow expression, you need not revisit the transformed version.

Exercise 6

Write a utility function called `skip-to-rightmost-leaf`. It's best explained with a picture:



When at the leftmost node in a tree, this moves to the very last leaf before zip/next would take the recursion out of the tree.

Note: skip-to-rightmost-leaf takes a zipper and returns a zipper.

Hint: zip/right is problematic because it can move you off the edge of a subtree (by returning nil if you try to go right when you're already on the rightmost node).

Exercise 7

There is an exception to the rule that Midje arrow expressions can't appear within other arrow expressions. They can if they're quoted:

```
1 (fact
2   (first '((+ 1 2) => 3))
3   => '(+ 1 2))
```

(This is a useful property for testing Midje itself.)

Make your version of transform obey that rule. That is:

```
1 user=> (def tree '(fact
2           (first '((+ 1 2) => 3))
3           => '(+ 1 2)))
4
5 user=> (transform tree)
6 (do (expect (first '((+ 1 2) => 3)) => '(+ 1 2)))
```

Note: remember that 'something is converted by the reader into (quote something), so it's that list form that your code will have to handle.

Hint: This might be trickier than it seems because the final quoted form is the final subtree in the tree.

Hint: The result of the last exercise might come in handy.

Exercise 8

Midje also contains a functional version of mock objects. Rather than faking out objects, Midje's mocks fake out functions. Here's an example of the syntax:

```
1 ;; First a trivial example of code under test
2 (def function-under-test
3   (fn [n] (- (subsidiary-function n))))
4
5 (fact
6   (function-under-test 3) => -88
7   (provided
8     (subsidiary-function 3) => 88))
```

The expansion of that fact looks like this:

```
1 (do
2   (expect (function-under-test 3) => -88
3           (fake (subsidiary-function 3) => 88)))
```

Change transform so that this works. For simplicity's sake, assume that a provided clause only contains one arrow form.

11.9 Thinking about zippers

I can't deny it: compared to direct mutation of trees, zippers are still awkward. That's mainly due to the fact that you get only one pointer into the tree. Instead of something like:

```
1 location.next.remove
2 location.next
```

... you have to capture zip/remove's return value, remember that it might be pointing deep into a subtree, and use the correct function to advance. And so on.

Were I working in a language with mutable data structures, I'd only use zippers when I had a need to look at past states of the tree. But it's comforting to know that I can do something moderately similar to what I'm used to, and in a purely immutable way.

Zippers also demonstrate an interesting bit of functional style: delayed evaluation. Notice the pattern of computation we've seen:

- move
- change
- move
- change
- move
- change
- use zip/root to magically make the edited tree appear

I like to think of code using zippers as the equivalent of writing a function with multiple steps. The final zip/root is like applying that function^{[1](#)}.

There's even an analog to partial application. Suppose you create a zipper and use it to perform a series of editing steps:

```
1 (def zipper (-> (zip/seq-zip tree) zip/down zip/remove ...))
```

That zipper can be passed all over the place. Multiple bits of code could take it, add different operations to the “program”, and create a custom tree with `zip/root`.

This idea of making a clear distinction between setting up a computation and using its results pops up frequently in functional programming. Another sort of data structure, the *future*, is an example.

Futures are my favorite parallelism construct. To show how they work, let me introduce Takeuchi's function, which has the distinction of being *very* recursive and *very* slow:

```
1 (defn tak [x y z]
2   (if (< y x)
3     (tak (tak (dec x) y z)
4         (tak (dec y) z x)
5         (tak (dec z) x y))
6     z))
```

`(tak 20 3 23)` takes two minutes and six seconds on my laptop. However, this takes no time at all:

```
1 (def t (future (tak 20 3 23)))
```

`future` spins off a computation in another thread (presumably running on some underused core) and returns a token that represents the future value of the computation when (and if) it finishes.

I can ask for the value of the future computation immediately, like this:

```
1 user=> (deref t)
```

In that case, the caller must wait for the computation to finish. But I wouldn't use a future if I wanted to so tightly couple the initiation the computation to the use of its results. Instead, I would write code that went

off and did other things. Only when it absolutely needed the results would it ask for them. If the computation had finished, they'd be immediately available.

This is an example of how distinguishing between a computation (as an abstract thing) and its results can produce interesting ideas.

11.10 Implementing zippers

In this section, I'm going to walk you through the creation of a version of the zipper data structure by giving you a sequence of exercises.

My solutions are in [solutions/build-zipper.clj](#). Although they add up to a working implementation of zippers, I didn't make a special effort to be efficient. A real implementation of such a common structure would, of course. [Clojure's](#) is such an implementation.

Exercise 1

The first version of our zipper will be able to do nothing but go down and produce subtrees. It will begin with this no-op data structure, where a zipper is just the original sequence.

```
1 (def seq-zip
2   (fn [tree] tree))
```

Given that, implement `zdown` and `znode` such that the following is true:

```
1 user=> (-> '(a b c)      seq-zip zdown      znode)
2 a
3 user=> (-> '( (+ 1 2) 3 4) seq-zip zdown      znode)
4 (+ 1 2)
5 user=> (-> '( (+ 1 2) 3 4) seq-zip zdown zdown znode)
6 +
```

Exercise 2

If the zipper is to support an “up” operation, it must have some record of what is above the current location. Since we might potentially need to go up several levels, it makes sense to maintain a list of all the parents of the current location. That suggests this data structure:

```

1 (def seq-zip
2   (fn [tree]
3     {:here tree
4      :parents '()}))

```

First, implement znode such that this continues to work:

```

1 user=> (-> '(a b c) seq-zip znode)
2 (a b c)

```

Now create a new version of zdown such that this works:

```

1 user=> (-> '(a b c) seq-zip zdown znode)
2 a

```

Next, create a zup such that this works:

```

1 user=> (-> '(a b c) seq-zip zdown zup znode)
2 (a b c)

```

Moving upward from the root should produce a nil:

```

1 user=> (-> '(a b c) seq-zip zup)
2 nil

```

Note: the first of an empty list is nil.

Moving down into an empty list should also produce a nil:

```

1 user=> (-> '() seq-zip zdown)
2 nil

```

Finally, implement zroot so that it delivers the entire tree:

```

1 user=> (-> '(a) seq-zip
2          zroot)
3 (a)
4 user=> (-> '(((a)) b c) seq-zip
5          zdown zdown zdown
6          zroot)
7 (((a)) b c)

```

Exercise 3

Now let's add lateral movement. That implies that the zipper data structure must keep track of what's to the left and right of the current node:

```
1 (def seq-zip
2   (fn [tree]
3     {:here tree
4      :parents '()
5      :lefts '()
6      :rights '()}))
```

Implement `zright`, and `zleft`. Change any other functions to make the following work:

```
1 user=> (-> (seq-zip '(a b c)) zdown zright znode)
2 b
3 user=> (-> (seq-zip '(a b c)) zdown zright zright zleft znode)
4 b
5 user=> (-> (seq-zip '(a b c)) zdown zleft)
6 nil
7 user=> (-> (seq-zip '(a b c)) zdown zright zright zright)
8 nil
9 user=> (-> (seq-zip '(a b c)) zdown zup znode)
10 (a b c)
```

Exercise 4

Produce a partial implementation of `zreplace` such that replacements work, so long as you don't worry about movement up.

```
1 user=> (-> (seq-zip '(a b c)) zdown zright (zreplace 3)
2         znode)
3 3
4 user=> (-> (seq-zip '(a b c)) zdown zright (zreplace 3)
5         zright zleft
6         znode)
7 3
8 user=> (-> (seq-zip '(a b c)) zdown zright (zreplace 3)
9         zleft zright zright
10        znode)
11 c
```

Exercise 5

Here's the clever part. Whenever a `zreplace` is done on a zipper, that zipper is marked as having been changed:


```

1 (def zreplace
2   (fn [zipper subtree]
3     (assoc zipper
4       :here subtree
5       :changed true))) ;; <==

```

When you move up from a changed node, two things have to happen:

1. The zipper for the destination branch must get a `:here` node that incorporates the `:here` of the changed node.
2. That new zipper itself must be marked as `:changed` so that any movement up will propagate the change. In particular, `zroot` must implement all the changes all the way to the top of the tree.

Here are examples of point 1:

```

1 user=> (-> (seq-zip '(a b c)) zdown zright (zreplace 3)
2           zup
3           znode)
4 (a 3 c)
5 user=> (-> (seq-zip '(a b c)) zdown zright (zreplace 3)
6           zright (zreplace 4) zup
7           znode)
8 (a 3 4)

```

Note that the new behavior for `:changed` does not affect the fact that `zup` returns `nil` if it tries to go too high:

```

1 user=> (-> (seq-zip '(a)) zdown (zreplace 3) zup zup)
2 nil

```

Here are examples of point 2:

```

1 user=> (-> (seq-zip '(a (b) c)) zdown zright zdown (zreplace 3)
2           zroot)
3 (a (3) c)
4 user=> (-> (seq-zip '(a (b) c)) zdown zright zdown (zreplace 3)
5           zup zright (zreplace 4)
6           zroot)
7 (a (3) 4)

```

Make all these examples work.

Exercise 6

Implement `znext`. (I suggest doing it in several steps, one group of examples at a time.)

1. When on a leaf with a rightward neighbor, `znext` moves right.

```
1 user=> (-> (seq-zip '(a b))      zdown znext  znode)
2 b
3 user=> (-> (seq-zip '(a ((b)))) zdown znext  znode)
4 ((b))
```

2. When on a branch, `znext` moves down:

```
1 user=> (-> (seq-zip '(a b)) znext znode)
2 a
```

... except that if the branch is empty, there's no first child for `znext` to move to, so it moves right:

```
1 user=> (-> (seq-zip '(() b)) znext znext znode)
2 b
```

To do make these examples work, you'll need to implement `zbranch?`. Using `sequential?` isn't right, because that's also true for vectors. You want `seq?`, which is true for lists but not vectors.

3. If `znext` needs to move further right and cannot, it should move up and try moving right again.

```
1 user=> (-> (seq-zip '((a) b)) zdown zdown znext znode)
2 b
```

Note that the attempt to move right could again fail, in which case `znext` should try moving up again:

```
1 user=> (-> (seq-zip '(((a)) b)) zdown zdown zdown znext znode)
2 b
```

We won't worry yet about what happens if `znext` "falls off the top".

4. If `znext` can move up no further, it should set a new key, `:end?`, to have the value `true`. That key is what `zend?` should check:

```

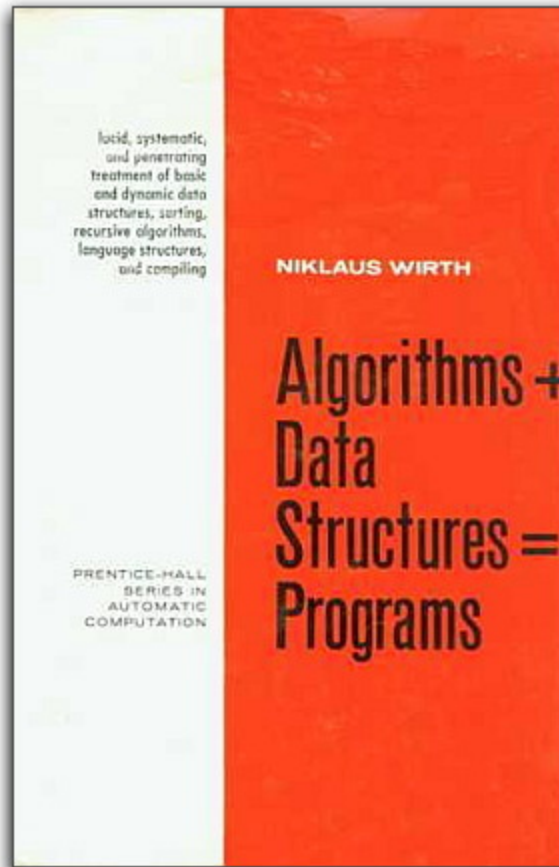
1 user=> (-> (seq-zip '()) zend?)
2 false
3 user=> (-> (seq-zip '()) znext zend?)
4 true
5 user=> (def zipper (-> (seq-zip '(a)) znext (zreplace 5) znext))
6 user=> (zend? zipper)
7 true
8 user=> (zroot zipper)
9 (5)

```

11.11 [The Worm Ouroborous](#)

In the previous exercise², you implemented, save for some details, a functional data structure that was first published by Huet in 1997. There's quite a cottage industry in the invention of functional data structures that mimic the behavior of the sort of data structures you're used to. For example, *finger trees* are a data structure that can be used to implement things like doubly-linked lists (including the property that it's just as fast to reach the last element as the first).

What that means for you is something of a return to an earlier era of software, where conferences would show off [nifty new data structures](#) rather than nifty new non-relational databases, and where books like this one:

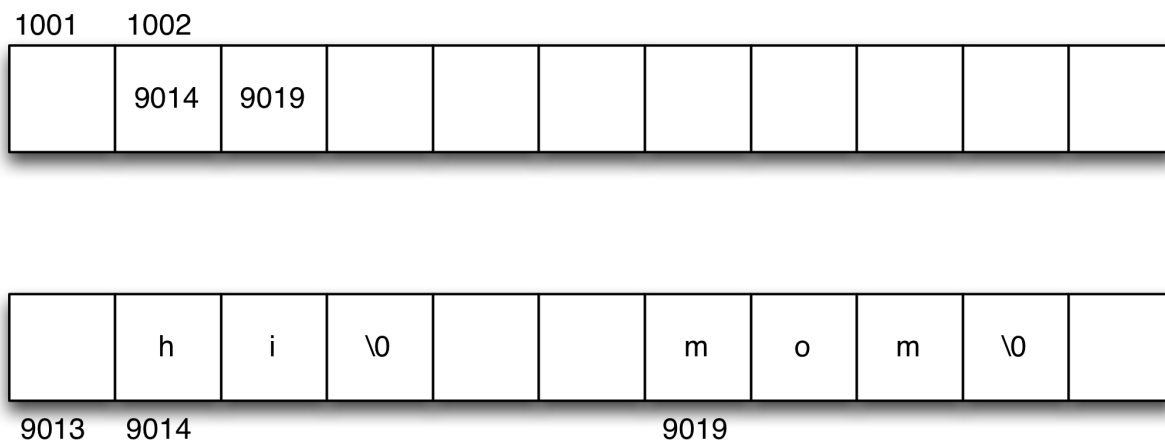


... weren't just for undergraduates but something a professional might read.

1. As you'll see in the next section, that's not a completely accurate metaphor for how zippers really work. zip/up can do some of the work I've attributed to zip/root. But how one thinks about the code is more important than implementation details and optimizations. ↵
2. You *are* still doing the exercises, right? *Right?* ↵

12. Pushing Bookkeeping into the Runtime: Laziness and Immutability

After I got my first post-college job, in 1981, I was a C programmer. C is a *brilliant* language, but it assumes you want to know a lot about your data. For example, if you have the vector of strings ["hi", "mom"], it assumes that you know that what you really have is an integer (1002, say) that's an index into a big 1-dimensional array of memory:



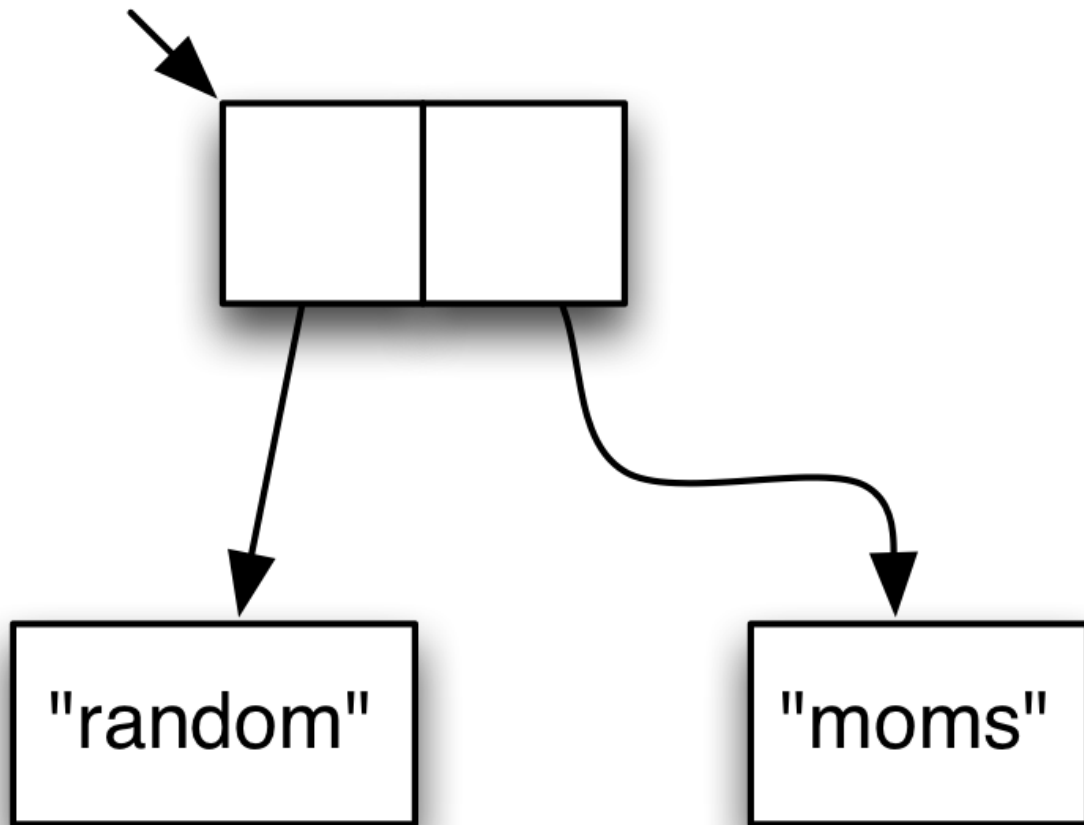
C does not pretend your vector is anything like an object. It's perfectly happy for you to add the integer 1 to it, giving you an index into its interior. You can think of that as a subvector if you like, but that's only your opinion about what a bunch of integers means: there's nothing in the runtime layout of memory to imply that. And if you add another 1, moving you past the "end" of your "two-element" "vector"... Well, best of luck to you if your code decides to interpret that as the index of another string.

C also lets you write new data on top of old. If you do that, it is entirely your responsibility to ensure that there's no code that assumes the locations still contain the old data.

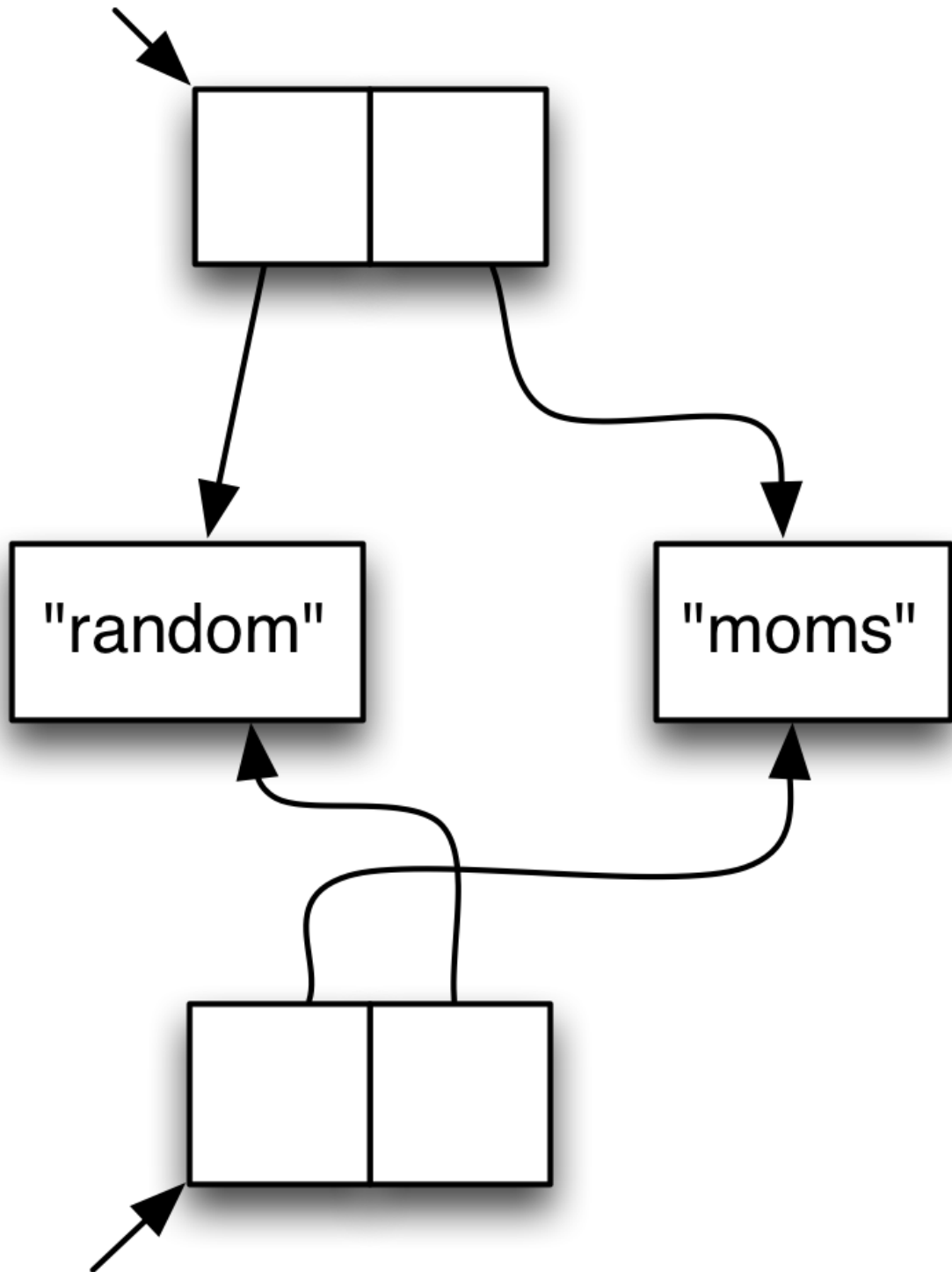
Back in 1981, the idea that garbage collection could be used in real applications was entertained only by lunatics. It took some fifteen years

before Java made garbage collection respectable.

But Java still permits you to make assumptions about your data. You know that a vector of strings looks something like this:



You don't know where things are in memory, and you can't just point into the middle of a vector and call that a new vector, but a Java programmer still knows that if she makes a sorted copy of a vector, the result will be something like this:



She knows other things, too:

- ... that there's no overlap between the old and new vectors. She can change either independently of the other.

- ... that the first element of the original vector is the identically same object as the second element of the sorted vector.
- ... that the copy-and-sort function won't return until it completes creating the new objects.

Functional languages give you even less knowledge of your data. Even more than object-oriented languages do, they hide from you the awkward truth that the story C tells is the true one¹.

In this chapter, I'll reveal more of the truth behind functional data structures.

12.1 Laziness

I've been promising for many chapters to explain why code like this:

```
1 user=> (first (range 0 1000))
```

... isn't crazy. Finally, I deliver on that promise. Before that, let me you remind you why it seems crazy.

We know how Clojure evaluation works. Given the expression above, the first step is to evaluate `first` to yield a function:

```
1 user=> first
2 #<core$first clojure.core$first@528a52b6>
```

Next, the `range` expression would be evaluated to yield a sequence:

```
1 user=> (range 0 1000)
2 (0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27
28 29\
3 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54
55 5\
4 6 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80
81 82 \
5 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105
106 1\
6 07 108 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125
126 1\
7 27 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144 145
146 1\
8 47 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162 163 164 165
```


166 1\
9 67 168 169 170 171 172 173 174 175 176 177 178 179 180 181 182 183 184 185
186 1\
10 87 188 189 190 191 192 193 194 195 196 197 198 199 200 201 202 203 204 205
206 2\
11 07 208 209 210 211 212 213 214 215 216 217 218 219 220 221 222 223 224 225
226 2\
12 27 228 229 230 231 232 233 234 235 236 237 238 239 240 241 242 243 244 245
246 2\
13 47 248 249 250 251 252 253 254 255 256 257 258 259 260 261 262 263 264 265
266 2\
14 67 268 269 270 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285
286 2\
15 87 288 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305
306 3\
16 07 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324 325
326 3\
17 27 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342 343 344 345
346 3\
18 47 348 349 350 351 352 353 354 355 356 357 358 359 360 361 362 363 364 365
366 3\
19 67 368 369 370 371 372 373 374 375 376 377 378 379 380 381 382 383 384 385
386 3\
20 87 388 389 390 391 392 393 394 395 396 397 398 399 400 401 402 403 404 405
406 4\
21 07 408 409 410 411 412 413 414 415 416 417 418 419 420 421 422 423 424 425
426 4\
22 27 428 429 430 431 432 433 434 435 436 437 438 439 440 441 442 443 444 445
446 4\
23 47 448 449 450 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465
466 4\
24 67 468 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485
486 4\
25 87 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504 505
506 5\
26 07 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522 523 524 525
526 5\
27 27 528 529 530 531 532 533 534 535 536 537 538 539 540 541 542 543 544 545
546 5\
28 47 548 549 550 551 552 553 554 555 556 557 558 559 560 561 562 563 564 565
566 5\
29 67 568 569 570 571 572 573 574 575 576 577 578 579 580 581 582 583 584 585
586 5\
30 87 588 589 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605
606 6\
31 07 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625
626 6\
32 27 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645
646 6\
33 47 648 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665

```

666 6\
34 67 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684 685
686 6\
35 87 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702 703 704 705
706 7\
36 07 708 709 710 711 712 713 714 715 716 717 718 719 720 721 722 723 724 725
726 7\
37 27 728 729 730 731 732 733 734 735 736 737 738 739 740 741 742 743 744 745
746 7\
38 47 748 749 750 751 752 753 754 755 756 757 758 759 760 761 762 763 764 765
766 7\
39 67 768 769 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785
786 7\
40 87 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805
806 8\
41 07 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823 824 825
826 8\
42 27 828 829 830 831 832 833 834 835 836 837 838 839 840 841 842 843 844 845
846 8\
43 47 848 849 850 851 852 853 854 855 856 857 858 859 860 861 862 863 864 865
866 8\
44 67 868 869 870 871 872 873 874 875 876 877 878 879 880 881 882 883 884 885
886 8\
45 87 888 889 890 891 892 893 894 895 896 897 898 899 900 901 902 903 904 905
906 9\
46 07 908 909 910 911 912 913 914 915 916 917 918 919 920 921 922 923 924 925
926 9\
47 27 928 929 930 931 932 933 934 935 936 937 938 939 940 941 942 943 944 945
946 9\
48 47 948 949 950 951 952 953 954 955 956 957 958 959 960 961 962 963 964 965
966 9\
49 67 968 969 970 971 972 973 974 975 976 977 978 979 980 981 982 983 984 985
986 9\
50 87 988 989 990 991 992 993 994 995 996 997 998 999)

```

And finally, the function would be applied to yield 0, with all 999 remaining values ignored.

Those evaluation rules are correct, but range doesn't return just any sequence. It returns a *lazy sequence*; that is, an object of a very particular Java type:

```

1 user=> (type (range 1000))
2 clojure.lang.LazySeq

```

Values of type LazySeq are usually called *lazyseqs*. I'll use that terminology except when I'm talking specifically about the Java class that implements

them.

A lazyseq only calculates its first value when some function like `first` asks for it. Therefore, in the previous example, the value of `range` is a lazyseq that stashes the start value `0` away and returns it when `first` asks for it. The value `1` is never calculated unless some code asks for the `first` of the rest of the lazyseq². All 1000 numbers would only be only calculated for code that explicitly asks for each element. `count` would be an example of such a function.

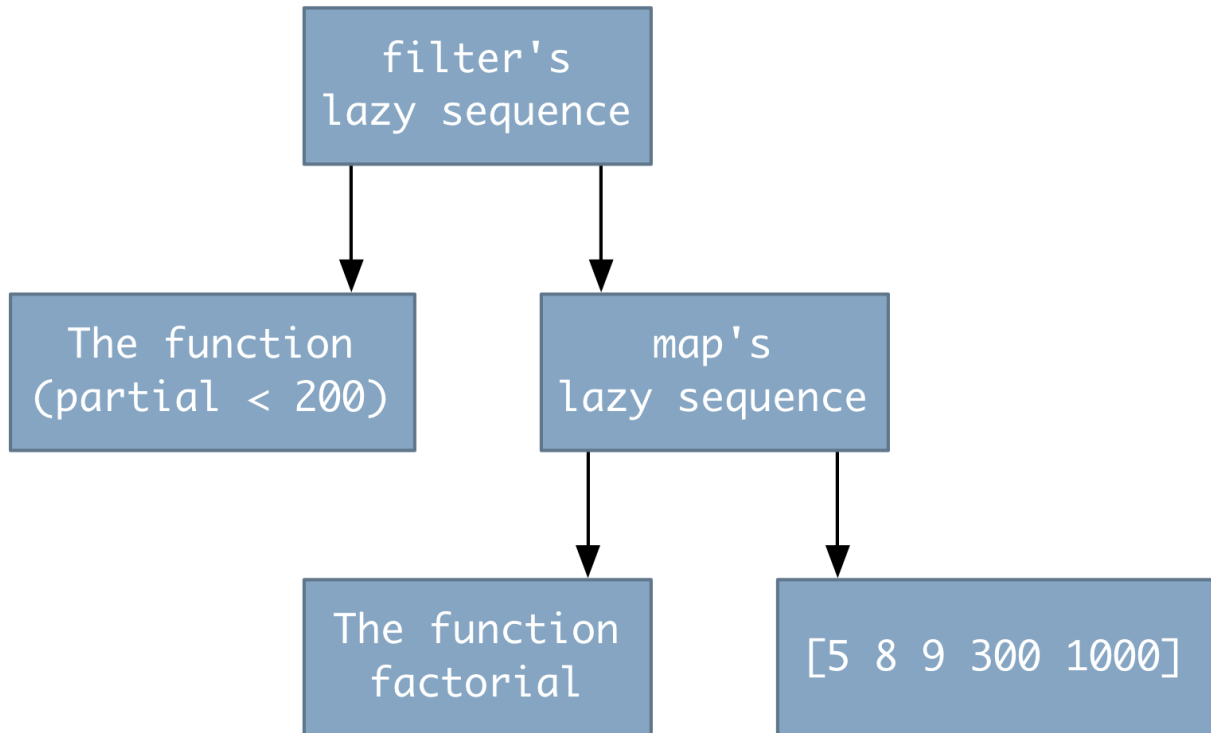
`range` is far from the only function that produces a lazyseq. Here are others you've been using throughout the book:

- `map`
- `filter`
- `take`
- `concat`
- `distinct`
- `keep`
- `interleave`
- `drop`
- `partition`
- `repeat`

Given how common those functions are, most any flow you start with a vector or a list ends up working with lazyseqs pretty quickly. Consider a function like this:

```
1 user=> (def some-numbers
2         (filter (partial < 200)
3                 (map factorial [5 8 9 300 1000])))
```

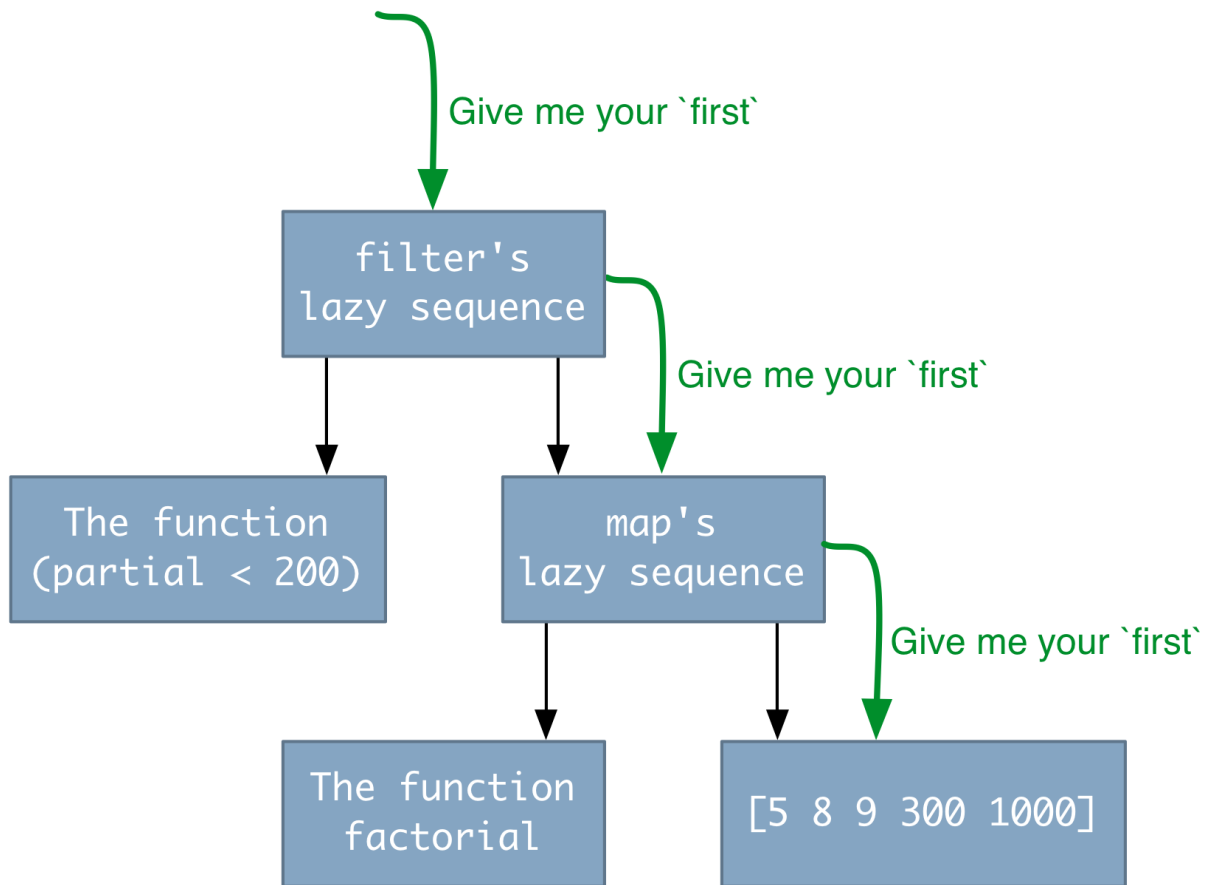
`map` initially does nothing with its arguments except produce a lazyseq. `filter` does the same: it produces a lazyseq that just points to the lazyseq it got from `map`. So no real computation happens here, just the creation of some data, which I'll represent like this:



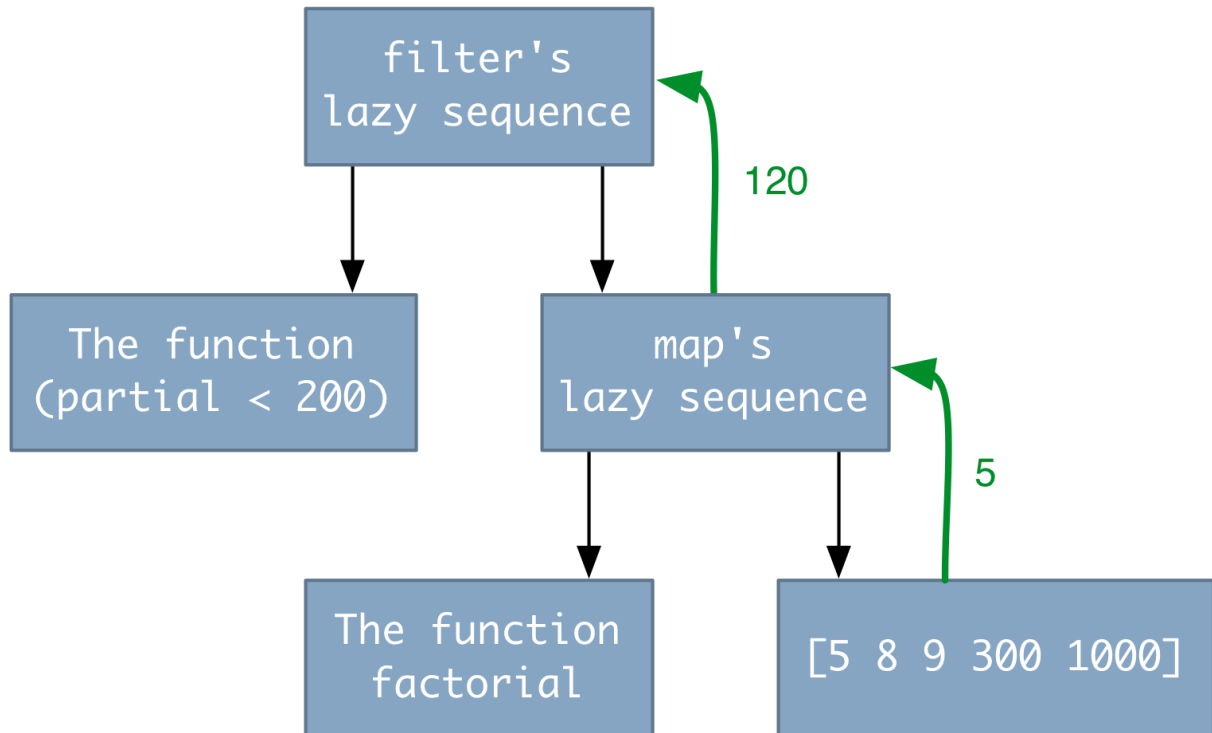
Now, let's execute this code:

```
1 user=> (first some-numbers)
```

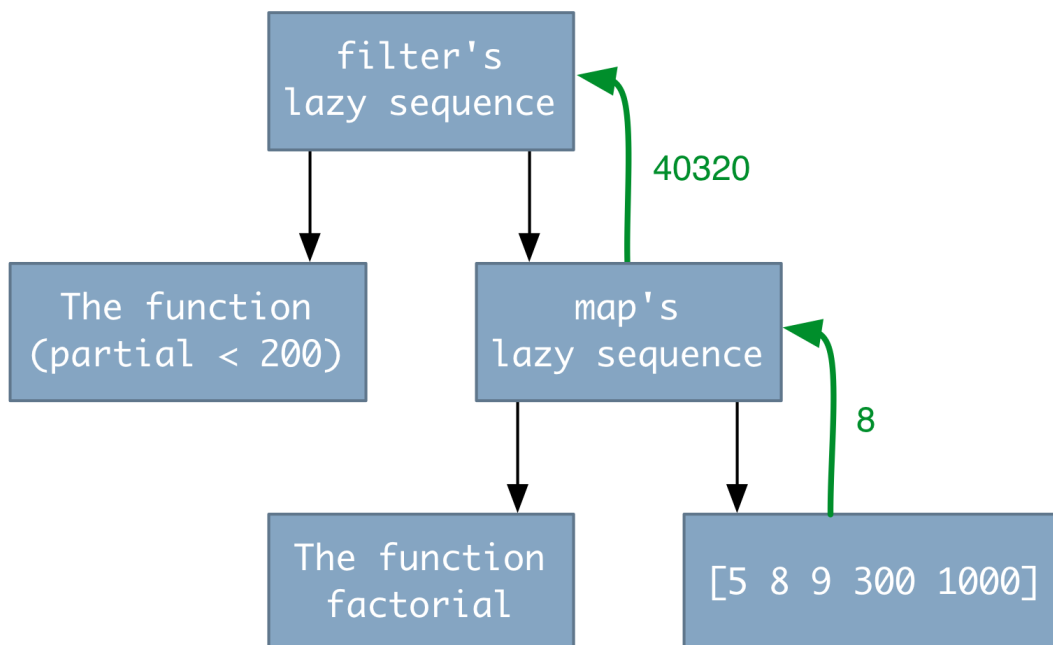
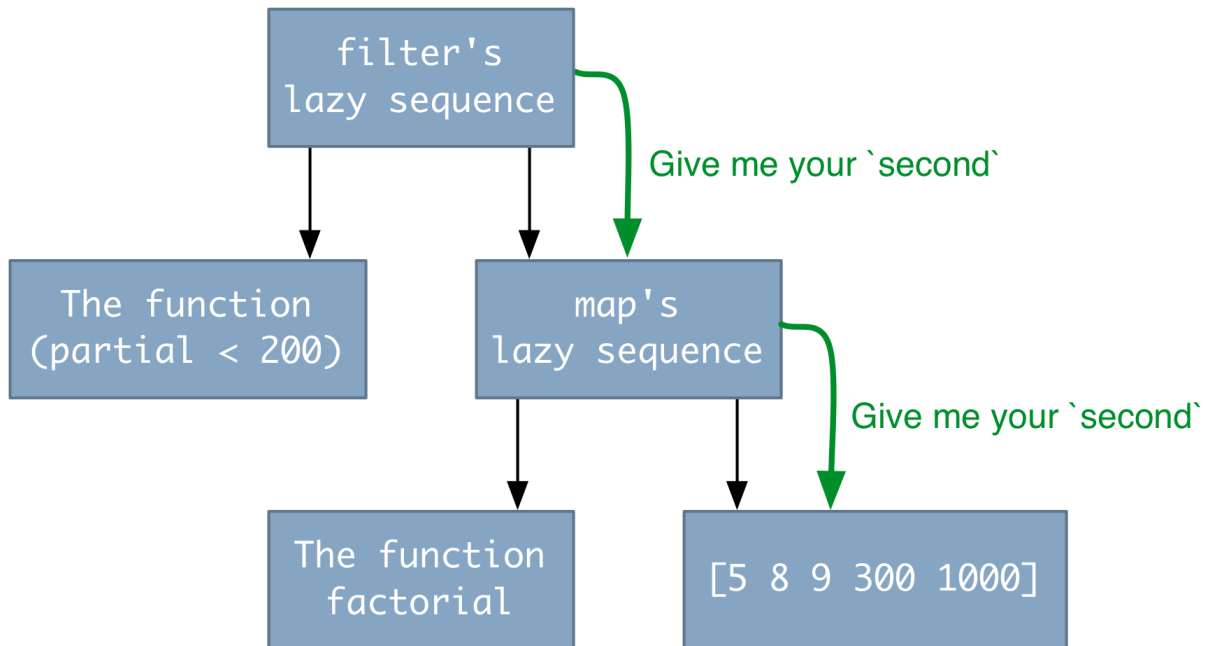
`first` asks the `filter lazyseq` for its first value. In order to calculate that, `filter` must ask the `map lazyseq` for its first value. In order to calculate that, `map` needs the first value of its sequence argument. Here's the picture so far:



Because map's sequence argument is a vector, the first value is readily available. map applies factorial to get 120, which it gives to filter:



120 doesn't pass filter's predicate (since $(< 200 \ 120)$ is false), so filter asks for the map's second element, which causes another flow of function calls down the hierarchy and values up it:



Since 40320 passes the filter, filter returns it to first. And the calculation is complete.

It's important to note that this sequence of calculations has collapsed what appears to be two loops (one for mapping factorial and one for filtering)

into what amounts to a single loop with an early exit:

```
1 for (int number in [5, 8, 9, 300, 1000]) {
2     int result = factorial(number);
3     if (number > 200) return number;
4 }
```

12.2 Infinite data

If `range` is not given a stopping argument, it produces a lazyseq of unbounded length. Here's a sequence of all the integers, starting with 0.

```
1 (def integers (range))
```

Here's a sequence of all the integers, starting with 1.

```
1 (def one-and-so-forth (map inc integers))
```

It would be unwise to try to print either of those sequences (since the printer has to produce each of the infinite number of integers). But as long as you avoid functions like printing, such infinite lists can come in handy.

In the [chapter on higher-order functions](#), you wrote code that checked the validity of ISBNs and other “self-checking numbers”. Here was one of those functions:

```
1 (def check-sum
2     (fn [sequence]
3         (apply + (map *
4                     (range 1 (inc (count sequence))) ; <== look here
5                     sequence))))
```

Notice that it goes to some trouble to make the second argument to `map` the correctly-sized sequence of integers. But that's unnecessary. Since `map` only processes until it runs out of elements in any of its sequence arguments, we can use an infinite lazyseq to simplify the code:

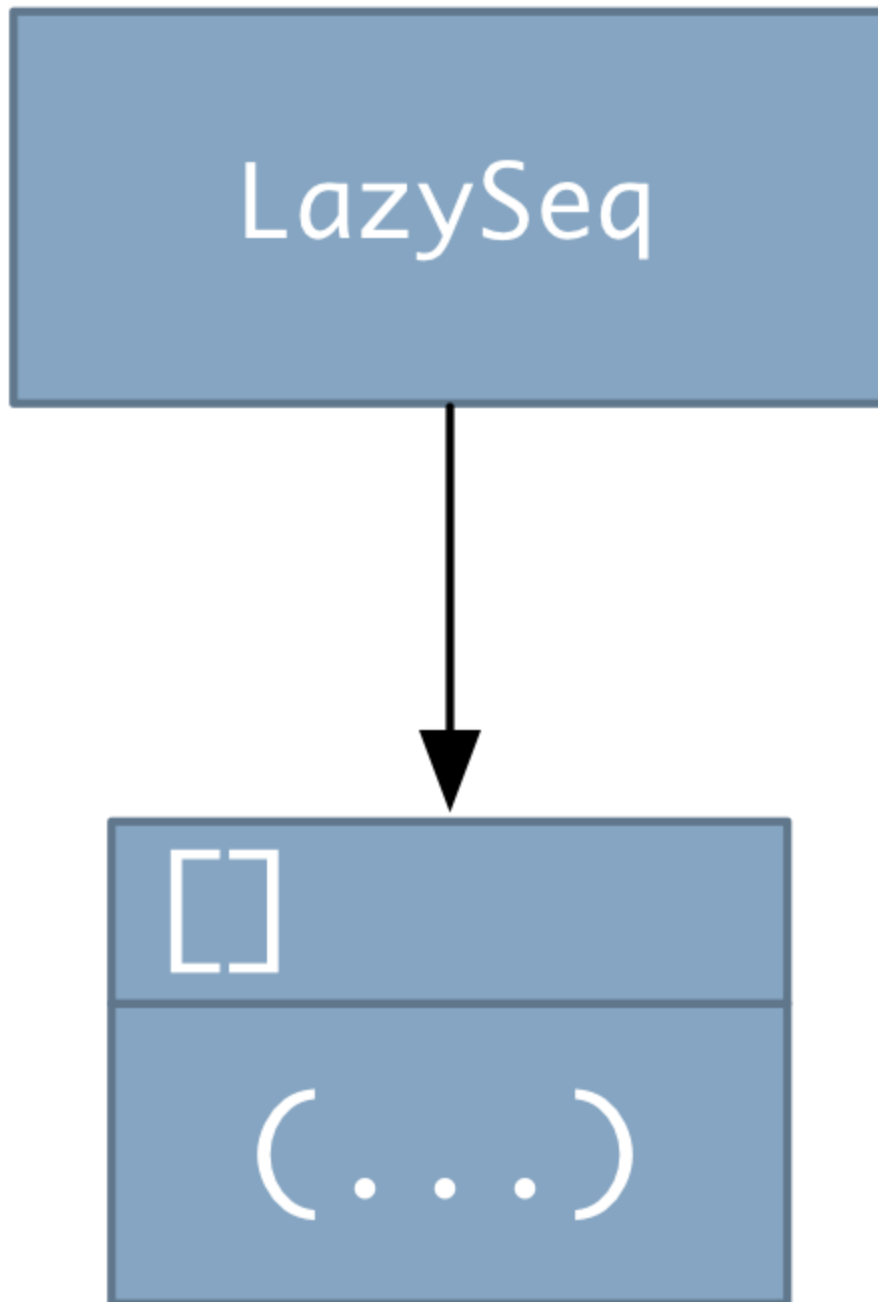
```
1 (def check-sum
2     (fn [sequence]
3         (apply + (map *
4                     one-and-so-forth ; <== here
5                     sequence))))
```


This exemplifies something that laziness gives you: you have the freedom to (appear to) generate *way* too much data, confident that only the amount actually needed will be created. That can eliminate tedious and error-prone bookkeeping.

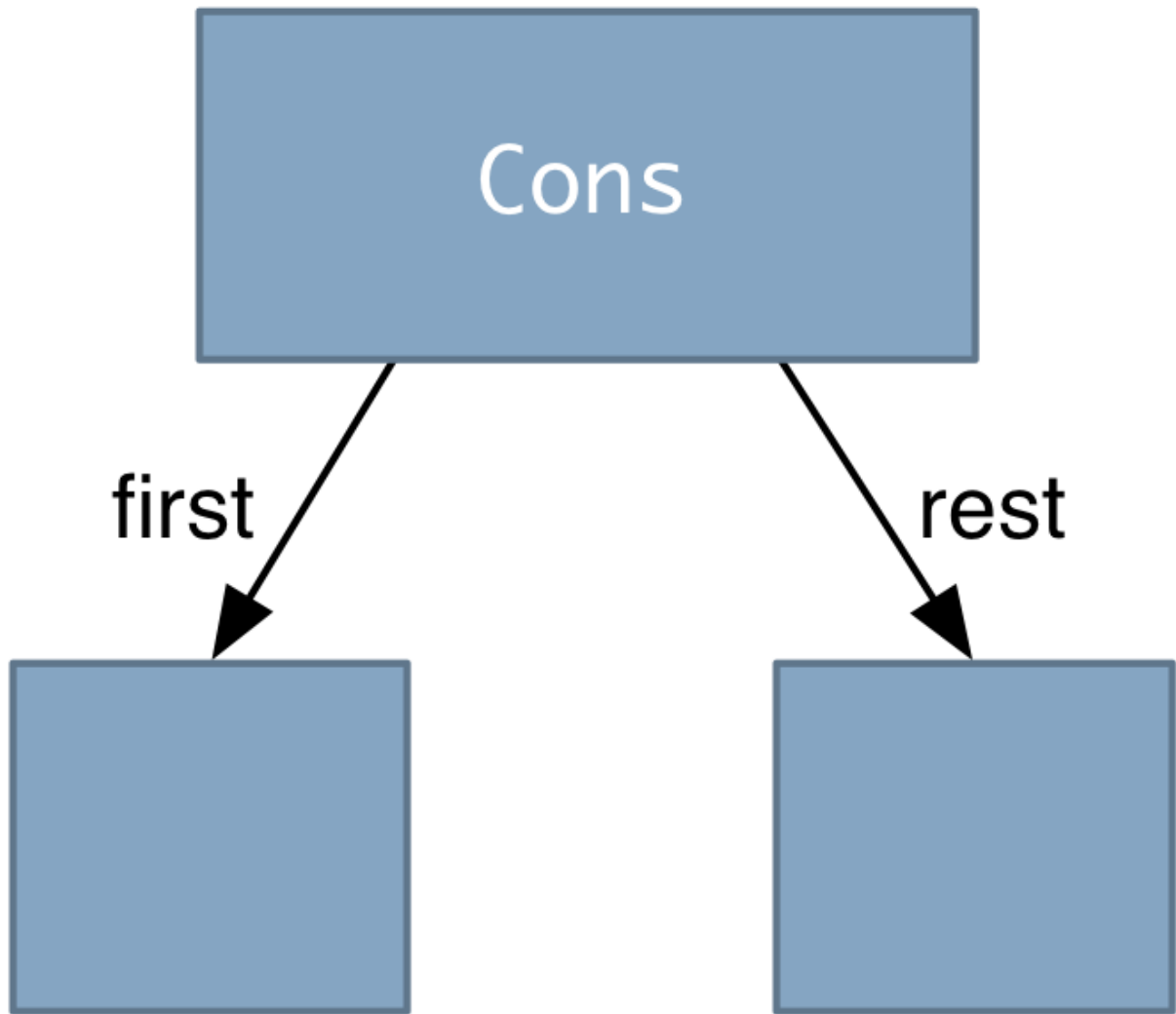
12.3 Implementing your own lazy sequences

To get a clearer idea of how lazyseqs work, it's helpful to see one being built, so in this section I'll implement a version of `range` called `rrange`.

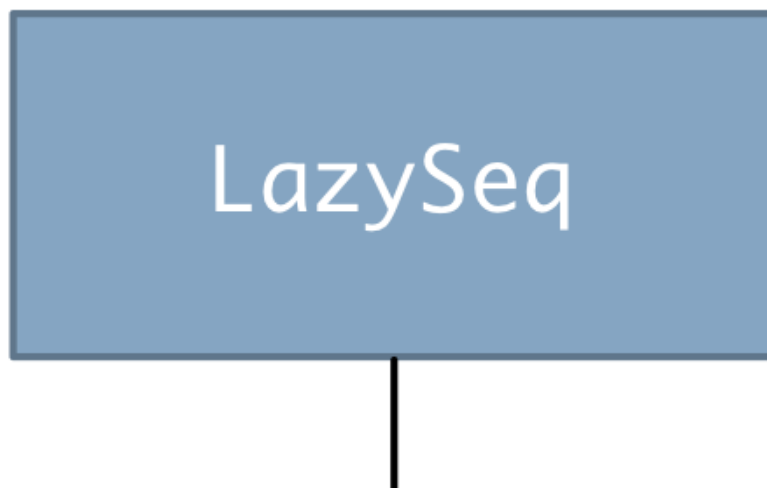
In its initial state, a lazyseq (that is, a `clojure.lang.LazySeq` object) holds a zero-parameter function:

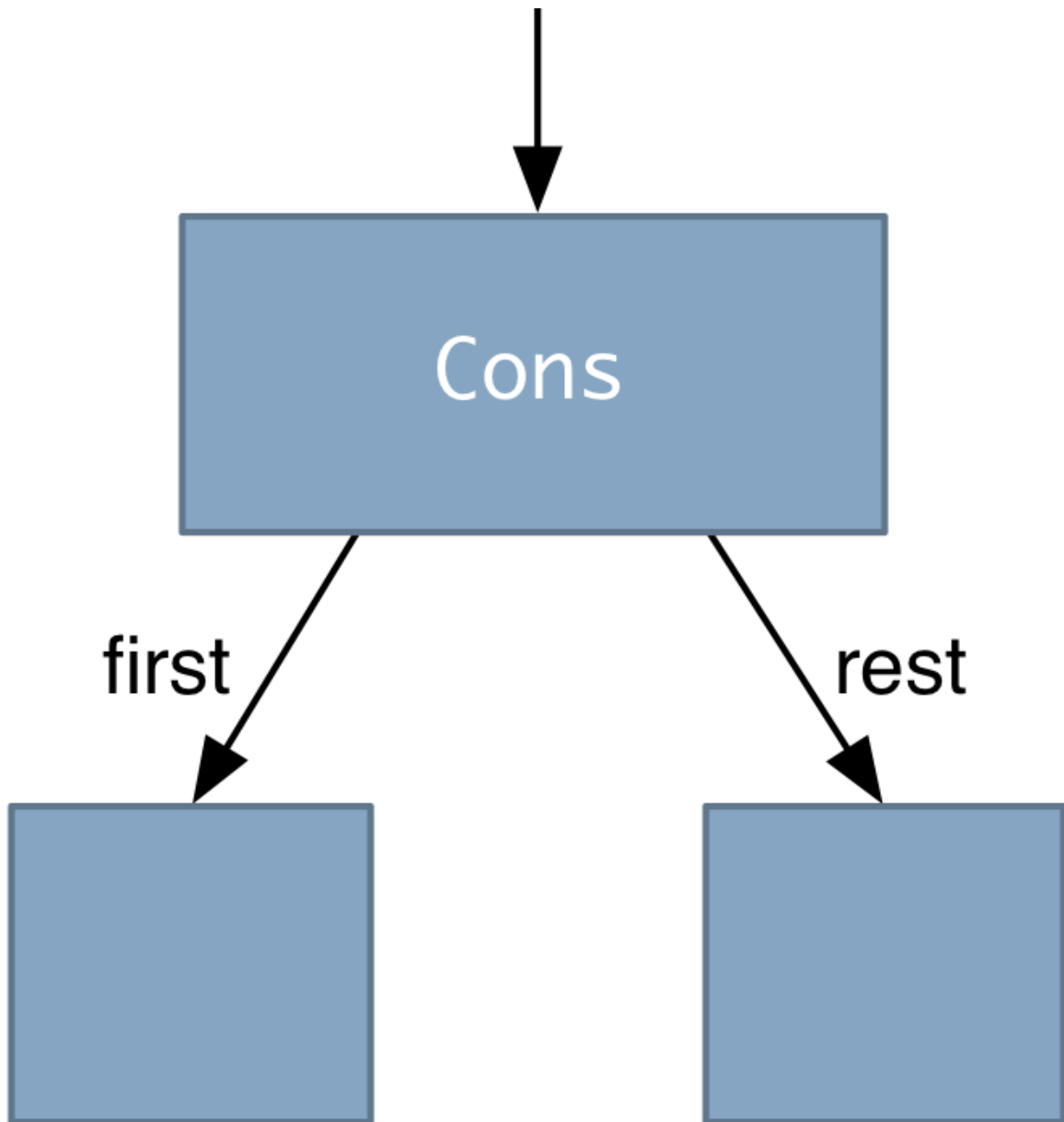


Clojure's first function sends a message to the LazySeq's first method, which calls the function. There are a number of possible objects it can return. A typical choice would be a `clojure.core.Cons` object, which happens to be the Java object the Clojure `cons` function produces. As a sequence, a Cons has a `first` and a `rest`, both of which are instance variables in the object:

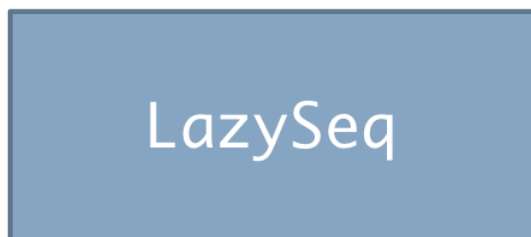


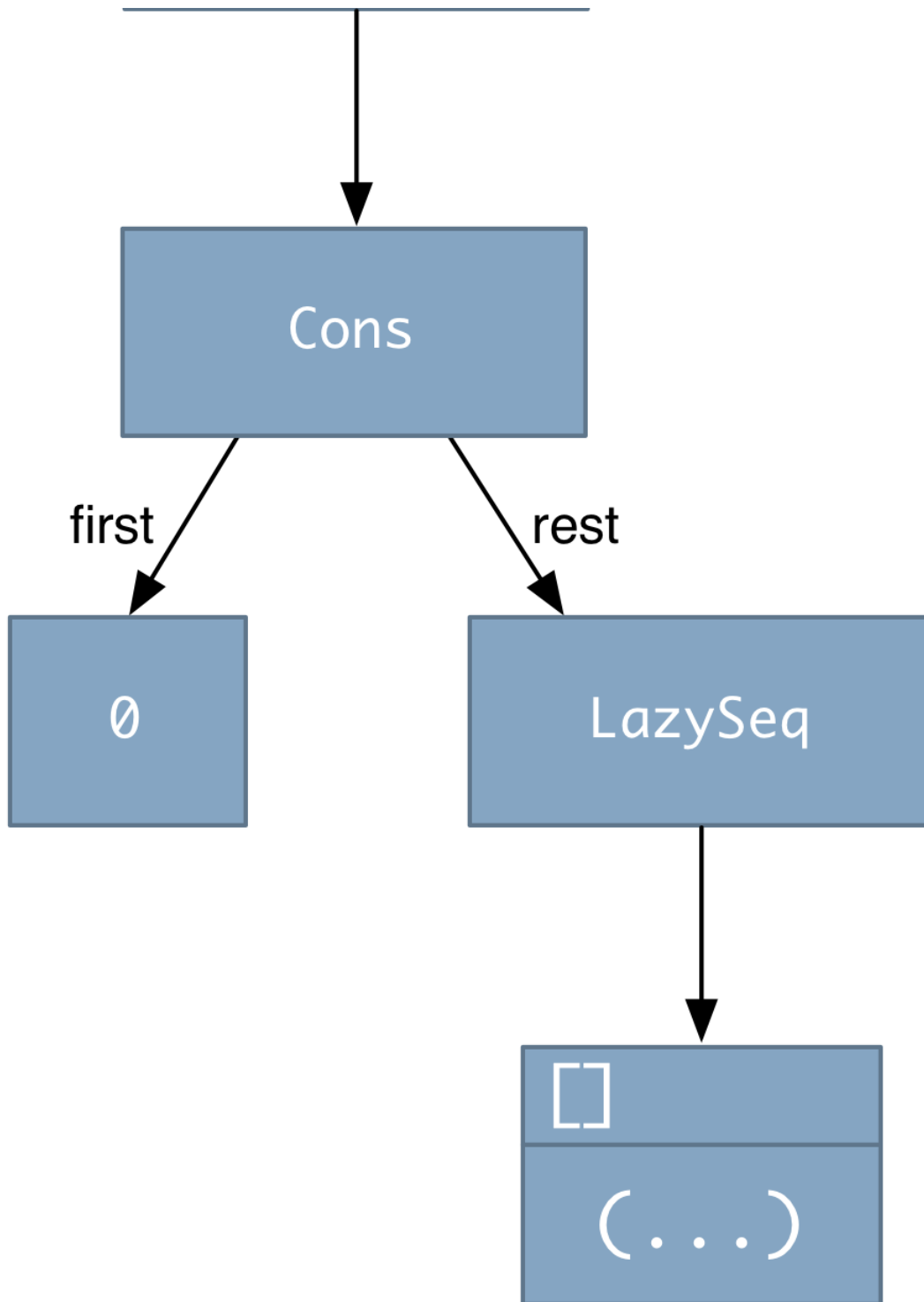
At this point, the function is discarded, and the LazySeq is changed so that it points to the Cons:





What does the cons contain? The first element is the first element of the range. In the case of our running example, that would be 0. The second is a LazySeq that will (via its attached function) calculate the next element of the sequence:





Let's turn those pictures into code. Since `rrange` returns a new Java object, it must new it up³:

```
1 (def rrange  
2   (fn [first past-end]
```

```
3      (new clojure.lang.LazySeq ...)))
```

LazySeq's constructor takes a function as its argument:

```
1 (def rrange
2   (fn [first past-end]
3     (new clojure.lang.LazySeq
4       (fn [] ...)))) ; <==
```

That function returns a Cons whose first element is the value passed in as first:

```
1 (def rrange
2   (fn [first past-end]
3     (new clojure.lang.LazySeq
4       (fn []
5         (cons first ...)))))) ; <==
```

What is the rest of that cons? It's a lazyseq whose function calculates (inc first). That is, it's a recursive rrange:

```
1 (def rrange
2   (fn [first past-end]
3     (new clojure.lang.LazySeq
4       (fn []
5         (cons first
6               (rrange (inc first) past-end)))))) ; <==
```

Oops. I forgot to use the past-end argument. A LazySeq indicates that the sequence is complete by returning nil:

```
1 (def rrange
2   (fn [first past-end]
3     (new clojure.lang.LazySeq
4       (fn []
5         (if (= first past-end) ; <==
6           nil ; <==
7           (cons first
8                 (rrange (inc first) past-end)))))))
```

Does that work? Why of course!

```
1 user=> (rrange 0 4)
2 (0 1 2 3)
3 user=> (type (rrange 0 4))
```

```
4 clojure.lang.LazySeq
5 user=> (first (rrange 0 4))
6 0
7 user=> (type (rest (rrange 0 4)))
8 clojure.lang.LazySeq
```

Notice one implication of this arrangement: such a lazyseq only calculates its first element once. If it is again asked for its first, it simply retrieves the already-calculated value. So lazyseqs are objects that use [lazy initialization](#).

12.4 Exercises

You can use `rrange` as a model for these exercises. It's in [sources/lazy.clj](#). My solutions are in [solutions/lazy.clj](#).

Exercise 1

Implement `map` using `clojure.lang.LazySeq`. You only need to implement the version that takes a single sequence.

Exercise 2

Implement `filter` using `clojure.lang.LazySeq`.

Exercise 3

Convert your implementation of `filter` into an *eager* version. (An eager sequence function is one that calculates all the elements of the result sequence before returning it.) How do your lazy and eager versions differ?

Since it's easy for sequence functions to “inherit” laziness from functions they use, `sources/lazy.clj` contains an `eager?` function you can use to check your work:

```
1 user=> (eager? filter)
2 false
3 user=> (eager? ffilter) ; my solution to the previous exercise
4 false
5 user=> (eager? eager-filter)
6 true
```

As you'll see, there's a close family resemblance between a recursive version of a function and a lazy version. Once you know how to write the recursive version, you can easily write the lazy one.

12.5 Turning the external world into a lazy sequence

Here's a function that prints a prompt, reads a line, and returns that line as a string:

```
1 (def prompt-and-read
2   (fn []
3     (print "> ")
4     (.flush *out*)
5     (.readLine
6       (new java.io.BufferedReader *in*))))
```

If you're curious, the `.readLine` notation is how you call a Java method in Clojure. In general, if you wrote this in Java:

```
1 object.message(arg1, arg2);
```

... you'd write it like this in Clojure:

```
1 (.message object arg1 arg2)
```

But those details won't be important in this book.

Our new function would be used like this⁴:

```
1 user=> (prompt-and-read)
2 > 0 wad some Pow'r the giftie gie us. To see oursels as ithers see us!
3 "0 wad some Pow'r the giftie gie us. To see oursels as ithers see us!"
```

`repeatedly` is a function that returns a lazyseq of calls to its single zero-argument function. So we can make a lazy list of inputs:

```
1 user=> (def inputs-starting-now (repeatedly prompt-and-read))
```

It would be unwise to try to print that sequence, since it never ends, but we can print just the first four values:

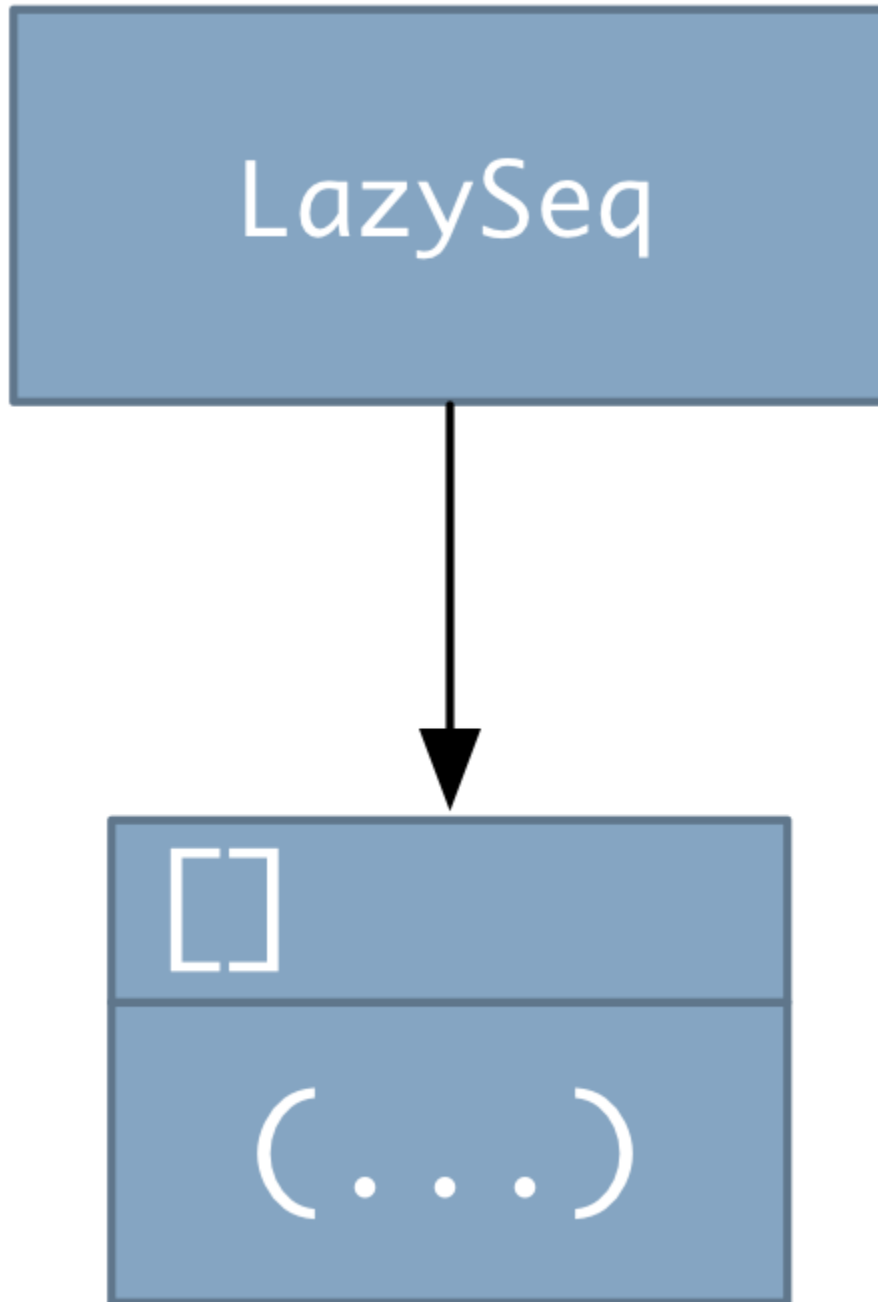

```
1 user=> (take 4 inputs-starting-now)
2 (> 1
3 > 2
4 "1" > 3
5 "2" > 4
6 "3" "4")
```

That's interestingly garbled output. What accounts for it?

Recall that the repl is basically an endless repetition of this code:

```
1 (print (eval (read *in*)))
```

`eval` executes the `take` expression. `take` returns a lazyseq that `print` is to print. At that point, nothing has been done to the lazyseq but setup. That is, `print` has just been given this:



What does print do? The first thing is to check the type of its argument to see how to print it. A lazyseq is printed just like a list, so print prints a parenthesis before even fetching the first value. Which looks like this:

```
1 user=> (take 4 inputs-starting-now)
2 (
```

Since it's printing a sequence, `print` now asks for its first element. Only now is the lazyseq triggered to evaluate its function. That means printing the prompt and reading what's typed. The state of your screen now looks like this:

```
1 user=> (take 4 inputs-starting-now)
2 (> 1
```

Because of the particulars of its implementation, the printing mechanism won't print the first element before checking if the second exists (and that check forces the second element to be read). That's why you see this:

```
1 user=> (take 4 inputs-starting-now)
2 (> 1
3 > 2
4 "1"
```

And before the printer prints the second element, it checks if the third exists:

```
1 user=> (take 4 inputs-starting-now)
2 (> 1
3 > 2
4 "1" > 3
5 "2"
```

And so on. When the sequence is complete (when the final `LazySeq`'s function produces `nil`), the remaining elements are printed.

The precise details are of no great interest. The more important point is that debugging-via-print-statements can get confusing in languages with lazy evaluation. In Clojure, one way to avoid that confusion is through the use of the `doall` function. If you give it a lazyseq, it forces all elements to be calculated before it returns. Here's how it's used:

```
1 user=> (doall (take 4 inputs-starting-now))
2 ("1" "2" "3" "4")
```

Did you find that result surprising? Did you expect prompts? That's an easy mistake to make. Remember that a `LazySeq` only applies its element-producing function once. After that, the cached value is returned. We

already gave the four values to four prompts, so `(take 4)` causes no new evaluations.

To cause a new prompt, we'd have to ask for the next unused element:

```
1 user=> (take 5 inputs-starting-now)
2 ("1" "2" "3" > 5
3  "4" "5")
```

Notice how all this “first this happens, then that happens, and if it happens again it doesn’t really happen” business seems clunkier than the use of `lazyseqs` earlier in the book (where I hope you never even noticed a difference between a `lazyseq` and a plain old list)? Why this sudden awkwardness? This is our first use of *mutable state* and functions with *side effects*⁵ (specifically, I/O). A functional enthusiast would claim that such awkwardness is *inevitable* when you have mutable state. We’re just noticing it because we’ve been free of it throughout the book. We’re like a friend of mine who had a job that made him fly across the Atlantic every week. He claimed he never got jet-lagged until he stopped flying and realized that, no, he’d *always* been jet-lagged.

I think that overstates the case, but I have to admit that mutable state seems odder and odder the more I use Clojure.

But let’s leave clunkiness aside and observe what we’ve done:

```
1 (def inputs-starting-now (repeatedly prompt-and-read))
```

We have abstracted an unbounded sequence of I/O actions into a *data structure*. Moreover, it’s a data structure that represents both the past—which is known and recorded and easily accessible via, say, `nth`—and also the future—which we can access, though only by stepping linearly through it. (That is, we can’t skip to the 100th input without stepping through the preceding 99. No time travel into the future.)

I think that’s just the bee’s knees... neat-o, daddy-o... cool... groovy... *legit*. I hope you do too.

12.6 Exercises

Starting code for these exercises is in [sources/lazy-world.clj](#). Except for the first exercise, my solutions are in [solutions/lazy-world.clj](#).

Exercise 1

Try the following code:

```
1 user=> (def inputs (repeatedly prompt-and-read))
2 user=> (def one-character-line? (fn [line] (= (count line) 1)))
3 user=> (def singles (filter one-character-line? inputs))
4 user=> (first singles)
5 > lkajsdflkjdsflkjlsdkjf
6 > 1
7 "1"
```

Now let's reset the inputs like this:

```
1 user=> (def inputs (repeatedly prompt-and-read))
```

What do you expect the result of the following to be? What do you actually see? Can you explain it?

```
1 user=> (first singles)
```

My solution to this exercise has pictures, so you can see it at the end of this section.

Exercise 2

Using `prompt-and-read`, create a lazyseq named `ys-and-ns` that represents an infinite list of inputs that begin with either “y” or “n”. Typing anything else produces new prompts until you type the right thing.

You can use `java.lang.String`'s `.startsWith` method to check your strings. Call it like this:

```
1 (.startsWith string "y")
```

Test your solution like this:

```
1 (take 2 ys-and-ns)
```

Exercise 3

Consider this function (which you can find in `sources/lazy-world.clj`):

```
1 (def counted-sum
2   (fn [number-count numbers]
3     (apply +
4             (take number-count
5                   numbers))))
```

It adds up the first `number-count` elements of `numbers`.

Write a version of `counted-sum` that takes no arguments. Instead, the `number-count` is the first number from the input. That value governs how many numbers are read and added. Strings that don't represent numbers (like "hi") are to be rejected (just as non "y"/"n" strings were in the previous exercise). Example:

```
1 user=> (counted-sum)
2 > oops
3 > 3
4 > Me encanta Dawn
5 > 100
6 > J'aime Dawn
7 > 200
8 > 300
9 600
```

To avoid wasting time on inessentials, use these two functions from `sources/lazy-world.clj`:

number-string?: returns true if Java can parse the string into an integer.

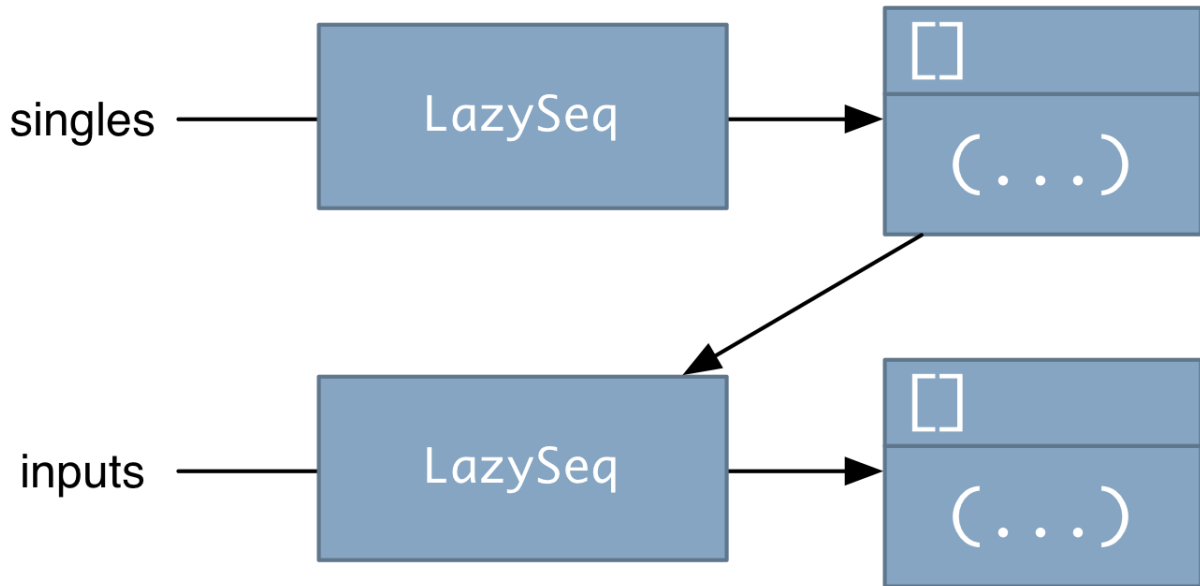
to-integer: converts a valid string into an integer.

Hint: How would you write an infinite sequence of syntactically valid strings?

Hint: Given an infinite sequence of syntactically valid strings, how would you write a sequence of integers?

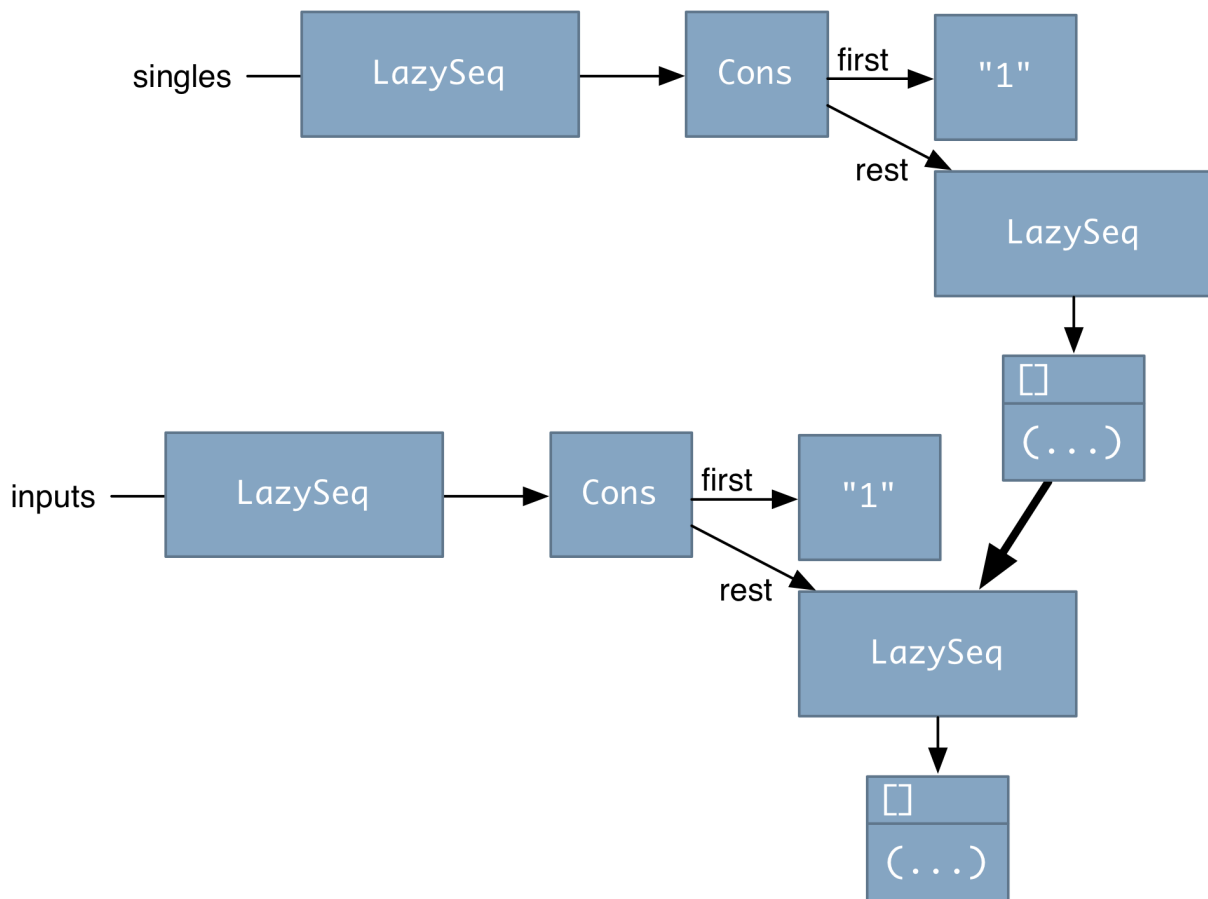
Solution to Exercise 1

Before the first input is read, the structure of the sequences looks like this:



The arrow pointing down from `singles` function to the `inputs lazyseq` is because that function must have a reference to `inputs` in order to ask it (via `first`) for its first element.

In order to produce its first value, the `singles` function must ask for the first element of the `inputs lazyseq`. That produces a structure like the following, which I explain in some detail below:



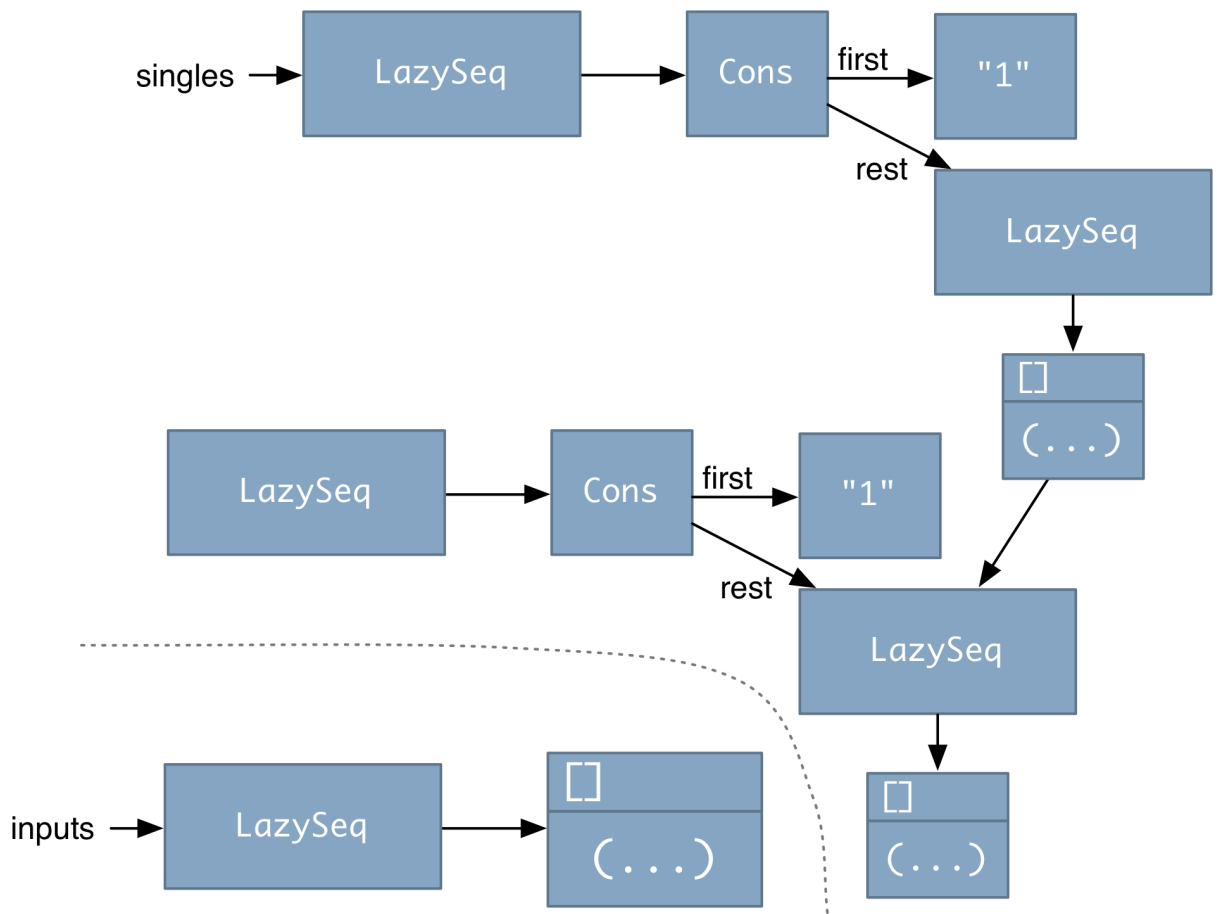
In response to a request from `singles`, the `inputs lazyseq` executed its function, producing a `Cons` that points to a cached value of the first string read—and also to a function that, when called, will produce the next `Cons` in the sequence. Because the whole `inputs` sequence was created with `repeatedly`, the same function can always be reused to create the next element.

When the `first` value is returned to the `singles LazySeq`, that object also caches the value in a `Cons`. That `Cons` also points to a function that can produce a next sequence value. In this case, though, that function cannot be the same as the one that created it. It must point to the `inputs lazyseq` that produces its second value.

The problem in this exercise was: what happens when you rest `inputs` with the following?

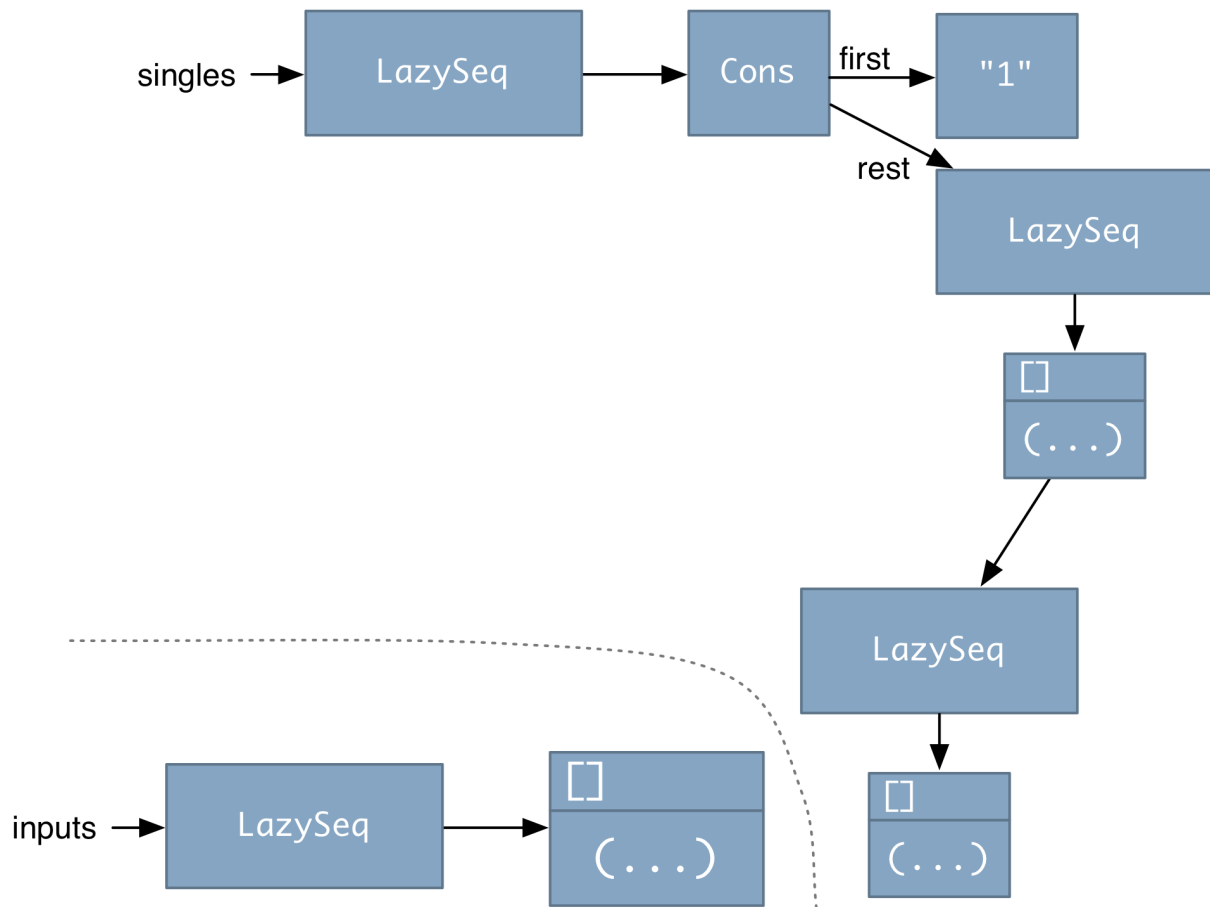
```
1 (def inputs (repeatedly prompt-and-read))
```


The effect of that form is to bind a new lazyseq to inputs. The new lazyseq is completely independent of the old. That situation looks like this:



You can see that retrieving the `first` value of `singles` will still return `"1"`. What `inputs` is bound to has no effect. (Such are the perils of code with state.)

One more small note: Since nothing points to the original `inputs` lazyseq, it will be garbage-collected away:



12.7 Immutability through structure sharing

I have to admit it: I worry about wasting memory. My first post-college programming job was on a computer that had 65,536 16-bit words, and it seems that warped me for life. A non-obsessive person would just say this about immutability:

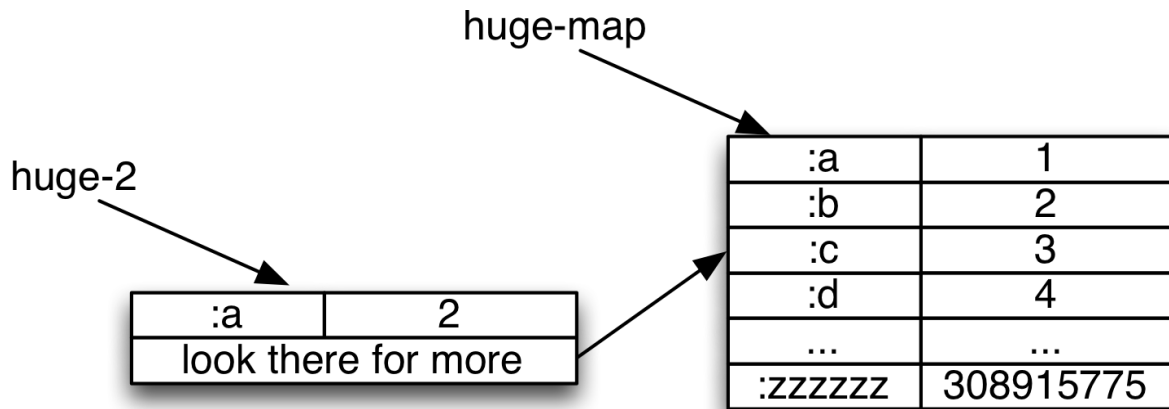
“Functional languages go to considerable effort to make sure that code like this:

```

1 (def huge-map {:a 1, :b 2, ... :zzzzzz 308915775})
2 (def huge-2 (assoc huge-map :a 2))

```

... does not in fact copy a 308,915,775-element array just to change a single entry. Instead, an operation like the above would produce something similar to this:



... except more cleverly laid out in a tree structure so that (in Clojure's case) lookup is logarithmic (base 32) for both maps and vectors."

In fact, I think I *will* say that. The algorithms are complicated enough that I don't care to learn them, and it would take too much time to explain them, so I'll force myself to operate on faith—and I think you should too. If you're not, there's a vast literature on efficient data structures for functional languages. For example, Clojure's `PersistentHashMap` data structure uses Phil Bagwell's Hash Array Mapped Trie, which you can find out about [here](#) and [here](#). The textbook on the topic is *Functional Data Structures* by Chris Okasaki.

Have at it!

12.8 Implications for the object-oriented programmer

What does this chapter mean for you?

I think the most important thing is that immutability and laziness are at least desirable: if you don't have to give up much to get them, why not?

Many languages have libraries that implement functional data structures. For example, Java has the [Functional Java](#), [totallylazy](#), and [Guava](#) libraries and a superset language, [Scala](#) (as well as embedded Clojure), Python has [funktown](#) and [pysistence](#), and Ruby has [Hamster](#). These are—at least—worth experimenting with.

Even without the libraries, you can still program in a functional style. For example:

1. In Ruby, you could use the build-in Hash class but simply always use merge (to create a copy with a difference) instead of []= (to modify a hash in place). In Java, you might work with your own subclass of HashMap that implements a copy-and-putAll version of merge. When the profiler tells you you have a bottleneck, optimize by using well-encapsulated mutation.
2. If you organize much of your program as a series of [flow functions](#) that mutate the structures that pass through them, I believe you'll be almost as safe as if the functions made copies. Since such functions only depends on their inputs, they won't break because of some unexpected change to shared state.

It's more awkward, I think, to simulate laziness. You can pass functions around instead of objects, or (in languages without functions) you can pass around small custom objects that are lazily initialized in response to their single public run method. But these are awkward because the consumer must explicitly trigger the computation (not, as with Clojure lazy sequences, trigger it as a side effect of normal access to an element).

(Note: this sort of hand-rolled laziness probably ought to be combined with immutability, else you're asking for weird bugs because of the order of initialization or computation.)

A special note for Ruby programmers

Ruby version 1.9 has the Enumerator class, which—with a little work—can behave like Clojure lazy sequences.^{[6](#)}

Here's how to create our old friend, all the integers, in Ruby:

```
1 def integers
2   retval = 0
3   Enumerator.new do | yielder |
4     loop do
5       yielder.yield(retval)
6       retval += 1
7     end
  end
```

```
8   end
9 end
```

Each time the `yielder` yields a value, the loop is suspended until the next time a value is called for. So here are the first 6 integers:

```
1 > integers.take(6)
2 => [0, 1, 2, 3, 4, 5]
```

The normal Ruby sequence functions continue to be eager rather than lazy. However, we can add lazy versions to `Enumerator` like this:

```
1 class Enumerator
2   def lazy_select(&predicate)    # select is like Clojure's filter
3     Enumerator.new do |yielder|
4       self.each do |value|
5         yielder.yield(value) if predicate.(val)
6       end
7     end
8   end
9
10  def lazy_collect(&transformer)  # collect is like Clojure's map
11    Enumerator.new do |yielder|
12      self.each do |value|
13        yielder.yield(transformer.(value))
14      end
15    end
16  end
17 end
```

With those, we can implement our earlier example of taking only the first factorial greater than 200. First, we make an enumerator based on an array, using a utility method:

```
1 > a = [5, 8, 9, 300, 1000].to_enum
2 => #<Enumerator: [5, 8, 9, 300, 1000]:each>
```

Now we can map and filter, just as in Clojure. In the following, I'm using a version of `factorial` that prints how it's called, so that you can see it's only called the needed number of times:

```
1 > a.lazy_collect { | n |
2 >   factorial(n)
3 > }.lazy_select { | n |
4 >   n > 200
```

```
5 > }.first
6 Factorial 5
7 Factorial 8
8 => 40320
```

1. Sort of. By presenting us with the appearance of a simple linear sequence of memory locations, the chip is hiding a more complicated reality.↵
2. To increase average efficiency, range's lazyseq actually calculates the first 32 values when asked for the first one.↵
3. There's a shorthand form—lazy-seq—for creating seqs of this sort, but it's easier to understand what's happening if the LazySeq's inner workings are made visible.↵
4. The line is from Robert Burns' "To a Louse. On seeing one on a lady's bonnet at church." It begins like this:
Ha! Whare ye gaun, ye crowlin ferlie?
Your impudence protects you sairly,
I canna say but ye strut rarely
Owre gauze and lace,
Tho' faith! I fear ye dine but sparely
On sic a place. ↵
5. Well, not quite. Whenever we used def to make a new binding to an old symbol (by, for example, redefining a function), we were causing side effects. That's a narrowly defined exception, though important for usability.↵
6. This idea is due to Brian Candler. ↵

13. Pattern Matching

We've seen that Clojure can *destructure* sequences in parameter lists. Suppose that we represent points as two-element vectors, and we want to add them together like so:

```
1 user=> (add-points [0, 1] [5, 4])
2 [5 5]
```

We do *not* have to write cumbersome code like this:

```
1 (def add-points
2   (fn [one two]
3     [ (+ (first one) (first two))
4       (+ (second one) (second two)) ])))
```

Instead, we can put a *pattern* for the two vectors into the argument list itself, naming each point's constituent parts:

```
1 (def add-points
2   (fn [ [x0 y0] [x1 y1] ]
3     [ (+ x0 x1) (+ y0 y1) ])))
```

This code is easier to read because it makes the structure of the data explicit, rather than something you infer from the functions applied to it^{[1](#)}.

Clojure's built-in destructuring parameter lists are limited compared to those of some other functional languages. Fortunately, because Clojure is a Lisp, it's easy enough to add more full-fledged pattern matching to it. That's what I've done, and that's what we'll discuss in this chapter.

13.1 Moving conditionals into arguments

Let's revisit `factorial`. Again, `factorial(5) = 5*4*3*2*1`, and (as a special case) `factorial(0) = 1`. In code, that can look like this:

```
1 (def factorial
2   (fn [n]
```

```

3      (cond (zero? n) 1
4            (= 1 n) 1
5            :else (* n (factorial (dec n)))))

```

For this chapter, I'll use Clojure's shorthand for defining functions:

```

1 (defn factorial
2   [n]
3   (cond (zero? n) 1
4         (= 1 n) 1
5         :else (* n (factorial (dec n)))))

```

... because my extensions for pattern-matching will look better modeled after that. As with the point example above, the structure inherent in the data is obscured by other code. So let's write a separate parameter list for each relevant "shape" of data:

```

1 (use 'patterned.sweet)
2
3 (defpatterned factorial
4   [0] 1
5   [1] 1
6   [n] (* n (factorial (dec n))))

```

That's the basic idea of pattern matching: a different parameter list for each important case of the data. Also notice that a parameter may be a constant (to be matched exactly) rather than a symbol (to be bound).

Pattern matching works particularly nicely for many recursive functions, because it distinguishes clearly between the ending case or cases (which are constants) and the recursive cases (which have symbols).

13.2 Sequence structure

We've seen a lot of recursive sequence functions. They follow a common pattern:

- Either you have an empty sequence, which is the ending case, or...
- ... you have a head and a tail. You work on the head and recurse on the tail.

Here's an example of how such a function can be written using pattern matching:

```
1 (defpatterned count-sequence
2   [ [] ] 0
3   [ [head & tail] ] (inc (count-sequence tail)))
```

Note the nesting of brackets. `count-sequence` takes a single argument, which is a sequence. Square brackets are doing double duty here: they surround parameter lists, as usual, but also indicate sequences within parameter lists.

Here's an example that has two arguments, one a collecting parameter. That may make the structure easier to see.

```
1 (defpatterned count-sequence
2   [so-far [ ] ] so-far
3   [so-far [head & tail] ] (count-sequence (inc so-far) tail))
```

Even though sequences are described with square brackets, they match any sort of sequence structure:

```
1 user=> (count-sequence 0 '(:a :b :c))
2 3
```

13.3 Arbitrary arguments

The earlier implementation of `factorial` will never finish if given a number less than zero. That can be prevented with a parameter list that matches any number less than zero. That requires use of a matching function, which looks like this:

```
1 (defpatterned factorial
2   [(:when neg? :bind n)] (oops! "No negative numbers" :n n)
3   [0] 1
4   [1] 1
5   [n] (* n (factorial (dec n))))
```

You might not want separate parameter lists for the `0` and `1` cases. A function could be used to match either of them, but there's a slightly nicer notation:

```

1 (defpatterned factorial
2   [(:when (partial > 0) :bind n)] (oops! "No negative numbers" :n n)
3   [(:in [0 1])] 1
4   [n] (* n (factorial (dec n))))

```

13.4 Summary

Pattern matching isn't wildly exciting. I've described it because its use reinforces two themes I've mentioned throughout this part of the book: thinking of computation as being about handling different shapes of data, and removing conditional expressions from view.

Pattern matching also generalizes to *generic functions*, the topic of the next chapter.

13.5 Exercises

You can find starting source for these exercises in [sources/pattern.clj](#). My solutions are in [solutions/pattern.clj](#).

Exercise 1

All of the functions above take a constant number of arguments. For example, the final version of `count-sequence` takes two arguments:

```

1 (defpatterned count-sequence
2   [so-far [          ] ] so-far
3   [so-far [head & tail] ] (count-sequence (inc so-far) tail))

```

Change that function so that it can also take a single argument, a sequence to count:

```

1 user=> (count-sequence '(:a :b :c))
2 3

```

Hint: Just add another parameter list and code snippet pair.

Exercise 2

Implement `reduce` as a `defpatterned` function, such that this code for reversing a sequence works:

```
1 user=> (pattern-reduce (fn [so-far elt] (cons elt so-far))
2                        []
3                        [:a :b :c])
4 (:c :b :a)
```

Hint: Generalize from the collecting parameter version of count - sequence.

1. You could argue that the structure of a point should be hidden behind specially-named accessor functions like `getX()` or `getY()`. However that argument applies to both solutions, so is a different topic than the one in this chapter. [↩](#)

14. Generic Functions

In Part 1, we went to great lengths to support object-oriented polymorphism. We made it possible for the same message to correspond to many methods, each attached to a different class. You’ve seen nothing comparable in Clojure, where each function name corresponds to only a single function.

Until now.

A *generic function* can have more than one *specialization*, in a way somewhat reminiscent of the way an abstract base class can have several concrete subclasses.



I’m unsure whether I want to include this chapter. I very much *like* the idea of generic functions, for reasons partly explained in [A Digression on Verbs](#) below. However, generic functions from the Lisp tradition (including Clojure’s) are quirky and burdened by their history. (They came from an era of kitchen-sink design and haven’t really overcome that; they still have remnants of the time when we poorly understood the tradeoffs between composition, inheritance, and delegation; and I think they’re too geared toward emulating object-oriented approaches rather than really striking out and inventing new ones.) Other implementations (that I’m aware of) of what might be called generic functions are really not that different from the pattern matching of the last chapter.

So this chapter is mainly an exploration of the less-quirky parts of Clojure’s generic functions. That’s useful if you’re a Clojure programmer—I use them all the time—but isn’t really applicable to other languages. My secret hope is that someone’s imagination catches fire because of this chapter and they go off to invent the *right* kind of a generic functions, especially one that points the way toward functional ways of modeling problems. But this chapter feels kind of dull for that, and inspiring invention isn’t really the point of this book.

So I’m putting it up to the readers. I’ve created [a poll](#) so that you can vote on what should happen to this chapter. I’ll repeat the link at the end of the chapter.

Clojure’s implementation of generic functions doesn’t work well in the repl—redefinitions don’t work the way you’re used to—so I’ve wrapped it in a

repl friendly way. You load my version of generic functions like this:

```
1 user=> (load-file "sources/generic.clj")
```

Here's a sample generic function:

```
1 user=> (defgeneric describe odd?)
```

The third symbol, `odd?`, is an example of a *classifier function*. When a generic function is called, the classifier function is used to convert the potentially very large number of possible argument lists into a few values. In this example, the classifier function converts all possible numbers into either `true` or `false`. Those values are then used to select one of the specialized functions. Here are two specialized functions:

```
1 user=> (defspecialized describe true
2         (fn [n] "odd"))
3 user=> (defspecialized describe false
4         (fn [n] "even"))
5 user=> (describe 3)
6 "odd"
```

`true` and `false` don't make for the clearest possible documentation of which case is which, so it's also common to return keywords:

```
1 (defgeneric describe
2   (fn [n]
3     (if (odd? n) :odd :even)))
4
5 (defspecialized describe :odd
6   (fn [n] "odd"))
7
8 (defspecialized describe :even
9   (fn [n] "even"))
```

Unless you instruct Clojure differently, it will throw an error if the classifier function produces a value that matches no specialization. If, however, you make a specialization for `:default`, that will be used for any non-matching value:

```
1 user=> (defgeneric describe identity)
2 user=> (defspecialized describe 1
3         (fn [n] "one"))
```

```

4 user=> (defspecialized describe 2
5         (fn [n] "two"))
6 user=> (defspecialized describe :default
7         (fn [n] "something else"))
8 user=> (describe 18)
9 "something else"

```

If you read other Clojure code or documentation, know that Clojure’s term for generic functions is “multimethods”. That’s a common term, but I consider it a historical hangover from the time when the Lisp world was explicitly trying to pull in ideas and terminology from the object-oriented world.

Clojure’s name for `defgeneric` is `defmulti`. Its name for `defspecialized` is `defmethod`, and the syntax is slightly different.

14.1 Generic functions for object-oriented programming

By storing “instance variables” in maps that have `:type` [metadata](#), you can emulate much of object-oriented programming with generic functions. Here’s a generic “constructor” for any sort of object:

```

1 user=> (def make
2         (fn [type value-map]
3           (with-meta value-map {:type type})))
4 user=> (def rim-griffon (make ::starship {:name "Rim Griffon" :speed 3}))
5 user=> (type rim-griffon)
6 :user/starship

```

Notice the odd double-colon version of a keyword: `::starship`. What’s the difference between that and the normal one-colon keywords?

A `:rose` is a `:rose` is a `:rose`, no matter where the keyword is seen. A `::rose`, however, is specific to the *namespace* it’s created in. You’ve been working in the `user` namespace throughout the book, so keywords like `::rose` print like this:

```

1 user=> ::rose
2 :user/rose

```

Importantly, `::rose` is a different value than `:rose`:

```
1 user=> (= :rose ::rose)
2 false
```

... and a `::rose` created in one namespace is different from a `::rose` created in another.

Why is this useful? Unlike many object-oriented languages, Clojure can have many different type hierarchies. Using namespace-qualified keywords keeps your hierarchy in your files from interfering with my hierarchy in my files.

Given type metadata, we can declare generic functions that use an object-oriented style of dispatch:

```
1 (def oo-style
2   (fn [this & args] (type this)))
3
4 (defgeneric nudge oo-style)
```

nudge increases the speed of ships:

```
1 (defspecialized nudge ::starship
2   (fn [this delta]
3     (assoc this :speed (+ delta (:speed this)))))
```

nudge would be used like this:

```
1 user=> (:speed rim-griffon)
2 3
3 user=> (-> (nudge rim-griffon 10) :speed)
4 13
```

Object-orientation isn't very interesting with only one type of object, so let's add asteroids:

```
1 user=> (def malse (make ::asteroid {:name "Malse" :speed 1}))
```

You can nudge an asteroid all you like, but you can't change their speed. They're too massive:

```
1 (defspecialized nudge ::asteroid
2   (fn [this delta]
3     this))
```

```
1 user=> (nudge malse 10000)
2 {:speed 1, :name "Malse"}
```

Generic functions also support inheritance. For example, notice that both asteroids and starships have names. Perhaps we should have a generic description function that applies to all named objects. Here's how you tell Clojure of a subtype relationship:

```
1 user=> (derive ::asteroid ::named)
2 user=> (derive ::starship ::named)
```

We can give the “base class” a specialization that applies to all `::named` maps:

```
1 user=> (defgeneric description oo-style)
2 user=> (defspecialized description ::named
3         (fn [this] (str "the " (name (type this)) " " (:name this))))
4 user=> (description malse)
5 "the asteroid Malse"
```

And we can override the base definition:

```
1 user=> (defspecialized description ::starship
2         (fn [this] (str "the spritely " (:name this))))
3 user=> (description rim-griffon)
4 "the spritely Rim Griffon"
```

At this point, I expect that you, my valued readers, are raising a rousing chorus of “So what?”. Indeed, it doesn't seem that generic functions, as presented so far, add much to object-orientation. True, but let me point out two things.

Dispatching on literal values

You don't have to work only with maps that have a type. The `defpatterned` [implementation](#) works on nothing you'd call an object. Instead, it uses a classification function (pattern-classification) that puts arbitrary Clojure forms (like `[a b & rest]`) into one of six categories. Each of the six specializations get the original form, not any sort of object, and does the right thing with it:


```

1 (defgeneric match-one? pattern-classification)
2
3 (defspecialized match-one? ::literal
4   (fn [pattern arg]
5     (= pattern arg)))
6
7 (defspecialized match-one? ::symbol
8   (fn [pattern arg] true))
9
10 (defspecialized match-one? ::nested
11   (fn [pattern arg]
12     (and (= (count pattern) (count arg))
13          (every? truthy? (map match-one? pattern arg)))))
14 ...

```

In an object-oriented language, I'd be tempted to make a `Pattern` class with a factory method that constructs one of six subclasses (`LiteralPattern`, `SymbolPattern`, etc.) based on the shape of the data. The `defpatterned` implementation bypasses that step of creating an object having a type: it works only with the raw data and a keyword classification of the same.

Code Organization

Object-oriented languages either force or strongly encourage you to clump together all of a class's non-inherited methods within a single class construct. Consider what that means for code understanding. Suppose we have two classes `Foo` and `Bar`, each with methods `collide` and `upgrade`. As is often the case, the two versions of `collide` share a family resemblance. They have differences, to be sure, but any notion of “collide” is probably about two objects that (metaphorically) try to occupy the same (metaphorical) space. And both `upgrade` methods are probably about improving something.

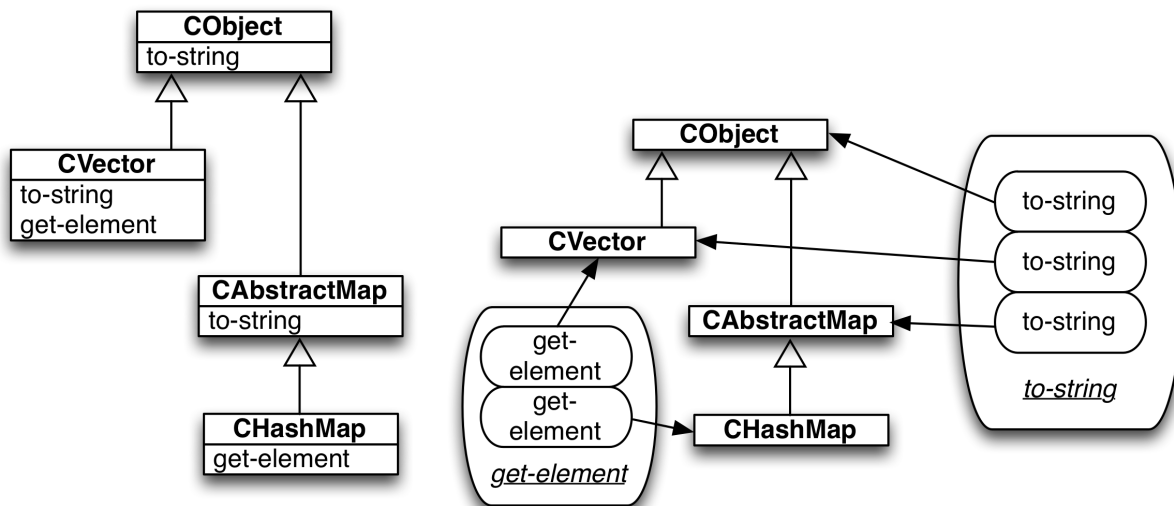
Clumping methods by class assumes that the easiest way to understand `Foo`'s `collide` is by reading it together with `Foo`'s other methods, such as `upgrade`. Sometimes that's true. But sometimes it's not: understanding `Foo`'s `collide` might be much easier if you could see it at the same time as `Bar`'s `collide`.

The `defpatterned` code, again, is an example. It has a generic function called `match-one?`. Patterns like `[a b]` and `[a b & rest]` are classified differently. But, because the `::nested-with-rest` specialization calls the

::nested one, they're best understood by reading them together. So it's nice that they can be one tiny eye-shift away from each other:

```
1 (defspecialized match-one? ::nested
2   (fn [pattern arg]
3     (and (= (count pattern) (count arg))
4           (every? truthy? (map match-one? pattern arg)))))
5
6 (defspecialized match-one? ::nested-with-rest
7   (fn [pattern arg]
8     (let [ [[pattern-required-part arg-required-part] _]
9             (partition-for-rest pattern arg)]]
10      (match-one? pattern-required-part arg-required-part))))
```

In cases where organization by classes is clearer, the separation of classification from specialization allows that too.



You choose

14.2 A digression on verbs

In US law, there's something called the "doctrine of attractive nuisance". Children find certain things attractive (trampolines, swimming pools, abandoned refrigerators) and they're not competent to understand the risks of playing with them. Therefore, it's the responsibility of the property owner to take reasonable care to prevent children from doing something stupid.

We all bring deep-seated assumptions to our work, and we're about as good at questioning them as children are at thinking "Could something bad happen if we're playing hide-and-seek and I climb inside that abandoned refrigerator and shut the door?"

One of those assumptions is that nouns name clear-cut categories. We *want* to believe there is a procedure that anyone could follow to distinguish, say, chairs from stools. And we assume that "bachelor" can be justifiably and equally applied to any of: the Pope, a 76-year-old widower, a never-married 45-year-old living with his mother, and a 28-year-old living in a Chicago loft who has three "friends with benefits" and a subscription to Maxim magazine.

A class-based language is one that has a boolean `is_a?` or `instanceof` predicate. Giving such a language to people like us is... an attractive nuisance. Just as with an unfenced swimming pool, quite often nothing bad happens. But sometimes we become too insistent on modeling the world with classes, on wedging the fuzzy world and the fuzzy ideas of our clients into an attractive inheritance structure. We climb inside the refrigerator of classes, slam the door shut, and only then notice we can't open it from the inside.

Is it possible functional languages offer a way out? People are much more tolerant of ambiguity in verbs than in nouns. We're perfectly happy with sentences like these:

- "I saw the dog."
- "I saw what she was getting at."
- "I saw the light."
- "I saw the light in her eyes."
- "I saw right through him."

... or these:

- "The clock ran out."
- "The boy ran out."
- "The water ran out of the basin."

We accept that there's no single thing that is "seeing" or "running", and that the meaning of a verb can't be found in isolation. Instead, you need to look at the surrounding nouns and possibly other words.

In effect, it doesn't bother us that verbs in practice don't have clear-cut definitions. Rather their uses share family resemblances. That suggests to me that generic functions might be a useful modeling tool in cases where the more noun-like, static approach of object modeling is inadequate.

I sometimes think of object orientation like this¹:



Like a good object-oriented design, the objects in the painting have *verisimilitude*. They don't have to be real, or even exactly realistic (that is, "true to life"), but they have to capture enough of the appearance of reality to allow the artist/designer to achieve his goals.

Also, the objects need to be placed in harmonious relationships. Those things (classes) that belong together should have easily comprehensible references to each other. Effects (like the play of light and shadow) should be uniform and predictable.

And I sometimes think of functional programming as like this²:



People who pay for programs want them to *do things* in the world. The point of this painting, Delacroix's "Liberty Leading the People", is to motivate ("to stimulate to action"). It both captures (statically) people in motion but also was intended to motivate the viewers to act on the ideals of the French Revolution.

I like the idea of something static—a painting, a program—that is both to represent and to cause a change in some part of the world. Functions would seem to do that better than classes, because functions are about change more than about relationships.

14.3 Polymorphism without a privileged argument

One of the situations objects have trouble modeling are cases where more than one object has equal weight in deciding what method needs to execute. This is the long-standing *double-dispatch* problem.

Let's create a generic function called `collide`. For simplicity, we'll use a ridiculous model of collisions. They affect only the `:speed` of objects:

- If two asteroids collide, their speeds are unchanged.
- If two ships collide, the faster nudges the slower up to its own speed.
- If a ship collides with an asteroid, the asteroid is unaffected and the ship—having been destroyed—slows down to speed zero³.

Here's an object-oriented model of collision:

```
1 (defgeneric collide oo-style)
2
3 (defspecialized collide ::starship
4   (fn [this other] ...))
5
6 (defspecialized collide ::asteroid
7   (fn [this other] ...))
```

What should the body of the first specialization look like? Its behavior depends on the second argument. Asteroids are treated differently than spaceships. Since putting an `if` check of `other`'s type in the body pushes against the whole point of object-orientation, the usual solution is to send a message to the other object, with the type of the first argument as part of the message name:

```
1 (defspecialized collide ::starship
2   (fn [this other]
3     (collide-with-starship other this)))
```

Since both asteroids and starships can collide with starships, we have a new generic function:

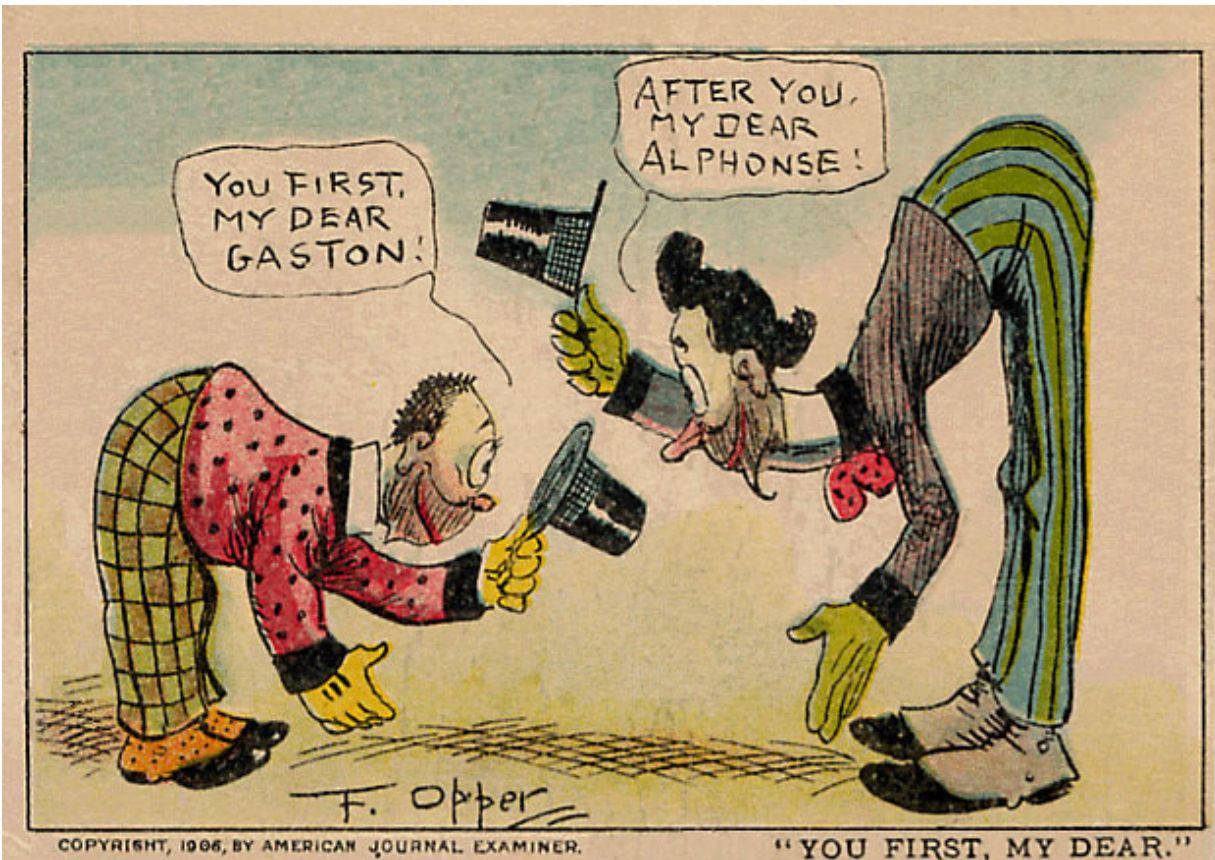
```
1 (defgeneric collide-with-starship oo-style)
2 (defspecialized collide-with-starship ::starship ...)
3 (defspecialized collide-with-starship ::asteroid ...)
```

And we have to do the same with asteroids, so the whole solution requires all these definitions:

```
1 (defgeneric collide oo-style)
2 (defgeneric collide-with-starship oo-style)
3 (defgeneric collide-with-asteroid oo-style)
4
5 (defspecialized collide ::starship
6   (fn [this other]
7     (collide-with-starship other this)))
8
9 (defspecialized collide ::asteroid
10  (fn [this other]
11    (collide-with-asteroid other this)))
12
13 (defspecialized collide-with-starship ::starship
14   (fn [& ships] ...))
15 (defspecialized collide-with-starship ::asteroid
16   (fn [asteroid ship] ...))
17
18 (defspecialized collide-with-asteroid ::asteroid
19   (fn [& asteroids] ...))
20 (defspecialized collide-with-asteroid ::starship
21   (fn [ship asteroid] ...))
```

This is really complicated. When I was writing sample code for this, I messed it up several times because the argument order in `collide-with-*` is the opposite of `collide`'s, but the results have to be in the original order.

Moreover, having one object's `collide` not do anything except ask the other object to do something reminds me of the 1901 "Alphonse and Gaston" comic strip that featured two overly-polite Frenchmen:



It all seems faintly ridiculous. “Collide” is a verb that applies to two objects of equal status. Wedging it into an object-oriented style, where the receiver is privileged over other arguments, makes coding awkward and overly verbose.

Since specializations can match any Clojure value, we can dispatch off the types of both arguments and return the result as a vector:

```
1 (defgeneric collide (fn [one two] [(type one) (type two)]))
```

That allows us to have one generic function with four specializations:

```
1 (defspecialized collide [::asteroid ::asteroid]
2   (fn [& asteroids] asteroids))
3
4 (defspecialized collide [::starship ::starship]
5   (fn [& starships]
6     (let [speed (apply max (map :speed starships))]
7       (map (partial with-speed speed) starships))))
8
9 (defspecialized collide [::asteroid ::starship]
```



```

10 (fn [asteroid starship]
11     [asteroid (stopped starship)]))
12
13 (defspecialized collide [::starship ::asteroid]
14     (fn [starship asteroid]
15         [(stopped starship) asteroid]))

```

It's slightly annoying that we have two specializations for the asteroid-starship case, but that's forced by the need to return the results in the same order as the arguments.

14.4 On your own

I don't have any exercises for this chapter, since I'm not sure I want to keep it. If you want to see the source for the examples above, plus one or two additional ones, look in [sources/asteroids.clj](#).



Tell me [what you think](#) about this chapter.

1. The artist is Luis Egidio Meléndez (1716-1780), a Spanish painter famous for his still-lives. He was a master of light, shadow, volume, and texture. [↩](#)
2. Eugène Delacroix (1798-1863) was a French Romantic painter. He emphasized colour and movement rather than the formal clarity of painters like MelÃ©ndez. [↩](#)
3. Because of air resistance, doncha know. It's the same air that lets us hear explosions in space. [↩](#)

III CODA

Let me turn this over to Mercury astronaut Gus Grissom:

Asking Gus to “just say a few words” was like handing him a knife and asking him to open a main vein. But hundreds of workers are gathered in the main auditorium of the Convair plant to see Gus and the other six [astronauts], and they’re beaming at them, and the Convair brass say a few words and then the astronauts are supposed to say a few words, and all at once Gus realizes it’s his turn to say something, and he is petrified. He opens his mouth and out come the words: “Well... do good work!” It’s an ironic remark, implying “... because it’s my ass that’ll be sitting on your freaking rocket.” But the workers start cheering like mad. They started cheering as if they had just heard the most moving and inspiring message of their lives: Do good work! After all, it’s little Gus’s ass on top of our rocket! They stood there for an eternity and cheered their brains out while Gus gazed blankly on them from the Pope’s balcony. Not only that, the workers—the workers, not the management but the workers!—had a flag company make up a huge banner, and they strung it up high in the main work bay, and it said: DO GOOD WORK.’

— Tom Wolfe, *The Right Stuff*

Go off, have fun, and do good work. And thanks for reading.

IV A MITE MORE ON MONADS (OPTIONAL)

In this section, I'll walk you through the implementation of two well-known monads: the Sequence monad (which does what Clojure and Python's `for` does), and the State monad (which lets you pretend that an immutable language actually has assignment statements, in something like the way that the [Zipper data structure](#) lets you pretend immutable trees are mutable).

These chapters concentrate on monad implementation because that surfaces concepts you need to understand before you can use monads effectively. However, that implementation bias means that my treatment will be informal compared to others. Monads have a fairly lofty mathematical pedigree that I will *totally ignore*. For example, many (most?) treatments of monads make a point of mentioning three Laws that legitimate monads must follow. Except for this paragraph, I don't mention them. I treat the first two informally (though I believe correctly) and don't bother with the third at all.

I hope that this approach lets you read other descriptions of monads (such as [this](#) and [this](#) more easily. I have a different [description of the elephant](#), and multiple descriptions make it easier for you to integrate knowledge.

15. Implementing the Sequence Monad

[Recall](#) that Clojure's Sequence monad is used like this:

```
1 (with-monad sequence-m
2   (domonad [a (list 1 2 3)
3              b (list (- a) a)
4              c (list (+ a b) (- a b))])
5   (* a b c)))
```

(I'm using `list` instead of more idiomatic vectors like `[1 2 3]` because I want it to be clear that the steps are arbitrary computations, not just lists of values.)

In this chapter, we'll define our own version of `sequence-m`, called `sequence-monad`.

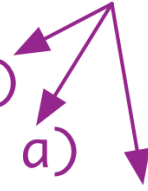
15.1 Monadic values and binding values

Before we look at defining `sequence-monad`, it's important to notice that the handling of step values seems different than that in the [earlier discussion of monads](#). Back there, our decider function accepted values from step calculations and passed them, unaltered, into a continuation. Something different has to be going on here. The steps produce sequences that the decider must *take apart* into numbers. It's those numbers that are passed into the continuation, not the sequences.

That is, there are two different types, or kinds, or shapes, of values at work in monads. I'm going to call those *monadic values* and *binding values*. Here is where the monadic values are calculated in the above `domonad`:

calculations that produce monadic values

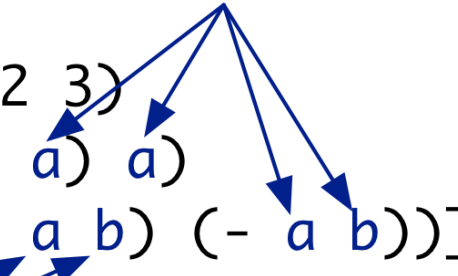
```
(domonad [a (list 1 2 3)
          b (list (- a) a)
          c (list (+ a b) (- a b))])
(* a b c))
```



And here are where binding values are used:

uses of binding values

```
(domonad [a (list 1 2 3)
          b (list (- a) a)
          c (list (+ a b) (- a b))])
(* a b c))
```



more uses of binding values



The body of the domonad is the only place you see a binding value being calculated (since the result is a number, not a list):

```

(domonad [a (list 1 2 3)
           b (list (- a) a)
           c (list (+ a b) (- a b))])
(* a b c))

```


 a calculation of a binding value

One more monadic value belongs in the picture, the final result:

```

an entire calculation that produces a monadic value
      ↓
user=> (domonad [a (list 1 2 3)
                 b (list (- a) a)
                 c (list (+ a b) (- a b))])
              (* a b c))
(0 -2 2 0 0 -16 16 0 0 -54 54 0)

```


 the result is a monadic value

For a monad to work properly, it must correctly coordinate the binding and monadic values. Let's look at what that means by examining the expansion of domonad.

Here's the top of it:

```

1 (-> (list 1 2 3)
2     (decider (fn [a] ...)))

```

First notice that the monadic value (1 2 3) is passed as the decider's first argument. (Its second argument is a continuation.) Since all deciders in a

domonad's expansion are the same function, we have a rule for any decider:

A decider must accept a monadic value as its first argument.

For example, our nested monad's decider function must accept a sequence.

Now notice that the result of the decider is not passed (via `->`) to any other function. It's the last function in this flow. (All the flows for the remaining steps are nested inside the first step's continuation.) That is: the top decider's result becomes the result of domonad. And since the result of the domonad is a monadic value, ...

A decider must produce a monadic value.

Let's now look at the interaction of one decider with the decider below it, using this deeper expansion:

```
1 (-> (list 1 2 3)                ;; <== first monadic calculation
2     (decider                     ;; <== first decider
3       (fn [a]
4         (-> (list (- a) a)        ;; <== second monadic calculation
5           (decider               ;; <== second decider
6             (fn [b] ...))))))
```

What happens here?

1. The first decider calls a continuation, giving it a binding value (for a).
2. That continuation calculates the second step's monadic value—something like `(- 1 1)`—and feeds it to the second decider.
3. When the second decider returns (what must be a monadic value), the continuation immediately returns that to the first decider.

Because an inner decider must produce a monadic value, a decider that “wraps” it must accept one.

A decider must accept a monadic value from its continuation.

Finally, let's look at the full expansion.

```
1 (-> (list 1 2 3)
2     (decider
3       (fn [a]
4         (-> (list (- a) a)
5             (decider
6               (fn [b]
7                 (-> (list (+ a b) (- a b))
8                     (decider
9                       (fn [c]
10                        (* a b c))))))))))))) ;; <== final decider
```

The final decider is in an odd position:

- Every decider above it expects to get a monadic value (a sequence) from its continuation.
- But *its* continuation returns a binding value (a number).

To resolve this, domonad's expansion inserts a “monadifier” function below the bottommost decider:

```
1 (-> (list 1 2 3)
2     (decider
3       (fn [a]
4         (-> (list (- a) a)
5             (decider
6               (fn [b]
7                 (-> (list (+ a b) (- a b))
8                     (decider
9                       (fn [c]
10                        (monadifier
11                          (* a b c))))))))))))) ;; <== New
```

The monadifier's job is to turn a binding value into a monadic value. In the Sequence monad, we can use `list` as the monadifier.

In cases where the monadic values and binding values have the same shape, the monadifier is just a function that does nothing: `identity`.

In Clojure's implementation of monads, what I'm calling `monadifier` is called `m-result`, and we saw it bound to `identity` in earlier definitions:


```

1 user=> (def maybe-monad
2         (monad [;; We call m-result "the monadifier".
3                 m-result identity
4
5                 ;; We call m-bind "the decider"
6                 m-bind (fn [monadic-value monadic-continuation]
7                         (if (nil? monadic-value)
8                             nil
9                             (monadic-continuation monadic-value))))
10        ]))

```

15.2 Defining the monad

Using `list` as the monadifier converts numbers into sequences. The decider needs to go in the reverse direction before it calls its continuation. A simple use of `map` might work:

```

1 (fn [monadic-value monadic-continuation]
2   (map monadic-continuation monadic-value))

```

That gives us this monad:

```

1 (def sequence-monad
2   (monad [m-result list
3           m-bind (fn [monadic-value monadic-continuation]
4                     (map monadic-continuation monadic-value))]))

```

Does it work?

```

1 user=> (with-monad sequence-monad
2         (domonad [a (list 1 2 3)
3                   b (list (- a) a)
4                   c (list (+ a b) (- a b))]
5                   (* a b c)))
6 (((((0) (-2)) ((2) (0))) (((0) (-16)) ((16) (0))) (((0) (-54)) ((54) (0)))))

```

Signs point to “no”.

Although I’ve been sloppy about saying it, we don’t really want the monadic values to be any old sequences. We want them to be sequences of *numbers*. Consider the bottommost use of `decider`, which might look something like this:

```

1 (-> (list 2 0)
2     (decider
3       (fn [c]
4         (monadifier
5           (* a b c)))))))))

```

1. The decider maps the continuation over `(list 2 0)`, so the continuation will be applied twice.
2. The first invocation will produce `0` from the multiplication.
3. The monadifier converts that to `(0)`, which is a valid monadic value.
4. In the second invocation of the continuation, `-2` is converted to `(-2)` and returned.
5. So the decider's map gets two monadic values to work with. map does what it always does: wraps those values into a sequence: `((0) (-2))`.

If a monadic value is a sequence of numbers, this ain't one. By returning it, decider is failing to honor its contract. It should be stripping away the outer level of parentheses that map puts on. That's easily done:

```

1 user=> (apply concat '( (0) (-2) ))
2 (apply concat '( (0) (-2) ))
3 (0 -2)

```

So we can redefine the decider like this:

```

1 (fn [monadic-value monadic-continuation]
2   (apply concat
3     (map monadic-continuation monadic-value)))

```

... or we can use the `mapcat` function, which is shorthand for the above. In point-free style, it'd be defined like this:

```

1 user=> (def mapcat (comp (partial apply concat) map))
2 user=> (map list [1 2 3])
3 ((1) (2) (3))
4 user=> (mapcat list [1 2 3])
5 (1 2 3)

```

Here's the decider's definition, using `mapcat`:

```

1 (fn [monadic-value monadic-continuation]
2   (mapcat monadic-continuation monadic-value))

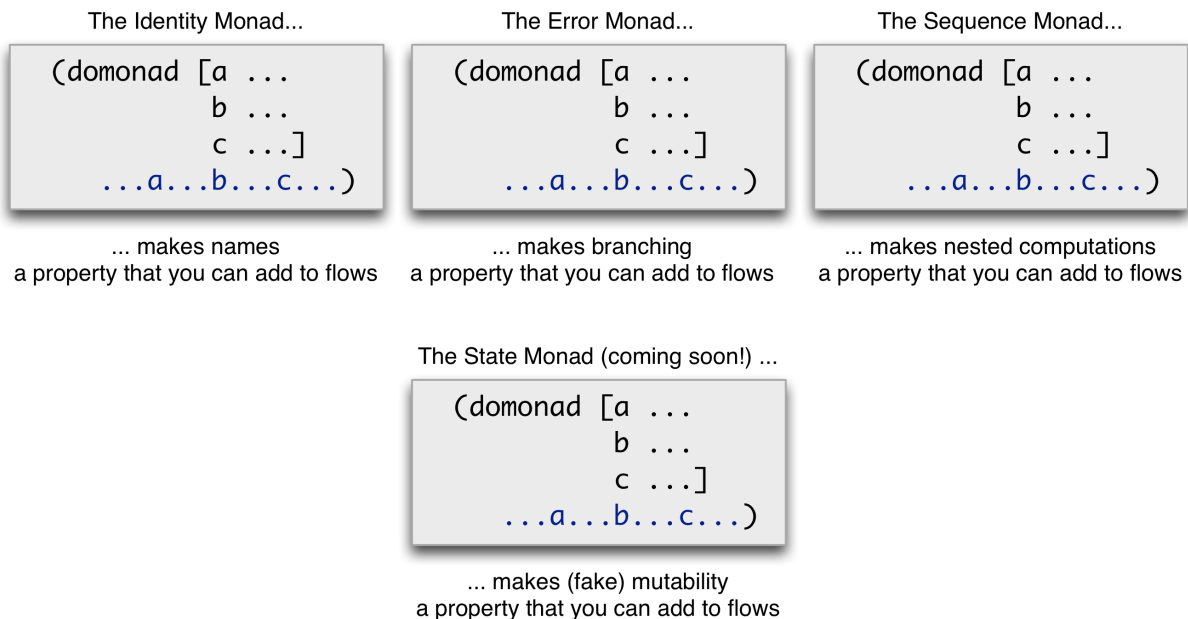
```

If we redefine the monad giving that function to m-bind, the result behaves as we want:

```
1 user=> (with-monad sequence-monad
2         (domonad [a (list 1 2 3)
3                   b (list (- a) a)
4                   c (list (+ a b) (- a b))])
5         (* a b c)))
6 (0 -2 2 0 0 -16 16 0 0 -54 54 0)
```

15.3 The monad bestiary

Let's take a moment to appreciate the variety of different behaviors we've abstracted out of a flow style:



15.4 Exercise

My solution is in [solutions/sequence-m.clj](#).

Here, again, is a definition of the Sequence monad:

```
1 (def sequence-monad-decider
2   (fn [monadic-value continuation]
3     (mapcat continuation monadic-value)))
4
5 (def sequence-monad-monomodifier list)
6
```

```

7 (def sequence-monad
8   (monad [m-result sequence-monad-monadifier
9           m-bind sequence-monad-decider]))

```

Starting from the source in [sources/sequence-m.clj](#), modify the decider so that it combines the behavior of the Maybe monad and the Sequence monad. That is, it should behave like the Sequence monad in the absence of `nil`:

```

1 user=> (with-monad maybe-sequence-monad
2         (domonad [a [1 2 3]
3                   b [-1 1]]
4                  (* a b)))
5 (-1 1 -2 2 -3 3)

```

However, this decider will not pass along a `nil` to its continuation.

```

1 user=> (with-monad maybe-sequence-monad
2         (domonad [a [1 nil 3]
3                   b [-1 1]]
4                  (* a b)))
5 (-1 1 nil -3 3)

```

Notice that there is only one `nil` in the output. If decider had passed the `nil` to the next step, and *then* that step had checked, seen the `nil`, and returned `nil` instead of `(* a b)`, there would be two `nil`s in the result.

Hint: The monadic values are lists of numbers or `nil`s.

Hint: One way to think about this is that the decider locally (via `let`) augments the continuation it's given to produce a new continuation that handles `nil` specially.

Hint:

```

1 user=> (concat [-1 1] nil [-3 3])
2 (-1 1 -3 3)
3 user=> (concat [-1 1] (list nil) [-3 3])
4 (-1 1 nil -3 3)

```

15.5 Refactoring to a monad transformer

In the previous exercise, you wrote a single monad that combined the behavior of the Maybe and Sequence monads.

[Earlier](#), I said that an advantage of the Sequence monad over Clojure's for is that you can combine the Maybe monad and Sequence monad like this:

```
1 user=> (def combined-monad (maybe-t sequence-m))
```

That monad behaves just like the one you hand-crafted.

```
1 user=> (with-monad combined-monad
2         (domonad [a [1 nil 3]
3                  b [-1 1]]
4                 (* a b)))
5 (-1 1 nil -3 3)
```

The `maybe-t monad transformer` will in fact work with any well-behaved monad. To show that it's not magic, we'll now extract it from the exercise solution. I'll call the extracted function `maybe-transform` to avoid clashing with the predefined `maybe-t``.

My solution to the exercise looked like the following series of expressions:

```
1 (def combined-monadifier list)
2
3 (def combined-decider
4   (fn [monadic-value continuation]
5     (let [maybe-ified-continuation
6           (fn [binding-value]
7             (if (nil? binding-value)
8                 (combined-monadifier binding-value)
9                 (continuation binding-value)))]
10       (mapcat maybe-ified-continuation monadic-value))))
11
12 (def combined-monad
13   (monad [m-result combined-monadifier
14          m-bind combined-decider]))
```

Let's start by working on the `combined-monadifier`. It's `list`, which we recognize as the monadifier for the Sequence monad. Instead of just hardcoding it, let's write code that fetches it from the Sequence monad.

How do we grab a monad's monadifier? Recall that with-monad wraps the functions that define a monad around code. That is, this:

```
1 (with-monad sequence-m ...anything...)
```

... is the same thing as this:

```
1 (let [m-result sequence-monadifier
2       m-bind sequence-decider]
3     "anything")
```

So we can just replace "anything" with m-result:

```
1 (with-monad sequence-m m-result))
```

That gives us this equivalent, but more general, definition for combined-monadifier:

```
1 (def combined-monadifier
2     (with-monad sequence-m m-result))
```

Now let's work on combined-decider:

```
1 (def combined-decider
2     (fn [monadic-value continuation]
3       (let [maybe-ified-continuation
4             (fn [binding-value]
5               (if (nil? binding-value)
6                   (combined-monadifier binding-value)
7                   (continuation binding-value)))]
8         (mapcat maybe-ified-continuation monadic-value))))
```

The last line looks suspiciously like sequence-m's decider, which is this:

```
1 (fn [monadic-value continuation]
2     (mapcat continuation monadic-value))))
```

So let's fetch that decider and use it specifically:

```
1 (def combined-decider
2     (fn [monadic-value continuation]
3       (let [maybe-ified-continuation
4             (fn [binding-value]
5               (if (nil? binding-value)
```

```

6             (combined-monadifier binding-value)
7             (continuation binding-value)))]
8   ( (with-monad sequence-m m-bind)      ; <== changed
9     monadic-value
10    maybe-ified-continuation))))

```

At this point, there are exactly two references to the Sequence monad in our code:

```

1 (def combined-monadifier
2   (with-monad sequence-m m-result))      ;; <== here
3
4 (def combined-decider
5   (fn [monadic-value continuation]
6     (let [maybe-ified-continuation
7           (fn [binding-value]
8             (if (nil? binding-value)
9               (combined-monadifier binding-value)
10              (continuation binding-value)))]
9     ( (with-monad sequence-m m-bind)      ;; <== here
10       monadic-value
11       maybe-ified-continuation))))

```

To start fixing that, let's first replace the two defs with one let:

```

1 (let [combined-monadifier      ;; <== here
2       (with-monad sequence-m m-result)
3
4       combined-decider        ;; <== here
5       (fn [monadic-value continuation]
6         (let [maybe-ified-continuation
7               (fn [binding-value]
8                 (if (nil? binding-value)
9                   (combined-monadifier binding-value)
10                  (continuation binding-value)))]
9         ( (with-monad sequence-m m-bind)
10           monadic-value
11           maybe-ified-continuation)))]
12
13 (def combined-monad
14   (monad [m-result combined-monadifier
15          m-bind combined-decider]))

```

Now we can abstract away the specific uses of sequence-m by wrapping everything in a function, maybe-transform, and expecting sequence-m to be passed to in as the source-monad:

```

1 (def maybe-transform
2   (fn [source-monad]
3     (let [combined-monadifier
4           (with-monad source-monad m-result)      ;; <== change
5
6           combined-decider
7           (fn [monadic-value continuation]
8             (let [maybe-ified-continuation
9                   (fn [binding-value]
10                     (if (nil? binding-value)
11                         (combined-monadifier binding-value)
12                         (continuation binding-value)))]
13               ( (with-monad source-monad m-bind) ;; <== change
14                 monadic-value
15                 maybe-ified-continuation)))]
16
17       (def combined-monad
18         (monad [m-result combined-monadifier
19                m-bind combined-decider])))

```

Having the def of combined-monad hardcoded inside maybe-transform is bad, so we can just use a monad expression as the body of the let and move the naming of the result outside maybe-transform:

```

1 (def maybe-transform
2   (fn [source-monad]
3     (let [...]
4       (monad [m-result combined-monadifier
5                m-bind combined-decider])))
6
7 (def maybe-sequence-monad (maybe-transform sequence-m))

```

Thus we create a transformer that adds “maybe” behavior to an existing monad, producing a new monad. This refactoring shows that a transformer has a fair amount of rote code and a core of core that requires thought to write. In maybe-transform, the only non-rote code is the combined-decider code that does a special check on a binding value:

```

1 (fn [binding-value]
2   (if (nil? binding-value)
3       (combined-monadifier binding-value)
4       (continuation binding-value)))

```

And even that code is tractable, if you understand the code for the Maybe monad’s decider and think very precisely:

1. The Maybe monad makes its decision based on whether a binding value is `nil`. (Note that it has to decide based on the binding value, not the monadic value, because the “shape” of the monadic value is decided by the source monad. As with the Sequence monad, the monadic value may hide `nil`s inside it. The Maybe monad is responsible for protecting the next step from getting a `nil` binding value.)
2. If the binding value isn't `nil`, it can be passed down in the normal way.
3. But a `nil` binding value must short-circuit all remaining steps and return some value compatible with whatever decider is above this step. That decider is a source monad decider wrapped in Maybe behavior. That means that the value passed up must be monadified in a way compatible with the source monad: and that is most easily done by using the source monad's monadifier.

I'm *not* claiming that writing a transformer function is trivial in the sense of seeing it once and being able to do it flawlessly. I do think it's like learning to drive: you fairly quickly get to the point where you can usually perform effectively without really thinking hard about it.

16. Functions as Monadic Values

People sometimes use the metaphor that a monad's monadic values are containers that hold the binding values. That doesn't apply to all monads (like the Maybe monad), but it's an OK way of thinking about the Sequence monad. Let's run with that metaphor for this chapter. Except that rather than using boring values like sequences to hold binding values, we'll wrap them inside of functions.

16.1 A function wrapper

In this section, we'll create a monad named `function-monad`. Like all monads, it needs a decider and a monadifier.

I'll choose a monadifier that converts a binding value into a zero-argument function that, when invoked, returns the binding value:

```
1 (def monadifier
2   (fn [binding-value]
3     (fn [] binding-value)))

1 user=> (def monadic-value (monadifier "hi!"))
2 user=> (monadic-value)
3 "hi!"
```

What's interesting about this is that, since a monad must return a monadic value, any use of this monad serves to define a function. So, rather than calculating results immediately (like all of our previous monads do), `function-monad` will, when it's finished, give us a “frozen” calculation:

```
1 user=> (def do-calculation
2         (with-monad function-monad
3           (domonad [a ...
4                     b ...]
5                     (+ a b))))
6 user=> (do-calculation)
```

I used ellipses in the above expression because we have to be careful with the right-hand side of the steps. Remember that steps must return *monadic* values, not binding values. (For example, in our earlier uses of the Sequence monad, the step values had to be sequences, not numbers.) In the case of this monad, they must return zero-argument functions.

The easy way to do that is to use the monadifier or (as `clojure.algo.monads` calls it) `m-result`:

```
1 user=> (def calculation
2         (with-monad function-monad
3           (domonad [a (m-result 8)
4                     b (m-result (+ a 88))]
5                     (+ a b))))
6 user=> (calculation)
7 96
```

I think that's ugly. Worse, the word `m-result` is so generic as to make it easy (for me, at least) to lose track of what's going on. So I'm going to use the synonym `frozen-step` from now on:

```
1 user=> (def calculation
2         (with-monad function-monad
3           (let [frozen-step m-result]           ;; <== A better API
4             (domonad [a (frozen-step 8)
5                       b (frozen-step (+ a 88))]
6                       (+ a b))))
7 user=> (calculation)
8 96
```

The above won't work yet, because we've only defined the monadifier, not the decider. What should the decider do? It needs to unwrap the monadic value and pass it to the continuation:

```
1 (def decider
2   (fn [monadic-value monadic-continuation]
3     (let [binding-value (monadic-value)]
4       (monadic-continuation binding-value))))
```

Does it need to do more? No, because we rely on the programmer to freeze the results of each step, and the `domonad` expansion freezes the result of the body. So the value the decider receives from its continuation is always

monadic (frozen). There's no reason to alter it: it can just be passed up to whatever lies above the decider.

Here, in summary, is the monad:

```
1 (def function-monad
2   (monad [m-result      ;; same as the body of `monadifier` above
3         (fn [binding-value]
4           (fn [] binding-value))
5
6         m-bind          ;; same as the body of `decider` above
7         (fn [monadic-value monadic-continuation]
8           (let [binding-value (monadic-value)]
9             (monadic-continuation binding-value))))))
```

That works fine, if by “fine” you mean “a convoluted definition that’s just like a `let` except you have to execute the result instead of just using it.”

However, that’s just the start. Because functions can take arguments.

16.2 A counting monad

Once upon a time, computer users were charged according to how much work the computer did when it “ran their job.” (No! It’s true!) Let us return to those primitive times for an example.

All jobs are functions created by a `domonad`. Charging starts with a flat charge of 3 units. Each additional step adds 1. The body is free. (Such a deal!) Here’s how a two-step “job” is prepared:

```
1 user=> (def run-and-charge
2         (with-monad charging-monad
3           (let [frozen-step m-result]
4             (domonad [a (frozen-step 8)
5                       b (frozen-step (+ a 88))]
6                       (+ a b))))))
```

The job is delivered to the computer operator (Mr. Repl), who puts it into execution and delivers the result. That looks like this:

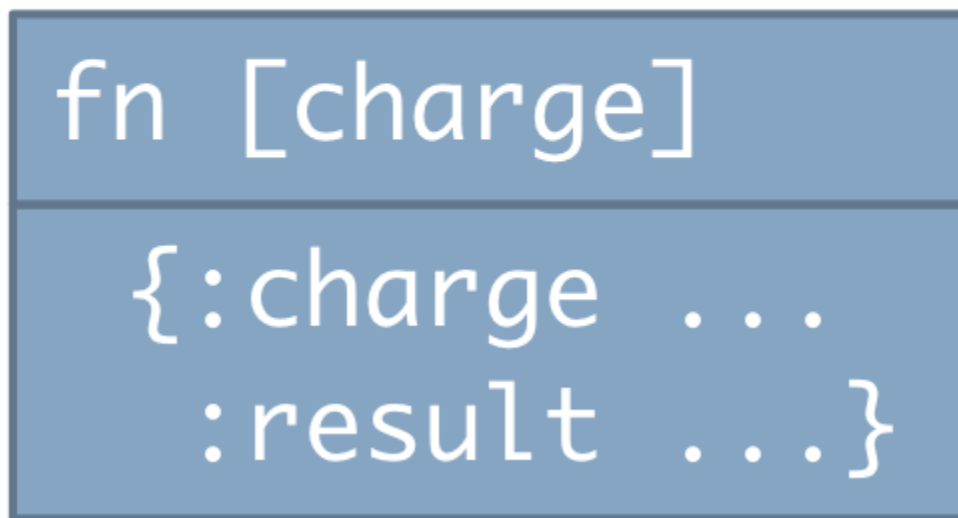
```
1 user=> (run-and-charge 3)
2 {:charge 5, :result 104}
```

It's our job to implement `charging-monad`. What facts do we know?

1. The binding values are numbers. (In the second step, one of them is added to 88.)
2. The monadic values are functions that take a charge and, when evaluated, produce a map with `:charge` and `:result` fields.

That's a complicated monadic value, and every piece of it will matter for developing the monad. As a terser reminder of the shape, I'll use this picture:

Monadic value



The monadifier has to create such a function from a binding value (a number). That means it *must* have this form:

```
1 (fn [result]
2   (fn [charge]
3     ...
4     {:charge ..., :result ...}))
```

The simplest way to fill in the ellipses would be this:

```
1 (fn [result]
2   (fn [charge]
```

```
3      {:charge charge, :result result}))
```

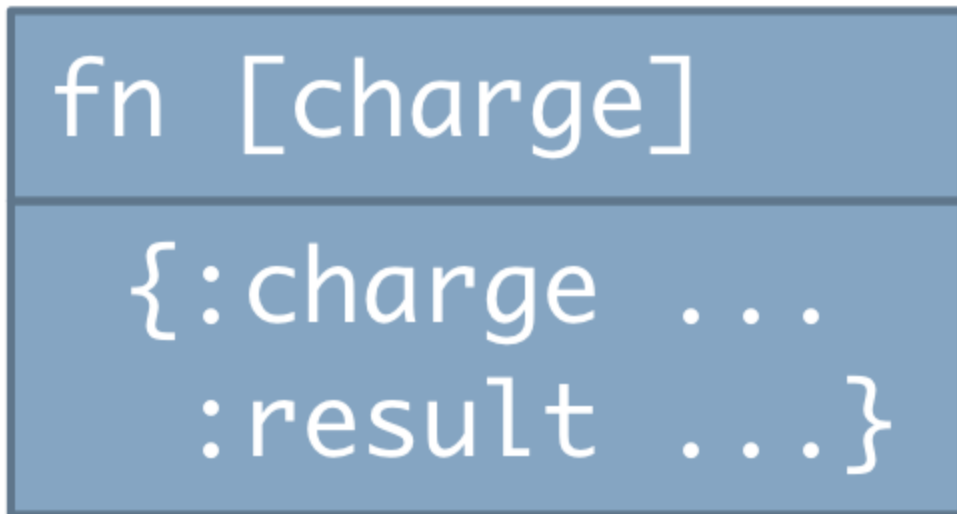
Let's see where that takes us.

Now let's consider the decider. We'll build it up from its simplest form, the decider that decides nothing:

```
1 (fn [monadic-value monadic-continuation]
2   (monadic-continuation binding-value))
```

The monadic value is one of our functions:

Monadic value



We need to get the `:result` out of the monadic value so we can pass it as the binding value for the continuation. An application of a monadic value *must* look like this:

```
1 (fn [monadic-value monadic-continuation]
2   (let [enclosed-map (monadic-value charge)
3         binding-value (:result enclosed-map)]
4     (monadic-continuation binding-value)))
```

I added a `charge` argument because that's what this kind of monadic value requires. But `charge` is an *unbound symbol*, meaning it hasn't been bound

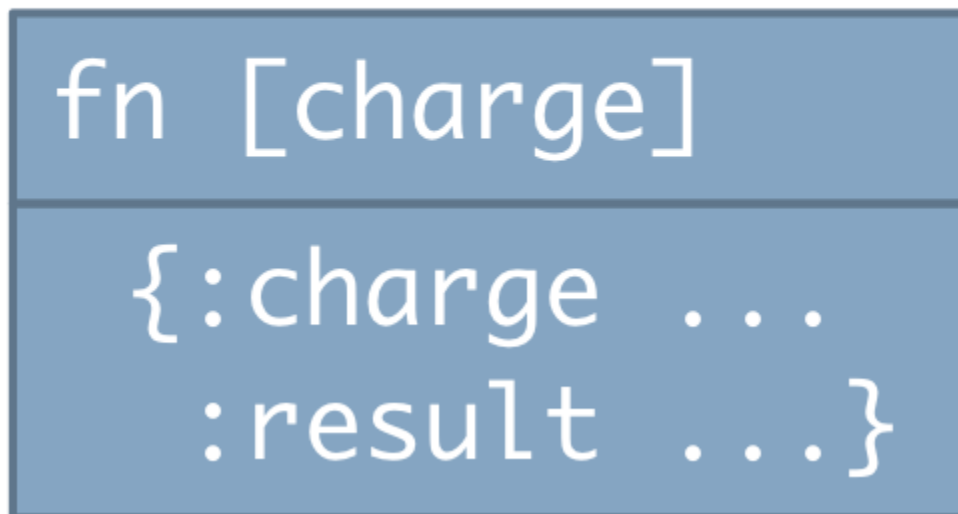
to any value by an enclosing `let` or function parameter list. We need to sneak in a binding somewhere. Where?

The decider must return a monadic value. If you ignore the unbound symbol for a second, you'll see that it does, because it returns the value of the continuation, which we know has to be a monadic value. However, nothing in the decider's contract says it has to return the continuation's monadic value. (Indeed, remember that the Sequence monad doesn't: it constructs a new monadic value with `mapcat`.) Let's have it make up a new monadic value to return:

```
1 (fn [monadic-value monadic-continuation]
2   (fn [charge] ;; <== Return a new function, not the continuation's
3     (let [enclosed-map (monadic-value charge)
4           binding-value (:result enclosed-map)]
5       (monadic-continuation binding-value))))
```

That has *part* of the right shape, namely the top half of this:

Monadic value



The bottom half is wrong, though, because our new wrapping function doesn't return a map. It returns the results of the continuation, which is a function—specifically, a monadic value.

Besides, we still need a place to increment the charge. How about passing an incremented version of it to the continuation's return value?

```
1 (fn [monadic-value monadic-continuation]
2   (fn [charge]
3     (let [enclosed-map (monadic-value charge)
4           binding-value (:result enclosed-map)]
5       ( (monadic-continuation binding-value) (inc charge)))))) ;; <==
```

That way, we get to increment, and we also extract the result map from the monadic value that the continuation returns.

Amazingly enough, this works. What it demonstrates, I think, is that while thinking of monads in terms of expansions into continuation-passing style is a good way to begin to understand monads, a meticulous consideration of the shapes of the monadic and binding values helps more once you start thinking about more complex monads.

By starting with the monadifier, you start by choosing the shape of a monadic value. By considering how the decider must (1) accept something of that shape as its first argument, (2) unwrap that shape for its continuation, (3) accept something of same shape from the continuation, and (4) pass something of that same shape up to its caller, even subtleties—and this *is* subtle—can fall into place without too much mental anguish.

16.3 Exercises

Exercise 1

Here's something that seems weird about our monad implementation:

```
1 (fn [monadic-value monadic-continuation]
2   (fn [charge]
3     (let [enclosed-map (monadic-value charge)      ;; <==
4           binding-value (:result enclosed-map)]
5       ( (monadic-continuation binding-value)
6         (inc charge))))))
```

We pass the charge into the monadic-value to get an enclosed-map containing the step's `:result`. But the enclosed-map also contains a

:charge. Yet we never use it. Could we pass in any old value to extract the :result?

```
1 (fn [monadic-value monadic-continuation]
2   (fn [charge]
3     (let [enclosed-map (monadic-value :any-old-value)      ;; <==
4           binding-value (:result enclosed-map)]
5       ( (monadic-continuation binding-value)
6         (inc charge))))))
```

Starting from the charging-monad source in [sources/function-monads.clj](#), make the change. Can you explain the results? (If you can't, the next exercise may help.)

Exercise 2

My source contains a version of our monad in which all functions print information about themselves when they run. Run the sample code. You might be surprised at the order in which things happen.

It might also be helpful to painstakingly hand-execute the monad.

Exercise 3

Move the inc of the charge from the decider to the monadifier. What affect does this have on the results?

(You can find my solution in [solutions/function-monads.clj](#). I modified the verbose version from exercise 2.)

16.4 The State monad

We can generalize the counting monad from the previous section. First, let's change the word charge to the more generic word state. That means that the monadifier (and, so, every step) must return this shape:

Monadic value

```
fn [state]
```

```
{:state ...  
 :result ...}
```

We'll also remove the `inc` from the decider. Instead, it now just passes the unmodified state when it calls monadic values:

```
1 (fn [monadic-value monadic-continuation]  
2   (fn [state]  
3     (let [enclosed-map (monadic-value state)      ;; <== as before  
4         binding-value (:result enclosed-map)]  
5       ( (monadic-continuation binding-value) state)))))) ;; <== no `inc`
```

Next, let's provide more flexibility to the steps. Right now, they all boringly freeze their results:

```
1 user=> (def run-and-charge  
2         (with-monad charging-monad  
3           (let [frozen-step m-result]  
4               (domonad [a (frozen-step 8)  
5                         b (frozen-step (+ a 88))]  
6                 (+ a b))))))
```

But the steps don't *have* to use `frozen-step` (the monadifier). They can use any function that produces the right shape. How about a function that makes the state available to later steps? That would look like this:

```

1 user=> (def calculation-with-initial-state
2         (with-monad state-monad
3         (let [frozen-step m-result]
4             (domonad [a (get-state)]           ;; <==
5                     (- a))))))
6
7 user=> (calculation-with-initial-state 1)
8 {:state 1, :result -1}

```

What does get-state look like? It takes no arguments and has to return a monadic value like this:

Monadic value

```

fn [state]

{:state ...
 :result ...}

```

It need only shift the state that it's given to be the :result part of the map. That's what becomes bound to the step's symbol and is thus available to all the later steps. I'll show both the monadifier and get-state to make the two easier to compare.

```

1 (def monadifier
2   (fn [result]
3     (fn [state]
4       {:state state, :result result}))))
5
6 (def get-state
7   (fn []

```

```

8      (fn [state]
9        {:state state, :result state})))

```

If we're going to write functions that do unusual things to the `:result` key, perhaps we should do the same for the `:state` key. As it stands, the decider allows no changes to the state, in that it ignores the `:state` key provided by the monadic value:

```

1 (fn [monadic-value monadic-continuation]
2   (fn [state]
3     (let [enclosed-map (monadic-value state)
4           binding-value (:result enclosed-map)] ;; <== :state key ignored.
5       ( (monadic-continuation binding-value) state))))))

```

Let's make it pass the `:state` key's value to the continuation:

```

1 (fn [monadic-value monadic-continuation]
2   (fn [state]
3     (let [enclosed-map (monadic-value state)
4           binding-value (:result enclosed-map)
5           new-state (:state enclosed-map)] ;; <== not ignored
6       ( (monadic-continuation binding-value) new-state))))))

```

This allows something like an assignment statement, albeit for a single (unnamed) variable. Let's suppose that `assign-state` is a function that modifies the current value of the state. Since functions have to return values, it'll return the previous value of the state. It could be used like this:

```

1 user=> (def calculation-with-initial-state
2         (with-monad state-monad
3           (domonad [original-state (assign-state 88)
4                    state (get-state)]
5                     (str "original state " original-state
6                          " was changed to " state))))
7
8 user=> (calculation-with-initial-state 1111)
9 {:state 88, :result "original state 1111 was changed to 88"}

```

And here's the definition:

```

1 (def assign-state
2   (fn [new-state]
3     (fn [state]
4       {:state new-state, :result state})))

```

Notice that state operations can be intermixed with ordinary binding operations. In the following, I'm using a common Clojure idiom, which is to use `_` where a symbol is required but the symbol's value will never be used.

```
1 user=> (def mixer
2         (with-monad state-monad
3           (let [frozen-step m-result]
4             (domonad [original (get-state)
5                       a (frozen-step (+ original 88))
6                       b (frozen-step (* a 2))
7                       _ (assign-state b)]
8                 [original a b])))
9 user=> (mixer -87)
10 {:state 2 :result [-87 1 2]}
```

The State monad can be the basis for a number of other monads. For example, a state monad that takes a sequence and adds values to it can be used for logging. Haskell builds on the State monad to do IO in a language more rigorous about immutability than Clojure is. That is, in Clojure, the following two expressions cause immediate input or output:

```
1 user=> (slurp "/etc/passwd")
2 "##\n# User Database\n# \n# Note that this file is consulted directly ..."
3 user=> (println "hello")
```

In Haskell, functions like `putStrLn` or `getLine` are like our `frozen-step`, `get-state`, and `transform-state`: they produce monadic values that are functions. Only within the context of Haskell's equivalent of `domonad` can those functions be evaluated to actually cause I/O.

16.5 Exercises

You can start from the source in [sources/function-monads.clj](#). (You can find my solutions in [solutions/function-monads.clj](#).)

Exercise 4

Write `transform-state`. It should take a function as an argument. That function is applied to the current state to produce the new state. It would be used like this:

```
1 user=> (def transform-state-example
2         (domonad [b (transform-state inc)]
3                 b))
4 user=> (transform-state-example 1))
5 {:state 2, :result 1}
```

Notice that the `:result` of `transform-state` is the old state.

Exercise 5

For fun, I once wrote a small program to run on an emulator for the [PDP-1](#), a computer that had only one register through which you could make changes to the state of memory¹. It was hard. From this, I conclude that if you're going to have state, you should have *lots* of it. Therefore, change the State monad so that state is represented by a map. The stateful functions should take a keyword that names the “variable” they work with:

```
1 (get-state :a)
2 (assign-state :b 3)
3 (transform-state :c inc)
```

All functions should only return the state of their variable argument. (That is, `transform-state` should return the old value of `:c` in the state, not the whole state.)

Write an example using all three. If I'm lucky, you'll experience a vague discomfort with having to think about both old and new values of variables. If so, you've been bitten by the immutability bug.

1. To be completely honest, you could sometimes use a second I/O register in addition. [↵](#)

V TO RUBY... AND BEYOND! (OPTIONAL)

Part 1's version of object-orientation was inspired by Java's. Java's implementation is rather impoverished, as object models go, so this part of the book will extend Part 1's implementation to add some features from the Ruby language:

- Making class into objects in their own right.
- Adding a class whose purpose is to make new classes.
- Adding *modules*, Ruby's version of multiple inheritance.
- By introducing dynamic binding, we can finally implement “call-super”, Ruby style.

As a final bonus, we'll peek at some of the surprising things languages like [Io](#) and [Self](#) turn into objects.

17. The Class as an Object

In this chapter, we'll be working up to the implementation in [sources/class-object.clj](#).

In Part 1, we made new objects using a function named `make`. I don't like it. I'd rather send a `:new` message to the class, like this:

```
1 user=> (def my-point (send-to Point :new 1 2))
2 user=> (send-to my-point :class-name)
3 Point
```

That unifies two ideas—sending to an object, and creating an instance—in one mechanism. It also makes it natural to add other methods to classes. For example, we might want a special message for creating the point at $(0, 0)$:

```
1 user=> (send-to Point :origin)
2 {:y 0, :x 0, :__class_symbol__ Point}
```

17.1 The wrong implementation

Some languages basically add a map of *class methods* to their classes. It sits alongside the instance methods they already hold. So, for example, their `Point` might look like this:

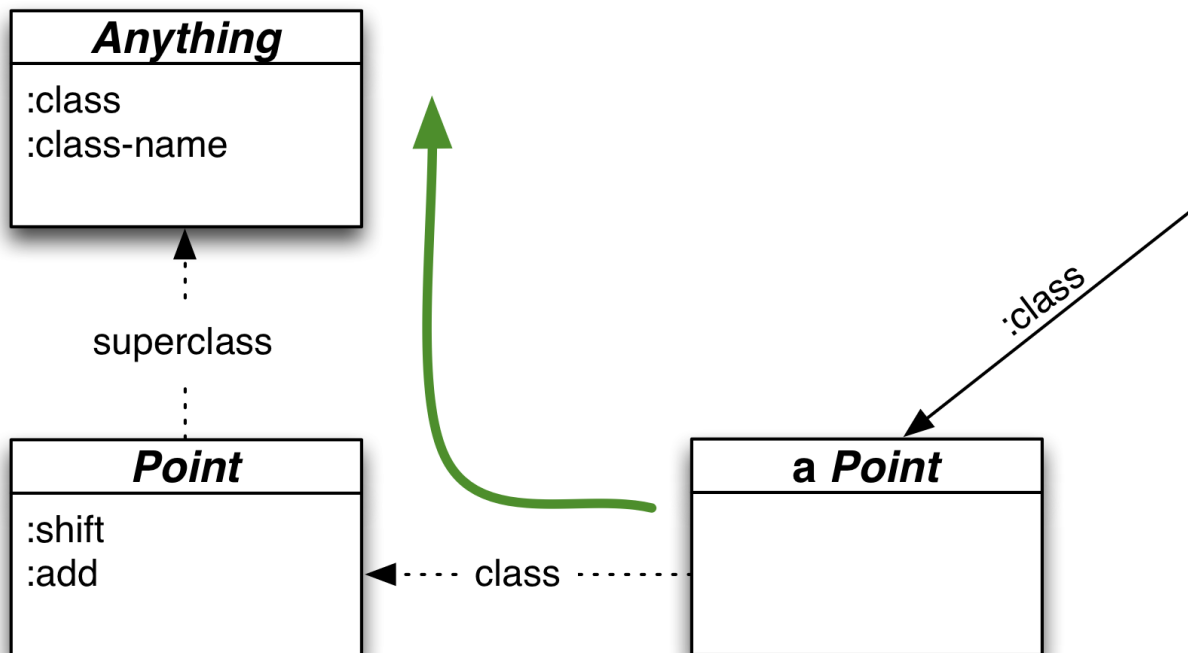
```
1 (def Point
2 {
3   :__own_symbol__ 'Point
4   :__superclass_symbol__ 'Anything
5   :__class_methods__
6   {
7     :origin (fn [class] (make Point 0 0))           ;; New
8   }
9   :__instance_methods__ {...}
10 })
```


Notice that class methods take an argument that plays something like the role of `this` in instance methods. `class` can be used to call one class method from another. There may or may not be “class variables” that play the same role as instance variables.

I’m not satisfied by this solution. Although the external syntax for sending messages to classes is the same as for instances, that’s just surface: the distinction is maintained in the implementation. We can do better.

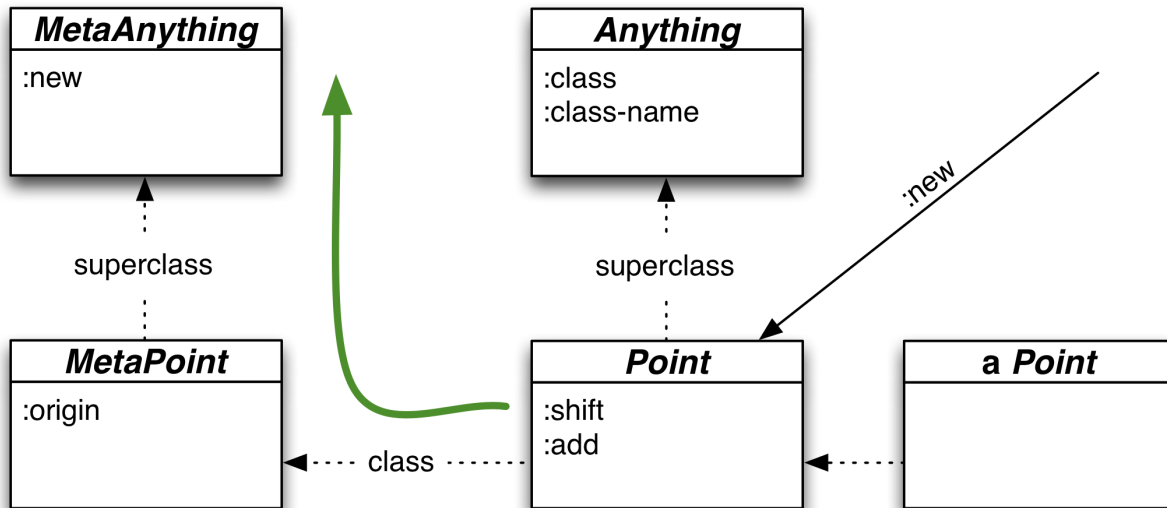
17.2 A better implementation

We already know how to handle a message sent to an object: we look it up in its lineage’s merged `:__instance_methods__` maps. Because, for whatever reason, I think of classes as being to the left of their instances, I visualize message dispatch as following a “look left, then up” rule:



Sending a message to an instance provokes a lookup

If classes *themselves* had classes to their left, we could use the same rule. Let’s call those “classes of classes” *metaclasses*. Here’s what that looks like:



Sending a message to a class provokes a lookup

Making this work requires no changes to send-to or the other dispatch function code. It requires only that:

1. Point and Anything have `:__class_symbol__` metadata, just like their instances do. They thereby become objects like any other:

```

1 (def Anything
2 {
3   :__own_symbol__ 'Anything
4   :__class_symbol__ 'MetaAnything      ;; <== new
5   ...
6
7 (def Point
8 {
9   :__own_symbol__ 'Point
10  :__class_symbol__ 'MetaPoint          ;; <== new
11  :__superclass_symbol__ 'Anything

```

2. The MetaAnything metaclass needs to exist. Its definition of `:new` is just the body of the old make:

```

1 (def MetaAnything
2 {
3   :__own_symbol__ 'MetaAnything
4   :__instance_methods__
5   {
6     :new
7     (fn [class & args]
8       (let [seeded {:__class_symbol__ (:__own_symbol__ class)}]
9         (apply-message-to class seeded :add-instance-values args)))

```

```
10     }
11  })
```

3. `MetaPoint` defines `:origin`. By not defining `:new`, it delegates that behavior to its superclass `MetaAnything`:

```
1 (def MetaPoint
2 {
3   :__own_symbol__ 'MetaPoint
4   :__superclass_symbol__ 'MetaAnything
5   :__instance_methods__
6   {
7     :origin (fn [class] (send-to class :new 0 0))
8   }
9 })
```

Here are some examples of how the behaviors you're used to still work:

```
1 user=> (send-to Anything :new)
2 {:__class_symbol__ Anything}
3 user=> (def point (send-to Point :new 1 2))
4 user=> point
5 {:y 2, :x 1, :__class_symbol__ Point}
6 user=> (send-to point :class-name)
7 Point
8 user=> (send-to point :shift 1 2)
9 {:y 4, :x 2, :__class_symbol__ Point}
```

I think that's neat: classes as objects themselves! How have we gotten here, though?

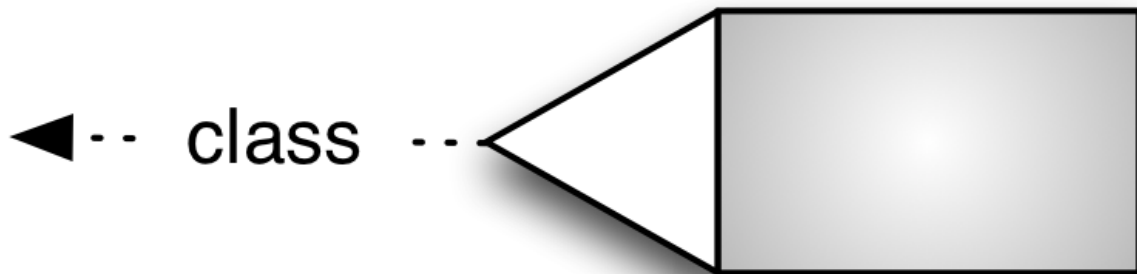
17.3 The story so far

We began with objects that were nothing but maps, operated on by global functions that took a `this` argument by convention. I'll draw that like this:



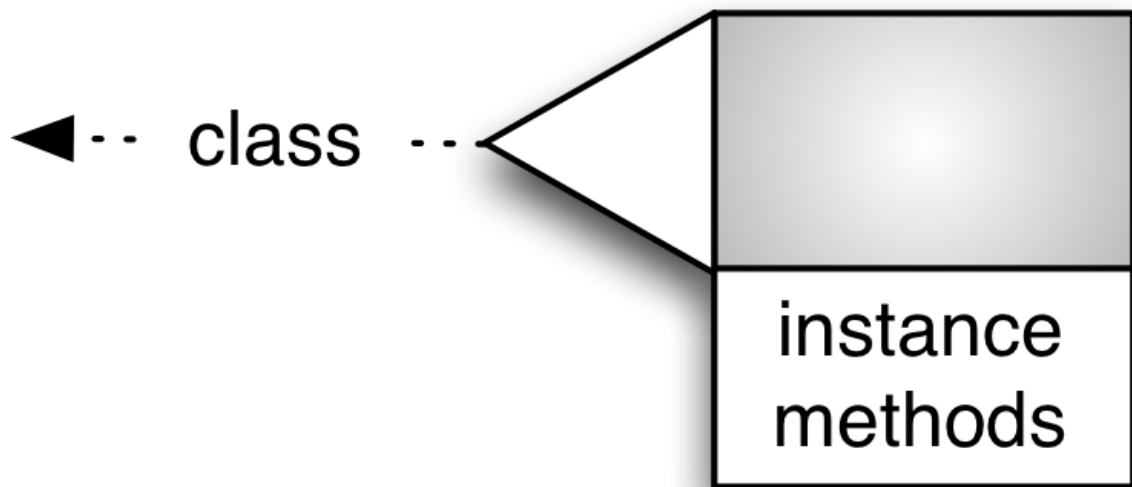
The shading indicates that the map is to be filled with what we'll call (by convention) instance variables (actually key/value pairs).

That was a bit too minimal, so we added a metadata key that named a class:



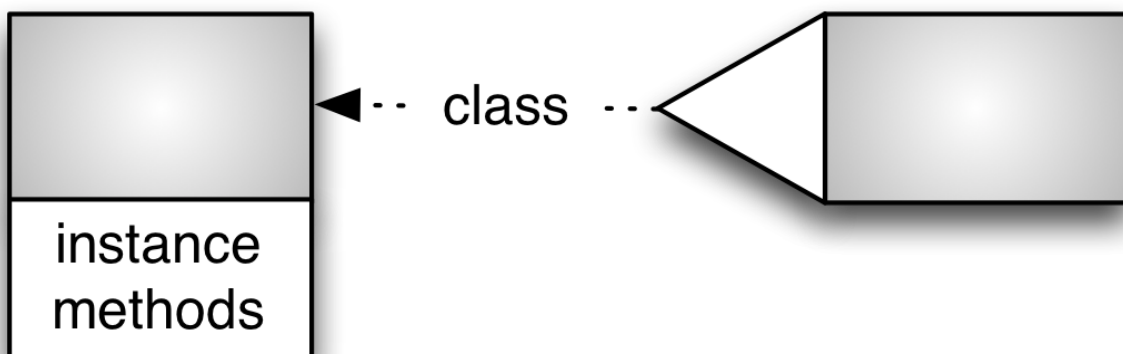
At first, though, the class was nothing but a symbol.

Since the global functions that used `this` really didn't look much like methods, we began moving them into objects. First, we added them to the objects themselves:

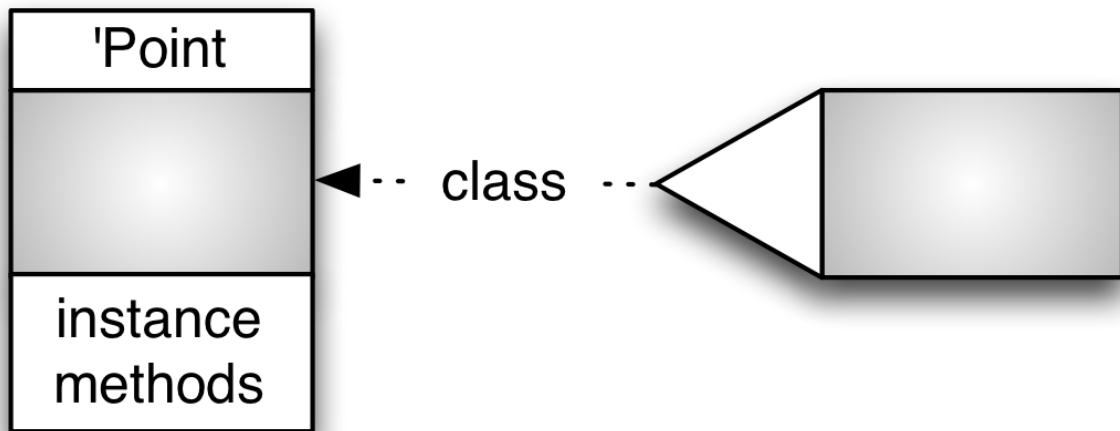


We created a simple dispatch function that knew how to convert from messages (keywords) to the methods (functions) that were nested inside the object.

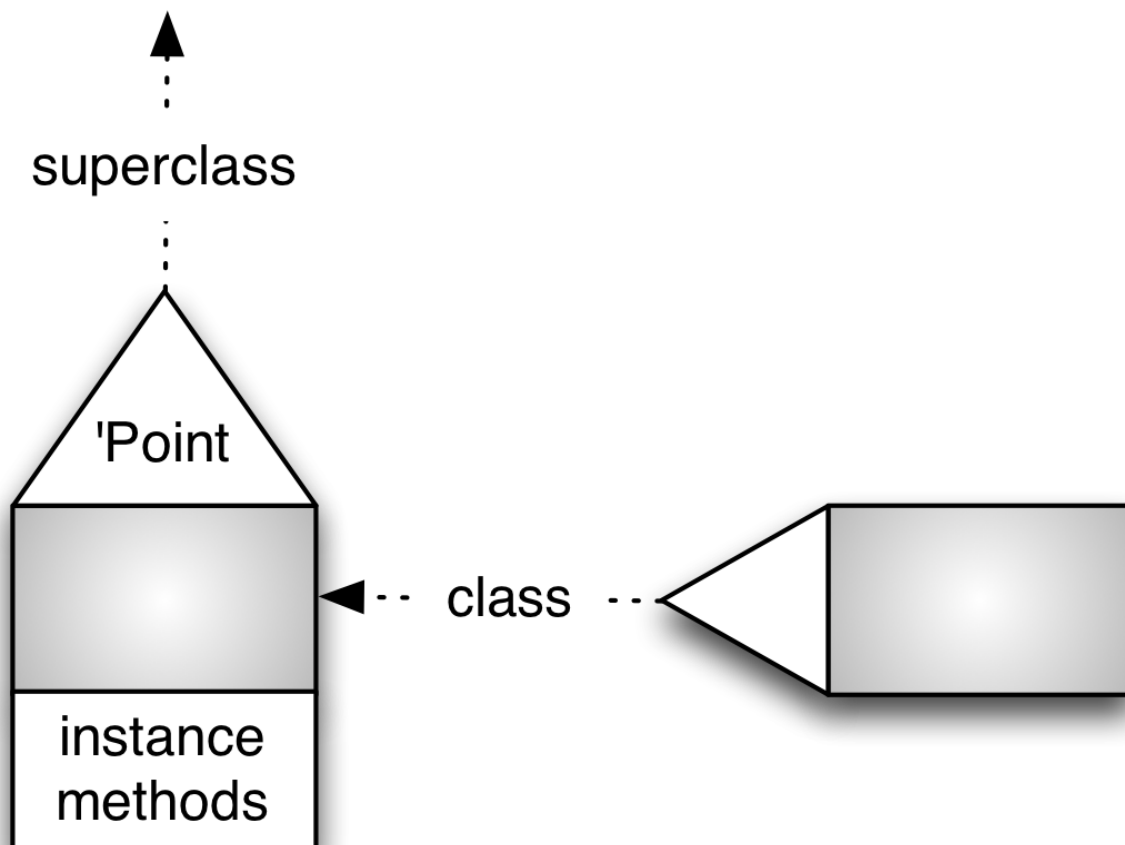
Then we decided that, since we already had the notion of a class that named all similar instances, we'd be better off putting the instance methods there (converting the class from a symbol to a map itself):



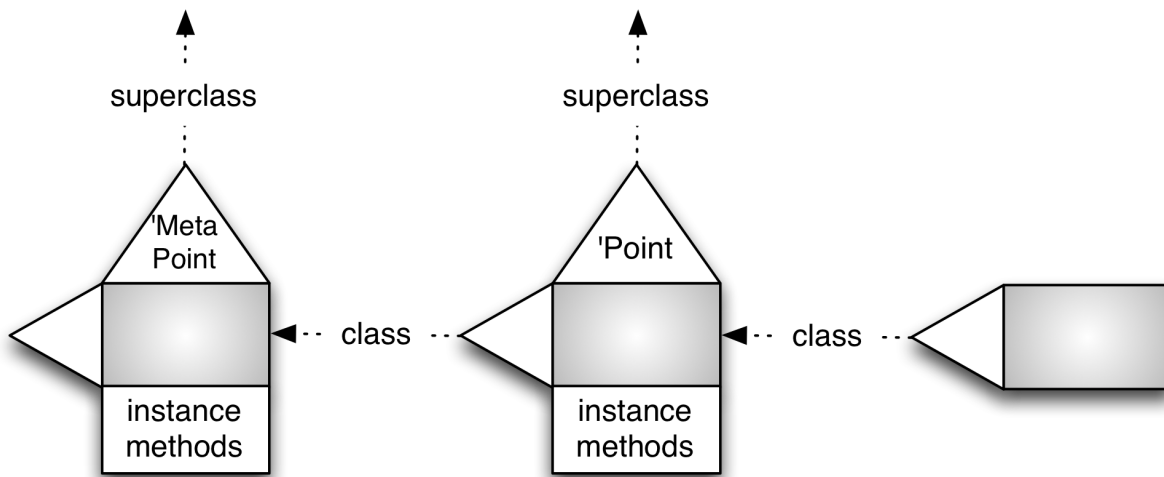
Because of our implementation, we needed the class to know its own name in order to create instances:



We next implemented inheritance by having a class point “up” to its superclass:



Finally, we decided that classes might as well be real objects themselves, which meant they too should have classes:



So: what's the difference between a class and other objects? All objects have a `:class-symbol`. Classes have, in addition, an `:own-symbol` (which is used by `:new` to give instances their `:class-symbol` values), a `:superclass-symbol`, and a map of `:__instance_methods__`.

The picture above matches Ruby's object model, except that we're giving metaclasses explicit names. Ruby doesn't (though you can still get to them if you need to). So this part of our object model is perhaps closer to Smalltalk's. That language gives metaclasses explicit names, emphasizing that they themselves are objects (and classes).

There are still some flaws in our model. You'll address them in the next set of exercises.

17.4 Exercises

You can find the starting source for these exercises in [sources/class-object.clj](#). My solutions are in [solutions/class-object.clj](#).

Exercise 1:

If no method in the class or in any superclass matches a message, arrange to send the `:method-missing` message to the instance, giving it the original

message and argument list. (Wondering why `:method-missing` gets the argument list? In Ruby, programmers frequently override `:method-missing` to implement some default behavior that uses the argument list.)

In the exercise source, I've already made `:method-missing` a method on the `Anything` class, so you needn't worry that `:method-missing` will itself be missing. (That assumes that all classes are subclasses of a single "root" class, usually called `Object` but in our case `Anything`. That assumption holds in most languages, but not in, for example, C++. You can go ahead and assume it.)

The `Anything` implementation of `:method-missing` raises a Java exception:

```
1 user=> (def something (send-to Anything :new))
2 user=> (send-to something :method-missing
3                                     :the-name-of-no-method ["arguments"])
4 Error A Anything does not accept the message :the-name-of-no-method.
```

... and inheritance works as for any other method:

```
1 user=> (def point (send-to Point :new 1 2))
2 user=> (send-to point :method-missing
3                                     :the-name-of-no-method ["arguments"])
4 Error A Point does not accept the message :the-name-of-no-method.
```

Make it so that the following message send produces the same error:

```
1 user=> (send-to point :the-name-of-no-method 1 2)
2 Error A Point does not accept the message :the-name-of-no-method.
```

Also: the exercise source defines a `MissingOverrider` class. Use it to check these requirements:

- A subclass can override `:method-missing`.
- The arguments are really passed correctly as a sequence.

Exercise 2:

`MetaPoint`'s superclass (`:__superclass_symbol__`) is `MetaAnything`. What should `MetaAnything`'s superclass be? After you adjust it, using code like this:


```
1 (def MetaAnything
2   (assoc MetaAnything :__superclass_symbol__ ???))
```

... write code that shows what kind of difference the superclass makes.

Hint: What happens if you send :to-string to Point? Or some unknown message to Anything?

Hint: Any object must be either a direct or indirect subclass of Anything. Metaclasses are objects.

Exercise 3:

MetaPoint is Point's class. (That is, it's the value of Point's :__class_symbol__ key.) What is MetaPoint's class?

MetaAnything is Anything's class. What is MetaAnything's class?

Write the code to set the :__class_symbol__ values of the two metaclasses, then demonstrate the difference the change made.

Hint: Does the following make sense?

```
1 user=> (send-to MetaPoint :class)
2 user=> (send-to MetaPoint :to-string)
```

Hint: The above are reasonable messages to send. That means that any metaclass's class must be a subclass of Anything.

Hint: Does the following make sense?

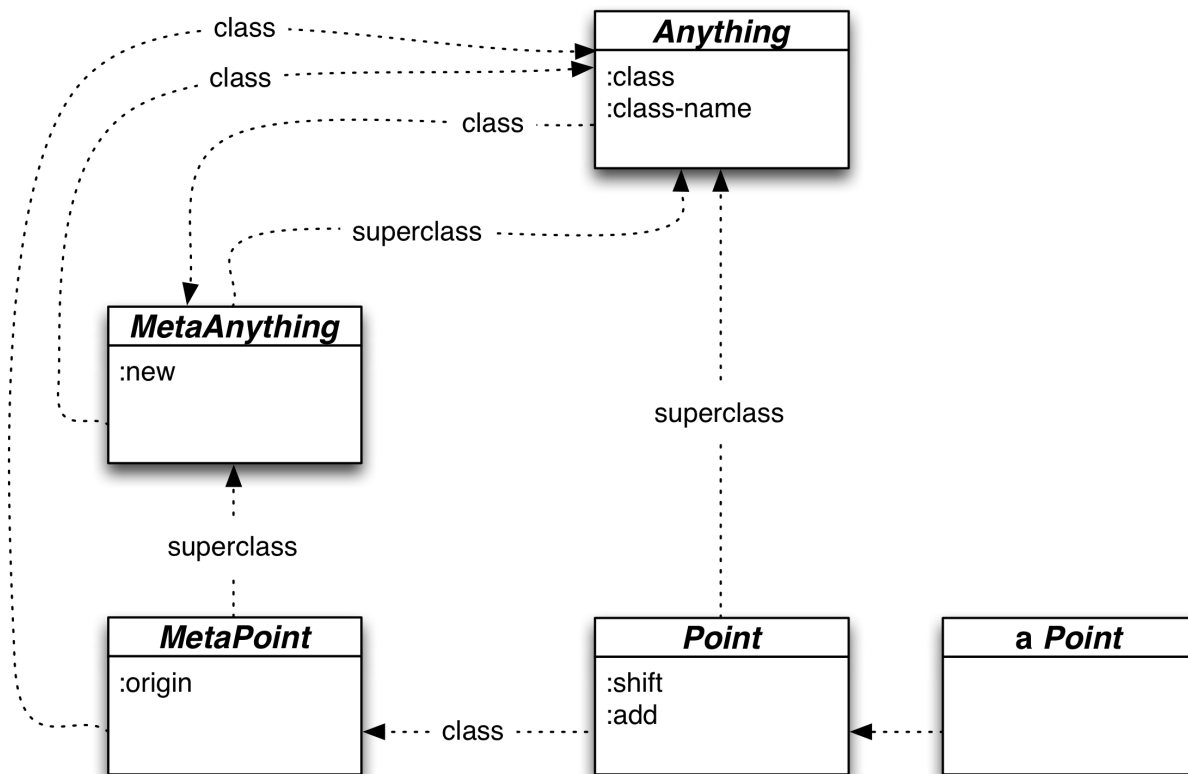
```
1 user=> (send-to MetaPoint :new ...)
```

Hint: I don't think a metaclass should respond to a :new message. (What would it mean to create a new MetaPoint?) Therefore, no class defining :new should be in MetaPoint's class's lineage.

Exercise 4

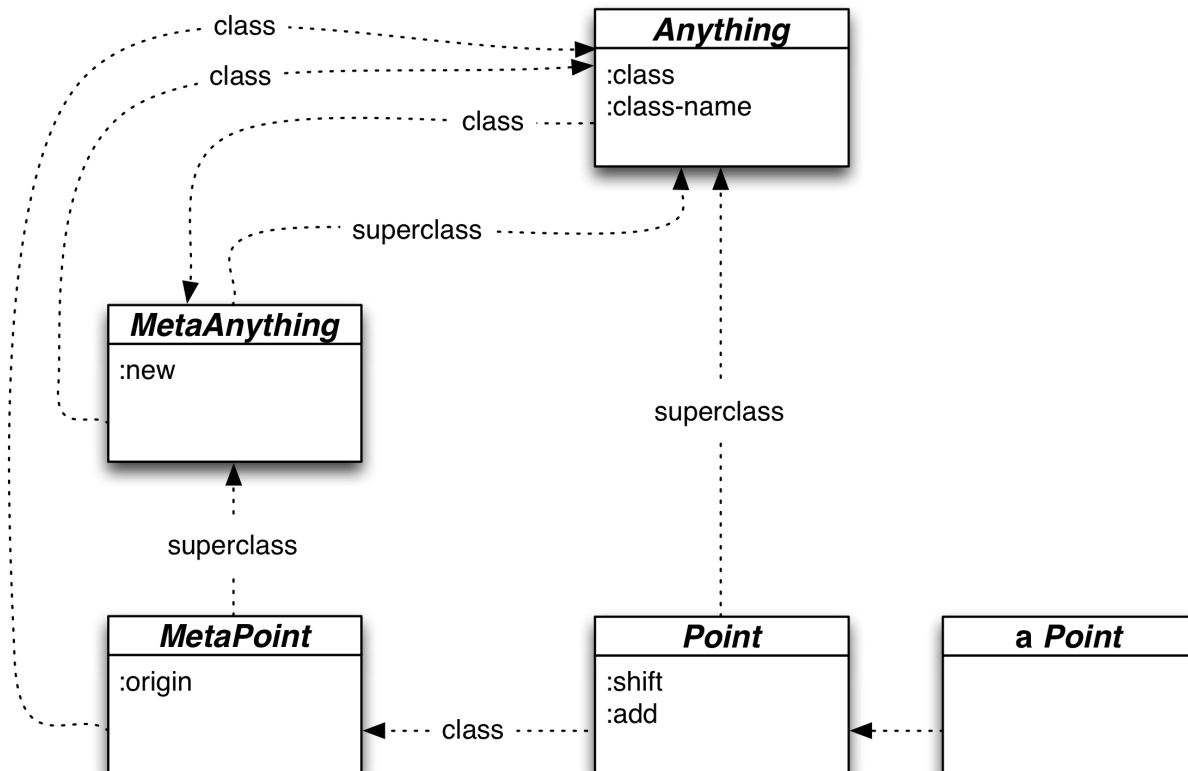
If you look back to the previous picture in this chapter, you'll see a class hierarchy. Revise it to include the changes you've made in these exercises.

My solution follows.



18. The Class That Makes Classes

In the previous chapter's exercises, you produced this class diagram:

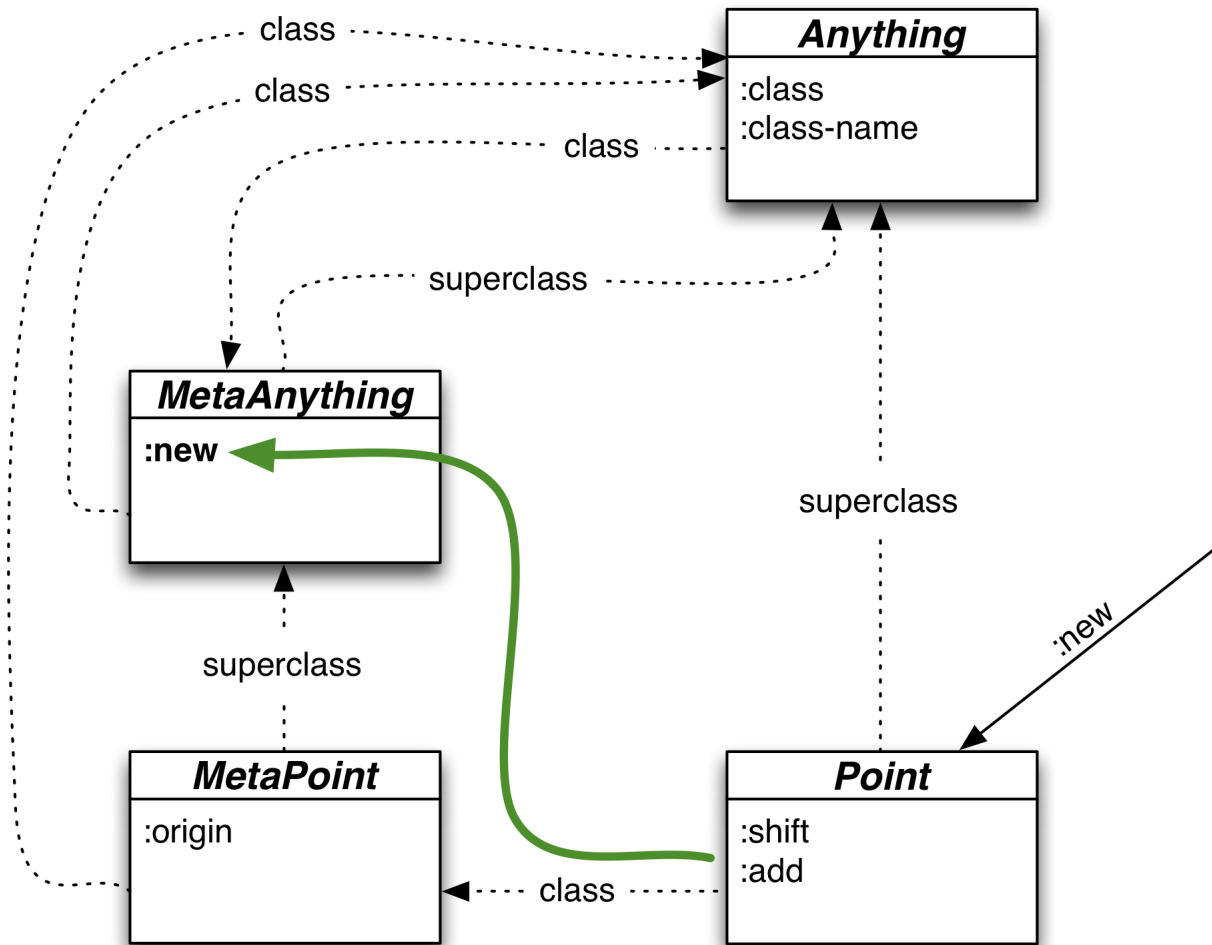


In this chapter, we'll modify it by adding a class that builds other classes. We'll do that in two stages.

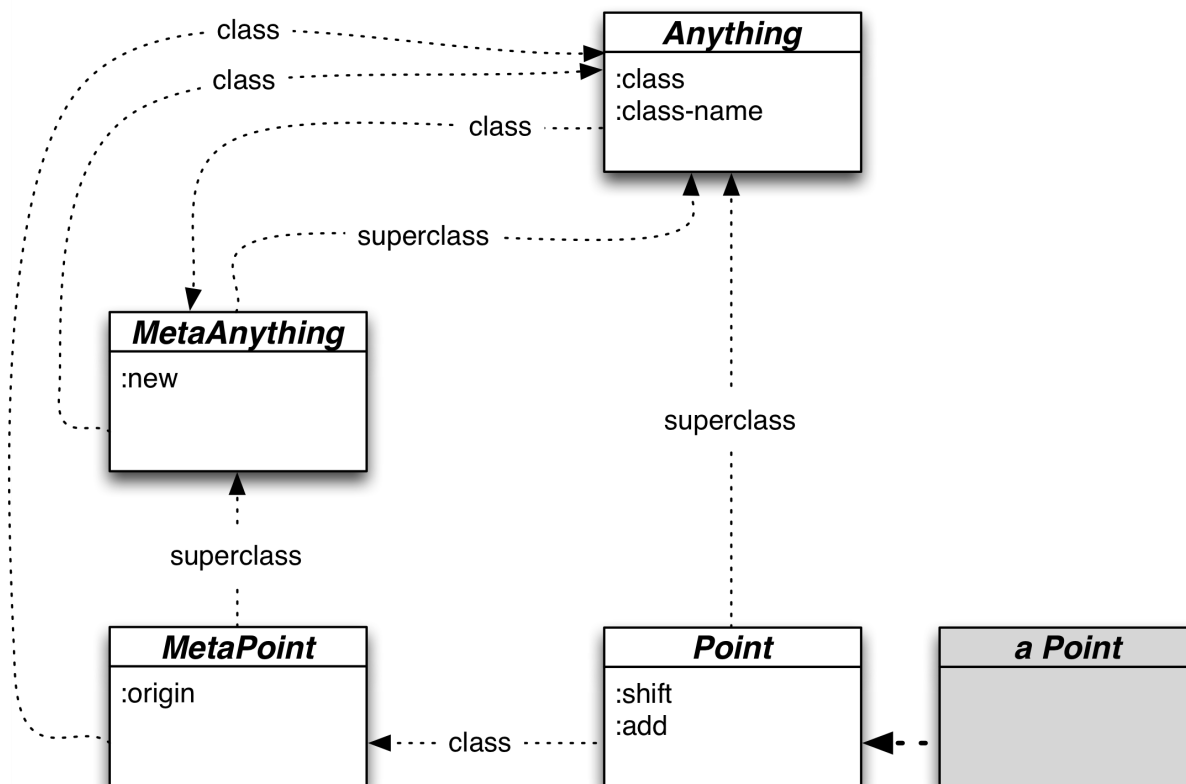
The first stage produces the code in [sources/klass-1.clj](#).

18.1 `klass` in pictures (version 1)

To review, once we have a `Point` class, we create a new point with `(send-to Point :new 1 2)`, which traces this dispatch path through the class structure:



As a result, a **Point** instance appears. It looks left (to its class) to find its methods:

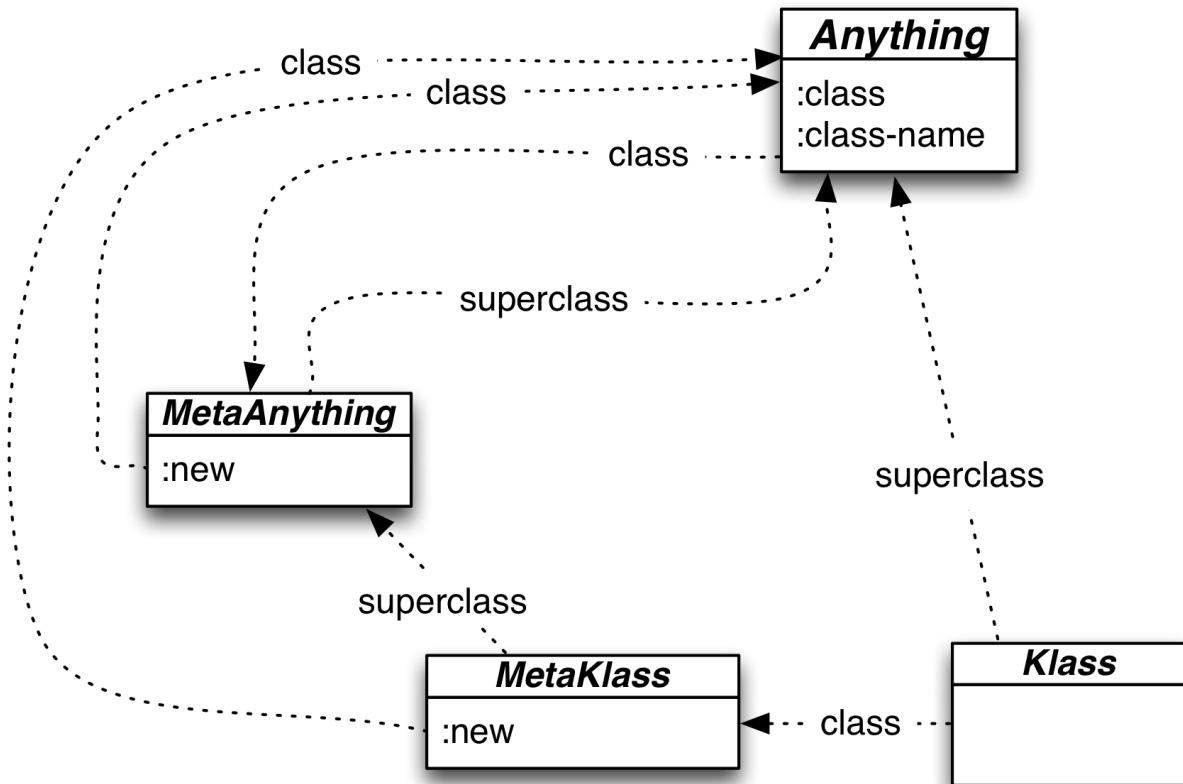


An inelegance with this structure is that there's no way within the object system to make a new class. If the `Point` class doesn't exist, you have to step outside the system, make a new map with a particular (finicky) structure, and then use `def` to give that class/map a symbol name. It would be better to be able to do something like this:

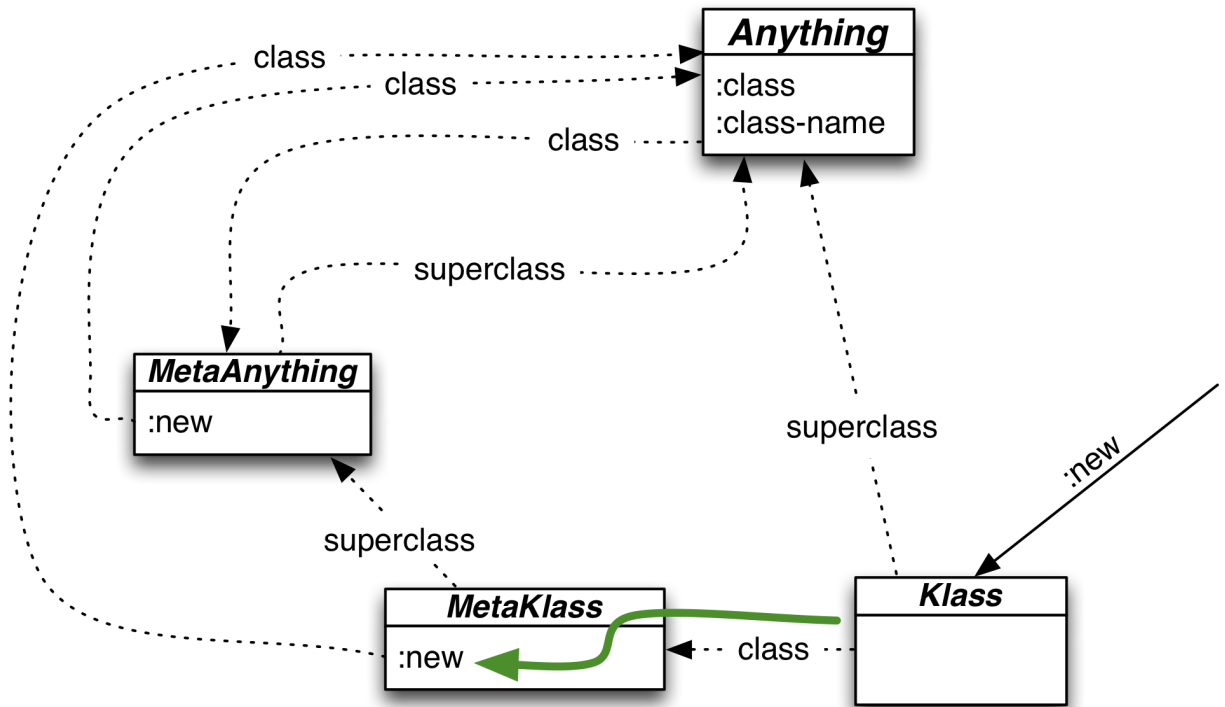
```
1 (send-to Klass :new 'Point ...)
```

(Note: It's more common to name this class `Class`, but—as with `Object`—Clojure reserves that to refer to the core Java class. So I reluctantly misspell it.)

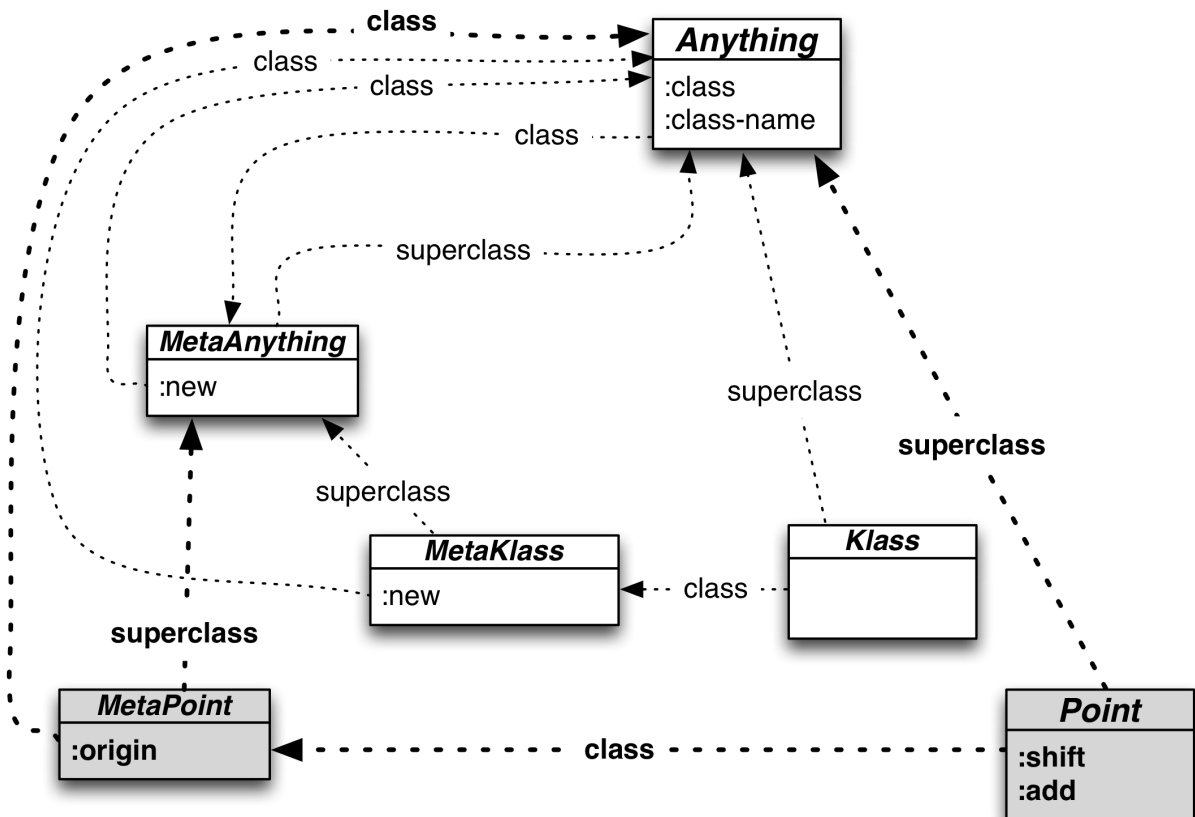
Here's a first version of what that class structure would look like.



Klass and MetaKlass fit into the diagram just as the earlier Point and MetaPoint did—they have the same superclass and class arrows. But there's an important difference: MetaKlass has a `:new` method. Therefore, as shown below, the `(send-to Klass :new 'Point ...)` example above would dispatch to that `:new`, not the one in MetaAnything.

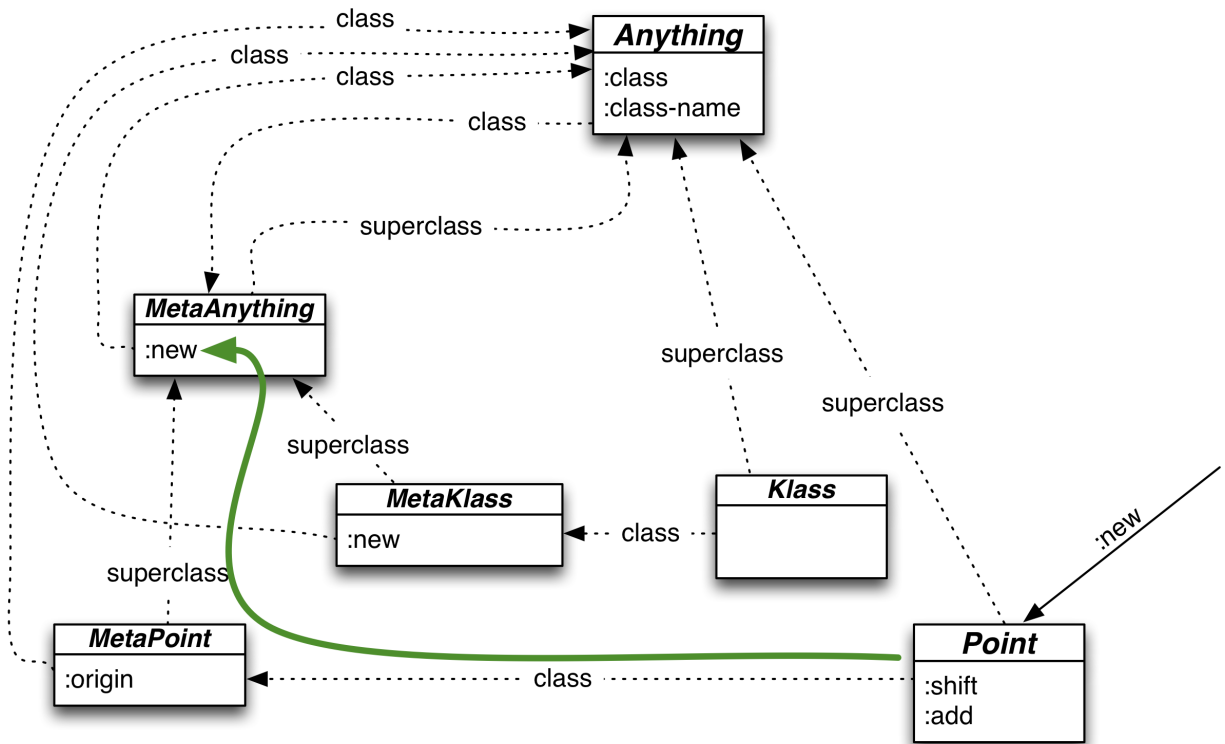


The code for this new version of `:new` (which you'll see in the next two sections) generates the familiar `Point` and `MetaPoint` classes:

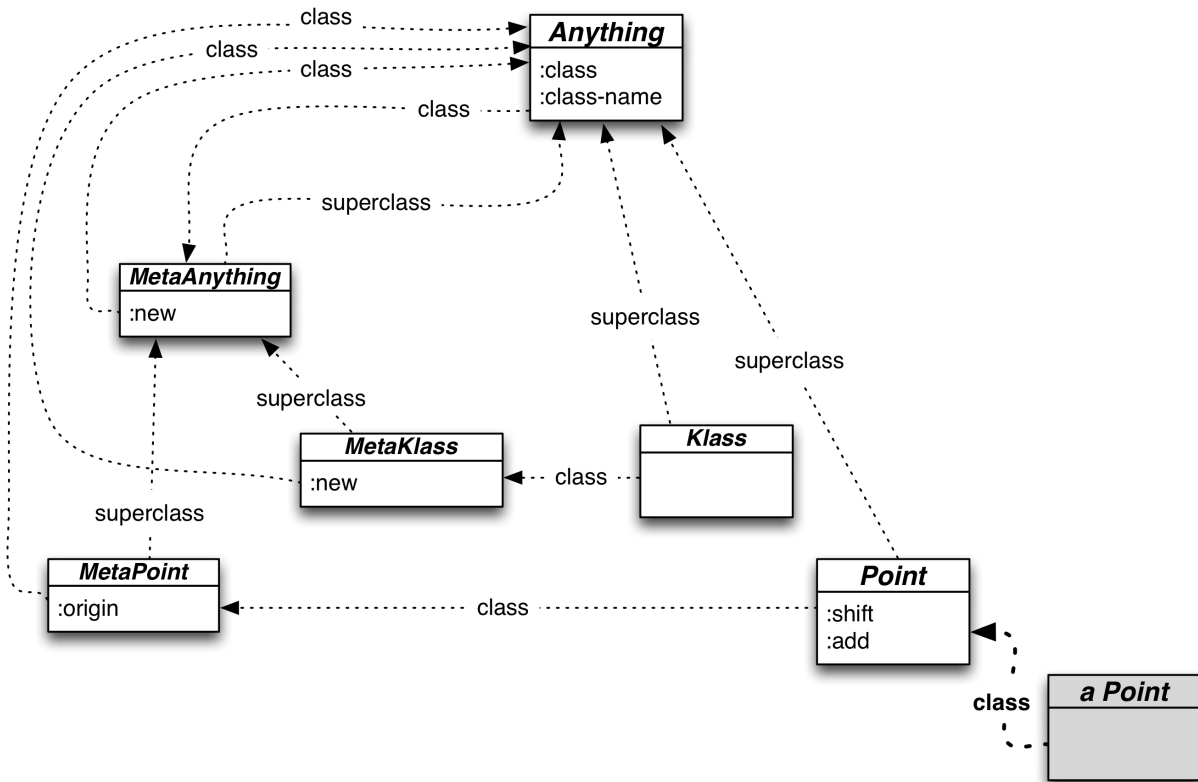


Note the implication of this: manually defining **Anything**, **MetaAnything**, **Klass**, and **MetaClass** bootstraps the object system to the point where we can use it to create all remaining classes.

Once **Klass** creates **Point** and **MetaPoint**, it has nothing more to do with them. Indeed, they have no pointers to it. So the dispatching to create a new **Point** is as before:



And the resulting class structure is as before (except for the two new `Klass` classes):



Now for the implementation.

18.2 The first `klass` implementation

The `:new` method will create both the class and its metaclass. It'll be invoked like this:

```

1 (send-to Klass :new
2   'Point 'Anything
3   {
4     :add-instance-values
5     (fn [this x y]
6       (assoc this :x x :y y))
7
8     :shift
9     (fn [this xinc yinc]
10      (let [my-class (send-to this :class)]
11        (send-to my-class :new
12                  (+ (:x this) xinc)
13                  (+ (:y this) yinc))))
14   }
15
16   {

```

```
17         :origin (fn [class] (send-to class :new 0 0))
18     })
```

It takes four arguments: the name of the class (from which the name of the metaclass will be derived), the name of the superclass, a map from message names to instance methods, and a map from message names to class methods.

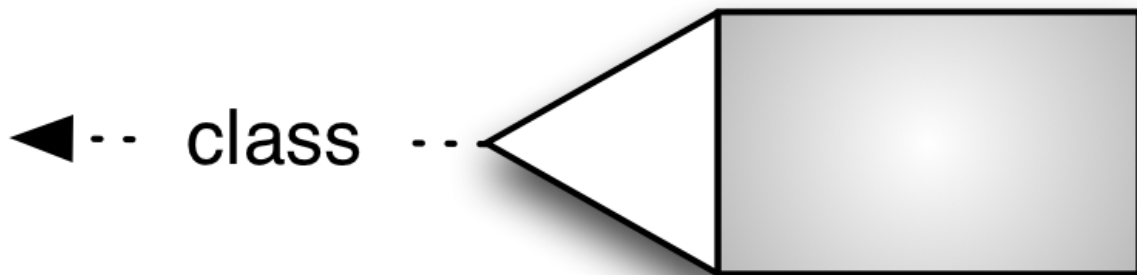
That still looks pretty clunky compared to the syntactically-sugared syntax in popular OO languages, but it's getting closer.

18.3 Class-defining functions

Before defining the method, let's define a few helper functions.

basic-object

`basic-object` makes an object with its bare-minimum map, containing nothing but its class. That is, it constructs this “shape”:



Here's the code:

```
1 user=> (def basic-object
2         (fn [class-symbol]
3           {:__class_symbol__ class-symbol}))
4
5 user=> (basic-object 'Point)
6 {:__class_symbol__ Point}
```

metasymbol

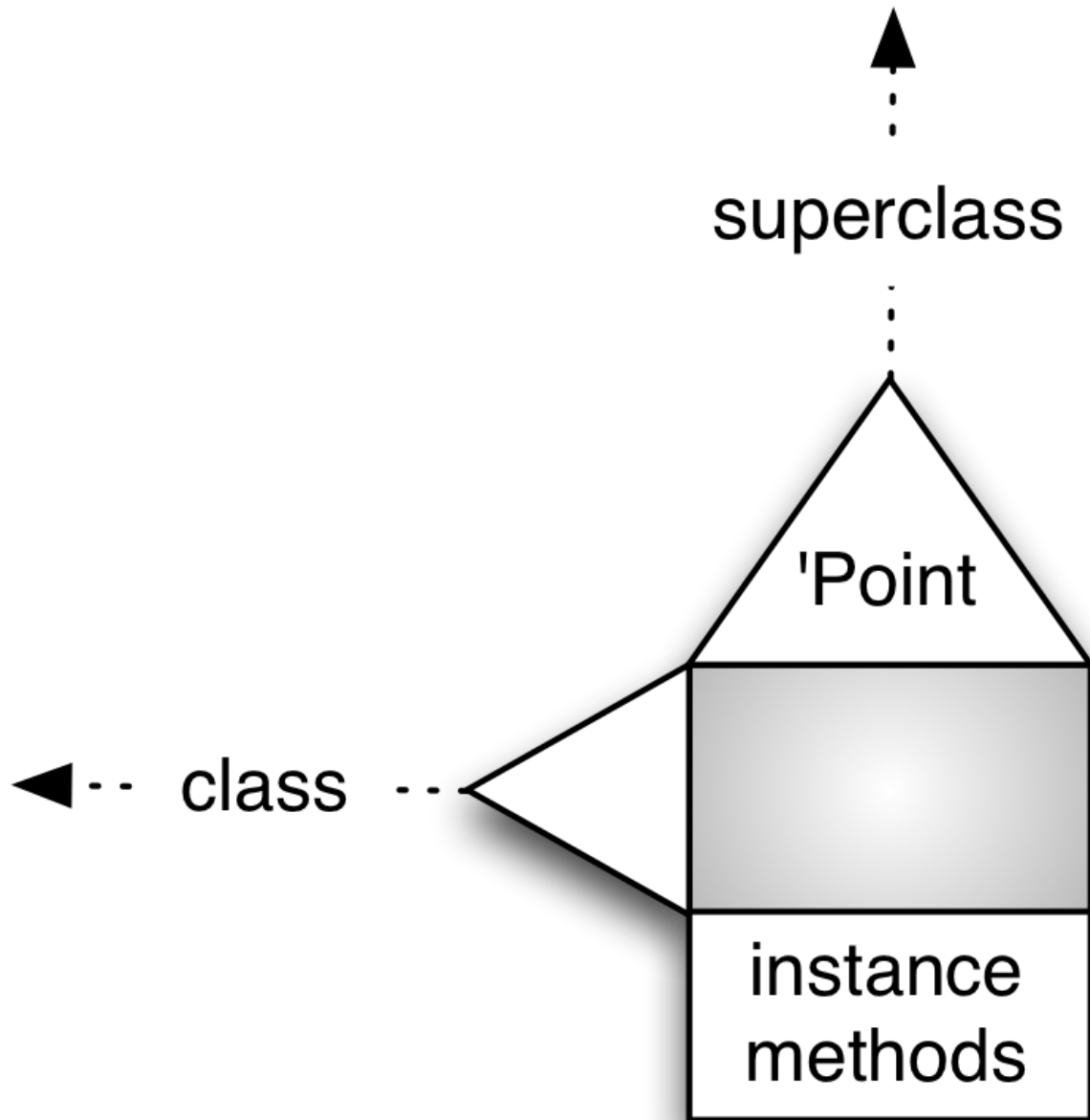
Since our convention for metaclasses is that they always begin with “Meta”, let’s make a function that creates a symbol naming a metaclass from a symbol naming a class:

```
1 (def metasymbol
2   (fn [some-symbol]
3     (symbol (str "Meta" some-symbol))))
```

That way, we won’t always have to pass metaclass and class names around together.

basic-class

`basic-class` makes a basic object that has a class’s three additional bits of metadata, as shown in this picture:



Because we'll have to use `basic-class` directly (not via `send-to Klass :new`) when creating `Anything`, `Klass`, and their metaclasses, I'll make its uses more readable by adding dummy keywords to the argument list:

```
1 user=> (basic-class 'Point ; Name of new class
2          :left 'MetaPoint ; Its class
3          :up 'Anything ; Its superclass
4          {:x :x}) ; Instance methods
5 {:__class_symbol__ MetaPoint
6   :__superclass_symbol__ Anything,
```

```

7  :__own_symbol__ Point,
8  :__instance_methods__ {:x :x}}

```

The code looks like this:

```

1  (def basic-class
2    (fn [my-name
3        _left left-symbol
4        _up up-symbol
5        instance-methods]
6      (assert (= _left :left))
7      (assert (= _up :up))
8      ;; Note that a class is just a basic object
9      ;; with more metadata
10     (assoc (basic-object left-symbol)
11            :__own_symbol__ my-name
12            :__superclass_symbol__ up-symbol
13            :__instance_methods__ instance-methods)))

```

install

Because (send Klass :new ...) creates both a class and its metaclass, we can't type something like this:

```

1 user=> (def Point (send-to Klass :new 'Point ...))

```

The :new method will have to create two bindings (Point and MetaPoint). It'll use an install method, something like this:

```

1      ...
2      :new
3      (fn [class-symbol superclass-symbol instance-methods
4          class-methods]
5        ...
6        (install (basic-class class-symbol...))
7        (install (basic-class superclass-symbol...))
8        ...)
9      ...

```

Here's a possible implementation of install:

```

1  (def install
2    (fn [class]
3      (def (:__own_symbol__ class) class)))

```

Won't work. The problem is that `def` is a special symbol that doesn't evaluate its first argument. Using this version of `install` would result in Clojure bitterly complaining that you gave a list where a symbol was expected.

Fortunately, there's a non-special function works equivalently:

```
1 user=> (intern *ns* 'some-symbol (+ 5 5))
2 user=> some-symbol
3 10
```

You haven't seen `*ns*` before. It has the current *namespace* as its value. A namespace is like a package or module in other languages: it makes name clashes less likely. In this book, you're doing all your work in the `user` namespace.

So here's `install`:

```
1 (def install
2   (fn [class]
3     (intern *ns* (::__own_symbol__ class) class)
4     class))
```

I'm returning the `class` argument because that will later be a convenient way to make the `:new` method return the class it created.

:new

That given, here's `:new`:

```
1 (install (basic-class 'MetaClass,
2           :left 'Anything,
3           :up 'MetaAnything,
4           {
5             :new
6             (fn [this
7                 new-class-symbol superclass-symbol
8                 instance-methods class-methods]
9               ;; Metaclass
10              (install
11                (basic-class (metasymbol new-class-symbol)
12                             :left 'Anything
13                             :up 'MetaAnything
14                             class-methods)))
```



```

15                                     ;; Class
16                                     (install
17                                     (basic-class new-class-symbol
18                                     :left (metasymbol new-class-
symbol)
19                                     :up superclass-symbol
20                                     instance-methods))))))

```

You can find the complete source for the new object system in [sources/klass-1.clj](#). In rough outline, it looks like this:

```

1  ;;; The two class/metaclass pairs from which everything else can be built
2
3  ;; Anything
4  (install (basic-class 'Anything ...))
5  (install (basic-class 'MetaAnything ...))
6
7  ;; Klass
8  (install (basic-class 'Klass ...))
9  (install (basic-class 'MetaKlass ...))
10
11 ;;; The remaining predefined classes
12
13 ;; Point
14 (send-to Klass :new
15             'Point 'Anything
16             {...}
17             {...}

```

This is starting to look *real*!

18.4 klass in pictures (version 2)

The rest of this chapter will add the code in [sources/klass-2.clj](#) to the code in [sources/klass-1.clj](#).

There's a deficiency in version 1 of this object model. Suppose we use `Klass :new` to make `Point`, `Circle`, `Alphabet`, and `Firehose` classes. What do they have in common? Why, that they're all classes. And that's indeed the answer you'd get from Ruby:

```

1 irb(main):004:0> Point.class
2 => Class

```

```
3 irb(main):005:0> Alphabet.class
4 => Class
```

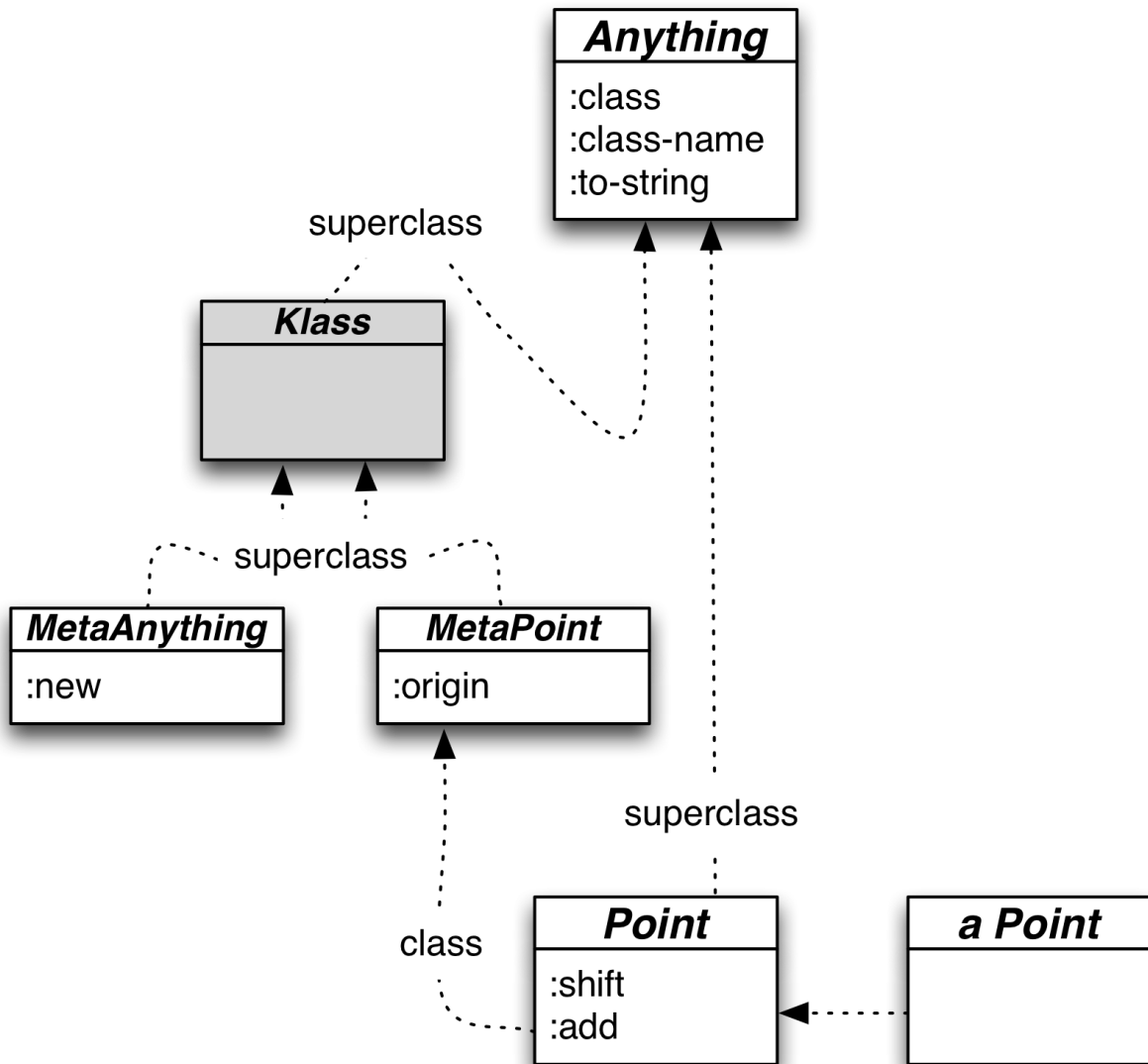
In our current implementation, we get something else:

```
1 user=> (send-to Point :class-name)
2 MetaPoint
3 user=> (send-to Alphabet :class-name)
4 MetaAlphabet
```

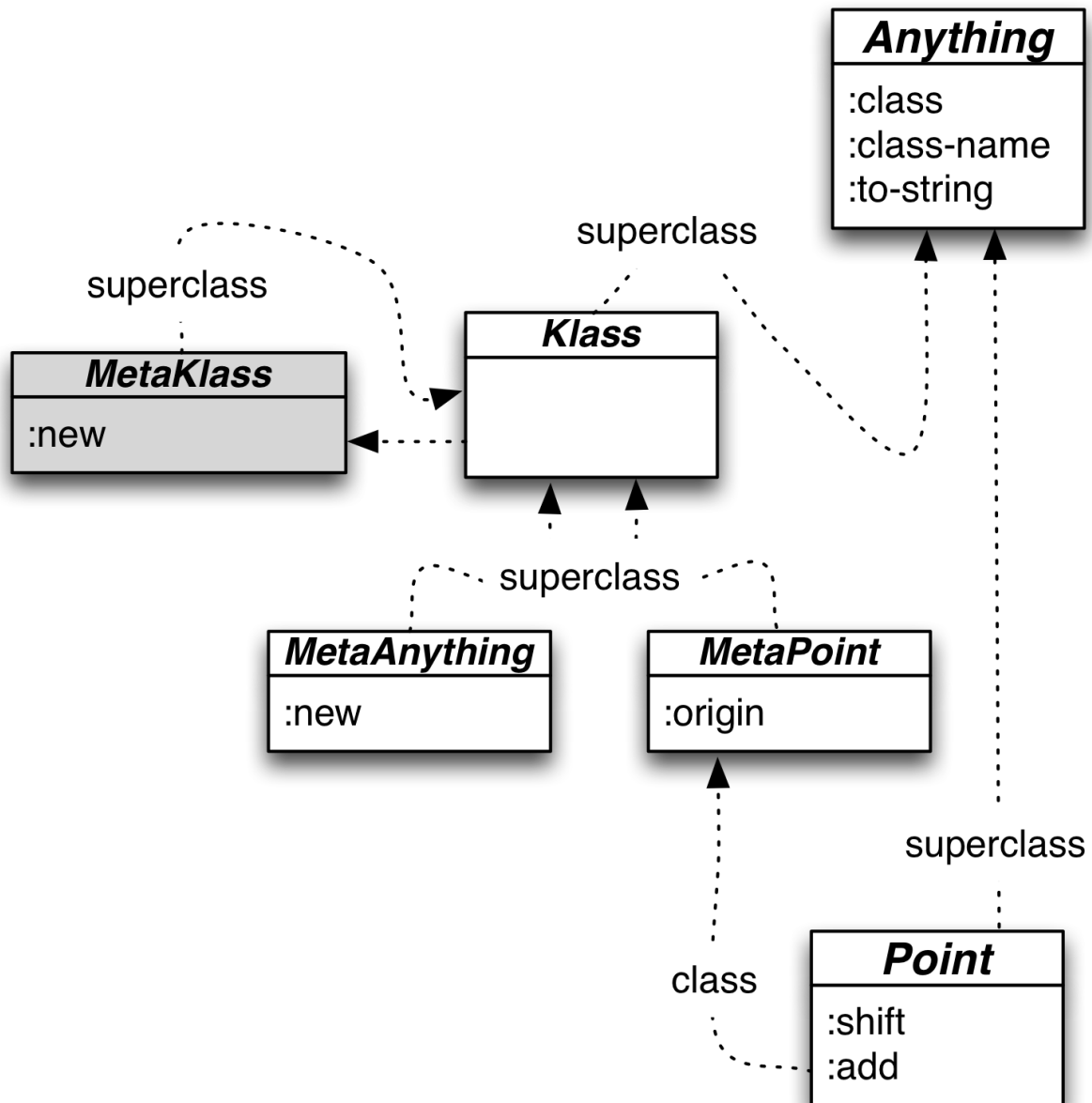
Given the one-to-one relationship between classes and metaclasses, this is not useful information. So we'll follow Ruby's lead and make metaclasses (mostly) invisible to the programmer. In an exercise, you'll change `:class-name` and `:class` so that they follow superclass links upward until they discover a visible class.

However, that's still not a complete solution, because the class above the now-invisible `MetaPoint` is the also-invisible `MetaAnything`. So searching up a superclass hierarchy for the first visible class would find... `Anything`. Which is no more useful than `MetaPoint`, since *everything* is an `Anything` and we want a name for what's special about `Point` (which is that it's a class).

To avoid this grim fate, we need something more informative below `Anything` in a metaclass's superclass chain. Specifically, we should—as pictured below—adjust the class structure to make `Klass` a superclass of all metaclasses. Therefore, a search upward would find it as the `:class-name` to return.



We need to add `MetaKlass` to the picture. It must stay to the left of `Klass` so that it can handle messages sent to `Klass`, most particularly `:new` to create a new class. But what is the superclass of `MetaKlass`? The same as any other metaclass: `Klass`. That gives us this:



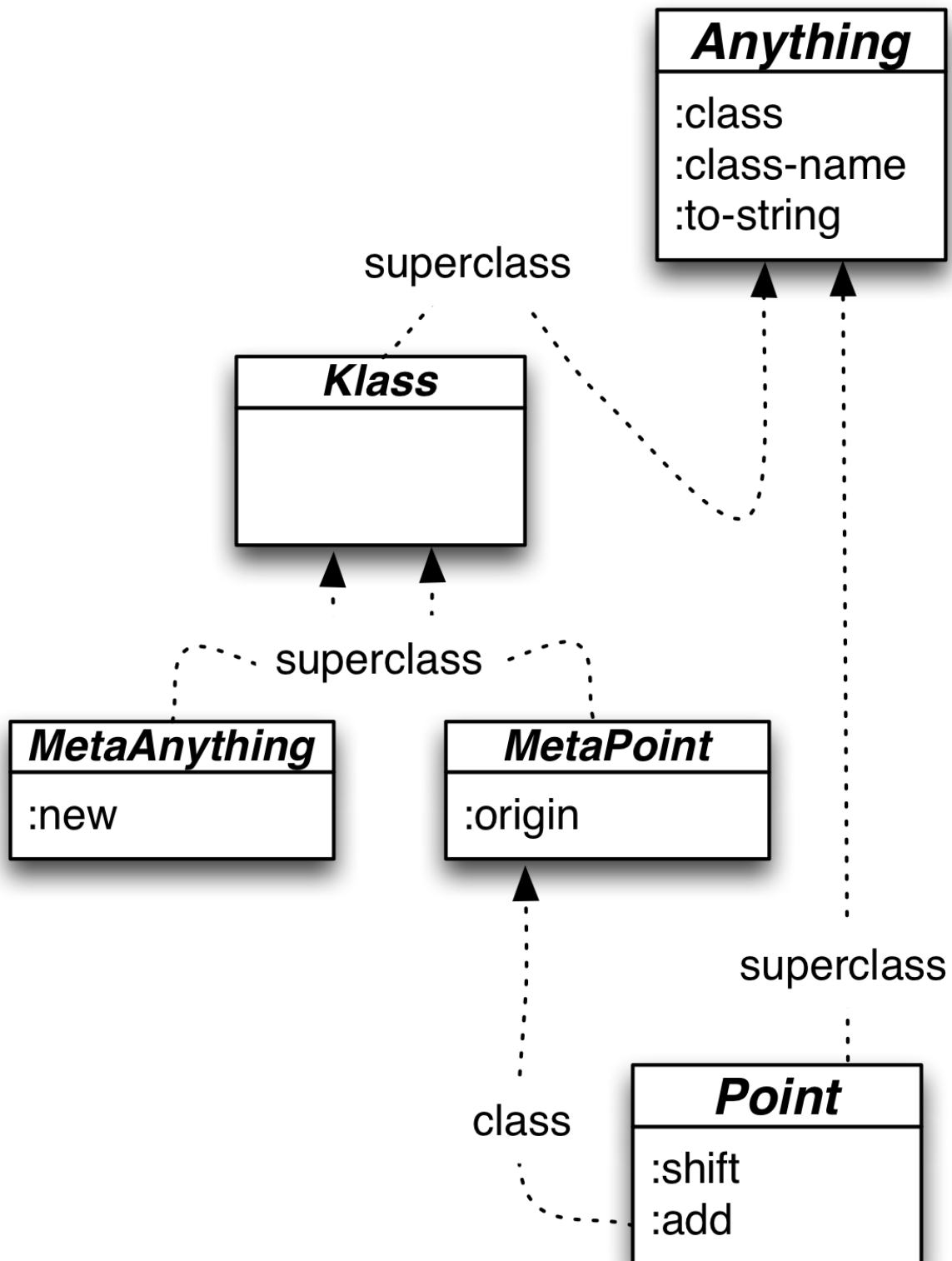
Note that the loop in the graph can't lead to a loop in message dispatch. Once a superclass link is followed, you never go left again via the class link.

18.5 The revised `klass` implementation

Although the many pictures in the previous section make the change from this chapter's beginning picture seem terribly complicated, in actuality the only change is three places where the superclass pointer gets changed from `Anything` to `Klass`. They are the links in the predefined `MetaAnything` and

MetaKlass, and to the code in the :new method that creates new metaclasses.

However, I overlooked a needed change. Consider this subset of the hierarchy:

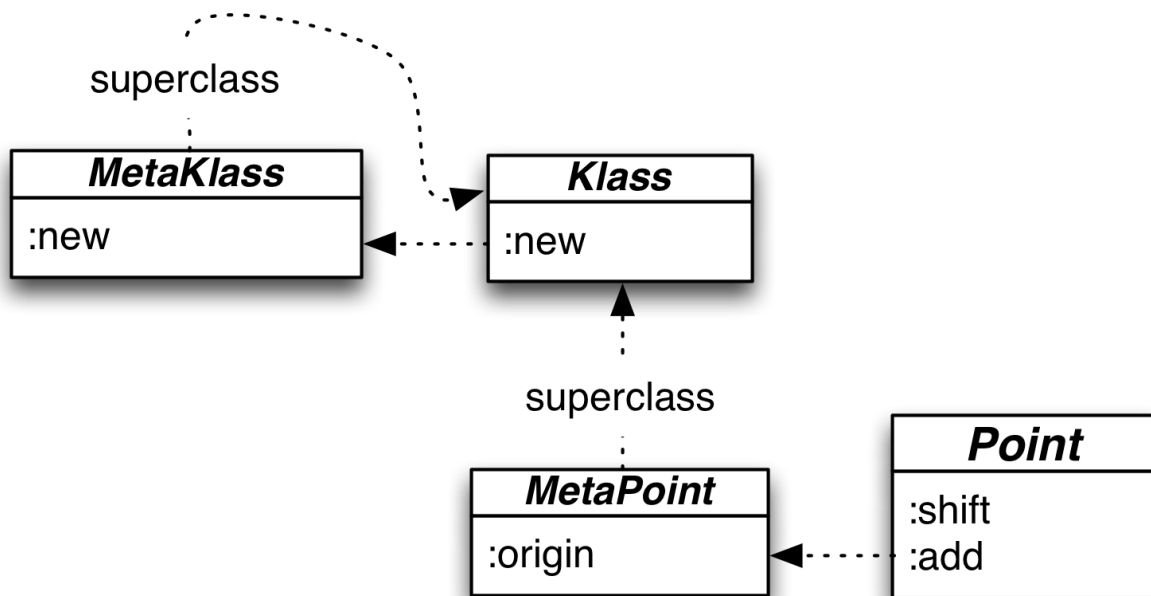


Trace what happens for this code:

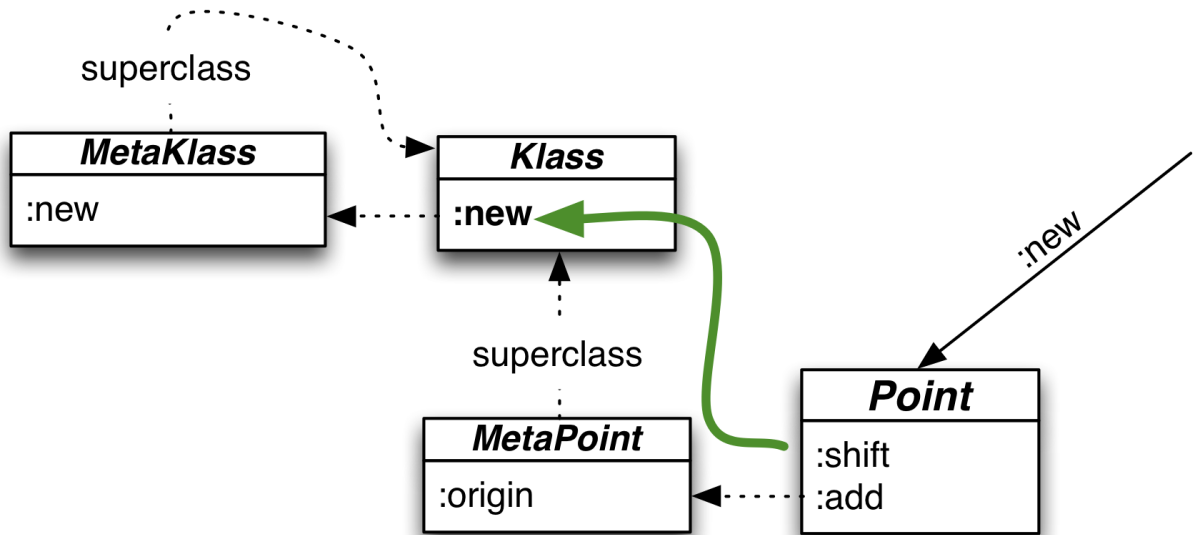
```
1 user=> (send-to Point :new 1 2)
```

You'll see that there's no way for the dispatch function to get to `MetaAnything` and find the `:new` there. `:new` needs to be moved into the path. Putting it in `MetaPoint` would be wrong, since `:new` should be behaviors shared by all classes. Putting it in `Anything` is inappropriate, since that would mean `:new` could be sent to an *instance* of `Point`, not just the `Point` class. (A `:new` sent to a `Point` instance would look left to `Point` and then up the superclass link to `Anything`.)

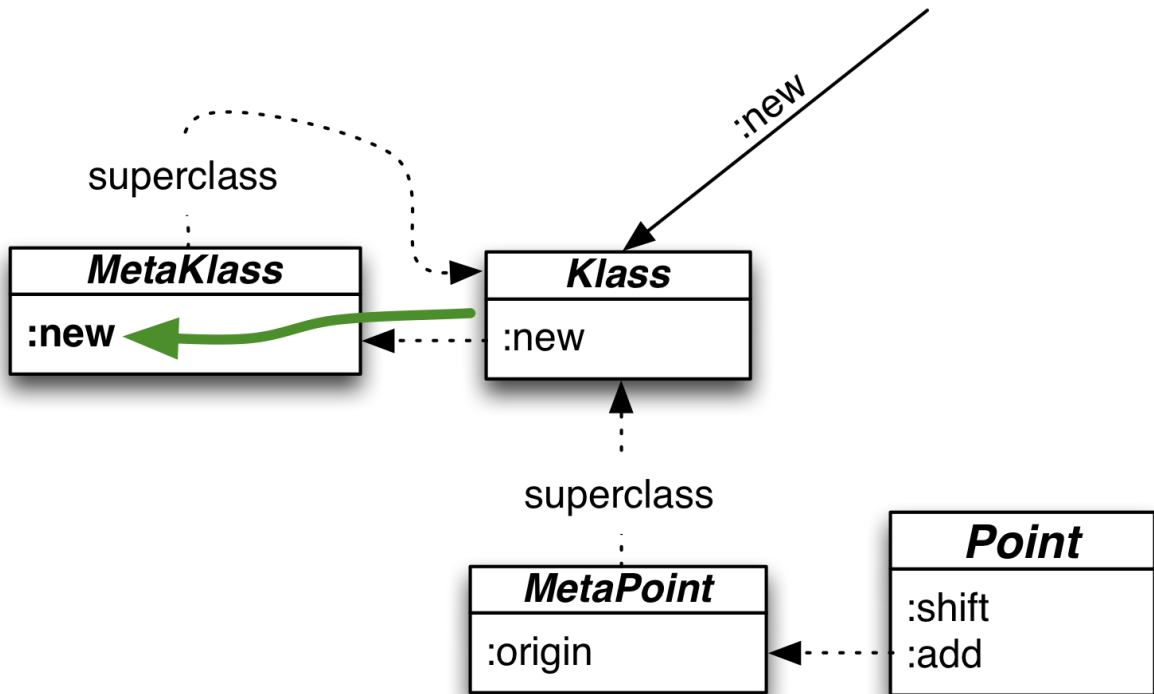
The right place to put it is in `Klass`, which leads to this appealingly compact situation:



The `Klass` pair of classes is responsible for all “newing” of objects. `Klass` acts (via the superclass link) as a metaclass for all objects-that-are-classes. Therefore `(send-to Point :new)`, `(send-to Alphabet :new)`, and so on all invoke `Klass`’s *instance* method to create new instances. Like so:



What happens when you send `:new` to `Klass`? The same thing as always: a look to the left and then (if necessary) up. In this case, the look to the left is enough:



That is:

1. In the initial system, all of Anything, Point, and Klass are classes: they are all capable of creating instances in response to :new.
2. Klass has its own version of :new, one that knows how to create other classes.
3. The proper definition of what it means to be a class is that it is an object whose metaclass is a descendent of Klass. (Klass is a descendent of itself because of the circularity between it and its metaclass.)

You can find the three superclass changes and the moved :new in [sources/klass-2.clj](#).

It's interesting that it took a lot of text and pictures to describe and justify a change that amounted to moving one method and changing three arrows. Things can get pretty subtle up near the top of the class hierarchy.

18.6 Exercises

My solution is in [solutions/klass.clj](#).

Exercise 1:

It would be nice to have a version of :to-string specialized for classes:

```
1 user=> (send-to Point :to-string)
2 "class Point"
3 user=> (send-to Klass :to-string)
4 "class Klass"
5 user=> ;; As before:
6 user=> (send-to (send-to Anything :new) :to-string)
7 "{:__class_symbol__ Anything}"
8 user=> (send-to (send-to Point :new 1 2) :to-string)
9 "{:y 2, :x 1, :__class_symbol__ Point}"
```

Implement it.

Exercise 2:

A class should be able to provide a sequence of its superclass symbols, omitting those that are supposed to be invisible (that is, metaclasses). Following Ruby, the class itself will be included in the sequence:

```

1 user=> (send-to Point :ancestors)
2 (Point Anything)
3 user=> (send-to ColoredPoint :ancestors)
4 (ColoredPoint Point Anything)
5 user=> (send-to Klass :ancestors)
6 (Klass Anything)

```

Note that the order of symbols is the opposite of the one lineage supplies.

Because metaclasses are not included in `:ancestors`, asking for the ancestors of a metaclass does *not* include it:

```

1 user=> (send-to MetaPoint :ancestors)
2 (Klass Anything)

```

Be sure to test your solution against both built-in classes and ones freshly created with `(send-to Klass :new)`.

Hint: You can always add metadata to classes. You can even add it after the class has been created, like this:

```

1 (def Point (assoc Point :__new_metadata__ "value"))

```

... or even like this:

```

1 (install
2   (assoc (basic-class ...)
3         :__new_metadata__ "value"))

```

Exercise 3:

Implement `class-name` and `class` so that they ignore invisible classes:

```

1 user=> (send-to Point :class-name)
2 Klass ; NOT MetaPoint---the class above it

```

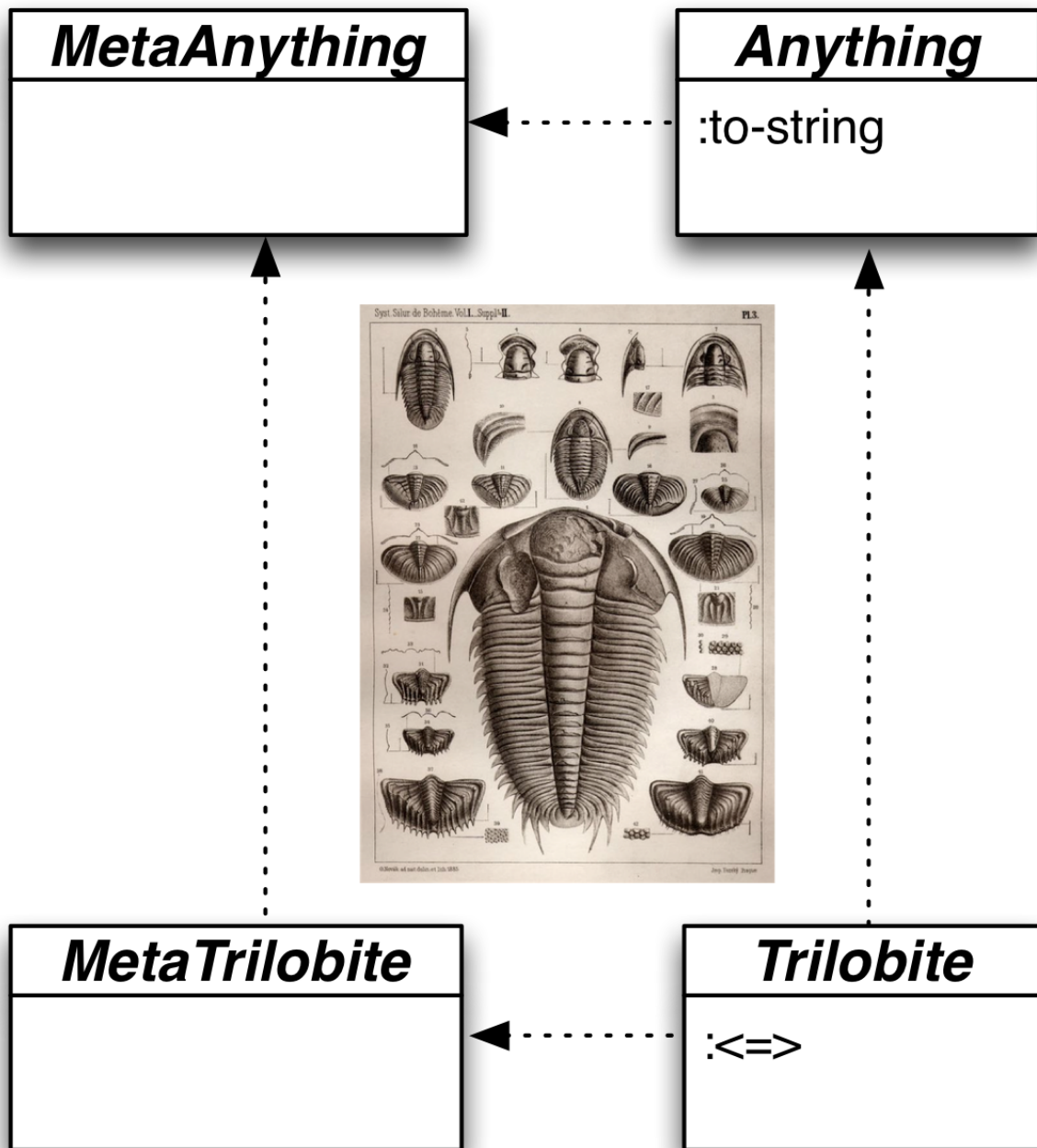
Hint: Use `:ancestors`.

Hint: I found it most convenient to implement `:class-name` first and have `:class` use it.

19. Multiple Inheritance, Ruby-style

In this chapter, I'll show how Ruby's variant of multiple inheritance would look in our object system. You'll implement it in the exercises.

Consider a class diagram for the trilobite, the favorite fossil of many a discerning enthusiast^{[1](#)}:



(I've simplified the class diagram by removing class `Klass`.)

If you have a set of trilobites, you can order them according to the number of lenses in an eye (which ranges from one to thousands). That means that the `Trilobite` class should provide methods like these: `:<`, `:<=`, `:=`, `:>`, `:>=`, and `:between?`.

When thinking about those methods, you might notice that they can all be defined in terms of a single one, `:<=>`, which Ruby calls “the spaceship method” (because it looks a little like a flying saucer). The spaceship method returns `-1` if `this` is smaller than the single argument, `0` if they’re equal, and `1` if the argument is greater.

Here are those definitions:

```
1      {:= (fn [this that] (zero? (send-to this :<=> that)))
2      :> (fn [this that] (= 1 (send-to this :<=> that)))
3      :>= (fn [this that] (or (send-to this := that)
4                           (send-to this :> that)))
5
6      :< (fn [this that] (send-to that :> this))
7      :<= (fn [this that] (send-to that :>= this))
8
9      :between?
10     (fn [this lower upper]
11       (and (send-to this :>= lower)
12            (send-to this :<= upper))))}
```

Imagine repeating those six definitions in every class that wants a “natural order” for instances. It would be far better to have such classes define only `:<=>` and get the other methods for free.

Ruby accomplishes that through its notion of *modules*. Modules are like classes, except that you can’t make instances of them. Instead, you “mix them in” or “include them into” classes. Doing so adds the module’s methods to the class. For example, here’s how a Ruby `Trilobite` class could include the `Comparable` module to get the six additional methods:

```
1 class Trilobite
2   def <=> [other]
3     self.eyes.count <=> other.eyes.count
4   end
5
6   include Comparable
7 end
```

In our embedded object language, I want to do this:

1. Begin with a spaceship operator for integers:

```

1 (def <=>
2   (fn [a-number another-number]
3     (max -1 (min 1 (compare a-number another-number)))))

```

2. It can be used to define how to compare trilobites:

```

1 (send-to Klass :new
2   'Trilobite 'Anything
3   {
4     ; ...
5     :<=>
6     (fn [this that]
7       (<=> (send-to this :facets)
8           (send-to that :facets)))
9   })

```

3. Then we should be able to tell Trilobite to define all six comparison functions in terms of its own :<=>:

```

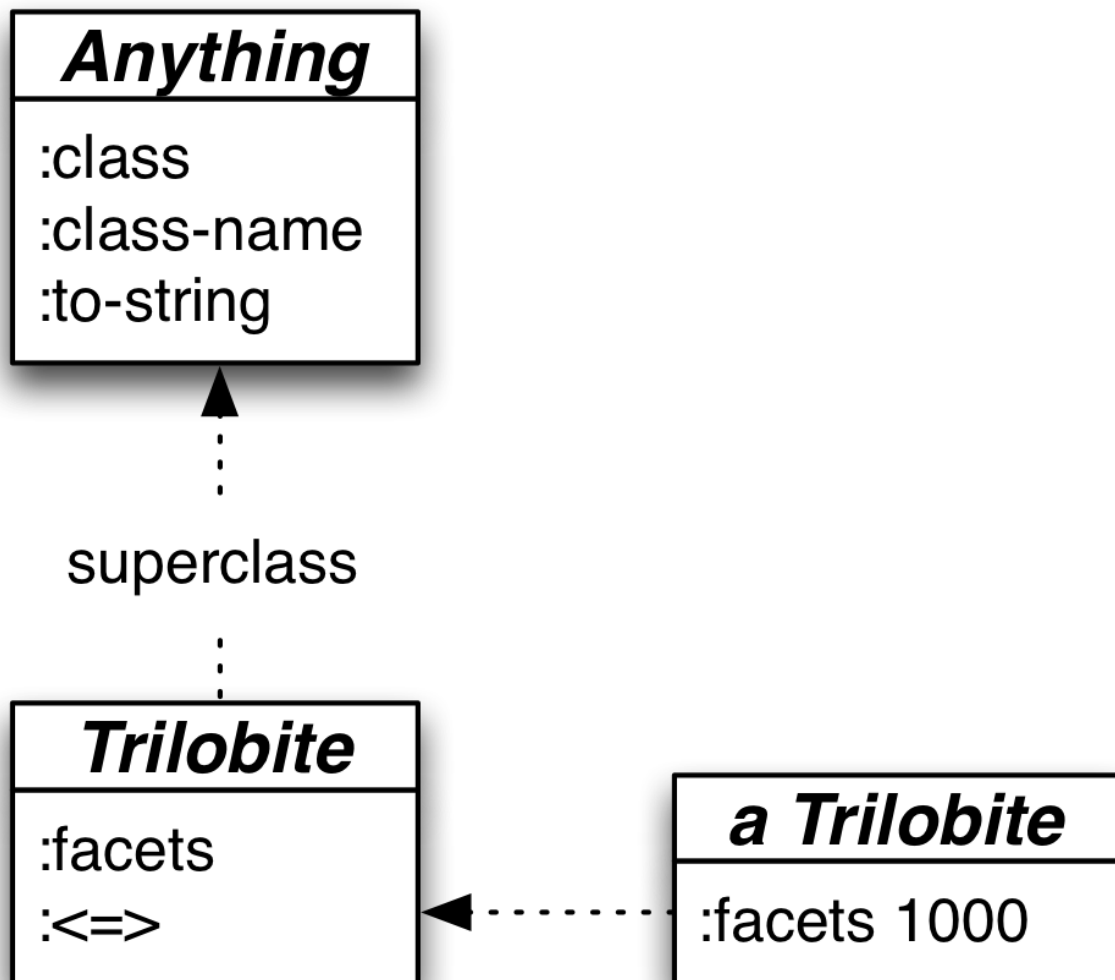
1 ;; `Komparable` has the six method definitions.
2 ;; The funny name is because Clojure already has a `Comparable`.
3 (send-to Trilobite :include Komparable)

```

Modules names are typically adjectives like Comparable, Enumerable, and observable. Think of them as modifying the meaning of the classes (nouns) they're included into.

19.1 New terminology

We're used to using the words "class" and "superclass" when describing how instances find their methods. For example, consider this diagram:



Let's call the `Trilobite` instance `fred`. If `fred` is sent the `:facets` message, the corresponding method is found by following `fred`'s `:__class_symbol__` left. If `fred` is sent the `:to-string` message, the method is found by following the `:__class_symbol__` left and then the `:__superclass_symbol__` up.

As you'll see shortly, the methods contained in modules are found by the same sort of "look left, then up" pattern. But that makes `:__class_symbol__` and `:__superclass_symbol__` bad names. Sometimes what's to the left is a module, not a class. The object above a module isn't usefully described as its superclass.

Because of that, I'm going to change the keywords used in the code:

Old	New
:__class_symbol__	:__left_symbol__
:__superclass_symbol__	:__up_symbol__

For the same reason, I'll be dropping the labels “class” and “superclass” from arrows in diagrams.

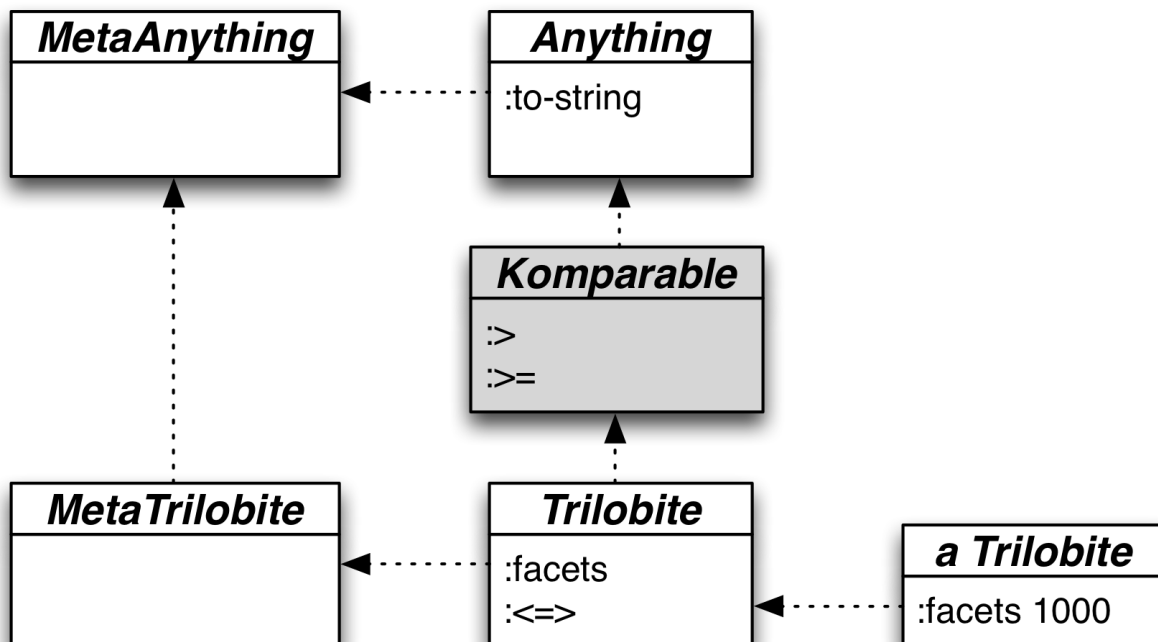
Finally, since the objects at the ends of those arrows might be either classes or modules, we need some collective term. From now on, we'll talk of *method holders*. Functions that used to have names like `class-symbol-` above will be renamed `method-holder-symbol-` above.

19.2 How modules work

Given this code:

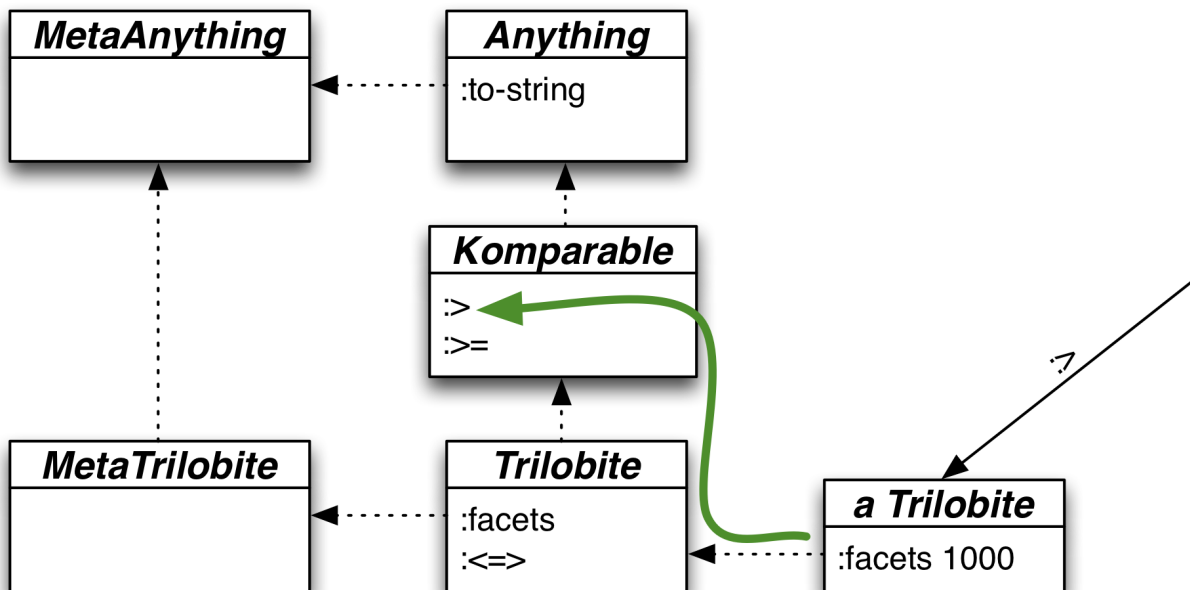
```
1 (send-to Trilobite :include Komparable)
```

... what does `:include` actually do? To a first approximation, this:



Let's step through what happens when a `Trilobite` instance receives the `>` message. As always, we look left and up, finding the version in

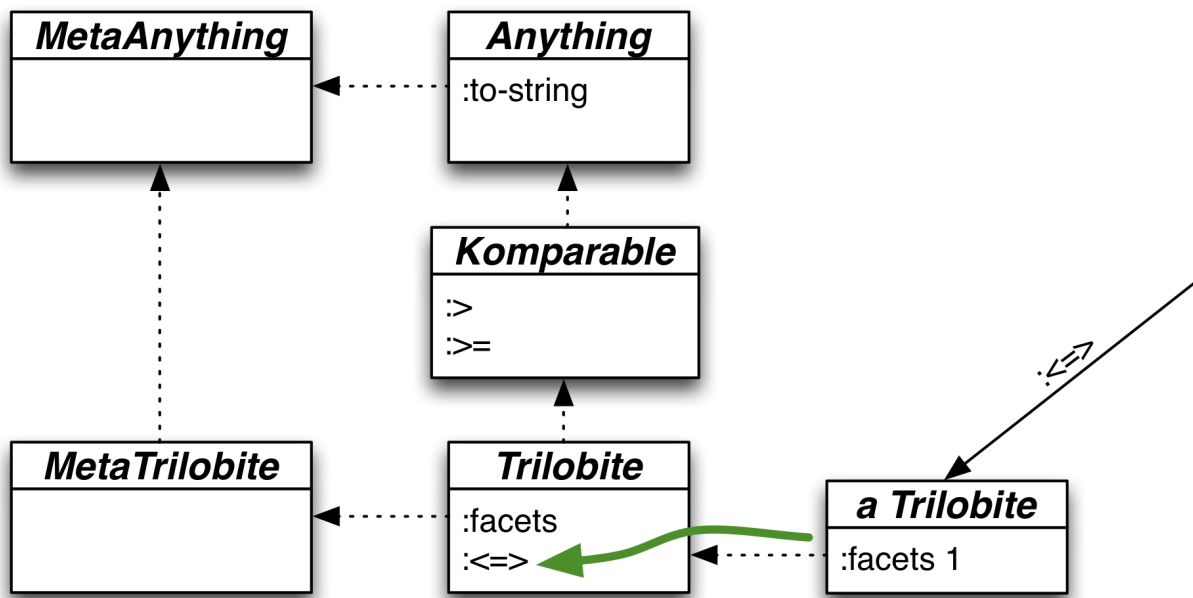
Komparable:



The `:>` method is defined like this:

```
1 (fn [this that] (= 1 (send-to this :<=> that)))
```

Therefore, the module sends a different method to the same `this` (the same instance). Like this:

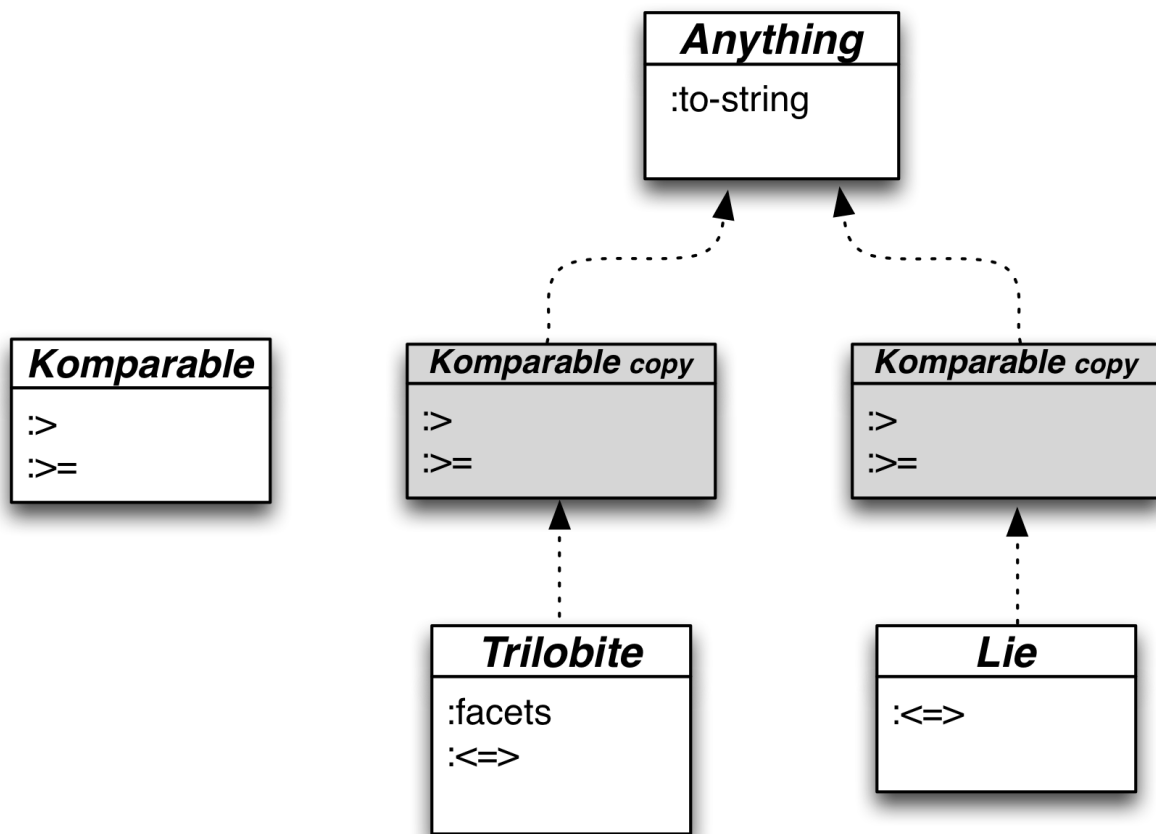


The module is behaving just like a class.

(Notice that module inclusion doesn't affect metaclasses. For that reason, I'll leave them out of diagrams from now on.)

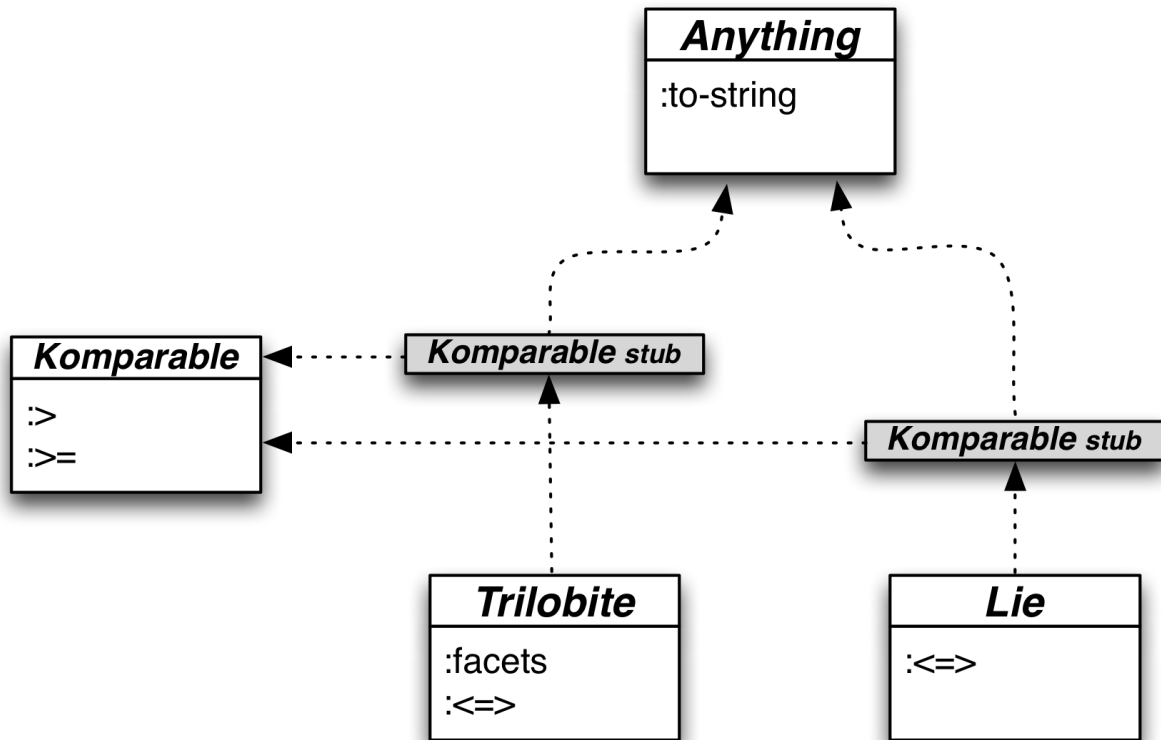
That first approximation falls apart quickly, though. It means that `Komparable` can only be included into one class, since it can only have one `:__up_symbol__` link. But the whole point of `Komparable` is that we should be able to `:include` it in *any* class that defines `:<=>`.

A solution to that problem would be to set the `:__up_symbol__` link in a copy of the module:



But that brings with it a second problem: now we can't change `Komparable` and have those changes be seen by `Trilobite` and `Lie`. In languages with repls (like Clojure and Ruby), being unable to redefine classes, functions, and other such values is bad form.

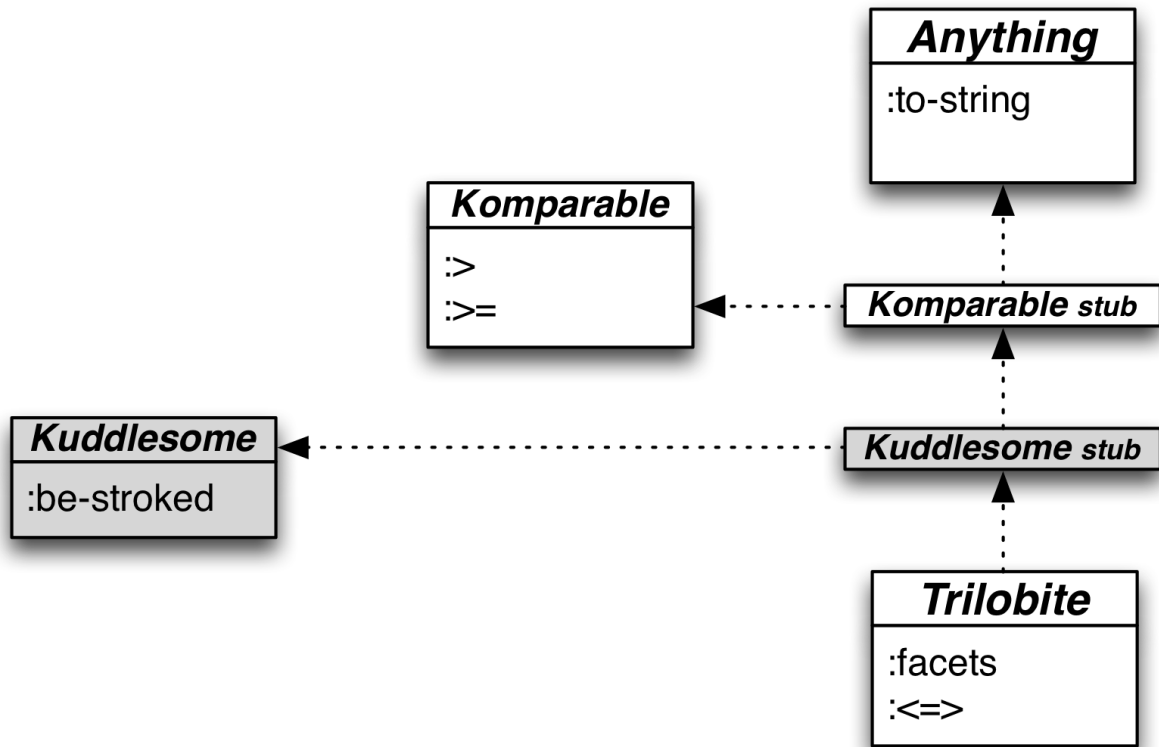
Therefore, Ruby adds a layer of indirection. `:include` doesn't put a module in the superclass chain. Instead, it constructs a small stub that points both upward and to the left:



There are two other complications you need to understand before implementation. First, a class may include more than one module, as shown here:

```
1 (send-to Trilobite :include Kuddlesome)
2 (send-to Trilobite :include Komparable)
```

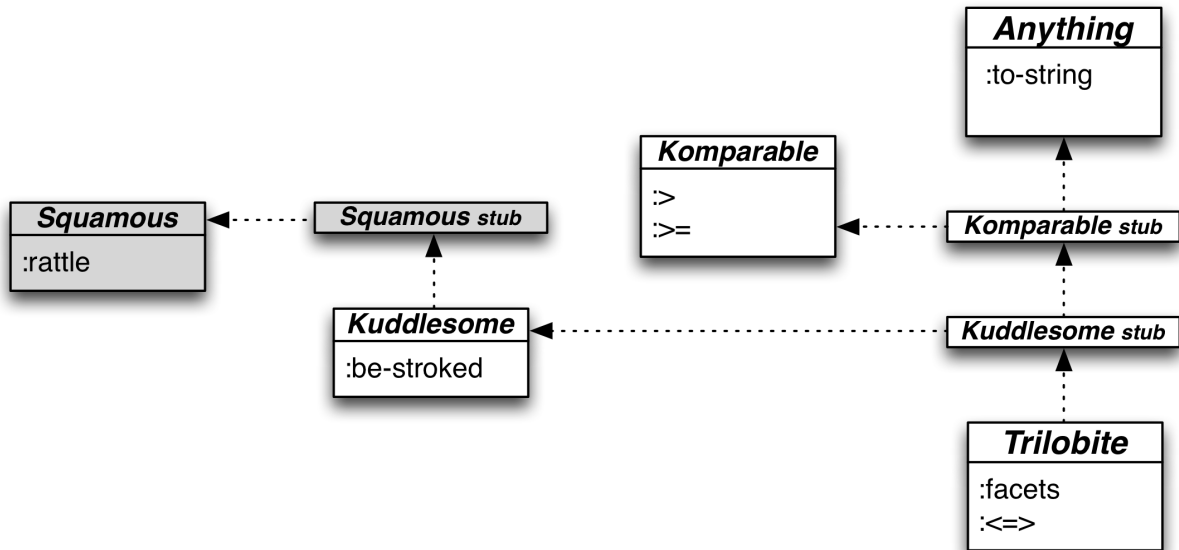
That should produce this structure:



Second, a module may include another module. For example, Kuddlesome may include Squamous:

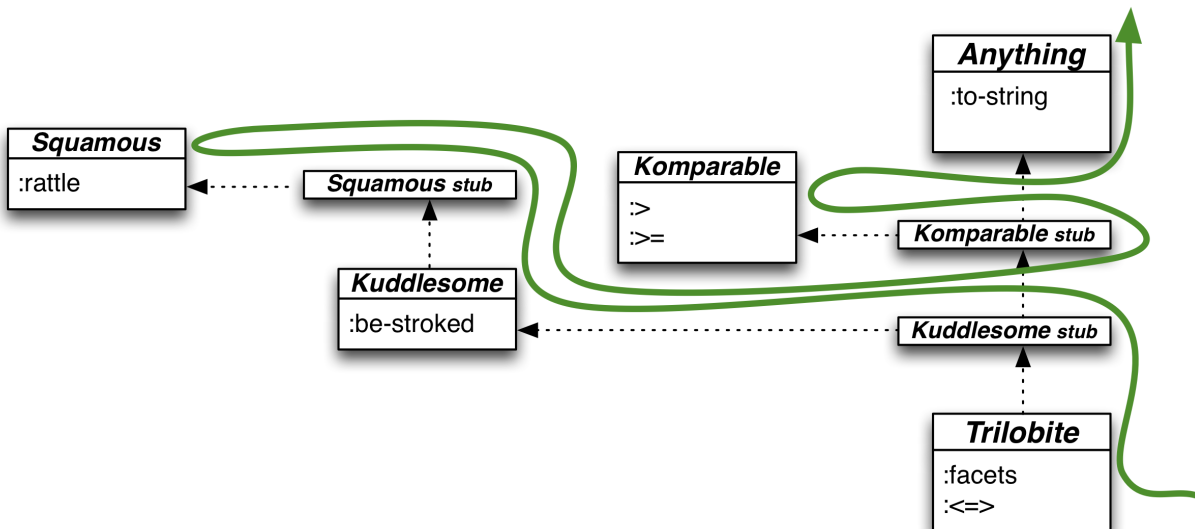
```
1 (send-to Kuddlesome :include Squamous)
```

That should produce this structure:

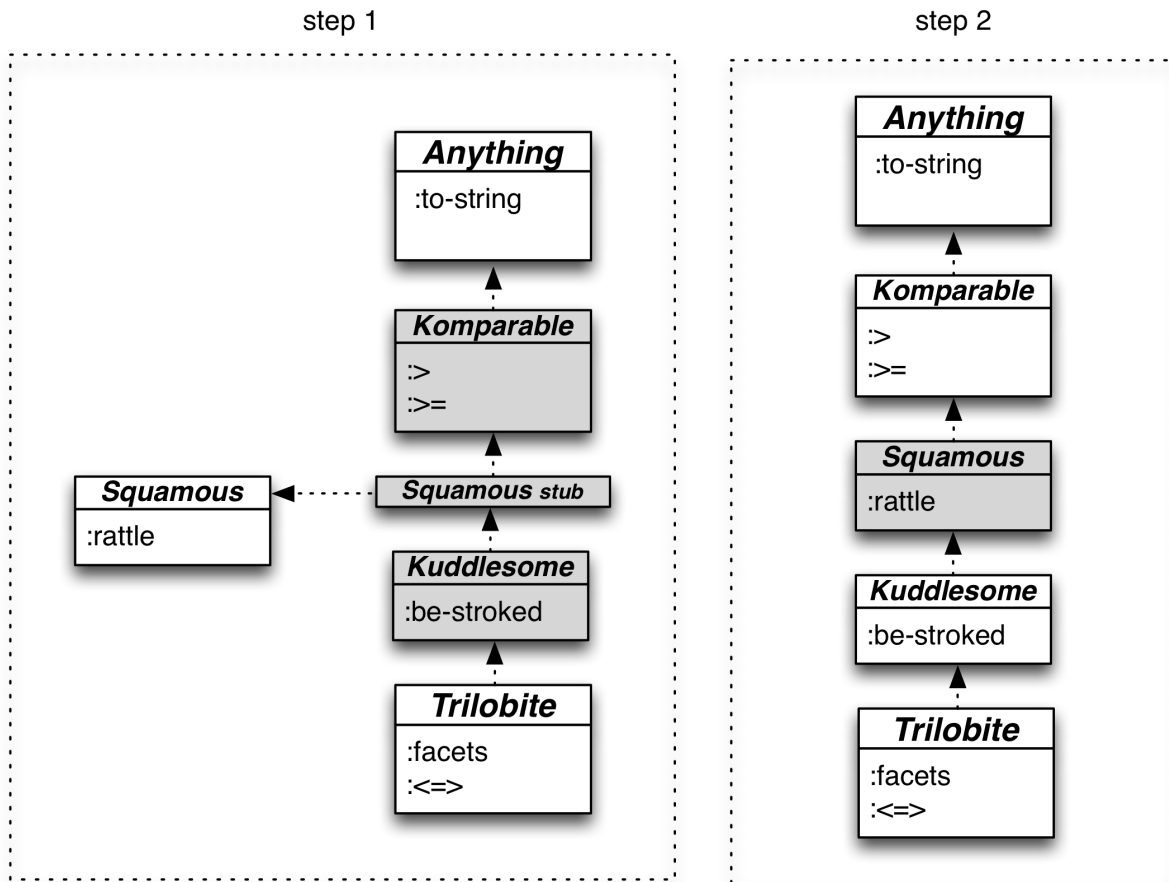


19.3 How dispatch works with modules

Our current version of lineage assumes that methods are found by going left than up. Modules make that lookup more complicated. One way to think about a lineage with modules is that the algorithm should recursively go left, then up, whenever it encounters a module stub:



Alternately, you can think of each stub being recursively replaced by what it points to on its left. That would look like this:



19.4 Adding module to the class structure

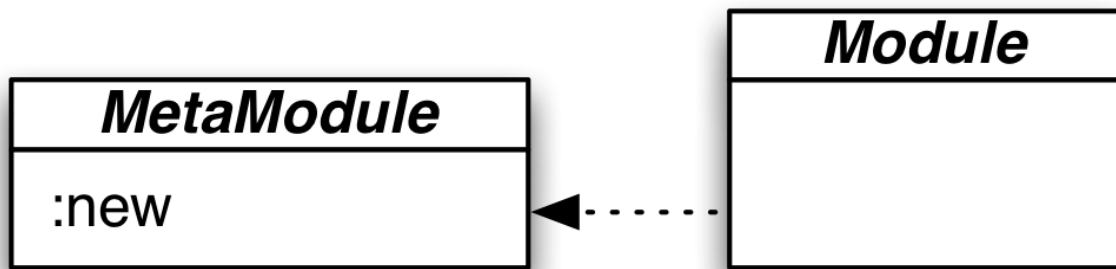
Modules like `Komparable` have to be created. It seems sensible to create them like this:

```

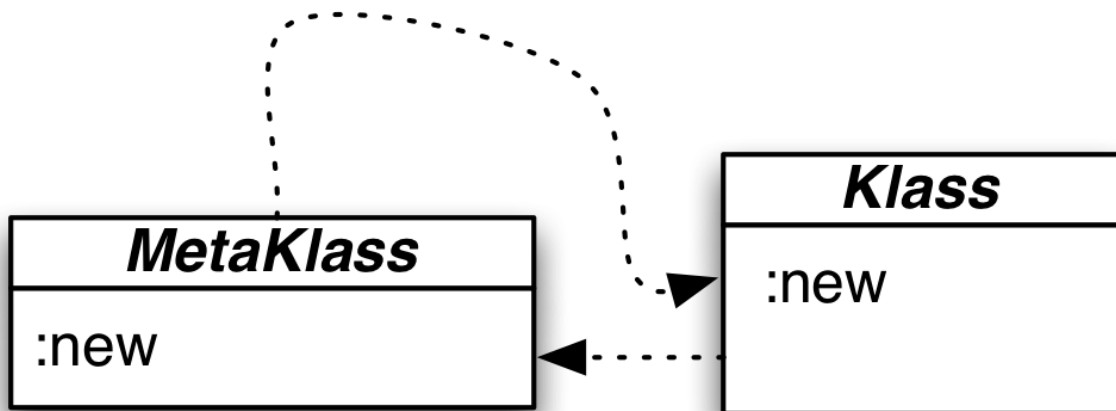
1 (send-to Module :new 'Komparable
2     {:= (fn [this that] ...)
3     := (fn [this that] ...)
4     ...})

```

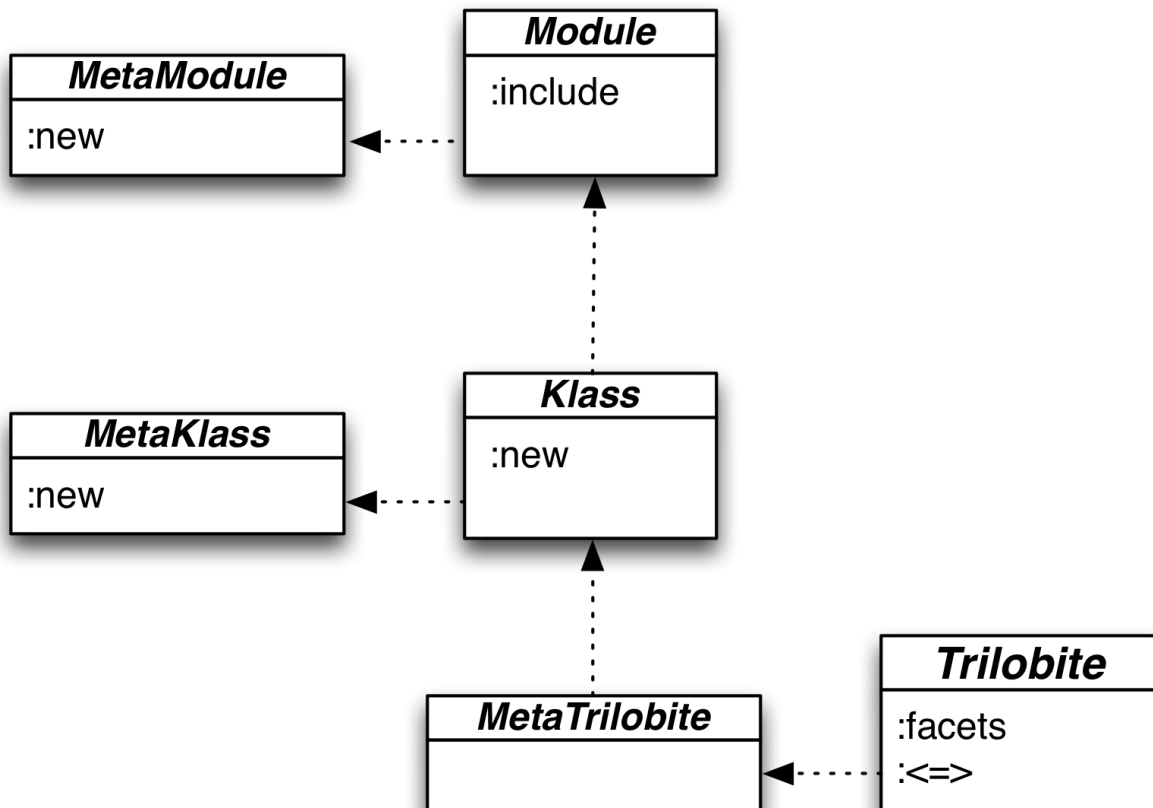
That means we have a new class/metaclass pair:



Let's contrast that to `Klass` and `MetaKlass`:



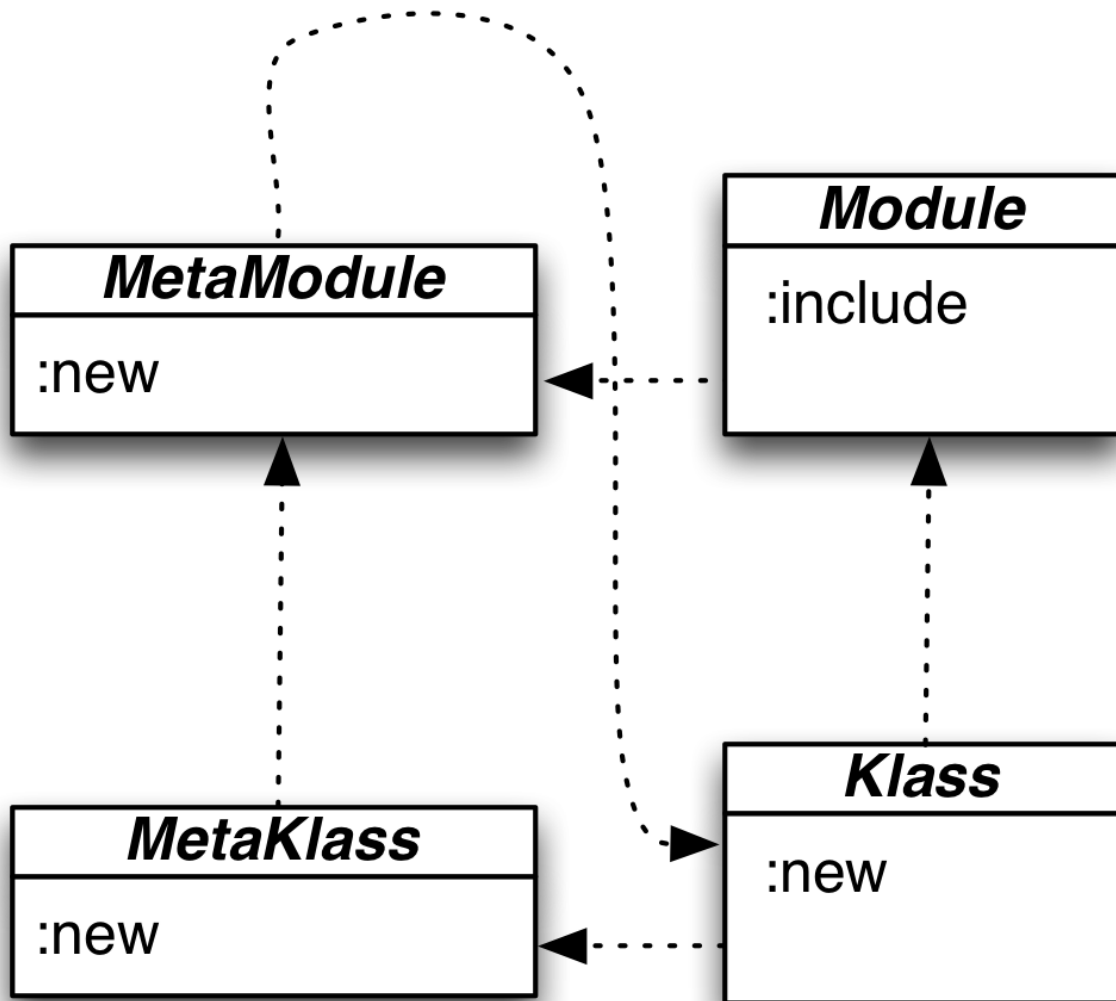
One difference between the two pairs is that classes can create instances but modules cannot. (The `Module` object does not contain a `:new` method.) A similarity (not yet shown in the diagram) is that both classes and modules respond to the `:include` message. We can tidily accommodate both those facts by making `Klass` a subclass of `Module`:



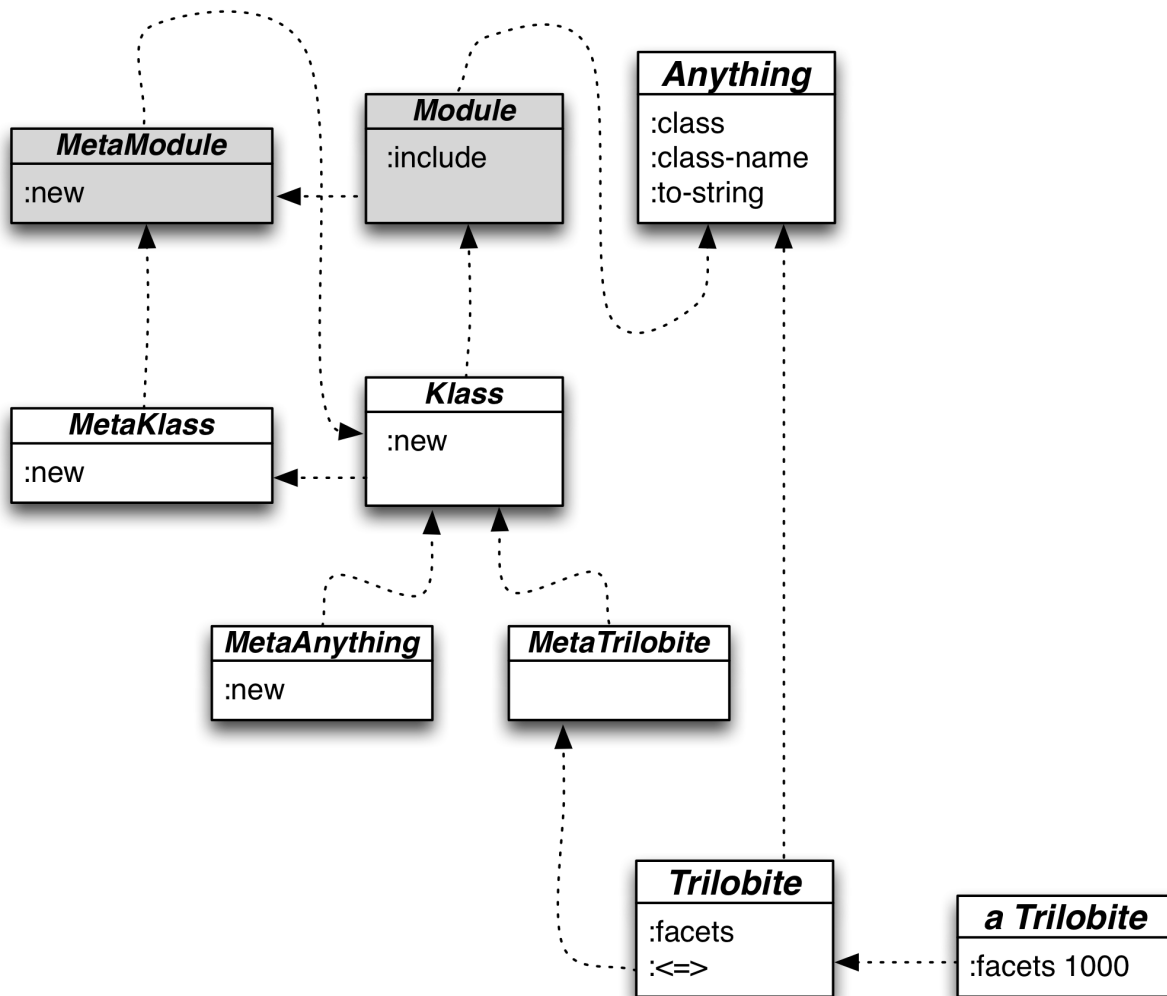
Let's review what that means:

Message	Behavior
(send-to Klass :new 'Trilobite)	MetaKlass makes a new class
(send-to Trilobite :new)	Klass makes a new instance
(send-to Module :new 'Komparable)	MetaModule makes a new module
(send-to Komparable :new)	Invalid, since Module has no :new method
(send-to Trilobite :include ...)	Module makes new methods available to Trilobite
(send-to Komparable :include...)	Module makes new methods available to Komparable

There are more arrows to adjust. MetaKlass used to have an upward link to Klass, which is appropriate because metaclasses are classes. It now has an upward link to MetaModule. To preserve MetaKlass's status as a class, and to make MetaModule a class itself, MetaModule should trace upward to Klass:



In effect, we've divided the behavior of Class into two parts: Class and Module, and put Module just above Class in the hierarchy:



So that's the plan. You'll do the work. In the exercises, you'll add `Module` to the hierarchy, implement its `:new` and `:include` methods, and modify the dispatch function to take module stubs into account.

19.5 Exercises

As I noted earlier, I've changed the names in the source now that we have both classes and modules. As a result, this is what a `Trilobite` instance looks like:

```

1 user=> (send-to Trilobite :new 3)
2 {[:facets 3, :__left_symbol__ Trilobite] ; :__left_symbol__ now, not :__class_sym\
3 bol__

```

And here's the `Trilobite` class:

```

1 user=> Trilobite
2 {:__up_symbol__ Anything,           ;; <== new name
3  :__own_symbol__ Trilobite,
4  :__left_symbol__ MetaTrilobite    ;; <== new name
5  :__methods__
6   {:add-instance-values ...
7    :facets :facets,
8    :<=> ...}}

```

(Notice that I also changed :__instance_methods__ to just plain :__methods__.)

The source is in [sources/modules.clj](#). My exercise solutions are in [solutions/modules.clj](#).

Exercise 1

Add a Module superclass above Klass. Implement rudimentary versions of :new and :include such that the following works:

```

1 user=> (def Kuddlesome (send-to Module :new 'Kuddlesome))
2 user=> Kuddlesome
3 {:__own_symbol__ Kuddlesome}
4 user=> (send-to Trilobite :include Kuddlesome)
5 "Module Kuddlesome will someday be included into Trilobite"

```

Exercise 2

In this exercise, you'll implement the real version of this:

```

1 user=> (send-to Module :new 'Kuddlesome
2         {:be_stroked (fn [this] "purrrrrrrr....")}))

```

This is similar to (send-to Klass :new...) in that it takes the name of the module and a map of methods. It's different because it doesn't take a superclass name or a map of class methods. (Since there are no class methods, there's no need to create a metaclass for the new module.)

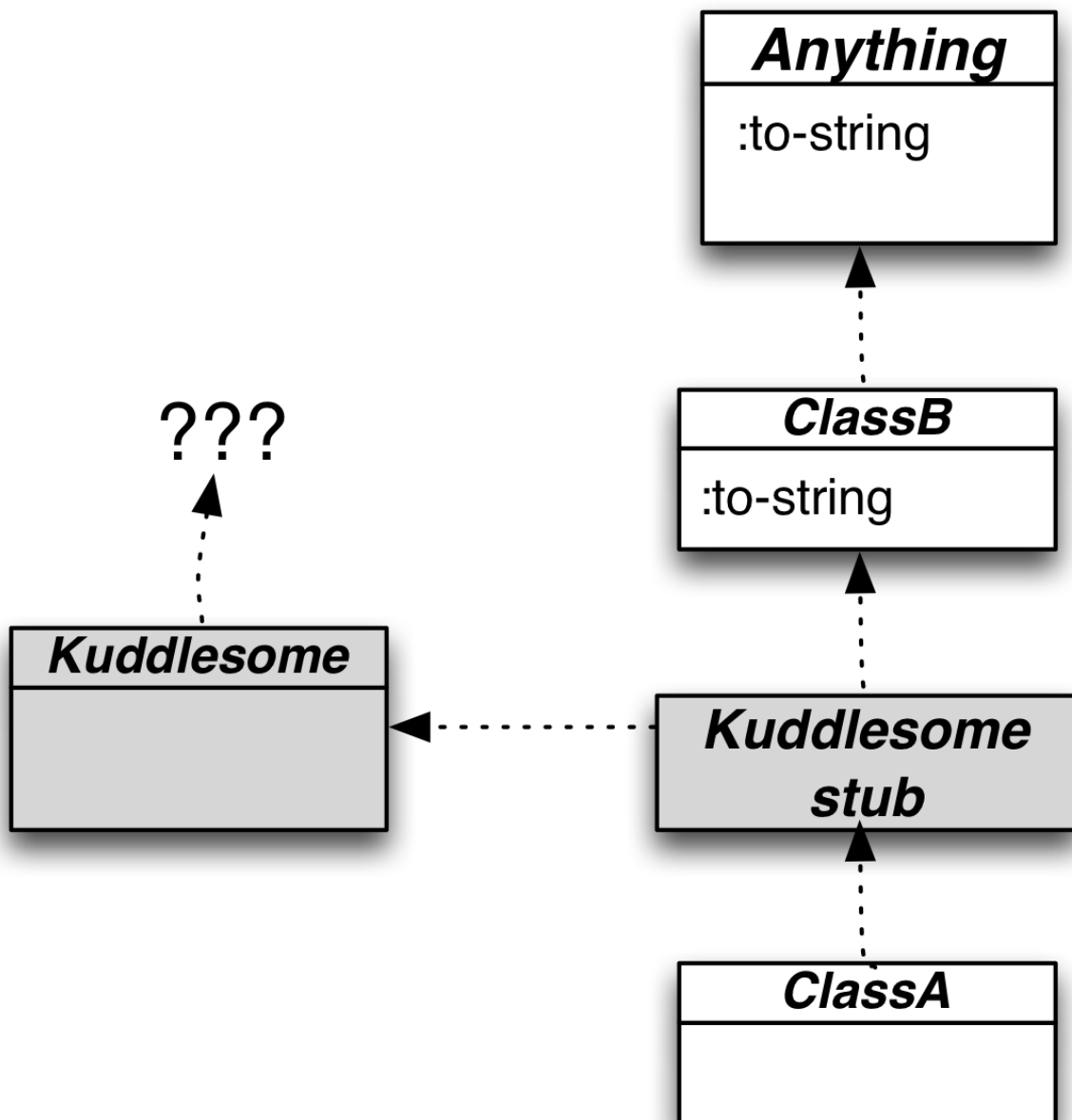
The challenging part of this exercise is to decide what :__left_symbol__ and :__up_symbol__ should be for the new module.

Hint: Use `method-holder` (the renamed version of `basic-class`) to create the module.

Hint: To think about what's to the left of a module, consider: that object is responsible for defining the method that responds to the following message:

```
1 (send-to Kuddlesome :include SomeOtherModule)
```

Hint: To think about what's above a module, consider this inheritance structure:



Given the following, which `:to-string` should run?

```
1 user=> (def a (send-to ClassA :new))
2 user=> (send-to a :to-string)
```

Exercise 3

In this exercise, you'll insert a module stub in an inheritance hierarchy. It will point up to whatever was above the class or module that included it, and point left to the actual module.

Our hierarchy is built from symbols, not direct pointers. That is, the superclass of `Trilobite` is the symbol `Anything`, not the map that symbol is bound to. That's useful because it lets us redefine a class and have the changed definition immediately take effect in its instances. (We don't have to recreate them.) But what name should we use for a module stub? Each one has to be unique (because it's spliced into a particular part of the inheritance chain).

Here's how you create a symbol that's guaranteed to be unique:

```
1 user=> (gensym 'Kuddlesome)
2 Kuddlesome73
```

When you've completed this exercise, you'll be able to type the following:

```
1 user=> (:__up_symbol__ Trilobite)
2 Anything
3 user=> (send-to Trilobite :include Kuddlesome)
4 user=> (:__up_symbol__ Trilobite)
5 Kuddlesome73
6 user=> (:__up_symbol__ Kuddlesome73)
7 Anything
8 user=> (:__left_symbol__ Kuddlesome73)
9 Kuddlesome
```

Make sure that this behavior also works when you include a module in a module.

Hint: Use the `install` function to install a modified class or new module.

Exercise 4

Add a new case to `lineage-1` that handles module stubs. Once that's done, inheritance, `:ancestors`, and `:class-name` should all work.

Hint: You first need a way to identify when a symbol is bound to a module stub (rather than a class or module).

Hint: Think recursion. When a module stub is encountered, the entire lineage to its left should be added to the lineage so far.

1. Trilobites, like insects and lobsters, were crustaceans. They were around for about 270 million years, finally dying out about 250 million years ago. They're notable for their compound eyes and their easily fossilized exoskeleton. They're such a popular fossil that attractive specimens are routinely assembled from partial specimens, or simply cast entirely in resin and glued to a rock. You've doubtless heard the phrase "as phony as a Moroccan *burmeisterella*"—now you know where it comes from. [↩](#)

20. Dynamic Scoping and send-super

In this chapter, we'll revise our implementation to eliminate the explicit this argument. That's a step toward implementing send-super, a function that lets a method delegate to a version of itself in a superclass.

To do that, I need to introduce one final Clojure feature.

20.1 Dynamic scope

Consider these functions:

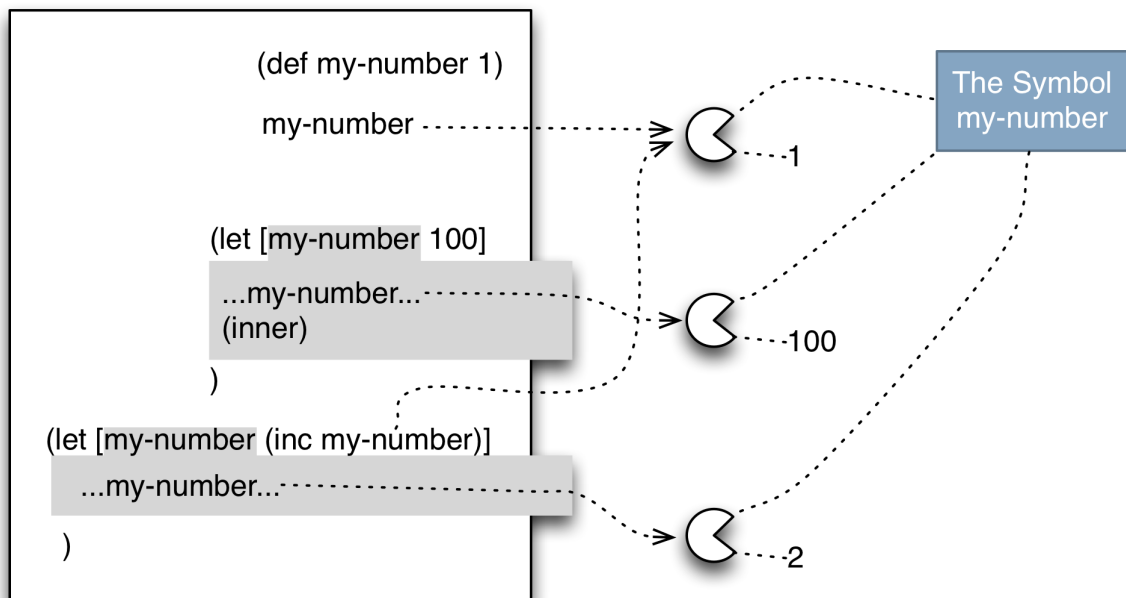
```
1 user=> (def my-number 1)
2 user=> my-number
3 1
4 user=> (def inside
5         (fn []
6           (println "starting value of my-number:" my-number)
7             (let [my-number (inc my-number)]
8               (println "rebound value of my-number:" my-number)))))
9 user=> (def outside
10        (fn []
11          (let [my-number 100] ; Has no effect on behavior of `inside`
12            (inside))))
13
14 user=> (outside)
15 starting value of my-number: 1
16 rebound value of my-number: 2
```

Although both outside and inside bind the same variable, my-number, they create *different* bindings that cannot affect one another. Even though inside is called from within the body of outside's let expression, which has bound my-number, the binding of my-number that inside's first println sees is that of the earlier def, the one before and outside of both outside and inside.

This happens because let (and function parameter lists) are *lexically scoped*. The bindings they produce follow visibility rules that are based on nesting levels in the *text* of the program.

What follows is a picture of these lexical scopes. The outermost box is the global scope. The gray areas are scopes created by `let` expressions. The function parameter lists also introduce scopes, but they don't bind anything. Because of that, I'm not showing them or their code.

The flow of control is from the top down: first the `def` is executed, then `outer` makes its binding. Within the scope of that binding, it calls `inner`, which makes its own binding:

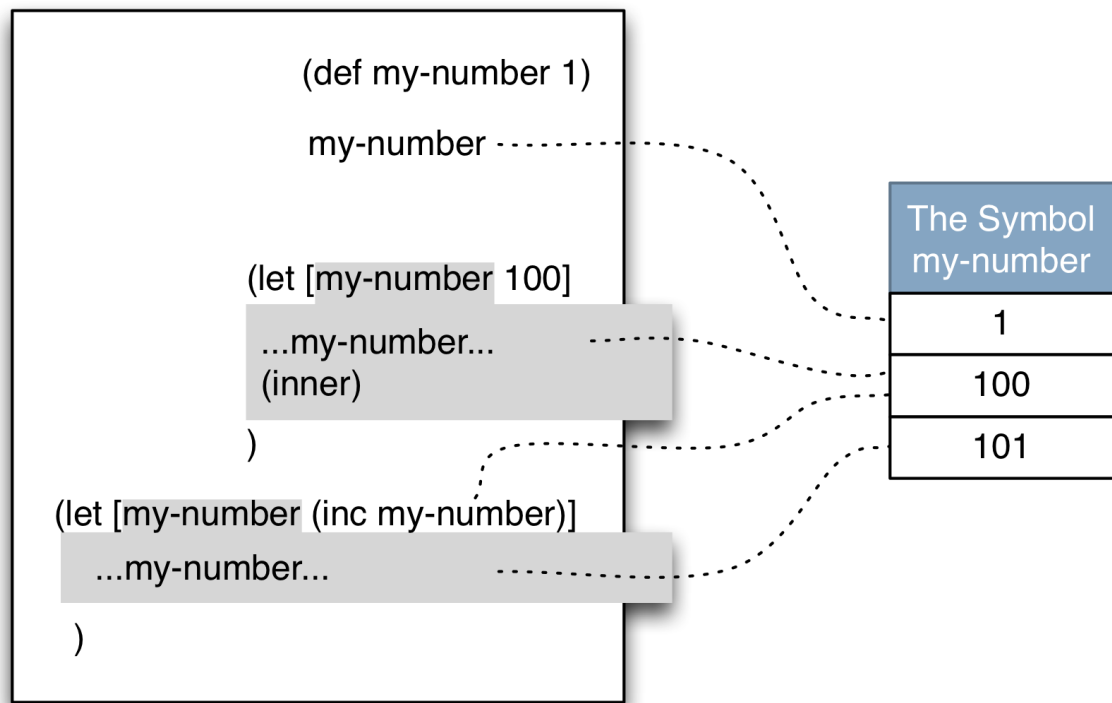


Lexical scoping

- Each `let` or parameter list introduces a new scope.
- Distinct bindings of the same symbol are independent.
- Unless code is textually nested within a binding construct, it cannot “see” it.

We're so used to lexical scoping that the idea `inside` could see into `outside` seems crazy. That's interesting because, if you look at the history of programming languages, it's amazing how long it took many extremely smart people to understand lexical scoping. Truly it's wonderful that what was so hard for the giants of the past has become “intuitively obvious” to us pygmies today.

If it took them a long time to understand lexical scoping, what did they use instead? It's called *dynamic scoping*. Think of it as if there is a single stack of bindings for each symbol (rather than multiple bindings that happen to refer to the same symbol). In the original Lisps, `let` expressions and function application would push new values onto that stack. Any reference to the bound value of a symbol, anywhere in the program, would get the top value. In this regime, the earlier example has a different picture:



Dynamic scoping

- Each `let` or parameter list introduces a new scope.
- All bindings of the same symbol share a stack of values.
- A scope can “see” a scope it’s not textually nested within, if that other scope bound a symbol the first scope uses.

With the exact same code, dynamic scoping produces a different result:

```

1 oldlisp=> (outside)
2 starting value of my-number: 100    ;; not 1
3 rebound value of my-number: 101    ;; not 2

```

In modern Lisps, like Clojure, lexical scoping is the default. But many of them support dynamic scoping as an alternative. In Clojure, you can make a symbol available for dynamic scoping like this:

```
1 user=> (def ^:dynamic this nil)
```

let continues to use lexical scoping, but there's a separate form that introduces a dynamic scope:

```
1 user=> (binding [this {:value 33}]
2         (:value this))
3 33
```

And that binding form allows us to have functions with an implicit this:

```
1 user=> (def increase-by-a-lot
2         (fn [] (assoc this
3                     :value (* 2 (:value this)))))
4
5 user=> (binding [this {:value 33}]
6         (increase-by-a-lot))
7 {:value 66}
```

20.2 Implicit this

This section builds the code in [sources/dynamic.clj](#).

An implicit this requires only one substantive change to our code. The function apply-message-to must bind this instead of passing it in. Here's the old version:

```
1 (def apply-message-to
2   (fn [method-holder instance message args]
3     (let [method (message (method-cache method-holder))]
4       (if method
5         (apply method instance args)           ;; <== old
6         (send-to instance :method-missing message args)))))
```

And the new one:

```

1 (def apply-message-to
2   (fn [method-holder instance message args]
3     (let [method (message (method-cache method-holder))]
4       (if method
5         (binding [this instance] (apply method args))    ;; <== new
6         (send-to instance :method-missing message args))))

```

All the methods need to be stripped of their `this` argument. Here's an example from `Point`:

```

1 :add
2 (fn [other]
3   (send-to this :shift (send-to other :x)
4     (send-to other :y)))

```

Somewhat annoyingly, we can no longer define methods just via symbols. `Point`'s accessors used to look like this:

```

1      :x :x
2      :y :y

```

Now they must look like this:

```

1      :x (fn [] (:x this))
2      :y (fn [] (:y this))

```

Another slight annoyance is that I used to name the first argument of class methods `class`:

```

1      :origin
2      (fn [class] (send-to class :new 0 0))

```

But now we have to use the implicit `this`. (Because class methods are instance methods of classes.)

```

1      :origin
2      (fn [this] (send-to this :new 0 0))

```

20.3 Ruby's `super`

A method *shadows* (or *overrides*) another if it has the same name as a method in a superclass. We'll give methods the ability to call the methods they shadow. Our design is patterned after Ruby's.

Ruby has two notations for calling shadowing methods. Here is the first:

```
1  def some_method(arg1, arg2)
2    ...
3    super(arg1)
4    ...
5  end
```

That calls a shadowed `some_method`, passing it a single argument.

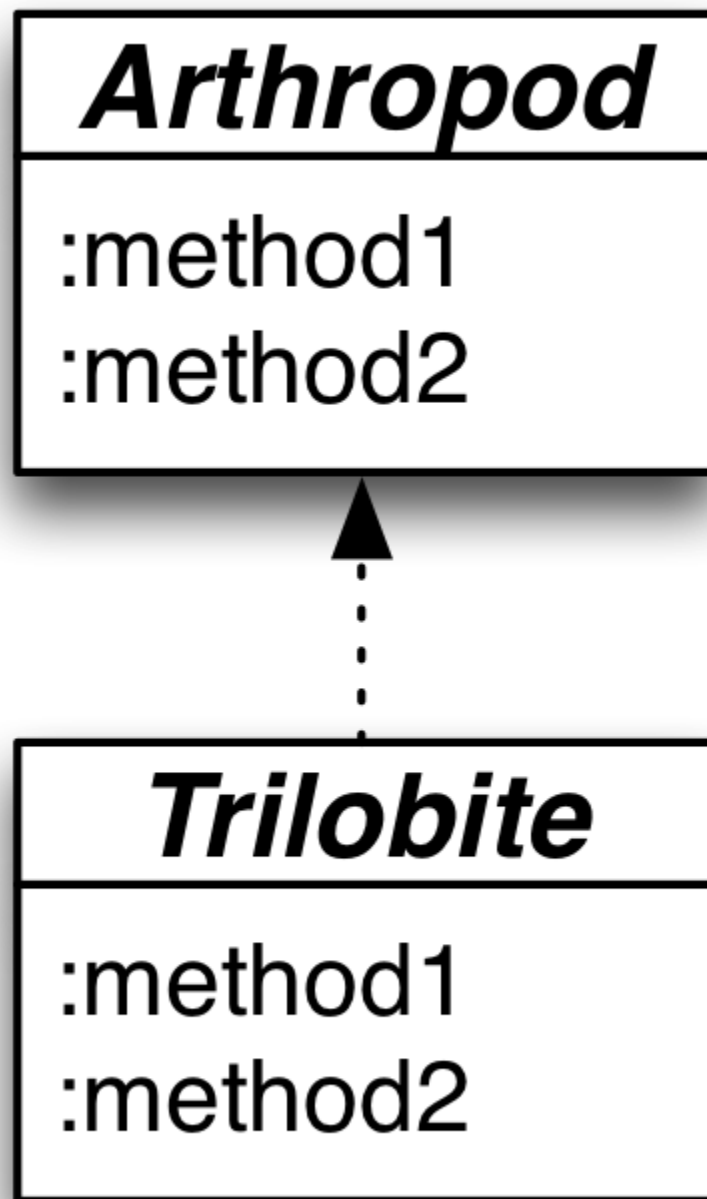
The second notation is shorthand for a common case:

```
1  def some_method(arg1, arg2)
2    ...
3    super
4    ...
5  end
```

That calls the shadowed `some_method`, giving it the same arguments as the shadowing method got. It's the same as this:

```
1  def some_method(arg1, arg2)
2    ...
3    super(arg1, arg2)
4    ...
5  end
```

Notice that Ruby's `super` doesn't let you send an arbitrary message to the superclass. That is, suppose you have this class structure:



... and you have a `Trilobite` object executing its `method1`. There is no use of `send-super` that allows that `method1` to invoke `Arthropod`'s `method2`. It can only delegate to `Arthropod`'s `method1`.

In our object system, we'll have two distinct functions, `send-super` and `repeat-to-super`. Here's the first:

```

1      :calculate
2      (fn [x y z]
3        (send-super x y))

```

Here's the second:

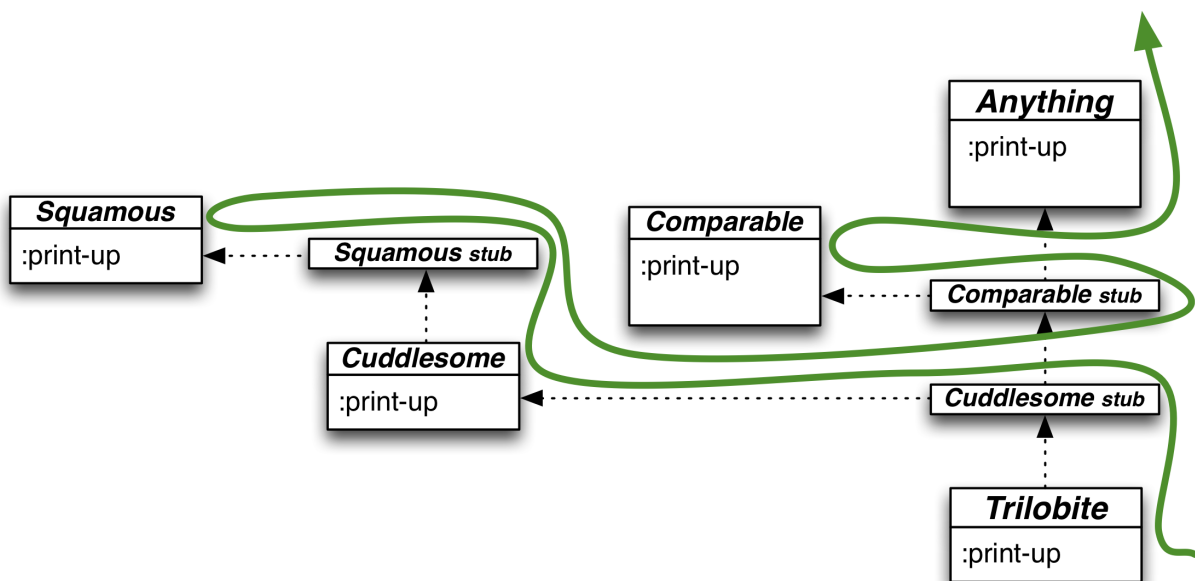
```

1      :calculate
2      (fn [x y z]
3        (* 10 (repeat-to-super))) ; passes x, y, and z

```

To correctly emulate Ruby, we also need to understand the interaction between super and modules. Ruby allows super inside module functions, and it allows super to pass control to module functions.

Consider this inheritance structure and its lineage:



Notice that each method holder has a :print-up method. Suppose each of them prints the name of the holder and then calls send-super. Then here would be the result of calling :print-up on a Trilobite instance:

```

1 In Trilobite
2 In Cuddlesome
3 In Squamous
4 In Comparable
5 In Anything
6 Error: No superclass method `:print-up` above `Anything`.

```

20.4 A design

Consider the example that ended the next section. When `:Cuddlesome` uses `send-super`, the search for the next method has to start with `Squamous`. That suggests we need a dynamically-bound symbol that records the method holder that holds the currently-executing method. We'll call that `holder-of-current-method`. A `send-super` search must always start at the method holder above it.

So now we have two dynamic variables:

```
1 (def ^:dynamic this nil)
2 (def ^:dynamic holder-of-current-method nil) ;;<== New
```

Now consider `send-super`:

```
1 :calculate
2 (fn [x y z]
3   (send-super x y))
```

In order to use `holder-of-current-method` to find the shadowed method, it has to know the shadowed method's name. Once again, we can stash that in a dynamically-bound symbol:

```
1 (def ^:dynamic this nil)
2 (def ^:dynamic holder-of-current-method nil)
3 (def ^:dynamic current-message nil) ;;<== New
```

Finally, consider `repeat-to-super`:

```
1 :calculate
2 (fn [x y z]
3   (* 10 (repeat-to-super))) ; passes x, y, and z
```

If it is to pass the current argument list to the shadowed method, it must have access to it. Another dynamic variable:

```
1 (def ^:dynamic this nil)
2 (def ^:dynamic holder-of-current-method nil)
3 (def ^:dynamic current-message nil)
4 (def ^:dynamic current-arguments) ;;<== New
```


(Does it seem to you that we have way too many distinct variables? If so, you're right. But we'll fix that in the next chapter.)

The implementation falls out of these bindings.

20.5 Exercises

You'll start with the sources in [sources/dynamic.clj](#). My solutions are in [solutions/send-super.clj](#).

I expect you'll find your solution uncomfortably messy. Mine is too. Don't worry about that—it'll be fixed in the next chapter.

Exercise 1

The current implementation of `apply-message-to` looks like this:

```
1 (def apply-message-to
2   (fn [method-holder instance message args]
3     (let [method (message (method-cache method-holder))] ;; <==
4       (if method
5         (binding [this instance] (apply method args))
6         (send-to instance :method-missing message args)))))
```

Recall that `method-cache` merges a bunch of maps together, which means it loses the information of where a matching method came from. Implement `find-containing-holder-symbol` such that the following work:

```
1 user=> (find-containing-holder-symbol 'Point :shift)
2 Point
3 ;; The following case is important: make sure you
4 ;; get the *first* method holder, not the last.
5 user=> (find-containing-holder-symbol 'Point :to-string)
6 Point
7 user=> (find-containing-holder-symbol 'Point :class-name)
8 Anything
9 user=> (find-containing-holder-symbol 'Point :nonsense)
10 nil
```

By building it on top of `lineage`, you'll ensure that it works with both classes and modules.

Change `apply-message-to` to use your new function.

Note that `apply-message-to` takes a message holder (a map), whereas `find-containing-holder-symbol` takes a symbol. That turns out to be handiest later.

Exercise 2

Change `apply-message-to` so that it binds `holder-of-current-method`, `current-message`, and `current-arguments` as well as this.

You can use the following class to confirm the bindings. It's defined in [sources/send-super-exercises.clj](#).

```
1 user=> (send-to Klass :new
2           'DynamicPoint 'Point
3           {
4             :shift
5             (fn [xinc yinc]
6               (println "Method" current-message "found in" holder-of-
current\
7 t-method)
8               (println "It has these arguments:" current-arguments))
9             })
10          {}})
11 user=> (def point (send-to DynamicPoint :new 1 2))
12 user=> (send-to point :shift 100 200)
13 Method :shift found in DynamicPoint
14 It has these arguments: (100 200)
```

Exercise 3

`send-super-exercises.clj` provides a function called `throw-no-superclass-method-error`. Using it, implement a function called `next-higher-holder-or-die`. Here are its two behaviors:

```
1 user=> (binding [current-message :to-string
2                 holder-of-current-method 'Point]
3         (next-higher-holder-or-die))
4 Anything
5
6 user=> (binding [current-message :shift
7                 holder-of-current-method 'Point]
8         (next-higher-holder-or-die))
9 Error No superclass method `:shift` above `Point`. [...]
```

Hint: You'll find the already-existing utility functions `method-holder-symbol-above` and `held-methods` useful.

Exercise 4

Implement `send-super`.

`send-super-exercises.clj` provides examples you can use to test your solution.

Hint: There are four dynamically-bound symbols. `send-super` can leave some of them alone, but must rebind others.

Hint: The expression in `send-super` that applies the superclass method is very similar to the `apply` expression in `apply-message-to`. The difference is that the values are dynamic instead of supplied to the function.

Exercise 5

Implement `repeat-to-super`. Again, look in `send-super-exercises.clj` for test cases.

21. Making an Object out of a Message in Transit

This chapter creates [sources/consolidation.clj](#).

The current code has too many globals: `this`, `holder-of-current-method`, `current-message`, and `current-arguments`. They all have something to do with the particulars of a message “in transit”. They should be lumped together. In this chapter, I’ll rewrite last chapter’s code to put them in a map, then I’ll let you do something interesting: converting such maps into `ActiveMessage` objects (giving you the odd experience of using message-sending to implement message-sending).

Ruby doesn’t have an `ActiveMessage` object, but other languages such as [Self](#), [Io](#), and [Ioke](#) do. It’s great fun to see how far you can ride the “everything should be an object” impulse. We won’t go as far as those languages do, but `ActiveMessage` will give you a taste of it.

21.1 The map

We’ll represent a message in transit with a map having these keys:

:target:

The receiver of the message. Because method writers will want to keep using `this`, that symbol will be made a synonym for this value.

:name:

The message name as a keyword. The second argument to `send-to`.

:args:

A sequence of the arguments sent with the message.

:holder-name:

The method holder wherein a match for the message name was found. `send-super` and `repeat-to-super` should start their search above this

value.

For this chapter, I'm going to call such a map an *active message*, and I'll say that such a message contains the information needed to *activate* a method (that is, apply it to the arguments). The terminology isn't particularly important: I'm mainly using it because you'll be basing your exercise solutions on some 350 lines of code, and I want the code you'll need to change to stand out.

Active messages will be dynamically bound to this symbol:

```
1 (def ^:dynamic *active-message* nil)
```

I'm using “earmuffs” in the name because that's the Clojure convention. I violated that convention in the previous chapter because I thought **this** would look too weird.

21.2 The programmer interface

Here's how the three main functions use **active-message**:

```
1 (def send-to
2   (fn [instance message-name & args]
3     (activate-method (fresh-active-message instance message-name args))))
4
5 (def repeat-to-super
6   (fn []
7     (activate-method (using-method-above *active-message*))))
8
9 (def send-super
10  (fn [& args]
11    (activate-method (assoc (using-method-above *active-message*)
12                           :args args))))
```

21.3 Implementation

Here are brief descriptions of each of the important functions. I expect this will be dull to read, so you might just want to use it for reference during the exercises.

activate-method looks up the method corresponding to the *:holder-name* and *:name*, then applies that method inside the context of a rebound

active-message and this.

```
1 (def activate-method
2   (fn [active-message]
3     (binding [*active-message* active-message
4               this (:target active-message)]
5       (apply (method-to-run active-message)
6               (:args active-message)))))
7
8 (def method-to-run
9   (fn [active-message]
10    (get (held-methods (:holder-name active-message))
11         (:name active-message))))
```

fresh-active-message creates the message map. To support method-missing, it constructs a fallback map when it can find no holder matching the message name:

```
1 (def fresh-active-message
2   (fn [target name args]
3     (let [holder-name (find-containing-holder-symbol
4                        (:__left_symbol__ target) name)]
5       (if holder-name
6           {:name name, :holder-name holder-name, :args args,
7            :target target}
8           (fresh-active-message target
9                                  :method-missing
10                                  (vector name args))))))y
```

using-method-above replaces the :holder-name argument with the next one up the hierarchy (or dies trying).

```
1 (def using-method-above
2   (fn [active-message]
3     (let [symbol-above (method-holder-symbol-above
4                        (:holder-name active-message))
5           holder-name (find-containing-holder-symbol
6                        symbol-above (:name active-message))]
7       (if holder-name
8           (assoc active-message :holder-name holder-name)
9           (throw (Error. (str "No superclass method `"
10                               (:name active-message)
11                               "` above `"
12                               (:holder-name active-message)
13                               "`."))))))
```

21.4 Exercises

Begin working with [sources/consolidation.clj](#). My solutions are in [solutions/message-class.clj](#). You can use classes in [sources/message-class-exercises.clj](#) for testing.

Exercise 1

First, implement an `ActiveMessage` class such that the following works:

```
1 user=> (def point (send-to Point :new 1 2))
2 user=> (def m (send-to ActiveMessage :new
3           :name :shift
4           :holder-name 'Point
5           :args [100 200]
6           :target point))
7 user=> (send-to m :name)
8 :shift
9 user=> (send-to m :holder-name)
10 Point
11 user=> (send-to m :args)
12 [100 200]
13 user=> (send-to m :target)
14 {:y 2, :x 1, :__left_symbol__ Point}
```

Hint: You can make a map with `(apply hash-map [:key1 "value1" :key2 "value2"])`.

Hint: There's no reason not to use `(send-to Klass :new)` to make `ActiveMessage`.

Exercise 2

Change `fresh-active-message` to construct and return a properly initialized `ActiveMessage` object. For fun, start by adding `(send-to ActiveMessage :new ...)` to its code. Can you make a new `Point`?

After that fails, try to come as close to a real message-send as you can without getting into an infinite loop.

There's a `Snooper` class in `message-class-exercises.clj` that lets you inspect the current `*active-message*`.

Hint: Looking at older versions of send-to might help.

Hint: Use basic-object to create a method, and call ActiveMessage's :add-instance-values directly.

Hint: Don't forget to bind this for the call to ActiveMessage's method.

Exercise 3

Let's start moving functions into ActiveMessage. We can't move fresh-active-message into ActiveMessage because that would cause an infinite loop. But what about using-message-above, which changes an existing ActiveMessage? It's used in repeat-to-super and send-super:

```
1 (def repeat-to-super
2   (fn []
3     (activate-method (using-method-above *active-message*))))
4
5 (def send-super
6   (fn [& args]
7     (activate-method (assoc (using-method-above *active-message*)
8                             :args args))))
```

Could we replace using-method-above in the above with a message send? Like this:

```
1 (def repeat-to-super
2   (fn []
3     (activate-method (send-to *active-message* :move-up))))
4
5 (def send-super
6   (fn [& args]
7     (activate-method (assoc (send-to *active-message* :move-up)
8                             :args args))))
```

Try it! Move the body of using-message-above into a :move-up method on ActiveMessage.

For this exercise, just make the smallest changes required.

The modified versions of repeat-to-super and send-super are in message-class-exercises.clj, along with a SubSnooper class that will help you test your solution.

Exercise 4

Show that, even though `ActiveMessage` is intimately tied into the internals of the system, it too can send messages. You'll do that by editing `:move-up`.

First, change code like this:

```
1      ... (:holder-name this) ... ; directly access an "instance variable"
```

... to this:

```
1      ... (send-to this :holder-name) ... ; use a getter method
```

Next, split `move-up` into more than one method. It's too big.

Exercise(ish) 5

I think you can see where this is going. The game is to find pieces of the system that can be moved into classes, creating new classes along the way. You win the game by having the smallest number of free-standing functions.

I won't take you any further down this path, but if this is the kind of game you like, I encourage you to make some more moves on your own.

I suspect my next move would be to work on `find-containing-holder-symbol`. It's called once from within the class `ActiveMessage` and once from the free-standing function `fresh-active-message`. We can't just change `fresh-active-message` to send a message to `ActiveMessage` (it would cause an infinite loop), which makes me think there should be another class that specializes in moving around the object hierarchy. `ActiveMessage` can delegate some of its work to that.

A test suite will help you. If you're using Leiningen (`lein repl`) and install the [Midje Leiningen plugin](#), you can run a halfway-decent set of tests for the object model like this:

```
1 734 $ lein midje solutions.ts-message-class-continued
2 All claimed facts (68) have been confirmed.
```

To make these tests use your solution, change the following line in [test/solutions/ts message class continued.clj](#).

```
1 ;;; You should replace this file with your own.
2 (load-file "solutions/message-class.clj")
```

Exercise 6

While the other readers are working on the previous exercise, we'll look at what else you can do with `ActiveMessage`.

Presented for your consideration: a tense true-crime drama. We begin with a criminal, feeling secure in his anonymity, who taunts a constable:

```
1 user=> (def criminal (send-to Criminal :new))
2 user=> (def police (send-to Police :new "Biggles"))
3 user=> (send-to criminal :taunt police)
4 Criminal: "Ha ha copper! You'll never catch me!"
```

Unbeknownst to him, a reader of this book (you) has implemented a mechanism by which the receiver of a message can discover its sender. Armed with that, the constable can execute this:

```
1 (let [evildoer (send-to *active-message* :sender)]
2   (send-to evildoer :give-yourself-up))
```

... which causes the thief to react:

```
1 Criminal: It's a fair cop, but society is to blame.
```

It's your job to implement a `:sender` method on `ActiveMessage` that returns the object that sent the message. That given, these are the classes (available in `message-class-exercises.clj`) that execute the drama:

```
1 (send-to Klass :new
2   'Criminal 'Anything
3   {
4     :taunt
5     (fn [copper]
6       (let [taunt "Ha ha copper! You'll never catch me!"]
7         (println "Criminal:" taunt)
8         (send-to copper :be-taunted taunt)))
9   })
```

```

10         :give-yourself-up
11         (fn []
12           (let [confession "It's a fair cop, but society is to blame."]
13             (println "Criminal:" confession)))
14       }
15     {}))

```

```

1 (send-to Klass :new
2   'Police 'Anything
3   {
4     :add-instance-values
5     (fn [name]
6       (assoc this :name name))
7
8     :name (fn [] (:name this))
9
10    :be-taunted
11    (fn [taunt]
12      (let [evildoer (send-to message :sender)]
13        (cl-format true "Detective ~A: Wot? Who?~%"
14                      (send-to this :name))
15        (println "<nab/>")
16        (send-to evildoer :give-yourself-up)))
17    }
18  {}))

```

Exercise 7

Give each ActiveMessage a link back to the message that came before it.

Add a `:trace` method to ActiveMessage. Here's an example of its output for the Top, Middle, and Bottom classes in `message-class-exercises.clj`:

```

1 user=> (def traceful
2         (send-to (send-to Bottom :new "one") :chained-message 1))
3 user=> (pprint (send-to traceful :trace))
4 ({:super-count 0,
5   :target {:value "one", :__left_symbol__ Bottom},
6   :holder-name Bottom,
7   :args (1),
8   :name :chained-message}
9  {:super-count 0,
10   :target {:value "two", :__left_symbol__ Bottom},
11   :holder-name Bottom,
12   :args (10),
13   :name :secondary-message}
14  {:super-count 1,
15   :target {:value "two", :__left_symbol__ Bottom},

```

```

16   :holder-name Middle,
17   :args (10),
18   :name :secondary-message}
19 ...

```

Note that `call-super` or `repeat-to-super` increments a new `:super-count` value in `ActiveMessage`. That supports a message trace printer I wrote, just for fun:

```

1 user=> (friendly-trace (send-to traceful :trace))
2 a-Bottom-1 =stands=for=> {:value "one", :__left_symbol__ Bottom}
3 a-Bottom-2 =stands=for=> {:value "two", :__left_symbol__ Bottom}
4 -----
5 (send-to a-Bottom-1 Bottom.chained-message 1)
6 (send-to a-Bottom-2 Bottom.secondary-message 10)
7   | delegate (10) to Middle
8   | delegate (100) to Top
9 (send-to a-Bottom-2 Middle.tertiary-message 1000)
10  | delegate (10000) to Top
11 -----
12 a-Bottom-1 =stands=for=> {:value "one", :__left_symbol__ Bottom}
13 a-Bottom-2 =stands=for=> {:value "two", :__left_symbol__ Bottom}

```

You can use `friendly-trace` to check your implementation. If it prints these results, you've formatted your `:trace` output correctly.

VI GLOSSARY

apply:

To apply a function to some [arguments](#) is to substitute each argument for its formal [parameter](#) and then [evaluate](#) (execute) the function. I'll also write “invoke a function” or “call a function” when they read better. They all mean the same thing.

argument:

In this book, I reserve “argument” for the actual [values](#) to which a function is [applied](#). I use [parameter](#) for the symbols in a function definition's parameter list.

atom:

In Clojure, an atom is a “container” for a value. The atom can be mutated to hold a different value (*not* to change the value within it). The change is made by fetching the current value, passing it to a function, and storing the function's return value. If two threads attempt to modify the atom at the same time, Clojure guarantees that one will complete before the other begins.

binding:

A binding associates a [symbol](#) with a [value](#).

binding value:

In this book, used to contrast with [monadic values](#). A [monad](#) accepts a monadic value, processes it, and then binds the resulting binding value to a [symbol](#) to make it available to later steps.

class:

A class describes a collection of similar [instances](#). It may describe the data those instances contain (by naming [instance variables](#)). It may

also describe the [methods](#) that act on those instance variables.

class method:

A class method is executed by [sending a message](#) to a [class](#), rather than to an [instance](#). In languages like Ruby and the embedded language of [Part 1](#), classes *are* instances, so when you send a message to an instance that happens to be a class, you get an instance method of that class object, which we call a class method. That is, there's no *implementation* difference between `a-point.foo` and `Point.new`. See [The Class as an Object](#) chapter.

classifier function:

The classifier takes the arguments to a [generic function](#) and usually converts them into a small number of values that are used to select a [specialized function](#).

closure:

A function that can be applied to arguments but that also has permanent access to all name/value [bindings](#) in its [environment](#) at *the moment of function creation*. As such, it can make use of named values defined “outside” itself, even after the names that refer to those values cease to do so.

collecting parameter:

In a [recursive function](#), a collecting parameter is one that is passed a closer approximation to the final solution in each nested recursive call. See [the explanation](#) in the book.

constructor:

A constructor creates an [instance](#) based on the information in a [class](#). The resulting instance is “of” that class.

continuation:

During a computation, the continuation is a description (in the form of a [function](#)) of the computation that remains to be done.

continuation-passing style:

Writing a computation as the calculation of one [value](#) that is then passed to a [function](#) that represents the [continuation](#) of the computation. See the [description](#) in the text.

dataflow style:

A programming style that emphasizes data flowing through a series of functions and being transformed at each stage.

depth-first traversal:

A tree traversal in which, if the traversal has a choice whether to go down first or right first, it chooses “down”.

destructuring binding:

When a sequence is passed as an argument, destructuring binding lets you bind parameter names to elements of the sequence without having to bind the whole sequence to a name and then pick it apart with code.

dispatch function:

When a name can refer to more than one function, the dispatch function decides which function to [apply](#) by examining the [argument list](#).

double dispatch

A kludge required in conventional object-oriented programming, used when the correct behavior depends on both this and another object. See the [discussion](#) in the book.

duck typing:

A way of defining class relationships used in languages without static types. Inspired by the saying “If it walks like a duck and talks like a duck, it’s a duck”. When duck typing, you don’t define one class as depending on another’s type but rather on particular [messages](#) it [responds to](#). It differs from (say) Java’s interfaces in that the sets of messages aren’t distinct named entities in the program, but rather implicit groups, one for each purpose.

dynamic scoping:

When a symbol is evaluated to find its bound value, the binding that's used is the one *most recently evaluated* during execution of the program. The position of the binding code in the program's text is irrelevant. Contrast to [lexical scoping](#).

eager evaluation:

The opposite of [lazy evaluation](#). Computation is performed immediately, rather than as values are demanded.

encapsulation:

Making the [binding](#) between a [symbol](#) and a [value](#) invisible to code outside a function or object boundary.

environment:

The environment collects all [symbol/value bindings](#) in effect at a particular moment.

evaluator:

An evaluator converts a data structure, usually obtained from the [reader](#), into a [value](#). See [the explanation in the text](#). Clojure's evaluator is named `eval`.

function:

In general terms, a function is some executable code that is given arguments and produces a value. In Clojure, a function is specifically a [closure](#).

future:

A future converts a computation into a [value](#). A computation wrapped in a future executes on a different thread. If the value of the future is ever referenced, and the computation is not finished, the referencing thread is paused until the value is computed.

generic function:

In conventional object-oriented programming, the [dispatch function](#) looks only at the type of the [object](#) given as the implicit "this" argument. Generic functions provide a different strategy, in which the

dispatch function is user-provided and can use any argument. In Clojure, generic functions are defined with `defmulti`. Generic functions encourage a verb-centered way of thinking about the world: there are actions that can apply very broadly. The specifics of an action depends on some properties (determined at runtime) of the values it's applied to.

global definition:

In a global definition, a function is [bound](#) to a symbol using `def`. Such a function can be used by any other function in the [namespace](#). Contrast with [local definition](#).

higher-order function:

A [function](#) that either takes a function as an [argument](#) or produces a function as its return value.

immutability:

In Clojure, data structures cannot be modified once created. Within functions and `let` forms, the association of a [symbol](#) to a [value](#) cannot be changed once made (because there is no assignment statement in Clojure).

instance:

Synonymous with [object](#), but emphasizes that the instance is one representative of a [class](#) (from which it is [instantiated](#)).

instance method:

The [method](#) applied in response to a [message](#) sent to an [instance](#). Used when a distinction between instance methods and [class methods](#) is useful. More usually, an unqualified “method” is used.

instance variable:

The data an [instance](#) holds can be thought of as a collection of name/value pairs. “Instance variable” can refer to the name part or to both parts. For example, “initialize the instance variable to 5” associates a value with the name. In Clojure and other languages with [immutable](#) data, instance variables don't ever vary.

instantiation:

Creating an [instance](#) by allocating space, associating runtime-specific [metadata](#) with it, and then calling a class-specific function to initialize [instance variables](#).

keyword:

A clojure datatype, written like `:my-keyword`. Keywords [evaluate](#) to themselves and are often used as the key in a [map](#). Keywords are [callable](#)s.

lazy evaluation:

In a fully lazy language, no computation is performed unless some other computation demands its results. In effect, [evaluation](#) is a “pull” process, where the need to print some output ripples “backward” to provoke only those computations that are needed. Clojure is not fully lazy, but it has the [lazyseq](#) data structure, which is.

lazy initialization:

In an object-oriented language, an [instance variable](#) is lazily initialized if its starting value is only calculated when some client code first asks for it.

lazyseq:

A Clojure [sequence](#) that uses [lazy evaluation](#).

lexical scoping:

The most common sort of binding in modern programming languages. When there is more than one binding for a symbol, evaluating that symbol uses the closest enclosing binding in *the text of the program*. Nothing in the execution of the program can change which binding is used. Contrast to [dynamic scoping](#).

list:

A clojure [sequence](#) that has the property that it takes longer to access the last element than the first. Lists are used both to hold data and to represent Clojure programs.

local definition:

A function definition that is either used immediately (as in `(partial + 1) 2`) and so has no name, or whose name is given in a `let` [binding](#) or a function's [parameter list](#). Whereas a function with a [global definition](#) can be used by any function in the [namespace](#), a local definition can be “seen” only within the body of its `let` or function definition.

macro:

A function that translates Clojure code into different clojure code. The transformed code is evaluated in the normal way. Macros are a way of inventing your own [special forms](#).

map:

As a noun, an unordered collection of key/value pairs, like a Java `HashMap` or a Ruby `Hash`. Maps are [callables](#).

As a verb, a function that [applies](#) a [callable](#) to each element of one or more collections. The return values are collected together and returned in a [lazyseq](#).

message:

A message is the name of a [method](#). When [functions](#) are used as methods, we use the metaphor that the program [sends a message](#) and [arguments](#) to an [object](#).

metaclass:

A [class](#) that describes a class in the same way that a class describes an [instance](#). Metaclasses store the [methods](#) invoked in response to a [message](#) sent to a class object.

metadata:

Data about data. An example in this book is the pointer from an [instance](#) to its [class](#), which the [dispatch function](#) uses when deciding which [method](#) to [apply](#).

method:

A method is a [function](#) with a (usually) implicit “this” or “self” argument that refers to an [instance](#) of a [class](#). Metaphorically, the method is invoked when a [message](#) of the same name is [sent](#) to the instance.

mock object:

A mock [object](#) is used to test whether [classes](#) use their collaborating classes correctly. It stands in for one of the object-under-test's neighbor objects. The test programs the mock to expect the object-under-test to [send it specific messages](#). If the mock object is not sent those messages, the test fails.

module:

In Ruby, a module is a class-like object that can be placed in the inheritance chain of a class

monad:

A set of functions that describes how to separate the steps of a computation from what happens between those steps.

monad transformer:

A function that takes one [monad](#) as its argument and produces another monad that has the properties of the argument monad plus different monad.

monadic function:

A function that takes a single [binding value](#) and converts it into a [monadic value](#).

monadic value:

The type (or shape) of value that a [monad](#) operates on. A monad accepts monadic values, may or may not do something to them, and provides the results to a computational step as a [binding value](#).

multimethod:

A synonym for [generic function](#).

multiple inheritance:

In multiple inheritance, an object can have more than one direct superclass, so its ancestors could form a complicated graph (with classes appearing more than once) rather than a simple sequence.

namespace:

Namespaces are Clojure's equivalent of packages or modules in other languages: a way of restricting the visibility of names to other parts of

a program. Roughly speaking, a namespace corresponds to a file. For purposes of this book, a namespace is a [map](#) from [symbols](#) to [values](#). (The reality is slightly more complicated.) There are functions that give one namespace access to values in another (by altering the client namespace's own map).

object:

Conventionally, [encapsulated mutable](#) state. Because of a [class](#) definition, certain [methods](#) can be applied to that state.

ORM:

Object-relational mapping. A library or framework that stores [objects](#) in a relational database and can reconstruct those objects later.

override (a method):

In an object-oriented language, a [method](#) defined in a subclass overrides a method with the same name in a superclass. In that case, the [dispatch function](#) applied to an [instance](#) of the subclass will pick the subclass version.

parameter:

In a function definition, the parameter list is a [vector](#) of [symbols](#). During function [application](#), those symbols are replaced with the corresponding [values](#) in the [argument list](#).

partial application:

Recasting a function of n arguments as one of $n-m$ arguments, where the m arguments are replaced by constants. In Clojure, `(partial + 3)` produces a function that adds three to its argument. Often also called “currying”, though that term is strictly incorrect.

point-free definitions:

Functions that are created without mentioning their [parameters](#). Creation is done with [higher-order-functions](#).

polymorphic:

When one name associated with potentially many functions (or [methods](#)). The [dispatch function](#) uses the [argument list](#) (perhaps

including the [receiver](#) of a [message](#)) to decide which function to [apply](#) to the arguments.

printer:

The printer converts the internal representation of data to output strings. See [the explanation in the text](#).

reader:

The reader converts text into Clojure's internal representation for data. See [the explanation in the text](#).

receiver:

In the [message/method](#) metaphor, the receiver is the particular [instance](#) to which a method is [applied](#).

recursion:

Traditionally, a book's definition of recursion reads "See [recursion](#)." Because I'm a humorless git, I'll point you to the [appropriate section](#) of this book.

repl:

The read-eval-print loop. It reads a Clojure expression, evaluates it, and prints the result. Also used to refer to Clojure's interactive interpreter. See [the explanation in the text](#).

respond to a message:

An [object](#) responds to a [message](#) if it has [method](#) with that name.

rest arguments:

When a function's [parameter list](#) contains an `&`, that signals that all remaining [arguments](#) should be collected into a [sequence](#) and associated with the parameter following the `&`. Those arguments are referred to as the "rest arguments".

send a message:

Sending a [message](#) is a stylized way of [applying](#) a function. The [dispatch function](#) uses the message, an [instance](#), and an [argument list](#) to find the function. That function is applied to an argument list composed of the original argument list and the instance.

seq:

Either a [list](#) or a [lazyseq](#).

sequence:

An umbrella term referring to Clojure's [list](#), [vector](#), and [seq](#) data types. All sequences can be indexed by integers (starting with 0).

set:

A datatype in Clojure that acts much like a mathematical set. In particular it's easy to test membership in a set. A set is a [callable](#). As such, it returns true iff its single argument is in the set.

shadowing:

When [symbols](#) can be defined to refer to [values](#) and a language allows such [binding](#) expressions to be nested, an enclosed definition shadows an enclosing one using the same name. In that case, [evaluation](#) of the symbol means the enclosed value.

In an object-oriented language, a [method](#) defined in a subclass shadows a method with the same name in a superclass. In that case, the [dispatch function](#) applied to an [instance](#) of the subclass will pick the subclass version.

side effect:

A “pure” function takes inputs, calculates a result, and does nothing else. A function with side effects can, during its calculation, change state in a way observable from outside the caller. For example, it may perform I/O. Or it may change the value of a global variable.

signature:

The name of a function (or [method](#)), together with its [parameter list](#).

software transactional memory:

Controlling read and write access to memory in a way similar to the way databases control write access to tables.

special symbol, special form:

When a [list](#) is being used to represent code, certain [symbols](#) are treated specially by the [evaluator](#). For example, `fn` heads a list used to create functions, and `quote` heads a list containing a [value](#) that should not be evaluated.

specialized function:

A specialized function is one that a [generic function](#) can [dispatch to](#). In Clojure, a specialized function is defined by `defmethod`.

state:

Data that can be [mutated](#), especially when the changes are made via [side effects](#).

structure sharing:

Languages that have only [immutable](#) data structures seem to require much wasteful copying. In fact, both the new and old copies will share most of their structure. This is analogous to video compression, where frame N+1 is stored as only what changed to the frame N.

substitution, substitution rule:

In a pure functional language, evaluation of code can (modulo optimization) be accomplished by successive substitution of values. See [the explanation in the text](#).

symbol:

A Clojure datatype that is typically used to refer to a [value](#).

syntactic sugar:

Special syntax in a language to make common operations easier to write. Often disparaged by purists. “Syntactic sugar causes cancer of the semicolon.”—Alan J. Perlis.

unbound symbol:

A [symbol](#) in an expression that is to be [evaluated](#) to yield a value—but no [binding](#) has been established for the symbol. (That is, it does not appear in an enclosing `let` or function [parameter list](#).)

value:

In this book, I use “value” to refer to any piece of Clojure data, be it an integer, a [list](#), a [vector](#), or whatever.

vector:

A Clojure [sequence](#) that has the property that the last element is as fast to access as the first. Vectors are [callable](#)s.

zipper:

A data structure that simulates random movement through, and editing of, immutable trees. They're explained in a [chapter](#).